

每个程序员都应该了解的内存知识

乌尔里希·德雷
珀 (Ulrich Drepper)
红帽公司

drepper@redhat.com
2007 年 11 月 21 日

抽象的

随着 CPU 内核变得更快、数量更多，大多数程序的限制因素现在并将在一段时间内是内存访问。硬件设计人员提出了更加复杂的内存处理和加速技术（例如 CPU 缓存），但如果没有程序员的帮助，这些技术就无法以最佳方式工作。不幸的是，大多数程序员都没有很好地理解使用计算机内存子系统或 CPU 高速缓存的结构和成本。本文解释了现代商品硬件上使用的内存子系统的结构，说明了为什么开发 CPU 缓存、它们如何工作以及程序应该如何利用它们来实现最佳性能。

1 简介

早期的计算机要简单得多。系统的各个组件，例如 CPU、内存、大容量存储和网络接口，是一起开发的，因此它们的性能相当平衡。例如，内存和网络接口在提供数据方面并不比 CPU 快（很多）。

一旦计算机的基本结构稳定下来并且硬件开发人员集中精力优化各个子系统，这种情况就会发生变化。突然间，计算机某些组件的性能明显落后，并出现了瓶颈。对于大容量存储和内存子系统尤其如此，由于成本原因，它们相对于其他组件的改进速度更慢。

大容量存储的缓慢问题主要是通过软件技术来解决的：操作系统将最常用（并且最有可能使用）的数据保存在主存储器中，其访问速度比硬盘快几个数量级。缓存存储已添加到存储设备本身，无需更改操作系统即可提高性能。¹出于本文的目的，我们不会详细讨论海量存储访问的软件优化。

与存储子系统不同，事实证明，消除主内存这一瓶颈要困难得多，而且几乎所有解决方案都需要更改硬件。到-

这些变化主要有以下几种形式：

- RAM 硬件设计（速度和并行性）。
- 内存控制器设计。
- CPU 缓存。
- 设备的直接内存访问 (DMA)。

在大多数情况下，本文档将讨论 CPU 缓存和内存控制器设计的一些影响。在探索这些主题的过程中，我们将探索 DMA 并将其带入更大的视野。然而，我们将从概述当今商品硬件的设计开始。这是理解有效使用内存子系统的问题和限制的先决条件。我们还将详细了解不同类型的 RAM，并说明为什么这些差异仍然存在。

本文件绝不是包罗万象的和最终的。它仅限于商用硬件，并进一步仅限于该硬件的子集。此外，许多主题将针对本文的目标进行足够详细的讨论。对于此类主题，建议读者查找更详细的文档。

当涉及到特定于操作系统的细节和解决方案时，本文专门描述了 Linux。它在任何时候都不会包含有关其他操作系统的任何信息。作者没有兴趣讨论对其他操作系统的影响。如果读者认为他/她必须使用不同的操作系统，他们必须去找他们的供应商并要求他们编写与此类似的文档。

开始前的最后一条评论。文本中多次出现“通常”一词和其他类似的限定词。这里讨论的技术是存在的

¹Changes are needed, however, to guarantee data integrity when using storage device caches.

现实世界中有很多很多的变体，本文仅讨论最常见、主流的版本。很少有人能够对这项技术做出绝对的陈述，因此也无法对其进行限定。

文件结构

本文档主要面向软件开发人员。它没有详细介绍硬件的技术细节，对面向硬件的读者来说没有多大用处。但在我们为开发人员提供实用信息之前，必须先打好基础。

为此，第二部分详细介绍了随机存取存储器 (RAM)。本节的内容很容易了解，但对于理解后面的部分来说并不是绝对重要的。在需要内容的地方添加了对该部分的适当反向引用，以便焦虑的读者可以首先跳过本部分的大部分内容。

第三部分详细介绍了 CPU 缓存行为的许多细节。使用图表可以避免文本变得枯燥无味。此内容对于理解文档的其余部分至关重要。第 4 节简要描述了虚拟内存是如何实现的。这也是其余部分所需的基础工作。

第 5 节详细介绍了非均匀内存访问 (NUMA) 系统。

第 6 节是本文的中心部分。它汇集了前面各节的所有信息，并为程序员提供了如何编写在各种情况下表现良好的代码的建议。非常不耐烦的读者可以从本节开始，如有必要，可以返回到前面的部分来刷新底层技术的知识。

第 7 节介绍了可以帮助程序员做得更好的工具。即使完全了解该技术，在一个重要的软件项目中问题出在哪里也远非显而易见。有些工具是必要的。

在第 8 节中，我们最后对在不久的将来可以预期的技术或可能只是简单地拥有的技术进行了展望。

报告问题

作者打算用一段时间来更新这个文档。这包括技术进步所必需的更新以及纠正错误。我们鼓励愿意报告问题的读者向作者发送电子邮件。他们被要求在报告中包含准确的版本信息。版本信息可以在文档的最后一页找到。

谢谢

我要感谢 Johnray Fuller 和 LWN 的工作人员（尤其是 Jonathan Corbet）承担了将作者的英语形式转变为更传统的形式的艰巨任务。Markus Armbruster 针对文本中的问题和遗漏提供了许多宝贵的意见。

关于本文档

本文的标题是对 David Goldberg 的经典论文“每个计算机科学家应该了解浮点运算的知识”的致敬 [12]。这篇论文仍然没有广为人知，尽管它应该是任何敢于接触键盘进行严肃编程的人的先决条件。

关于 PDF 的一句话：xpdf 绘制的一些图表相当糟糕。建议您仔细查看，或者如果确实需要的话，查看 Adobe 的程序。如果您使用 evince，请注意超链接在整个文档中广泛使用，即使查看者没有像其他人那样指出它们。

2 当今的商品硬件

了解商品硬件非常重要，因为专用硬件正在消退。如今，扩展通常是水平实现的，而不是垂直实现的，这意味着今天使用许多较小的、连接的商用计算机而不是一些非常大且异常快速（且昂贵）的系统更具成本效益。之所以会出现这种情况，是因为快速且廉价的网络硬件已广泛使用。在某些情况下，大型专用系统仍然占有一席之地，并且这些系统仍然提供商业机会，但总体市场与商品硬件市场相比相形见绌。红帽预计，截至 2007 年，对于未来的产品，大多数数据中心的“标准构建模块”将是一台最多具有四个插槽的计算机，每个插槽都装有一个四核 CPU，如果是英特尔 CPU，则将是超线程的。²这意味着数据中心的标准系统将拥有多达 64 个虚拟处理器。将支持更大的机器，但四插槽、四 CPU 核心机箱目前被认为是最佳选择，并且大多数优化都是针对此类机器。

由商品部件构成的计算机的结构存在很大差异。也就是说，我们将重点关注最重要的差异，涵盖 90% 以上的此类硬件。请注意，这些技术细节往往会迅速变化，因此建议读者考虑本文的撰写日期。

多年来，个人计算机和小型服务器标准化的芯片组由两部分组成：北桥和南桥。图 2.1 显示了该结构。

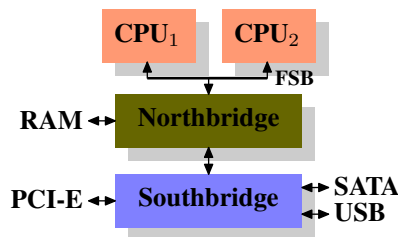


图 2.1：北桥和南桥结构

所有 CPU（前面的示例中为两个，但可以有更多）都通过公共总线（前端总线，FSB）连接到北桥。除其他外，北桥还包含内存控制器，其实现决定了计算机使用的 RAM 芯片的类型。不同类型的 RAM，例如 DRAM、Rambus 和 SDRAM，需要不同的内存控制器。

要访问所有其他系统设备，北桥必须与南桥进行通信。南桥，通常称为 I/O 桥，处理通信

通过各种不同的总线与设备进行灰化。如今，PCI、PCI Express、SATA 和 USB 总线最为重要，但南桥也支持 PATA、IEEE 1394、串行和并行端口。较旧的系统具有连接到北桥的 AGP 插槽。这样做是出于与北桥和南桥之间的连接速度不够快相关的性能原因。然而，如今 PCI-E 插槽全部连接到南桥。

这样的系统结构有许多值得注意的后果：

- 从一个 CPU 到另一个 CPU 的所有数据通信都必须通过用于与北桥通信的同一总线进行。
- 所有与 RAM 的通信都必须通过北桥。
- RAM 只有一个端口。³
- CPU 和连接到南桥的设备之间的通信通过北桥进行路由。

此设计中的几个瓶颈立即显而易见。其中一个瓶颈涉及设备对 RAM 的访问。在 PC 的早期，与任一桥上的设备的所有通信都必须通过 CPU，这对整体系统性能产生了负面影响。为了解决这个问题，一些设备变得能够直接内存访问（DMA）。DMA 允许设备在北桥的帮助下直接在 RAM 中存储和接收数据，而无需 CPU 的干预（及其固有的性能成本）。如今，连接到任何总线的所有高性能设备都可以使用 DMA。虽然这大大减少了 CPU 的工作负载，但由于 DMA 请求与 CPU 的 RAM 访问竞争，因此也造成了北桥带宽的争用。因此，必须考虑到这个问题。

第二个瓶颈涉及从北桥到 RAM 的总线。总线的具体细节取决于所部署的内存类型。在较旧的系统上，只有一条总线连接所有 RAM 芯片，因此不可能进行并行访问。最新的 RAM 类型需要两个独立的总线（或通道，因为它们被称为 DDR2，请参见第 8 页），这使可用带宽加倍。北桥通过通道交错进行内存访问。最近的内存技术（例如 FB-DRAM）增加了更多通道。

由于可用带宽有限，以最小化延迟的方式安排内存访问对于性能非常重要。正如我们将看到的，处理器速度要快得多

²Hyper-threading enables a single processor core to be used for two or more concurrent executions with just a little extra hardware.

³We will not discuss multi-port RAM in this document as this type of RAM is not found in commodity hardware, at least not in places where the programmer has access to it. It can be found in specialized hardware such as network routers which depend on utmost speed.

尽管使用了 CPU 缓存，但必须等待访问内存。如果多个超线程、内核或处理器同时访问内存，则内存访问的等待时间会更长。对于 DMA 操作也是如此。

然而，访问内存不仅仅是并发性。访问模式本身也极大地影响内存子系统的性能，特别是对于多个内存通道。在 2.2 节中，我们将介绍 RAM 访问模式的更多细节。

在一些更昂贵的系统上，北桥实际上并不包含内存控制器。相反，北桥可以连接到多个外部内存控制器（在以下示例中为四个）。

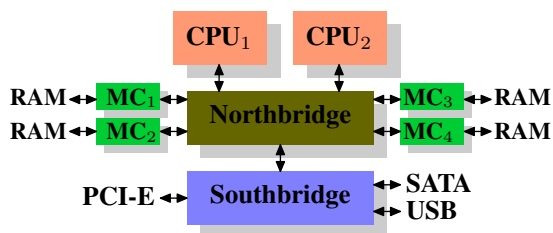


图 2.2: 带有外部控制器的北桥

这种架构的优点是存在多个内存总线，因此总可用带宽增加。这种设计还支持更多的内存。并发存储器访问模式通过同时访问不同的存储器组来减少延迟。当多个处理器直接连接到北桥时尤其如此，如图 2.2 所示。对于此类设计，主要限制是北桥的内部带宽，这对于该架构（来自英特尔）来说是非常显着的。⁴

使用多个外部内存控制器并不是增加内存带宽的唯一方法。另一种日益流行的方法是将内存控制器集成到 CPU 中并将内存连接到每个 CPU。这种架构因基于 AMD Opteron 处理器的 SMP 系统而变得流行。图 2.3 显示了这样一个系统。英特尔将从 Nehalem 处理器开始支持通用系统接口 (CSI)；这基本上是相同的方法：集成内存控制器，每个处理器都可以使用本地内存。

对于这样的架构，可用的存储体数量与处理器的数量一样多。在四 CPU 机器上，内存带宽增加了四倍，而不需要具有巨大带宽的复杂北桥。将内存控制器集成到 CPU 中还有一些额外的优点；我们不会挖

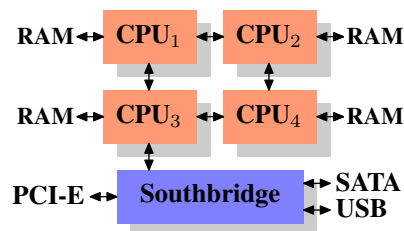


图 2.3: 集成内存控制器

在这里更深入地了解这项技术。

这种架构也有缺点。首先，由于机器仍然必须使系统的所有内存可供所有处理器访问，因此内存不再统一（因此称为 NUMA - 非统一内存架构 - 这种架构）。可以以通常的速度访问本地内存（连接到处理器的内存）。当访问连接到另一个处理器的内存时，情况会有所不同。在这种情况下，必须使用处理器之间的互连。要从 CPU₁ 访问连接到 CPU₂ 的内存，需要通过一个互连进行通信。当同一个 CPU 访问连接到 CPU₄ 的内存时，必须交叉两个互连。

每个这样的通信都有相关的成本。当我们描述访问远程内存所需的额外时间时，我们谈论“NUMA 因素”。图 2.3 中的示例架构对于每个 CPU 都有两个级别：直接相邻的 CPU 和一个远离两个互连的 CPU。对于更复杂的机器，级别的数量可以显着增加。还有一些机器架构（例如 IBM 的 x445 和 SGI 的 Altix 系列）存在不止一种连接类型。CPU 被组织成节点；在一个节点内，访问内存的时间可能是统一的，或者只有很小的 NUMA 因子。但是，节点之间的连接可能非常昂贵，并且 NUMA 系数可能非常高。

商品 NUMA 机器现已存在，并且可能在未来发挥更大的作用。预计从 2008 年底开始，每台 SMP 机器都将使用 NUMA。与 NUMA 相关的成本使得识别程序何时在 NUMA 计算机上运行非常重要。在第 5 节中，我们将讨论更多的机器架构和 Linux 内核为这些程序提供的一些技术。

除了本节其余部分描述的技术细节之外，还有几个影响 RAM 性能的其他因素。它们无法由软件控制，这就是本节不介绍它们的原因。有兴趣的读者可以在 2.1 节中了解其中一些因素。他们实际上只是为了更全面地了解 RAM 技术，并可能在购买计算机时做出更好的决策。

⁴For completeness it should be mentioned that such a memory controller arrangement can be used for other purposes such as “memory RAID” which is useful in combination with hotplug memory.

以下两节讨论门级硬件细节以及内存控制器和 DRAM 芯片之间的访问协议。程序员可能会发现这些信息很有启发性，因为这些细节解释了为什么 RAM 访问是这样工作的。不过，这是可选知识，渴望了解与日常生活更直接相关的主题的读者可以跳至第 2.2.5 节。

2.1 内存类型

多年来，RAM 出现了多种类型，每种类型都存在差异，有时甚至差异很大。如今，只有历史学家才对较旧的类型感兴趣。我们不会探讨这些细节。相反，我们将专注于现代 RAM 类型；我们只会触及表面，探索内核或应用程序开发人员通过其性能特征可见的一些细节。

第一个有趣的细节集中在为什么同一台机器上有不同类型的 RAM 的问题上。更具体地说，为什么会有静态 RAM (SRAM⁵) 和动态 RAM (DRAM)。前者速度更快并且提供相同的功能。为什么不是所有 RAM 都在机器 SRAM 中？正如人们所预料的那样，答案是成本。SRAM 的生产和使用比 DRAM 昂贵得多。这两个成本因素都很重要，第二个成本因素的重要性越来越高。为了理解这些差异，我们研究一下 SRAM 和 DRAM 的存储位的实现。

在本节的剩余部分中，我们将讨论 RAM 实现的一些底层细节。我们将保持尽可能低的细节水平。为此，我们将在“逻辑级别”而不是硬件设计人员必须使用的级别讨论信号。对于我们这里的目的而言，这种详细程度是不必要的。

2.1.1 静态RAM

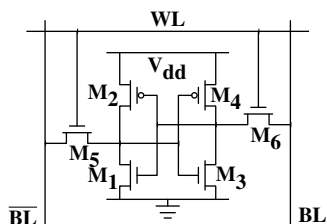


图 2.4: 6-T 静态 RAM

图 2.4 显示了 6 晶体管 SRAM 单元的结构。该单元的核心由四个晶体管 M1 至 M4 构成，这四个晶体管构成两个交叉耦合的反相器。它们有两个稳定状态，分别代表 0 和 1。只要 Vdd 上电可用，状态就稳定。

⁵在其他情况下，SRAM 可能意味着“同步 RAM”。

如果需要访问单元的状态，则字访问线 WL 被拉高。这使得单元的状态可以立即在 BL 和 BL_上 读取。如果必须覆盖单元状态，则首先将 BL 和 BL_上 线设置为所需值，然后提高 WL。由于外部驱动器比四个晶体管（M1 到 M4）更强，因此可以覆盖旧状态。

有关细胞工作方式的更详细描述，请参阅[20]。对于下面的讨论，重要的是要注意

- 一个单元需要六个晶体管。有四个晶体管的变体，但它们有缺点。
- 维持细胞状态需要恒定的功率。
- 一旦字访问线 WL 上升，单元状态几乎可以立即读取。该信号与其他晶体管控制信号一样是矩形信号（在两个二进制状态之间快速变化）。
- 单元状态稳定，不需要刷新周期。

还有其他速度较慢且耗电较少的 SRAM 形式可用，但这些形式在这里不感兴趣，因为我们正在研究快速 RAM。这些慢速变体之所以有趣，主要是因为它们的接口更简单，因此比动态 RAM 更容易在系统中使用。

2.1.2 动态RAM

动态 RAM 的结构比静态 RAM 简单得多。图 2.5 显示了常用 DRAM 单元设计的结构。它仅由一个晶体管和一个电容器组成。当然，这种复杂性的巨大差异意味着它的功能与静态 RAM 有很大不同。

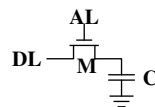


图 2.5: 1-T 动态 RAM

动态 RAM 单元将其状态保存在电容器 C 中。晶体管 M 用于保护对状态的访问。为了读取单元的状态，将访问线 AL 拉起；这要么导致电流在数据线 DL 上流动，要么不流动，具体取决于电容器中的电荷。为了写入单元，数据线 DL 被适当设置，然后 AL 升高足够长的时间以对电容器充电或耗尽。

动态 RAM 的设计存在许多复杂性。使用电容器意味着读取

电池对电容器放电。该过程不能无限地重复，电容器必须在某个时刻重新充电。更糟糕的是，为了容纳大量的单元（具有 10^9 或更多单元的芯片现在很常见），电容器的容量必须很低（在飞法范围或更低）。充满电的电容器可容纳数十或数千个电子。尽管电容器的电阻很高（几兆欧姆），但电容只需要很短的时间即可耗散。这个问题称为“泄漏”。

这种泄漏就是 DRAM 单元必须不断刷新的原因。对于现在的大多数 DRAM 芯片来说，这种刷新必须每 64 毫秒发生一次。在刷新周期期间，不可能访问存储器，因为刷新只是存储器读取操作，其中结果被丢弃。对于某些工作负载，此开销可能会导致高达 50% 的内存访问停滞（请参阅 [3]）。

微小电荷导致的第二个问题是从电池读取的信息不能直接使用。数据线必须连接到一个检测放大器，该放大器可以在整个电荷范围内区分存储的 0 或 1，但仍必须计为 1。

第三个问题是读取单元会导致电容器的电荷耗尽。这意味着每次读取操作之后都必须执行对电容器充电的操作。这是通过将检测放大器的输出反馈回电容器来自动完成的。但这确实意味着，阅读记忆内容需要额外的精力，更重要的是，需要时间。

第四個问题是电容器的充电和放电不是瞬时的。检测放大器接收到的信号不是矩形的，因此必须对单元输出何时可用进行保守估计。电容器充电和放电的公式为

$$Q_{\text{Charge}}(t) = Q_0(1 - e^{-\frac{t}{RC}})$$

$$Q_{\text{Discharge}}(t) = Q_0 e^{-\frac{t}{RC}}$$

这意味着电容器充电和放电需要一些时间（由电容 C 和电阻 R 决定）。这也意味着检测放大器可以检测到的电流并不是立即可用的。图 2.6 显示了充电和放电曲线。X 轴以 RC （电阻乘以电容）为单位进行测量， RC 是时间单位。

与静态 RAM 的情况不同，在静态 RAM 的情况下，当字访问线上升时，输出立即可用，而电容器总是需要一些时间才能充分放电。这种延迟严重限制了 DRAM 的速度。

这种简单的方法也有其优点。主要优点是尺寸。一个 DRAM 单元所需的芯片空间比 SRAM 小很多倍

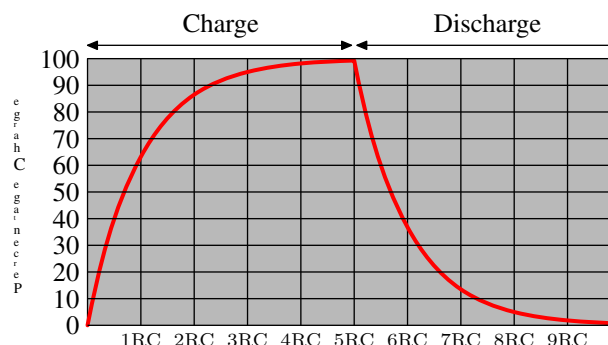


图 2.6: 电容器充电和放电时序

细胞。SRAM 单元还需要单独的电源来维持晶体管的状态。DRAM 单元的结构也更简单、更规则，这意味着将许多单元紧密地封装在芯片上更简单。

总体而言，成本上的（相当显著的）差异获胜。除了专用硬件（例如网络路由器）之外，我们还必须使用基于 DRAM 的主存储器。这对程序员有着巨大的影响，我们将在本文的其余部分讨论这一点。但首先我们需要研究 DRAM 单元实际使用的更多细节。

2.1.3 DRAM 访问

程序使用虚拟地址选择内存位置。处理器将其转换为物理地址，最后内存控制器选择与该地址相对应的 RAM 芯片。为了选择 RAM 芯片上的单个存储单元，部分物理地址以多个地址线的形式传递。

从内存控制器单独寻址内存位置是完全不切实际的：4GB RAM 将需要 2^{32} 地址线。相反，地址是使用较小的一组地址线编码为二进制数传递的。通过这种方式传递到 DRAM 芯片的地址必须首先进行多路分解。具有 N 地址线的多路分配器将具有 2^N 输出线。这些输出线可用于选择存储单元。对于小容量的芯片来说，使用这种直接的方法并不是什么大问题。

但如果细胞数量增加，这种方法就不再适用了。具有 1Gbit⁶ 容量的芯片需要 30 条地址线和 2 条³⁰ 选择线。当不牺牲速度时，解复用器的大小随着输入线的数量呈指数增长。除了解复用器的复杂性（大小和时间）之外，用于 30 个地址线的解复用器还需要大量的芯片空间。更重要的是，传输

⁶我讨厌那些国际单位制前缀。对我来说，千兆位永远是 2^{30} 位，而不是 10^9 位。

地址线上同步传输 30 个脉冲比“仅”传输 15 个脉冲要困难得多。需要以完全相同的长度或适当的时间布置更少的线。⁷

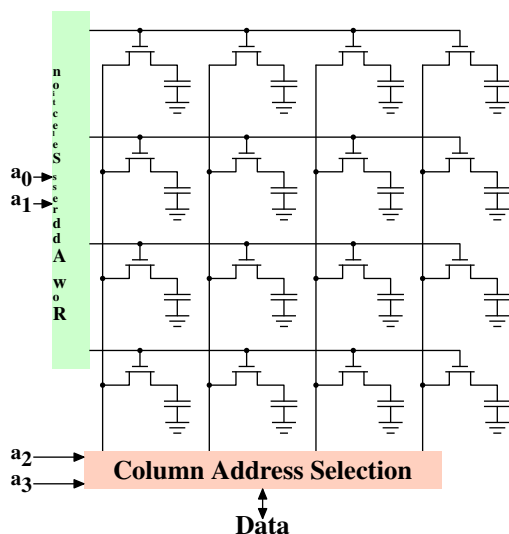


图 2.7: 动态 RAM 原理图

图 2.7 显示了非常高水平的 DRAM 芯片。DRAM 单元按行和列组织。它们可以全部排列成一排，但 DRAM 芯片将需要一个巨大的解复用器。通过阵列方法，设计可以使用一个解复用器和一个尺寸减半的复用器。⁸这在所有方面都是巨大的节省。在示例中，地址线 a0 和 a1 通过 *row address selection (RAS)*⁹ 解复用器选择整行单元的地址线。读取时，所有单元的内容因此可供 *column address selection (CAS)*⁹ 多路复用器使用。基于地址线 a2 和 a3，一列的内容可用于 DRAM 芯片的数据引脚。这种情况在多个 DRAM 芯片上并行发生多次，以产生与数据总线宽度相对应的总位数。

对于写入，新的单元值被放置在数据总线上，并且当使用 RAS 和 CAS 选择单元时，它被存储在单元中。一个非常简单的设计。事实上，显然，还有更多的复杂情况。需要指定在信号之后有多少延迟才能在数据总线上读取数据。如上一节所述，电容器不会立即卸载。来自细胞的信号非常微弱，需要被放大。对于写入，必须指定在 RAS 和 CAS 完成后数据必须在总线上可用多长时间才能成功地将新值存储在单元中（再次，电容 —— ——

⁷Modern DRAM types like DDR3 can automatically adjust the timing but there is a limit as to what can be tolerated.

⁸Multiplexers and demultiplexers are equivalent and the multiplexer here needs to work as a demultiplexer when writing. So we will drop the differentiation from now on.

⁹The line over the name indicates that the signal is negated.

它不会立即填充或排出）。这些时序常数对于 DRAM 芯片的性能至关重要。我们将在下一节中讨论这个问题。

第二个可扩展性问题是每个 RAM 芯片连接 30 条地址线也是不可行的。芯片的管脚是宝贵的资源。数据必须尽可能并行传输（例如，以 64 位批次），这已经够“糟糕”了。内存控制器必须能够对每个 RAM 模块（RAM 芯片的集合）进行寻址。如果出于性能原因需要并行访问多个 RAM 模块，并且每个 RAM 模块需要自己的一组 30 条或更多地址线，那么对于 8 个 RAM 模块，内存控制器需要拥有多达 240+ 引脚，仅用于地址处理。

为了解决这些次要的可扩展性问题，DRAM 芯片长期以来一直对地址本身进行复用。这意味着地址分两部分传输。第一部分由地址位（图 2.7 示例中的 a0 和 a1）组成，用于选择行。该选择在撤销之前一直有效。然后第二部分，地址位 a2 和 a3，选择列。关键的区别在于只需要两条外部地址线。还需要几行来指示 RAS 和 CAS 信号何时可用，但这只是将地址线数量减少一半所付出的很小的代价。然而，这种地址复用也带来了其自身的一系列问题。我们将在 2.2 节中讨论它们。

2.1.4 结论

如果本节中的详细信息有点令人难以承受，请不要担心。从本节中需要了解的重要内容是：

- 并非所有内存都是 SRAM 是有原因的
- 需要单独选择存储单元才能使用
- 地址线的数量直接影响内存控制器、主板、DRAM 模块和 DRAM 芯片的成本
- 读取或写入操作的结果需要一段时间才能获得

以下部分将详细介绍访问 DRAM 内存的实际过程。我们不会讨论访问 SRAM 的更多细节，SRAM 通常是直接寻址的。发生这种情况是为了速度并且因为 SRAM 存储器的大小有限。SRAM 目前用于 CPU 缓存和片上，其中连接很小并且完全由 CPU 设计者控制。CPU 缓存是我们稍后讨论的一个主题，但我们需要知道的是 SRAM 单元具有一定的最大速度，这取决于 SRAM 上花费的精力。速度可能会稍慢一些

比CPU核心要慢一两个数量级。

2.2 DRAM 访问技术细节

在介绍 DRAM 的部分中，我们看到 DRAM 芯片对地址进行复用，以地址引脚的形式节省资源。我们还发现，访问 DRAM 单元需要时间，因为这些单元中的电容器不会立即放电以产生稳定的信号；我们还看到 DRAM 单元必须刷新。现在是时候将所有这些放在一起，看看所有这些因素如何决定 DRAM 访问必须如何发生。

我们将专注于现有技术；我们不会讨论异步 DRAM 及其变体，因为它们根本不再相关。对这个主题感兴趣的读者可以参考[3]和[19]。我们也不会谈论 Rambus DRAM (RDRAM)，即使该技术并未过时。它只是没有广泛用于系统内存。我们将专门关注同步 DRAM (SDRAM) 及其后继产品双倍数据速率 DRAM (DDR)。

同步 DRAM，顾名思义，是相对于时间源工作的。内存控制器提供时钟，其频率决定前端总线 (FSB) ——DRAM 芯片使用的内存控制器接口的速度。截至撰写本文时，频率为 800MHz、1,066MHz 甚至 1,333MHz，下一代更高频率 (1,600MHz) 也已推出。这并不意味着总线上使用的频率实际上如此高。相反，今天的总线是双泵或四泵的，这意味着每个周期数据传输两次或四次。由于销量较高，制造商喜欢将四泵浦 200MHz 总线宣传为“有效的”800MHz 总线。

对于当今的 SDRAM，每个数据传输由 64 位 – 8 个字节组成。因此，FSB 的传输速率为 8 字节乘以有效总线频率（四泵浦 200MHz 总线为 6.4GB/s）。听起来很多，但这就是爆发速度，是永远无法超越的最高速度。正如我们现在将看到的，当无法传输数据时，与 RAM 模块通信的协议会产生大量停机时间。我们必须了解并尽量减少这种停机时间，以实现最佳性能。

2.2.1 读访问协议

图 2.8 显示了 DRAM 模块的某些连接器上的活动，该活动发生在三个不同颜色的阶段。像往常一样，时间从左向右流动。很多细节被遗漏了。这里我们只讨论总线时钟、RAS 和 CAS 信号以及地址和数据总线。读周期从存储器控制器使行地址在地址上可用开始。

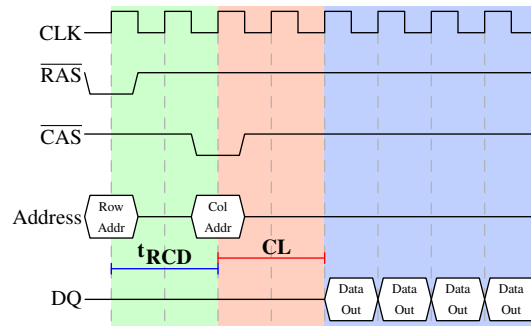


图 2.8: SDRAM 读访问时序

总线并降低 RAS 信号。所有信号均在时钟 (CLK) 的上升沿读取，因此只要信号在读取时稳定，信号是否不完全方波也没关系。设置行地址会使 RAM 芯片开始锁存所寻址的行。

CAS 信号可以在 t_{RCD} (RAS 到 CAS 延迟) 时钟周期后发送。然后通过在地总线上提供列地址并拉低 CAS 线来传输列地址。在这里，我们可以看到地址的两部分（或多或少是一半，其他没有意义）如何通过同一地址总线进行传输。

现在寻址已完成，可以传输数据了。RAM 芯片需要一些时间为此做好准备。该延迟通常称为 CAS 延迟 (CL)。在图 2.8 中，CAS 延迟为 2。它可以更高或更低，具体取决于内存控制器、主板和 DRAM 模块的质量。延迟也可以有半值。使用 $CL=2.5$ ，第一个数据将在蓝色区域的第一个 *falling* 侧翼提供。

通过所有这些获取数据的准备工作，仅传输一个数据字将是一种浪费。这就是 DRAM 模块允许内存控制器指定要传输多少数据的原因。通常选择在 2、4 或 8 个单词之间。这允许在高速缓存中填充整行，而无需新的 RAS/CAS 序列。内存控制器也可以发送新的 CAS 信号而不重置行选择。通过这种方式，可以显着更快地读取或写入连续的内存地址，因为不必发送 RAS 信号，也不必停用该行（见下文）。保持行“打开”是内存控制器必须决定的事情。推测性地使其始终保持打开状态对于现实世界的应用程序来说是不利的（参见[3]）。发送新的 CAS 信号仅受 RAM 模块的命令速率影响（通常指定为 T_x ，其中 x 是 1 或 2 之类的值；对于每个周期接受新命令的高性能 DRAM 模块，该值将为 1）。

在此示例中，SDRAM 每个周期吐出一个字。这就是第一代所做的。DDR 每个周期能够传输两个字。这减少了传输时间，但不会改变延迟。原则上

例如，DDR2 的工作原理相同，尽管实际上看起来不同。这里无需赘述。值得注意的是，DDR2 可以变得更快、更便宜、更可靠，并且更节能（更多信息请参阅 [6]）。

2.2.2 预充电和激活

图 2.8 并未涵盖整个周期。它仅显示访问 DRAM 的整个周期的一部分。在发送新的 RAS 信号之前，必须停用当前锁存的行，并且必须对新行进行预充电。我们在这里可以集中讨论通过显式命令完成此操作的情况。该协议有一些改进，在某些情况下可以避免这个额外的步骤。不过，预充电带来的延迟仍然会影响操作。

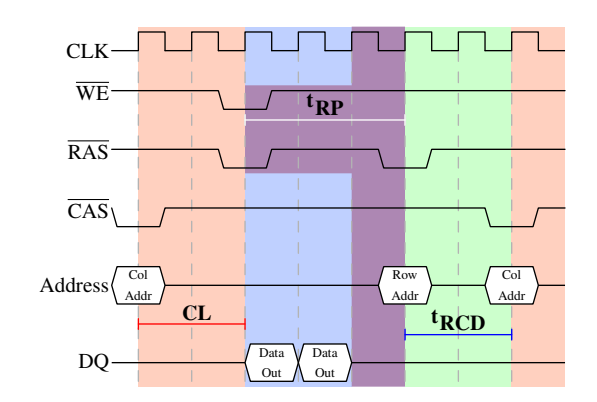


图 2.9: SDRAM 预充电和激活

图 2.9 显示了一个从 CAS 信号开始到另一行的 CAS 信号的活动。在 CL 周期之后，第一个 CAS 信号请求的数据与以前一样可用。在示例中，请求两个字，在简单的 SDRAM 上，需要两个周期来传输。或者，想象 DDR 芯片上有四个字。

即使在命令速率为 1 的 DRAM 模块上，也无法立即发出预充电命令。传输数据需要等待多久。在这种情况下需要两个周期。这恰好与 CL 相同，但这只是巧合。预充电信号无专用线；相反，一些实现通过同时降低写使能（WE）和 RAS 线来发出它。这种组合本身没有任何有用的意义（有关编码详细信息，请参阅 [18]）。

发出预充电命令后，需要 tRP（行预充电时间）周期才能选择该行。在图 2.9 中，大部分时间（由紫色表示）与内存传输（浅蓝色）重叠。这很好！但 tRP 大于传输时间，因此下一个 RAS 信号会停滞一个周期。

如果我们继续图中的时间线，我们会发现下一个数据传输发生在上一个数据传输停止后 5 个周期。这意味着数据总线是

七次循环中仅使用两次。将此值乘以 FSB 速度，800MHz 总线的理论 6.4GB/s 变为 1.8GB/s。这是不好的，必须避免。第 6 节中描述的技术有助于提高这个数字。但程序员通常必须尽自己的一份力。

SDRAM 模块还有一个我们尚未讨论的时序值。在图 2.9 中，预充电命令仅受数据传输时间的限制。另一个限制是 SDRAM 模块在收到 RAS 信号后需要一段时间才能对另一行进行预充电（表示为 tRAS）。这个数字通常相当高，大约是 tRP 值的两到三倍。如果在 RAS 信号之后仅跟随一个 CAS 信号并且数据传输在几个周期内完成，则这是一个问题。假设在图 2.9 中，初始 CAS 信号直接位于 RAS 信号之前，并且 tRAS 为 8 个周期。那么预充电命令就必须再延迟一个周期，因为 tRCD、CL 和 tRP 之和（因为它大于数据传输时间）仅为 7 个周期。

DDR 模块通常使用特殊符号来描述：w-x-y-z-T。例如：2-3-2-8-T1。这意味着：

w	2	CAS Latency (CL)
x	3	RAS-to-CAS delay (tRCD)
y	2	RAS Precharge (tRP)
z	8	Active to Precharge delay (tRAS)
T	T1	Command Rate

还有许多其他定时常数会影响命令的发出和处理方式。不过，这五个常数实际上足以确定模块的性能。

有时，了解这些信息对于使用中的计算机能够解释某些测量结果很有用。在购买计算机时了解这些细节绝对有用，因为它们与 FSB 和 SDRAM 模块速度一起，是决定计算机速度的最重要因素之一。

非常喜欢冒险的读者也可以尝试调整系统。有时 BIOS 允许更改部分或全部这些值。SDRAM 模块具有可编程寄存器，可以在其中设置这些值。通常 BIOS 会选择最佳的默认值。如果 RAM 模块的质量较高，则有可能减少其中一种或另一种延迟，而不会影响计算机的稳定性。互联网上的许多超频网站都提供了充足的文档来执行此操作。不过，这样做的风险由您自己承担，并且不要说您没有收到警告。

2.2.3 充电

当涉及 DRAM 访问时，一个最容易被忽视的话题是充电。如第 2.1.2 节所述，DRAM 单元必须不断刷新。这不会发生

对于系统的其余部分完全透明。有时，当 row¹⁰ 充电时，无法进行访问。[3]中的研究发现 “[s]urprisingly, DRAM re-fresh organization can影响 performance dramatically”。

根据 JEDEC（联合电子设备工程委员会）规范，每个 DRAM 单元必须每 64 毫秒刷新一次。如果 DRAM 阵列有 8,192 行，这意味着内存控制器必须平均每 $7.8125\mu\text{s}$ 发出一次刷新命令（刷新命令可以排队，因此实际上两个请求之间的最大间隔可以更高）。内存控制器负责调度刷新命令。DRAM 模块会跟踪最后刷新行的地址，并针对每个新请求自动增加地址计数器。

对于刷新和发出命令的时间点，程序员实际上无能为力。但在解释测量结果时，记住 DRAM 生命周期的这一部分非常重要。如果必须从当前正在刷新的行中检索关键字，则处理器可能会停滞相当长的时间。每次刷新需要多长时间取决于 DRAM 模块。

2.2.4 内存类型

值得花一些时间研究当前和即将使用的当前内存类型。我们将从 SDR（单倍数据速率）SDRAM 开始，因为它们是 DDR（双倍数据速率）SDRAM 的基础。特别提款权非常简单。存储单元和数据传输速率是相同的。

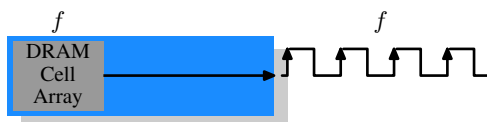


图 2.10: SDR SDRAM 操作

在图 2.10 中，DRAM 单元阵列可以以与通过内存总线传输相同的速率输出内存内容。如果 DRAM 单元阵列可以工作在 100MHz，则单个单元总线的数据传输速率为 100Mb/s。所有分量的频率 f 都是相同的。提高 DRAM 芯片的吞吐量是昂贵的，因为能耗随着频率的增加而增加。对于大量的阵列单元，这是非常昂贵的。¹¹实际上，这甚至是一个更大的问题，因为增加频率通常还需要增加电压来维持系统的稳定性。DDR SDRAM（追溯称为 DDR1）设法在不增加任何相关频率的情况下提高吞吐量。

¹⁰Rows are the granularity this happens with despite what [3] and other literature says (see [18]).

¹¹Power = Dynamic Capacity $\times$ Voltage² $\times$ Frequency.

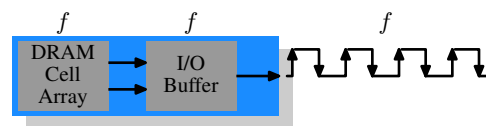


图 2.11: DDR1 SDRAM 操作

SDR 和 DDR1 之间的区别如图 2.11 所示并从名称中猜出，每个周期传输的数据量是 DDR1 的两倍。即 DDR1 芯片在上升 and 下降沿传输数据。这有时称为“双泵”总线。为了在不增加单元阵列频率的情况下实现这一点，必须引入缓冲器。该缓冲器每条数据线保存两位。这又要求在图 2.7 的单元阵列中，数据总线由两条线组成。实现这一点很简单：只需对两个 DRAM 单元使用相同的列地址并并行访问它们。实现此目的对元胞数组的更改也很小。

SDR DRAM 只需通过其频率即可得知（例如，PC100 表示 100MHz SDR）。为了让 DDR1 DRAM 听起来更好，营销人员必须想出一个新方案，因为频率没有改变。它们的名称包含 DDR 模块（它们有 64 位总线）可以维持的字节传输速率：

$$100\text{MHz} \times 64\text{bit} \times 2 = 1,600\text{MB/s}$$

因此频率为 100MHz 的 DDR 模块被称为 PC1600。1600 > 100 满足所有营销要求；听起来好多了，尽管改进实际上是 *only* 两倍。¹²

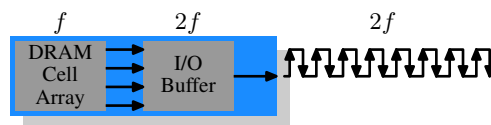


图 2.12: DDR2 SDRAM 操作

为了充分利用内存技术，DDR2 包含了更多创新。图 2.12 中最明显的变化是总线频率加倍。频率加倍意味着带宽加倍。由于频率加倍对于单元阵列来说并不经济，因此现在要求 I/O 缓冲区在每个时钟周期中获取四个位，然后将其发送到总线上。这意味着对 DDR2 模块的更改包括仅使 DIMM 的 I/O 缓冲区组件能够以更高的速度运行。这当然是可能的，并且不需要更多的能量，它只是一个微小的部件，而不是整个模块。名字的 mar-

¹²我会采用二的因数，但我不必喜欢夸大的数字。

keters 为 DDR2 提出的名称与 DDR1 名称类似，只是在计算值时将因子 2 替换为 4（我们现在有一个四泵浦总线）。表 2.1 显示了当今使用的模块的名称。

Array Freq.	Bus Freq.	Data Rate	Name (Rate)	Name (FSB)
133MHz	266MHz	4,256MB/s	PC2-4200	DDR2-533
166MHz	333MHz	5,312MB/s	PC2-5300	DDR2-667
200MHz	400MHz	6,400MB/s	PC2-6400	DDR2-800
250MHz	500MHz	8,000MB/s	PC2-8000	DDR2-1000
266MHz	533MHz	8,512MB/s	PC2-8500	DDR2-1066

表 2.1: DDR2 模块名称

命名还有一个变化。CPU、主板和 DRAM 模块使用的 FSB 速度通过使用 *effective* 频率来指定。也就是说，它考虑了时钟周期两侧的传输，从而夸大了数字。因此，具有 266MHz 总线的 133MHz 模块的 FSB “频率”为 533MHz。

DDR3 的规范（真正的规范，而不是显卡中使用的假 GDDR3）要求在向 DDR2 过渡的过程中进行更多更改。电压将从 DDR2 的 1.8V 降至 DDR3 的 1.5V。由于功耗方程是使用电压的平方计算的，因此仅此一项就带来了 30% 的改进。再加上芯片尺寸的减小以及其他电气方面的进步，DDR3 可以在相同的频率下将功耗降低一半。或者，使用更高的频率，可以达到相同的功率包络。或者用双倍的容量可以实现相同的热量排放。

DDR3 模块的单元阵列将以外总线速度的四分之一运行，这需要 8 位 I/O 缓冲区，而 DDR2 需要 4 位。原理图见图 2.13。

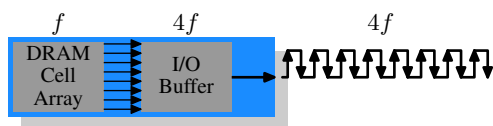


图 2.13: DDR3 SDRAM 操作

最初，DDR3 模块的 CAS 延迟可能会稍高，因为 DDR2 技术更加成熟。这将导致 DDR3 仅在比 DDR2 所能达到的频率更高的频率下才有用，而且即使如此，大多数情况下还是在带宽比延迟更重要的情况下。已经有传言称 1.3V 模块可以实现与 DDR2 相同的 CAS 延迟。无论如何，由于更快的总线而实现更高速度的可能性将超过增加的延迟。

DDR3 的一个可能的问题是，对于 1,600Mb/s 或更高的传输速率，每个通道的模块数量

nel 可能会减少到只有 1。在早期版本中，这一要求适用于所有频率，因此人们可以希望在某个时候针对所有频率取消该要求。否则系统的容量将受到严重限制。

表 2.2 显示了我们可能看到的 DDR3 模块的名称。JEDEC 到目前为止就前四种类型达成了一致。鉴于英特尔 45nm 处理器的 FSB 速度为 1,600Mb/s，超频市场需要 1,866Mb/s。在 DDR3 生命周期结束时，我们可能会看到更多这样的情况。

Array Freq.	Bus Freq.	Data Rate	Name (Rate)	Name (FSB)
100MHz	400MHz	6,400MB/s	PC3-6400	DDR3-800
133MHz	533MHz	8,512MB/s	PC3-8500	DDR3-1066
166MHz	667MHz	10,667MB/s	PC3-10667	DDR3-1333
200MHz	800MHz	12,800MB/s	PC3-12800	DDR3-1600
233MHz	933MHz	14,933MB/s	PC3-14900	DDR3-1866

表 2.2: DDR3 模块名称

所有 DDR 内存都存在一个问题：总线频率的增加使得创建并行数据总线变得困难。DDR2 模块有 240 个引脚。所有与数据和地址引脚的连接都必须经过布线，以便它们具有大致相同的长度。更严重的问题是，如果在同一总线上以菊花链方式连接多个 DDR 模块，则每增加一个模块，信号就会变得越来越失真。DDR2 规范仅允许每条总线（也称为通道）使用两个模块，而 DDR3 规范仅允许一个高频模块。每个通道有 240 个引脚，单个北桥无法合理驱动两个以上的通道。另一种方法是使用外部内存控制器（如图 2.2 所示），但这很昂贵。

这意味着商用主板最多只能容纳四个 DDR2 或 DDR3 模块。此限制严重限制了系统可以拥有的内存量。即使是旧的 32 位 IA-32 处理器也可以处理 64GB RAM，而且家庭使用的内存需求也在不断增长，因此必须采取一些措施。

一个答案是向每个处理器添加内存控制器，如第 2 节中所述。AMD 使用 Opteron 产品线来实现这一点，英特尔将使用其 CSI 技术来实现这一点。只要处理器能够使用的合理内存量可以连接到单个处理器，这就会有所帮助。在某些情况下，情况并非如此，这种设置将引入 NUMA 架构及其负面影响。对于某些情况，需要另一种解决方案。

至少目前，英特尔针对大型服务器机器的这一问题的解决方案被称为全缓冲 DRAM (FB-DRAM)。FB-DRAM 模块使用与当今 DDR2 模块相同的内存芯片，这使得它们的生产成本相对较低。不同之处在于与内存控制器的连接。FB-DRAM 使用串行总线代替并行数据总线 (Rambus DRAM 有

那时，SATA 是 PATA 的后继者，PCI Express 也是 PCI/AGP 的后继者）。串行总线可以以更高的频率驱动，从而消除串行化的负面影响，甚至增加带宽。使用串行总线的主要作用是

- 1.每通道可使用更多模块。
- 2. 每个北桥/内存控制器可以使用更多通道。
- 3. 串行总线设计为全双工（两条线）。
- 4. 实现差分总线（每个方向两条线）足够便宜，因此可以提高速度。

FB-DRAM 模块只有 69 个引脚，而 DDR2 有 240 个引脚。菊花链 FB-DRAM 模块要容易得多，因为可以更好地处理总线的电气效应。FB-DRAM 规范允许每个通道最多 8 个 DRAM 模块。

与双通道北桥的连接要求相比，现在可以用更少的引脚驱动 6 个通道的 FB-DRAM：2 个 × 240 引脚与 6 个 × 69 引脚。每个通道的布线更加简单，这也有助于降低主板的成本。

对于传统 DRAM 模块来说，全双工并行总线的成本过高，复制所有这些线路的成本也太高。对于串行线路（即使它们是差分的，如 FB-DRAM 所要求的），情况并非如此，因此串行总线被设计为完全双工，这意味着在某些情况下，理论上带宽可以单独加倍。但这并不是使用并行性来增加带宽的唯一地方。由于 FB-DRAM 控制器可以同时运行多达 6 个通道，因此即使对于 RAM 量较小的系统，也可以通过使用 FB-DRAM 来增加带宽。如果具有四个模块的 DDR2 系统有两个通道，则可以使用普通 FB-DRAM 控制器通过四个通道处理相同的容量。串行总线的实际带宽取决于 FB-DRAM 模块上使用的 DDR 2（或 DDR3）芯片的类型。

我们可以这样总结其优点：

	DDR2	FB-DRAM
Pins	240	69
Channels	2	6
DIMMs/Channel	2	8
Max Memory ¹³	16GB ¹⁴	192GB
Throughput ¹⁵	~10GB/s	~40GB/s

¹³ Assuming 4GB modules.
¹⁴ An Intel presentation, for some reason I do not see, says 8GB...
¹⁵ Assuming DDR2-800 modules.

如果在一个通道上使用多个 DIMM，FB-DRAM 会存在一些缺点。信号在链中的每个 DIMM 处都会延迟（尽管延迟很小），从而增加了延迟。第二个问题是驱动串行总线的芯片需要大量的能量，因为频率非常高并且需要驱动总线。但对于相同数量、相同频率的内存，FB-DRAM 始终比 DDR2 和 DDR3 更快，因为最多四个 DIMM 可以各自获得自己的通道；对于大型内存系统，DDR 根本无法使用商品组件。

2.2.5 结论

本节应该表明访问 DRAM 并不是一个任意快速的过程。至少与处理器运行速度以及访问寄存器和高速缓存的速度相比并不快。请务必记住 CPU 和内存频率之间的差异。以 2.933GHz 运行的 Intel Core 2 处理器和 1.066GHz FSB 的时钟比为 11:1（注意：1.066GHz 总线是四泵浦的）。内存总线上每个周期的停顿意味着处理器停顿 11 个周期。对于大多数机器来说，实际使用的 DRAM 速度较慢，从而增加了延迟。当我们在接下来的部分讨论摊位时，请记住这些数字。

读取命令的时序图显示 DRAM 模块能够实现高持续数据速率。无需停顿即可传输整个 DRAM 行。数据总线可以保持100%占用。对于 DDR 模块，这意味着每个周期传输两个 64 位字。对于 DDR2-800 模块和两个通道，这意味着速率为 12.8GB/s。

但是，除非以这种方式设计，否则 DRAM 访问并不总是顺序的。使用非连续存储区域，这意味着需要预充电和新的 RAS 信号。这是事情变慢并且 DRAM 模块需要帮助的时候。预充电发生得越早并且发送的 RAS 信号在实际使用该行时的损失就越小。

硬件和软件预取（参见第 6.3 节）可用于在时序中创建更多重叠并减少停顿。预取还有助于及时转移内存操作，以便在稍后实际需要数据之前减少争用。当必须存储一轮中产生的数据并且必须读取下一轮所需的数据时，这是一个常见问题。通过及时移动读取，写入和读取操作不必基本上同时发出。

2.3 其他主存用户

除了 CPU 之外，还有其他可以访问主存储器的系统组件。高性能卡（例如网络和大容量存储控制器）无法满足

福特通过管道传输他们需要的或通过CPU提供的所有数据。相反，它们直接从主内存读取数据或将数据写入主内存（直接内存访问，DMA）。在图 2.1 中，我们可以看到卡可以通过南桥和北桥直接与内存进行通信。其他总线（例如 USB）也需要 FSB 带宽（即使它们不使用 DMA），因为南桥也通过 FSB 通过北桥连接到处理器。

虽然 DMA 肯定是有益的，但这意味着对 FSB 带宽的竞争更加激烈。在 DMA 流量较高时，CPU 在等待来自主内存的数据时可能会比平时停顿得更多。只要有合适的硬件，就有一些方法可以解决这个问题。利用如图 2.3 所示的架构，可以确保计算使用不受 DMA 影响的节点上的内存。还可以将南桥连接到每个节点，将负载平均分配到所有节点的 FSB 上。有无数的可能性。在第 6 节中，我们将介绍有助于实现软件改进的技术和编程接口。

最后应该提到的是，一些廉价系统的图形系统没有单独的专用视频 RAM。这些系统使用部分主内存作为视频 RAM。由于对视频 RAM 的访问很频繁（对于 60Hz 下 16 bpp 的 1024x768 显示器，我们所说的速度为 94MB/s），并且系统内存与显卡上的 RAM 不同，没有两个端口，这会严重影响系统性能，尤其是延迟。当性能优先时，最好忽略此类系统。它们带来的麻烦比它们带来的价值还要多。购买这些机器的人知道他们不会获得最佳性能。

3 个 CPU 高速缓存

如今的 CPU 比 25 年前更加复杂。当时，CPU 核心的频率与内存总线的频率相当。内存访问仅比寄存器访问慢一点。但这种情况在 90 年代初发生了巨大的变化，当时 CPU 设计者提高了 CPU 核心的频率，但内存总线的频率和 RAM 芯片的性能并没有成比例地提高。正如上一节所述，这并不是因为无法构建更快的 RAM。这是可能的，但不经济。与当前 CPU 内核一样快的 RAM 比任何动态 RAM 贵几个数量级。

如果在 RAM 非常少、速度非常快的机器和具有大量相对快速 RAM 的机器之间进行选择，则考虑到工作集大小超过小 RAM 大小以及访问硬盘驱动器等辅助存储介质的成本，后者始终会获胜。这里的问题是辅助存储（通常是硬盘）的速度，必须使用它来保存工作集的换出部分。访问这些磁盘甚至比访问 DRAM 慢几个数量级。

幸运的是，这不一定是个全有或全无的决定。计算机除了大量的 DRAM 之外还可以拥有少量的高速 SRAM。一种可能的实现方式是将处理器地址空间的某个区域指定为包含 SRAM，其余区域为 DRAM。操作系统的任务就是优化数据分布以利用 SRAM。基本上，SRAM 在这种情况下充当处理器寄存器组的扩展。

虽然这是一种可能的实现，但它并不可行。忽略将这种 SRAM 支持的内存的物理资源映射到进程的虚拟地址空间的问题（这本身就非常困难），这种方法将要求每个进程在软件中管理该内存区域的分配。内存区域的大小可能因处理器而异（即，处理器具有不同数量的昂贵的 SRAM 支持内存）。组成程序一部分的每个模块都将要求其共享的快速内存，这会因同步要求而引入额外的成本。简而言之，拥有快速内存所带来的好处将被管理资源的开销完全吞噬。

因此，SRAM 不再受操作系统或用户的控制，而是成为由处理器透明使用和管理的资源。在此模式下，SRAM 用于在主存储器中制作可能很快被处理器使用的数据的临时副本（换句话说，就是缓存）。这是可能的，因为程序代码和数据具有时间和空间局部性。这意味着，在短时间内，相同的代码或数据很有可能被获取

重复使用。对于代码来说，这意味着代码中很可能存在循环，因此相同的代码会一遍又一遍地执行（*spatial locality* 的完美情况）。理想情况下，数据访问也仅限于小区域。即使短时间内使用的内存并不紧密相连，相同的数据也很有可能很快就会被重用（*temporal locality*）。例如，对于代码来说，这意味着在循环中进行函数调用，并且该函数位于地址空间的其他位置。该函数在内存中可能很遥远，但对该函数的调用在时间上会很接近。对于数据来说，这意味着一次使用的内存总量（工作集大小）在理想情况下是有限的，但由于 RAM 的 *random* 访问性质，所使用的内存并不是紧密相连的。认识到局部性的存在是我们今天使用的 CPU 缓存概念的关键。

一个简单的计算就可以显示缓存理论上的有效性。假设访问主存储器需要 200 个周期，访问高速缓冲存储器需要 15 个周期。然后，如果没有缓存，则使用 100 个数据元素各 100 次的代码将在内存操作上花费 2,000,000 个周期，如果所有数据都可以缓存，则仅花费 168,500 个周期。提高了 91.5%。

用于高速缓存的 SRAM 的大小比主内存小很多倍。根据作者对带有 CPU 缓存的工作站的经验，缓存大小始终约为主内存大小的 1/1000（目前：4MB 缓存和 4GB 主内存）。仅此一点并不构成问题。如果工作集（当前处理的数据集）的大小小于缓存大小，则没关系。但计算机没有理由没有大的主存储器。工作集必然大于缓存。对于运行多个进程的系统尤其如此，其中工作集的大小是所有单独进程和内核大小的总和。

处理缓存大小有限的问题需要一组好的策略来确定在任何给定时间应该缓存什么。由于并非工作集中的所有数据都在 *exactly* 上同时使用，因此我们可以使用技术暂时用其他数据替换缓存中的某些数据。也许这可以在实际需要数据之前完成。这种预取将消除访问主内存的一些成本，因为它与程序的执行异步发生。所有这些技术以及更多技术都可以用来使缓存看起来比实际更大。我们将在 3.3 节中讨论它们。一旦所有这些技术都被利用，就需要程序员来帮助处理器。如何做到这一点将在第 6 节中讨论。

3.1 CPU 缓存概览

在深入研究 CPU 缓存实现的技术细节之前，一些读者可能会发现首先了解缓存如何融入现代计算机系统的“大局”的更多细节是有用的。

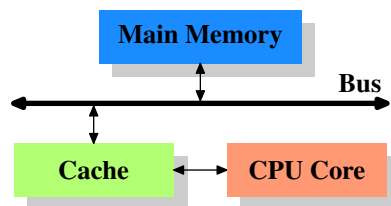


图 3.1: 最小缓存配置

图 3.1 显示了最小缓存配置。它对应于部署 CPU 缓存的早期系统中的架构。CPU 核心不再直接连接到主存。¹⁶所有加载和存储都必须经过缓存。CPU 核心和缓存之间的连接是一种特殊的、快速的连接。在简化的表示中，主存储器 and 高速缓存连接到系统总线，该总线也可用于与系统的其他组件进行通信。我们将系统总线称为“FSB”，这是今天使用的名称；参见第 2.2 节。在本节中我们忽略北桥；假设它的存在是为了促进 CPU 与主存储器的通信。

尽管过去几十年来大多数计算机都使用冯·诺依曼架构，但经验表明，将用于代码和数据的缓存分开是有利的。自 1993 年以来，英特尔一直使用独立的代码和数据缓存，并且从未回头。代码和数据所需的内存区域几乎彼此独立，这就是独立缓存工作得更好的原因。近年来出现了另一个优势：最常见处理器的指令解码步骤很慢；缓存解码指令可以加快执行速度，特别是当管道由于错误预测或无法预测分支而为空时。

引入缓存后不久，系统变得更加复杂。高速缓存和主存之间的速度差异再次增大，以至于添加了另一级高速缓存，比第一级高速缓存更大且更慢。出于经济原因，仅增加一级缓存的大小并不是一个选择。如今，甚至有经常使用具有三级缓存的机器。具有这种处理器的系统如图 3.2 所示。随着单个 CPU 核心数量的增加，未来缓存级别的数量可能会进一步增加。

图 3.2 显示了三个级别的缓存，并介绍了我们将在文档的其余部分中使用的术语。L1d 是一级数据缓存，L1i 是一级指令缓存等。注意，这是示意图；实际上，数据流从核心到主程序的途中不需要经过任何更高级别的缓存

¹⁶In even earlier systems the cache was attached to the system bus just like the CPU and the main memory. This was more a hack than a real solution.

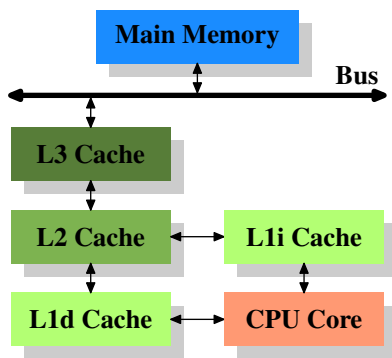


图 3.2: 具有 3 级高速缓存的处理器

记忆。CPU 设计者在设计高速缓存接口时有很大的自由度。对于程序员来说，这些设计选择是不可见的。

此外，我们还有具有多个核心的处理器，每个核心可以有多个“线程”。核心和线程之间的区别在于，单独的核心具有（几乎¹⁷）所有硬件资源的单独副本。内核可以完全独立运行，除非它们同时使用相同的资源（例如与外部的连接）。另一方面，线程共享几乎所有处理器资源。英特尔的线程实现只有单独的线程寄存器，即使这是有限的，一些寄存器也是共享的。因此，现代 CPU 的完整情况如图 3.3 所示。

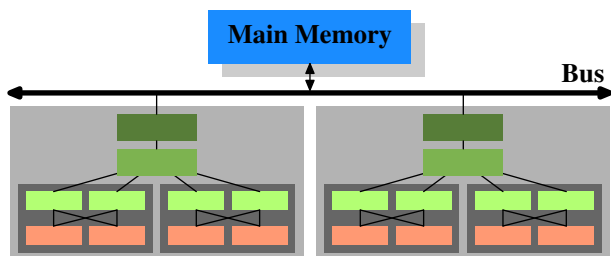


图 3.3: 多处理器、多核、多线程

在此图中，我们有两个处理器，每个处理器有两个内核，每个内核有两个线程。线程共享 1 级缓存。核心（深灰色阴影）具有单独的 1 级缓存。CPU 的所有核心共享更高级别的缓存。两个处理器（浅灰色阴影下的两个大盒子）当然不共享任何缓存。所有这些都很重要，特别是当我们讨论缓存对多进程和多线程应用程序的影响时。

¹⁷Early multi-core processors even had separate 2nd level caches and no 3rd level cache.

3.2 高层缓存操作

为了了解使用缓存的成本和节省，我们必须将第 2 节中有关机器架构和 RAM 技术的知识与上一节中描述的缓存结构结合起来。

默认情况下，CPU 核心读取或写入的所有数据都存储在缓存中。有些内存区域无法缓存，但这只是操作系统实现者必须关心的事情；它对于应用程序员来说是不可见的。还有一些指令允许程序员故意绕过某些缓存。这将在第 6 节中讨论。

如果 CPU 需要数据字，则首先搜索高速缓存。显然，缓存不能包含整个主存的内容（否则我们就不需要缓存了），但由于所有内存地址都是可缓存的，所以每个缓存条目都是使用主存中数据字的地址的 *tagged*。这样，读取或写入地址的请求可以在缓存中搜索匹配的标签。此上下文中的地址可以是虚拟地址或物理地址，具体取决于缓存实现。

由于对于标签来说，除了实际的内存之外，还需要额外的空间，因此选择一个字作为缓存的粒度是低效的。对于 x86 机器上的 32 位字，标签本身可能需要 32 位或更多。此外，由于空间局部性是缓存所依据的原则之一，因此不考虑这一点是很糟糕的。由于相邻内存可能会一起使用，因此也应该一起加载到缓存中。请记住我们在第 2.2.1 节中学到的内容：如果 RAM 模块能够连续传输许多数据字而无需新的 CAS 甚至 RAS 信号，那么它们会更加有效。因此，存储在缓存中的条目不是单个单词，而是几个连续单词的“行”。在早期的缓存中，这些行的长度为 32 字节；现在标准是 64 字节。如果内存总线是 64 位宽，这意味着每个高速缓存行有 8 次传输。DDR 有效地支持这种传输模式。

当处理器需要内存内容时，整个缓存行将加载到 L1d 中。每个缓存行的内存地址是通过根据缓存行大小屏蔽地址值来计算的。对于 64 字节高速缓存行，这意味着低 6 位被清零。丢弃的位用作高速缓存行的偏移量。在某些情况下，其余位用于在高速缓存中定位该行并用作标记。实际上，地址值被分为三个部分。对于 32 位地址，它可能如下所示：



当缓存行大小为 20 时，使用低 O 位

作为缓存行的偏移量。接下来的 S 位选择“缓存集”。我们很快就会更详细地了解为什么缓存行使用集合而不是单个插槽。现在只要理解有 2S 组缓存线就足够了。这留下了形成标签的前 $32 - S - O = T$ 位。这些 T 位是与每个缓存行相关联的值，用于区分在同一缓存集中缓存的所有 *aliases*¹⁸。用于寻址高速缓存组的 S 位不必被存储，因为它们对于同一组中的所有高速缓存线都是相同的。

当指令修改内存时，处理器仍然必须首先加载高速缓存行，因为没有指令会立即修改整个高速缓存行（规则的例外：写组合，如第 6.1 节中所述）。因此，必须在写入操作之前加载缓存行的内容。高速缓存不可能保存部分高速缓存行。已写入且尚未写回主存储器的高速缓存行被称为“脏”。一旦写入，脏标志就被清除。

为了能够在缓存中加载新数据，几乎总是首先需要缓存中腾出空间。从 L1d 逐出会将缓存行下推到 L2（使用相同的缓存行大小）。这当然意味着必须在 L2 中腾出空间。这反过来可能会将内容推入 L3 并最终推入主内存。每次驱逐的费用都逐渐增加。这里描述的是现代 AMD 和 VIA 处理器首选的 *exclusive cache* 模型。Intel 实现了 *inclusive caches*¹⁹，其中 L1d 中的每个缓存行也存在于 L2 中。因此从 L1d 中驱逐要快得多。有了足够的二级缓存，浪费存储在两个地方的内容的内存的缺点就很小了，并且在驱逐时得到了回报。独占高速缓存的一个可能的优点是加载新的高速缓存行只需接触 L1d 而不是 L2，这可能会更快。

只要为处理器架构定义的内存模型不改变，CPU 就可以按照自己的意愿管理缓存。例如，处理器利用很少或没有内存总线活动并主动将脏高速缓存行写回主内存是完全可以的。x86 和 x86-64 处理器之间、制造商之间甚至同一制造商的型号内的高速缓存架构多种多样，都证明了内存模型抽象的强大功能。

在对称多处理器 (SMP) 系统中，CPU 的缓存无法彼此独立工作。所有处理器都应该始终看到相同的内存内容。维护这种统一的内存视图称为“缓存一致性”。如果处理器只看它自己的缓存和主存储器

¹⁸All cache lines with the same S part of the address are known by the same alias.

¹⁹This generalization is not completely correct. A few caches are exclusive and some inclusive caches have exclusive cache properties.

理论上它不会看到其他处理器中脏高速缓存行的内容。提供从一个处理器对另一个处理器的高速缓存的直接访问将是非常昂贵的并且是一个巨大的瓶颈。相反，处理器会检测另一个处理器何时想要读取或写入某个缓存行。

如果检测到写访问并且处理器在其高速缓存中具有高速缓存行的干净副本，则该高速缓存行被标记为无效。将来的引用将需要重新加载缓存行。请注意，另一个 CPU 上的读取访问不需要失效，可以很好地保留多个干净副本。

更复杂的缓存实现允许另一种可能性发生。假设一个处理器的高速缓存中的高速缓存行是脏的，并且第二个处理器想要读取或写入该高速缓存行。在这种情况下，主存储器已过时，发出请求的处理器必须从第一个处理器获取高速缓存行内容。通过监听，第一个处理器注意到这种情况并自动向请求处理器发送数据。此操作绕过主存储器，尽管在某些实现中，存储器控制器应该注意到此直接传输并将更新的缓存行内容存储在主存储器中。如果访问是为了写入第一个处理器，则使其本地高速缓存行的副本无效。

随着时间的推移，已经开发了许多缓存一致性协议。最重要的是 MESI，我们将在 3.3.4 节中介绍。所有这一切的结果可以总结为几个简单的规则：

- 脏高速缓存行不存在于任何其他处理器的高速缓存中。
- 同一高速缓存行的干净副本可以驻留在任意多个高速缓存中。

如果可以维护这些规则，即使在多处理器系统中，处理器也可以有效地使用其缓存。所有处理器需要做的就是监视彼此的写访问并将地址与本地缓存中的地址进行比较。在下一节中，我们将详细介绍有关实施的更多细节，尤其是成本。

最后，我们至少应该对与缓存命中和未命中相关的成本有一个印象。这些是英特尔列出的 Pentium M 的编号：

To Where	Cycles
Register	≤ 1
L1d	~ 3
L2	~ 14
Main Memory	~ 240

这些是以 CPU 周期为单位测量的实际访问时间。有趣的是，对于片上 L2 缓存

很大一部分（甚至可能是大部分）访问时间是由线路延迟引起的。这是一个物理限制，随着缓存大小的增加，它只会变得更糟。只有工艺缩小（例如，英特尔产品线中的 Merom 从 60 纳米缩小到 Penryn 的 45 纳米）才能改善这些数字。

表中的数字看起来很高，但幸运的是，不必为每次发生的缓存加载和未命中支付全部成本。部分成本可能会被隐藏。今天的处理器都使用不同长度的内部管道，其中指令被解码并准备执行。准备工作的一部分是从内存（或缓存）加载值（如果它们被传输到寄存器）。如果内存加载操作可以在管道中足够早地启动，则它可能与其他操作并行发生，并且加载的整个成本可能会被隐藏。这对于 L1d 通常是可能的；对于一些具有 L2 长管道的处理器也是如此。

尽早开始记忆读取有很多障碍。这可能很简单，因为没有足够的资源用于内存访问，也可能是由于另一条指令的结果，加载的最终地址较晚才可用。在这些情况下，无法（完全）隐藏负载成本。

对于写操作，CPU 不一定要等到该值安全存储在内存中。只要执行以下指令看起来与将值存储在内存中具有相同的效果，就没有什么可以阻止 CPU 采取捷径。它可以提前开始执行下一条指令。在影子寄存器的帮助下，影子寄存器可以保存常规寄存器中不再可用的值，甚至可以更改要在不完整的写入操作中存储的值。

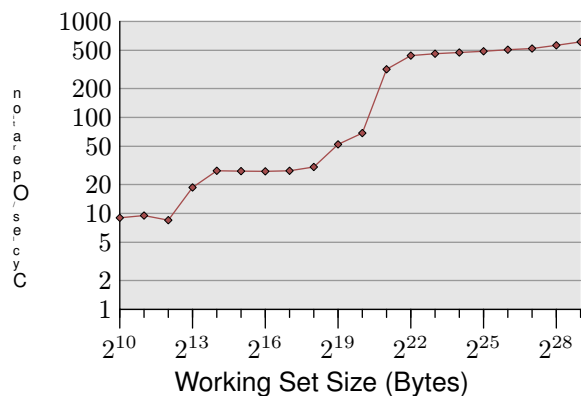


图 3.4：随机写入的访问时间

有关缓存行为影响的说明，请参见图 3.4。我们稍后会讨论生成数据的程序；它是一个程序的简单模拟，该程序以随机方式重复访问可配置的内存量。每个数据项都有固定的大小。元素的数量取决于所选的工作 -

ing 设置大小。Y 轴显示处理一个元素所需的平均 CPU 周期数；请注意，Y 轴的刻度是对数的。这同样适用于所有此类 X 轴图表。工作集的大小始终以 2 的幂表示。

该图显示了三个不同的平台。这并不奇怪：特定处理器有 L1d 和 L2 缓存，但没有 L3。根据一些经验，我们可以推断出 L1d 的大小为 2^{13} 字节，L2 的大小为 2^{20} 字节。如果整个工作集适合 L1d，则每个元素上的每次操作周期数低于 10。一旦超过 L1d 大小，处理器就必须从 L2 加载数据，平均时间会突然上升到 28 左右。一旦 L2 不再足够，时间就会跳到 480 个周期甚至更多。这是许多或大多数操作必须从主存储器加载数据的时候。更糟糕的是：由于数据被修改，脏缓存行也必须写回。

该图应该提供足够的动力来研究有助于提高缓存使用的编码改进。我们这里讨论的并不是百分之几的差异；而是差异。我们谈论的是有时可能实现的数量级改进。在第 6 节中，我们将讨论允许编写更高效代码的技术。下一节将详细介绍 CPU 缓存设计。拥有这些知识固然很好，但对于本文的其余部分来说并不是必需的。所以这一部分可以跳过。

3.3 CPU缓存实现细节

缓存实现者面临的问题是，巨大的主内存中的每个单元都可能需要缓存。如果程序的工作集足够大，这意味着有许多主内存位置争夺高速缓存中的每个位置。之前有人指出，高速缓存与主内存大小的比例为 1 比 1000 的情况并不罕见。

3.3.1 关联性

可以实现一个缓存，其中每个缓存行都可以保存任何内存位置的副本（见图 3.5）。这称为 *fully associative cache*。为了访问高速缓存行，处理器内核必须将每个高速缓存行的标签与所请求地址的标签进行比较。该标签将由地址的整个部分组成，该地址不是高速缓存行的偏移量（这意味着第 15 页图中的 S 为零）。

有些缓存是这样实现的，但是通过查看目前使用的 L2 的数量，我们会发现这是不切实际的。给定 4MB 缓存和 64B 缓存行，缓存将有 65,536 个条目。为了获得足够的性能，缓存逻辑必须能够在几个周期内从所有这些条目中挑选出与给定标签匹配的条目。努力

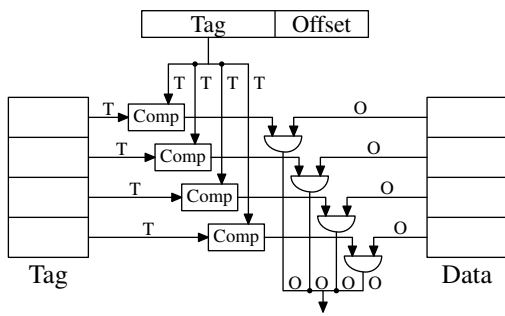


图 3.5: 全关联缓存原理图

实施这一点将是巨大的。

对于每个缓存行，需要一个比较器来比较大标签（注意，S 为零）。每个连接旁边的字母表示位宽。如果没有给出，则它是单个位线。每个比较器必须比较两个 T 位宽的值。然后，根据结果，选择适当的缓存行内容并使其可用。这需要合并与缓存桶一样多的 O 数据线组。实现单个比较器所需的晶体管数量很大，特别是因为它必须工作得非常快。没有可用的迭代比较器。节省比较器数量的唯一方法是通过迭代比较标签来减少比较器的数量。这不适合，其原因与迭代比较器不适合的原因相同：它花费的时间太长。

全关联缓存对于小型缓存非常实用（例如，某些 Intel 处理器上的 TLB 缓存是全关联的），但这些缓存很小，非常小。我们最多谈论几十个条目。

对于 L1i、L1d 和更高级别的缓存，需要不同的方法。能做的就是限制搜索。在最极端的限制中，每个标签恰好映射到一个缓存条目。计算很简单：给定具有 65,536 个条目的 4MB/64B 缓存，我们可以使用地址的第 6 到 21 位（16 位）直接寻址每个条目。低 6 位是高速缓存行的索引。

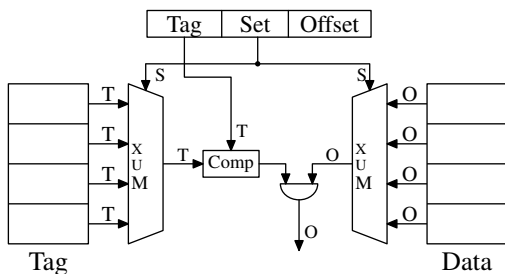


图 3.6: 直接映射缓存原理图

这样的 *direct-mapped cache* 速度很快并且相对容易实现，如图 3.6 所示。它需要

恰好有一个比较器、一个多路复用器（图中有两个多路复用器，其中标签和数据是分开的，但这不是设计的硬性要求）以及一些仅选择有效缓存行内容的逻辑。由于速度要求，比较器很复杂，但现在只有一个；因此，可以花更多的精力来加快速度。这种方法的真正复杂性在于多路复用器。简单多路复用器中晶体管的数量以 $O(\log N)$ 的速度增长，其中 N 是缓存行的数量。这是可以容忍的，但可能会变慢，在这种情况下，可以通过在多路复用器中的晶体管上花费更多空间来并行某些工作并提高速度来提高速度。晶体管总数会随着缓存大小的增加而缓慢增长，这使得该解决方案非常有吸引力。但它有一个缺点：只有当程序使用的地址相对于用于直接映射的位均匀分布时，它才能正常工作。如果不是（通常是这种情况），则某些缓存条目会被大量使用，因此会被重复驱逐，而其他缓存条目则几乎不会使用或保持为空。

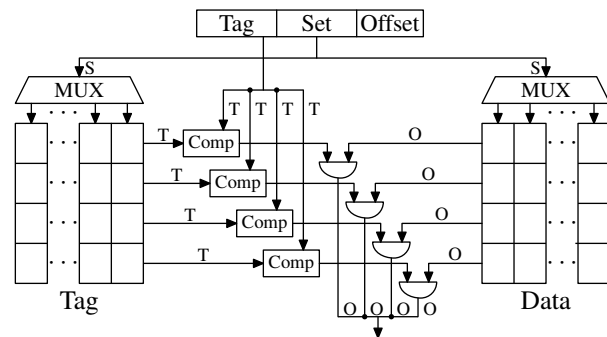


图 3.7: 组关联缓存原理图

这个问题可以通过将缓存设为 *set associative* 来解决。组关联缓存结合了全关联缓存和直接映射缓存的优点，在很大程度上避免了这些设计的弱点。图 3.7 显示了组关联缓存的设计。标签和数据存储被分成几组，其中一组由高速缓存行的地址选择。这与直接映射缓存类似。但是，缓存中的每个设置值不是只有一个元素，而是为同一设置值缓存少量值。并行比较所有集成员标签，这类似于全关联缓存的功能。

结果是高速缓存不会轻易被不幸或故意选择具有相同设置编号的地址所破坏，同时高速缓存的大小不受可以经济地实现的比较器数量的限制。如果缓存增长（在此图中），则仅增加列数，而不是行数。仅当缓存的关联性增加时，行数（以及比较器数）才会增加。如今，处理器对 L2 缓存或更高级别的缓存使用高达 24 的关联级别。L1 缓存通常有 8 组。

L2 Cache Size	Associativity							
	Direct		2		4		8	
	CL=32	CL=64	CL=32	CL=64	CL=32	CL=64	CL=32	CL=64
512k	27,794,595	20,422,527	25,222,611	18,303,581	24,096,510	17,356,121	23,666,929	17,029,334
1M	19,007,315	13,903,854	16,566,738	12,127,174	15,537,500	11,436,705	15,162,895	11,233,896
2M	12,230,962	8,801,403	9,081,881	6,491,011	7,878,601	5,675,181	7,391,389	5,382,064
4M	7,749,986	5,427,836	4,736,187	3,159,507	3,788,122	2,418,898	3,430,713	2,125,103
8M	4,731,904	3,209,693	2,690,498	1,602,957	2,207,655	1,228,190	2,111,075	1,155,847
16M	2,620,587	1,528,592	1,958,293	1,089,580	1,704,878	883,530	1,671,541	862,324

表 3.1: 高速缓存大小、关联性和行大小的影响

考虑到我们的 4MB/64B 缓存和 8 路组关联性，我们剩下的缓存有 8,192 个组，并且仅使用标签的 13 位来寻址缓存组。为了确定缓存集中的哪些条目（如果有）包含已寻址的缓存行 8 标签，必须进行比较。这在很短的时间内是可行的。通过实验我们可以看出这是有道理的。

表 3.1 显示了程序（根据 Linux 内核人员的说法，在本例中为 gcc，这是所有程序中最重要基准）在更改高速缓存大小、高速缓存行大小和关联集大小时的 L2 高速缓存未命中次数。在 7.2 节中，我们将介绍用于模拟本测试所需的缓存的工具。

以防万一这还不明显，所有这些值的关系是缓存大小是

缓存行大小 × 关联性 × 组数

地址通过使用映射到缓存中

$O = \log_2 \text{cache line size}$

$S = \log_2 \text{number of sets}$

如第 15 页上的图所示。

图 3.8 使表中的数据更容易理解。它显示了 32 字节固定缓存线大小的数据。查看给定缓存大小的数字，我们可以看到关联性确实有助于显着减少缓存未命中的数量。对于 8MB 缓存，从直接映射到 2 路组关联缓存可减少近 44% 的缓存未命中。与直接映射缓存相比，使用集关联缓存，处理器可以在缓存中保留更多的工作集。

在文献中，人们偶尔会读到引入关联性与将缓存大小加倍具有相同的效果。在某些极端情况下确实如此，从 4MB 缓存跳转到 8MB 缓存即可看出。但对于进一步加倍关联性来说，情况肯定不正确。从数据中我们可以看出，连续增益是

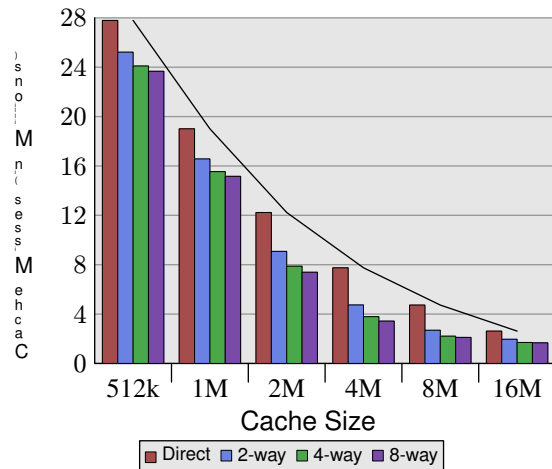


图 3.8: 缓存大小与关联性 (CL=32)

小得多。不过，我们不应该完全忽视其影响。在示例程序中，峰值内存使用量为 5.6M。因此，对于 8 MB 缓存，同一缓存集不太可能有多次（超过两次）使用。工作集越大，节省的成本就越高，正如我们可以从较小的缓存大小的关联性带来的更大好处中看到的那样。

一般来说，将缓存的关联性提高到 8 以上似乎对单线程工作负载影响不大。随着共享一级缓存的超线程处理器和使用共享二级缓存的多核处理器的引入，情况发生了变化。现在，基本上有两个程序访问同一个缓存，这导致实践中的关联性减半（或四核处理器的四分之一）。因此可以预期，随着核心数量的增加，共享缓存的关联性也会增加。一旦这不再可能（16 路组关联性已经很困难），处理器设计人员必须开始使用共享 L3 缓存及其他缓存，而 L2 缓存可能由核心子集共享。

我们可以在图 3.8 中研究的另一个影响是缓存大小的增加如何帮助提高性能。如果不了解工作原理，则无法解释此数据

设置大小。显然，与主内存一样大的缓存会比较小的缓存产生更好的结果，因此通常对最大缓存大小没有限制，但可以带来可衡量的好处。

上面已经提到，峰值时工作集的大小为 5.6M。这并没有给我们任何最大有益缓存大小的绝对数字，但它允许我们估计这个数字。问题在于，并非所有使用的内存都是连续的，因此，即使使用 16M 缓存和 5.6M 工作集，也会发生冲突（参见 2 路集关联 16M B 缓存相对于直接映射版本的优势）。但可以肯定的是，在相同的工作负载下，32MB 缓存的优势可以忽略不计。但谁说工作集必须保持不变？工作负载随着时间的推移而增长，缓存大小也应随之增长。购买机器时，必须选择愿意支付的缓存大小，因此测量工作集大小是值得的。为什么这很重要可以从第 21 页的图中看出。

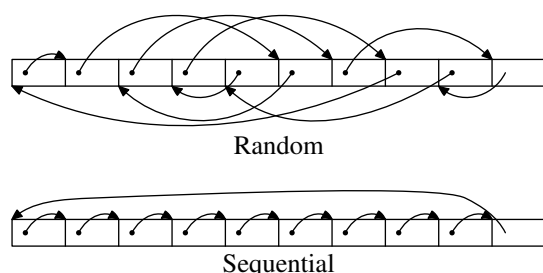


图 3.9：测试内存布局

运行两种类型的测试。在第一个测试中，元素是按顺序处理的。测试程序遵循指针 n ，但数组元素是链接的，以便按照它们在内存中找到的顺序遍历它们。这可以在图 3.9 的下半部分看到。最后一个元素有一个反向引用。在第二个测试（图的上半部分）中，数组元素以随机顺序遍历。在这两种情况下，数组元素形成一个循环单链表。

3.3.2 缓存效果的测量

所有数据都是通过测量一个程序创建的，该程序可以模拟任意大小的工作集、读写访问以及顺序或随机访问。我们已经在图 3.4 中看到了一些结果。该程序创建一个与该类型元素的工作集大小相对应的数组：

```
结构l { 结构l *n;长整数填充[
NPAD]; };
```

所有条目都使用 n 元素以顺序或随机顺序链接在循环列表中。从一个条目前进到下一个条目始终使用指针，即使元素是按顺序布局的。pad 元素是有效负载，它可以任意增大。在某些测试中，数据被修改，而在其他测试中，程序仅执行读取操作。

在性能测量中，我们讨论的是工作集大小。工作集由 struct l 元素的数组组成。 2^N 字节的工作集包含

$$2^N / \text{sizeof}(\text{struct l})$$

元素。显然 $\text{sizeof}(\text{struct l})$ 取决于 NPAD 的值。对于 32 位系统，NPAD=7 表示每个数组元素的大小为 32 字节，对于 64 位系统，大小为 64 字节。

单线程顺序访问 最简单的情况是简单遍历列表中的所有条目。列表元素按顺序排列，密集排列。无论处理顺序是向前还是向后并不重要，处理器可以同样好地处理两个方向。我们在这里以及在所有以下测试中测量的是处理单个列表元素所需的时间。时间单位是处理器周期。图 3.10 显示了结果。除非另有说明，所有测量均在 Pentium 4 机器上以 64 位模式进行，这意味着带有 NPAD=0 的结构 l 的大小为 8 个字节。

前两次测量受到噪声污染。测量的工作负载太小，无法过滤掉系统其余部分的影响。我们可以放心地假设这些值都处于 4 个周期的水平。考虑到这一点，我们可以看到三个不同的层次：

- 工作集大小最大为 2^{14} 字节。
- 从 2^{15} 字节到 2^{20} 字节。
- 从 2^{21} 字节及以上。

这些步骤很容易解释：处理器有 16kB L1d 和 1MB L2。我们在从一个级别到另一级别的过渡中看不到明显的边缘，因为高速缓存也由系统的其他部分使用，因此高速缓存并非专门用于程序数据。具体来说，L2 缓存是统一缓存，也用于指令（注意：英特尔使用包容性缓存）。

也许不太预期的是不同工作集大小的实际时间。L1d 命中的时间是预期的：L1d 命中后的加载时间约为 P4 上的 4 个周期。但是 L2 访问呢？一旦 L1d 不足以保存数据，人们可能会期望每个元素需要 14 个周期或更多，因为这是 L2 的访问时间。但结果表明只需要大约 9 个周期。这种差异

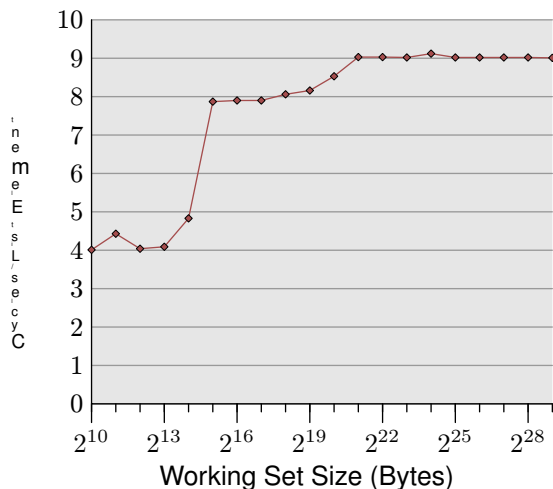


图 3.10: 顺序读取访问, NPAD=0

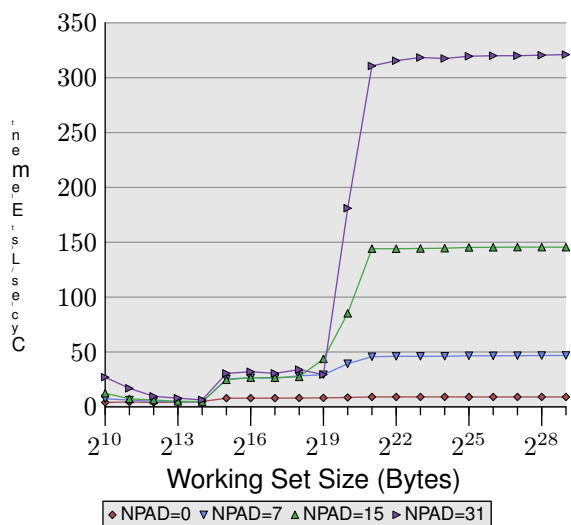


图 3.11: 多种大小的顺序读取

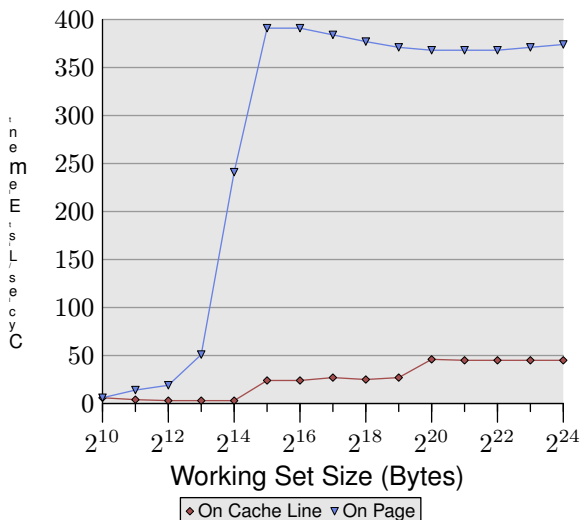


图 3.12: TLB 对顺序读取的影响

可以用处理器中的高级逻辑来解释。由于预计将使用连续的内存区域, 处理器 *prefetches* 将使用下一个高速缓存行。这意味着当实际使用下一行时, 它已经加载了一半。因此, 等待加载下一个高速缓存行所需的延迟远小于 L2 访问时间。

一旦工作集大小超过 L2 大小, 预取的效果就更加明显。之前我们说过一次主存访问需要 200+ 个周期。只有有效的预取, 处理器才有可能将访问时间保持在 9 个周期以下。从 200 和 9 之间的差异可以看出, 效果很好。

我们可以在预取时观察处理器, 至少是间接的。在图 3.11 中, 我们看到了相同工作集大小的时间, 但这次我们看到了结构 1 的不同大小的图表。这意味着列表中的元素更少但更大。不同的大小会导致 (仍然连续的) 列表中的 n 个元素之间的距离增大。在图中的四种情况下, 距离分别为 0、56、120 和 248 字节。

在底部我们可以看到图 3.10 中的线, 但这次它或多或少看起来是一条平线。其他情况的情况则要糟糕得多。我们也可以在这张图中看到三个不同的级别, 并且我们在小工作集大小的测试中看到了很大的错误 (再次忽略它们)。只要仅涉及 L1d, 这些行或多或少都相互匹配。无需预取, 因此每次访问时所有元素大小都会达到 L1d。

对于 L2 缓存命中, 我们看到这三个新行几乎彼此匹配, 但它们处于更高的级别 (大约 28)。这是 L2 的访问时间级别。这意味着从 L2 预取到 L1d 基本上被禁用。即使使用 NPAD=7, 我们也需要为循环的每次迭代提供一个新的缓存行; 相反, 对于 NPAD=0, 循环必须迭代八次才能需要下一个缓存行。预取逻辑无法在每个周期加载新的缓存行。因此, 我们会在每次迭代中看到从 L2 加载的停滞。

一旦工作集大小超过 L2 容量, 事情就会变得更加有趣。现在所有四条线都有很大差异。不同的元件尺寸显然对性能差异起着很大的作用。处理器应该识别步幅的大小, 并且不会为 NPAD=15 和 31 获取不必要的缓存行, 因为元素大小小于预取窗口 (请参阅第 6.3.1 节)。元素大小阻碍预取工作的地方是硬件预取限制的结果: 它不能跨越页面边界。每增加一次大小, 我们就会将硬件调度程序的效率降低 50%。如果允许硬件预取器跨越页面边界并且下一页不驻留或无效, 则操作系统将不得不参与定位

页。这意味着程序会遇到不是它自己发起的页面错误。这是完全不可接受的，因为处理器不知道页面是否存在或不存。在后一种情况下，操作系统将不得不中止该进程。无论如何，对于 NPAD=7 及更高版本，我们每个列表元素都需要一个缓存行，而硬件预取器无法做太多事情。根本没有时间从内存加载数据，因为处理器所做的只是读取一个字，然后加载下一个元素。

速度减慢的另一个重要原因是 TLB 缓存未命中。这是一个缓存，其中存储虚拟地址到物理地址的转换结果，如第 4 节中更详细地解释的。TLB 缓存非常小，因为它必须非常快。如果重复访问的页面数量多于 TLB 缓存的条目，则必须不断重复从虚拟地址到物理地址的转换。这是一个非常昂贵的操作。对于较大的元素大小，TLB 查找的成本会分摊到较少的元素上。这意味着每个列表元素必须计算的 TLB 条目总数更高。

为了观察 TLB 效果，我们可以运行不同的测试。对于一项测量，我们像往常一样按顺序布置元素。我们对占用整个缓存行的元素使用 NPAD=7。对于第二次测量，我们将每个列表元素放置在单独的页面上。每个页面的其余部分保持不变，我们不将其计入工作集大小的总数中。²⁰ 结果是，对于第一次测量，每个列表迭代都需要一个新的缓存行，并且对于每 64 个元素，需要一个新页面。对于第二次测量，每次迭代都需要加载新页面上的新缓存行。

结果如图 3.12 所示。测量是在与图 3.11 相同的机器上进行的。由于可用 RAM 的限制，工作集大小必须限制为 2^{24} 字节，这需要 1GB 将对象放置在单独的页面上。下面的红色曲线与图 3.11 中的 NPAD=7 曲线完全对应。我们看到显示 L1d 和 L2 缓存大小的不同步骤。第二条曲线看起来完全不同。重要的特征是当工作集大小达到 2^{13} 字节时开始出现巨大的峰值。这就是 TLB 缓存溢出的时候。当元素大小为 64 字节时，我们可以计算出 TLB 缓存有 64 个条目。不存在影响成本的页面错误，因为程序锁定内存以防止其被换出。

可以看出，计算物理地址并将其存储在 TLB 中所需的周期数非常高。图 3.12 中的图表显示了极端情况，但现在应该很清楚，缓慢的一个重要因素是

对于较大的 NPAD 值，TLB 缓存的效率会降低。由于必须先计算物理地址，然后才能读取 L2 或主存储器的高速缓存行，因此地址转换的代价会增加存储器访问时间。这部分解释了为什么 NPAD=31 每个列表元素的总成本高于 RAM 的理论访问时间。

通过查看修改列表元素的测试运行数据，我们可以了解预取实现的更多细节。图 3.13 显示了三行。在所有情况下，元素宽度均为 16 字节。第一行是现在熟悉的列表遍历，用作基线。第二行标记为 “Inc”，在继续下一个元素之前简单地增加当前元素的 pad[0] 成员。第三行标记为 “Addnext0”，获取 next 元素的 pad[0] 列表元素，并将其添加到当前列表元素的 pad[0] 成员。

天真的假设是 “Addnext0” 测试运行速度较慢，因为它有更多工作要做。在前进到下一个列表元素之前，必须加载该元素的值。这就是为什么令人惊讶的是，对于某些工作集大小，该测试实际上运行得比 “Inc” 测试更快。对此的解释是，来自下一个列表元素的加载基本上是强制预取。每当程序前进到下一个列表元素时，我们就可以确定该元素已经在 L1d 缓存中。结果，我们看到只要工作集大小适合二级缓存，“Addnext0” 就可以像简单的 “Follow” 测试一样执行。

不过，“Addnext0” 测试运行 L2 的速度比 “Inc” 测试要快。它需要从主存储器加载更多数据。这就是为什么 “Addnext0” 测试达到 2^{21} 字节工作集大小的 28 个周期级别。28 个周期的水平是 “跟随” 测试达到的 14 个周期水平的两倍。这也很容易解释。由于其他两个测试会修改内存，因此为新的缓存行腾出空间的 L2 缓存驱逐不能简单地丢弃数据。相反，它必须被写入内存。这意味着 FSB 上的可用带宽减半，因此将数据从主存储器传输到 L2 所需的时间加倍。

顺序、高效的缓存处理的最后一个方面是缓存的大小。这应该是显而易见的，但仍然应该指出。图 3.14 显示了具有 128 字节元素的增量基准测试的时序（64 位计算机上的 NPAD=15）。这次我们看到了三台不同机器的测量结果。前两台机器是 P4，最后一台是 Core2 处理器。前两者的区别在于具有不同的缓存大小。第一个处理器具有 32k L1d 和 1M L2。第二个有 16k L1d、512k L2 和 2M L3。Core2 处理器具有 32k L1d 和 4M L2。

该图有趣的部分不一定是 Core2 处理器相对于其他两个处理器的性能如何（尽管它令人印象深刻）。这里的主要兴趣点是工作集大小的区域

²⁰Yes, this is a bit inconsistent because in the other tests we count the unused part of the struct in the element size and we could define NPAD so that each element fills a page. In that case the working set sizes would be very different. This is not the point of this test, though, and since prefetching is ineffective anyway this makes little difference.

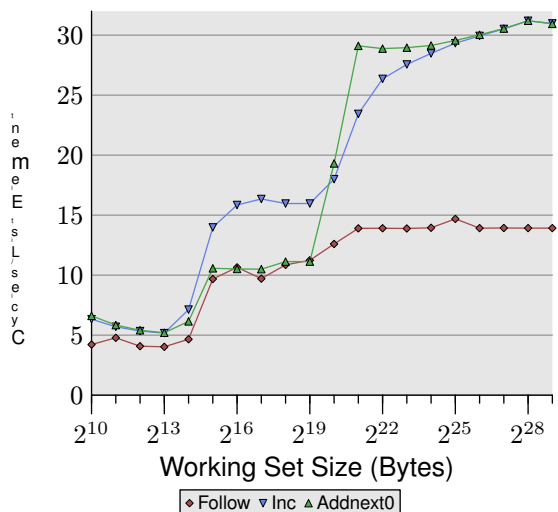


图 3.13: 顺序读写, NPAD=1

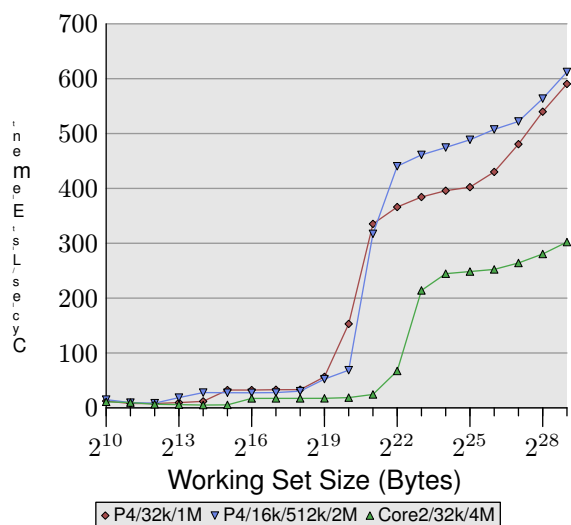


图 3.14: 更大的 L2/L3 缓存的优势

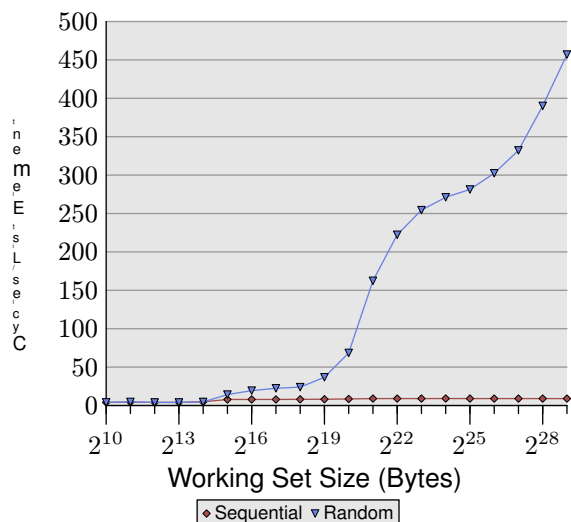


图 3.15: 顺序读取与随机读取, NPAD=0

对于相应的最后一级缓存来说太大, 并且主内存会大量参与。

正如预期的那样, 最后一级缓存越大, 曲线停留在与 L2 访问成本相对应的低水平的时间就越长。需要注意的重要部分是它提供的性能优势。第二个处理器 (稍旧) 可以在 2^{20} 字节工作集上执行工作, 速度是第一个处理器的两倍。这一切都归功于最后一级缓存大小的增加。具有 4M L2 的 Core2 处理器性能甚至更好。

对于随机工作负载来说, 这可能意义不大。但是, 如果工作负载可以根据最后一级缓存的大小进行调整, 则程序性能可以显著提高。这就是为什么有时花额外的钱购买具有更大缓存的处理器是值得的。

多线程随机访问 我们已经看到, 处理器能够通过将缓存行预取到 L2 和 L1d 来隐藏大部分主内存, 甚至隐藏 L2 访问延迟。不过, 只有当内存访问是可预测的时候, 这才能很好地发挥作用。

如果访问模式是不可预测的或随机的, 情况就完全不同了。图 3.15 将顺序访问的每个列表元素时间 (与图 3.10 相同) 与列表元素随机分布在工作集中的时间进行了比较。顺序由随机链表决定。处理器无法可靠地预取数据。只有当紧接着使用的元素在内存中也彼此接近时, 这才可能偶然起作用。

图 3.15 中有两点需要注意。首先是增加工作集大小所需的大量周期。该机器可以在 200-300 个周期内访问主存储器, 但这里我们达到 450 个周期甚至更多。我们以前见过这种现象 (比较图 3.11)。自动预取实际上在这里有一个缺点。

第二个有趣的点是, 曲线在各个平台上并不像顺序访问情况那样平坦。The curve keeps on rising.为了解释这一点, 我们可以测量程序在不同工作集大小下的 L2 访问。结果如图 3.16 和表 3.2 所示。

该图显示, 当工作集大小大于 L2 大小时, 缓存未命中率 (L2 访问/L2 未命中) 开始增长。该曲线的形式与图 3.15 中的曲线类似: 快速上升, 略微下降, 然后再次开始上升。与每个列表元素图的周期有很强的相关性。L2 缺失率将会不断增长, 直至最终接近 100%。给定足够大的工作集 (和 RAM), 任何随机选取的高速缓存行位于 L2 中或正在加载的过程中的概率都可以任意降低。

Set Size	Sequential					Random				
	L2 Hit	L2 Miss	#Iter	Ratio Miss/Hit	L2 Accesses per Iteration	L2 Hit	L2 Miss	#Iter	Ratio Miss/Hit	L2 Accesses per Iteration
2 ²⁰	88,636	843	16,384	0.94%	5.5	30,462	4721	1,024	13.42%	34.4
2 ²¹	88,105	1,584	8,192	1.77%	10.9	21,817	15,151	512	40.98%	72.2
2 ²²	88,106	1,600	4,096	1.78%	21.9	22,258	22,285	256	50.03%	174.0
2 ²³	88,104	1,614	2,048	1.80%	43.8	27,521	26,274	128	48.84%	420.3
2 ²⁴	88,114	1,655	1,024	1.84%	87.7	33,166	29,115	64	46.75%	973.1
2 ²⁵	88,112	1,730	512	1.93%	175.5	39,858	32,360	32	44.81%	2,256.8
2 ²⁶	88,112	1,906	256	2.12%	351.6	48,539	38,151	16	44.01%	5,418.1
2 ²⁷	88,114	2,244	128	2.48%	705.9	62,423	52,049	8	45.47%	14,309.0
2 ²⁸	88,120	2,939	64	3.23%	1,422.8	81,906	87,167	4	51.56%	42,268.3
2 ²⁹	88,137	4,318	32	4.67%	2,889.2	119,079	163,398	2	57.84%	141,238.5

表 3.2: 顺序游走和随机游走的 L2 命中和未命中, NPAD=0

不断增加的缓存未命中率本身就解释了一些成本。但还有另一个因素。查看表 3.2, 我们可以在 L2/#Iter 列中看到, 程序每次迭代使用的 L2 总数正在增长。每个工作集都是之前的两倍大。因此, 如果没有缓存, 我们预计主内存访问量会增加一倍。凭借缓存和 (几乎) 完美的可预测性, 我们看到顺序访问数据中显示的 L2 使用量略有增加。增加是由于工作集大小的增加而不是其他原因。

对于随机访问, 工作集大小每增加一倍, 每个元素的访问时间就会增加一倍以上。这意味着每个列表元素的平均访问时间会增加, 因为工作集大小仅增加了一倍。这背后的原因是 TLB 未命中率不断上升。在图 3.17 中, 我们看到了 NPAD=7 随机访问的成本。只是这一次随机化被修改了。在正常情况下, 整个随机化列表作为一个块 (由标签 ∞ 表示), 其他 11 条曲线显示在较小的块中执行的随机化。对于标记为 “60” 的曲线, 每组 60 页 (245.760 字节) 单独随机化。这意味着在转到下一个块中的元素之前会遍历该块中的所有列表元素。这会导致任一时间使用的 TLB 条目数量受到限制。

NPAD=7 的元素大小为 64 字节, 对应于缓存行大小。由于列表元素的随机顺序, 硬件预取器不太可能产生任何影响, 尤其是对于少数元素而言。这意味着二级缓存未命中率与一个块中整个列表的随机化没有显著差异。随着块大小的增加, 测试的性能逐渐接近单块随机化的曲线。这意味着后一个测试用例的性能受到 TLB 未命中的显著影响。如果 TLB 未命中率可以降低, 则性能会显著提高

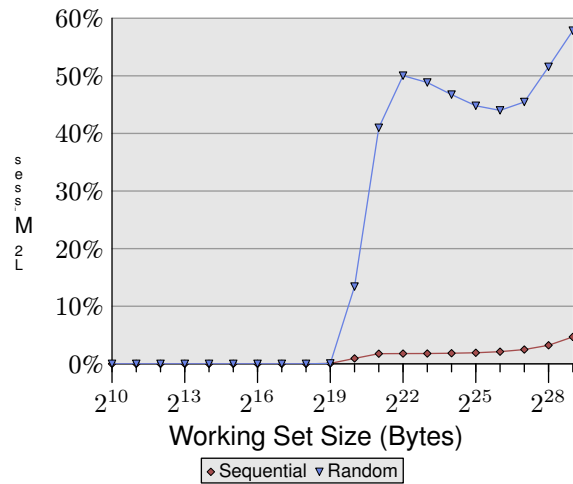


图 3.16: L2d 缺失率

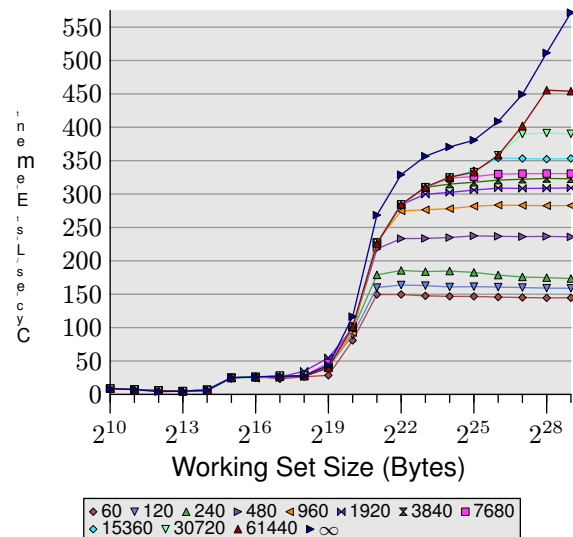


图 3.17: 逐页随机化, NPAD=7

(在一项测试中我们稍后会看到高达 38%)。

3.3.3 写入行为

在我们开始研究多个执行上下文(线程或进程)使用相同内存时的缓存行为之前,我们必须探索缓存实现的细节。缓存应该是一致的,并且这种一致性对于用户级代码应该是完全透明的。内核代码则是另一回事;有时需要刷新缓存。

这具体意味着,如果修改了缓存行,则该时间点之后系统的结果与根本没有缓存并且主内存位置本身已被修改一样。这可以通过两种方式或政策来实现:

- 直写式缓存实现;
- 回写式缓存实现。

直写式缓存是实现缓存一致性的最简单方法。如果写入高速缓存行,处理器也会立即将高速缓存行写入主存储器。这确保主内存和缓存始终保持同步。每当替换高速缓存线时,可以简单地丢弃高速缓存内容。这种缓存策略很简单,但速度不是很快。例如,一个反复修改局部变量的程序会在 FSB 上产生大量流量,即使这些数据可能不会在其他地方使用并且可能是短暂的。

回写策略更加复杂。这里处理器不会立即将修改后的缓存行写回主内存。相反,缓存行仅被标记为脏。当将来某个时刻从高速缓存中删除高速缓存行时,脏位将指示处理器当时将数据写回,而不是仅仅丢弃内容。

Write-back caches have the chance to be significantly better performing, which is why most memory in a system with a decent processor is cached this way.处理器甚至可以在必须清空高速缓存行之前利用 FSB 上的可用容量来存储该行的内容。这允许清除脏位,并且当需要高速缓存中的空间时,处理器可以仅删除高速缓存行。

但回写的实现存在一个重大问题。当多个处理器(或核心或超线程)可用并访问同一内存时,仍然必须确保两个处理器始终看到相同的内存内容。如果一个处理器上的高速缓存行是脏的(即,尚未写回),并且第二个处理器尝试读取相同的内存位置,则读取操作不能直接发送到主内存。而是第一个处理器缓存的内容

需要线路。在下一节中,我们将了解当前是如何实现的。

在我们讨论这个之前,还有两个缓存策略需要提及:

- 写组合;和
- 不可缓存。

这两种策略都用于不受实际 RAM 支持的地址空间的特殊区域。内核为地址范围设置这些策略(在使用内存类型范围寄存器, MTRR 的 x86 处理器上),其余的事情会自动发生。MTRR 还可用于在直写和回写策略之间进行选择。

写组合是一种有限的缓存优化,更常用于显卡等设备上的 RAM。由于设备的传输成本远高于本地 RAM 访问,因此避免进行过多的传输就显得尤为重要。如果下一个操作修改了下一个字,仅仅因为行中的一个字已被写入而传输整个高速缓存行是浪费的。人们可以很容易地想象这是一种常见的情况,屏幕上水平相邻像素的内存存在大多数情况下也是相邻的。顾名思义,写组合在写出缓存行之前组合多个写访问。在理想情况下,整个缓存行会逐字修改,并且只有在写入最后一个字后,缓存行才会写入设备。这可以显着加快对设备上 RAM 的访问速度。

最后还有不可缓存的内存。这通常意味着内存位置根本不受 RAM 支持。它可能是一个特殊地址,经过硬编码以在 CPU 外部实现某些功能。对于商用硬件,最常见的情况是内存映射地址范围,它转换为对连接到总线(PCIe 等)的卡和设备的访问。在嵌入式板上有时会找到这样一个内存地址,可用于打开和关闭 LED。缓存这样的地址显然是一个坏主意。在这种情况下,LED 用于调试或状态报告,人们希望尽快看到这一点。PCIe 卡上的内存可以在没有 CPU 交互的情况下发生变化,因此不应缓存该内存。

3.3.4 多处理器支持

在上一节中,我们已经指出了多处理器发挥作用时遇到的问题。即使多核处理器也存在那些不共享的缓存级别(至少是 L1d)的问题。

提供从一个处理器到另一处理器的高速缓存的直接访问是完全不切实际的。首先,连接速度不够快。实际的替代方案是将缓存内容传输到

另一个处理器，以备需要时使用。请注意，这也适用于不在同一处理器上共享的缓存。

现在的问题是缓存线传输什么时候必须发生？这个问题很容易回答：当一个处理器需要一个在另一个处理器的缓存中是脏的缓存行来读取或写入时。但是处理器如何确定另一个处理器的缓存中的缓存行是否脏呢？假设它只是因为缓存行被另一个处理器加载将是次优的（充其量）。通常大多数内存访问都是读访问，并且生成的缓存行不是脏的。处理器对高速缓存行的操作很频繁（当然，否则我们为什么会有这篇论文？）这意味着在每次写访问后广播有关已更改高速缓存行的信息是不切实际的。

多年来开发出来的是 MESI 缓存一致性协议（修改的、独占的、共享的、无效的）。该协议以使用 MESI 协议时缓存线可以处于的四种状态命名：

已修改：本地处理器已修改缓存行。这也意味着它是任何缓存中的唯一副本。

独占：高速缓存行未被修改，但已知不会加载到任何其他处理器的高速缓存中。

共享：高速缓存行未被修改，并且可能存在于另一个处理器的高速缓存中。

无效：缓存行无效，即未使用。

多年来，该协议从更简单的版本发展而来，这些版本不太复杂，但效率也较低。通过这四种状态，可以有效地实现回写式缓存，同时还支持在不同处理器上并发使用只读数据。

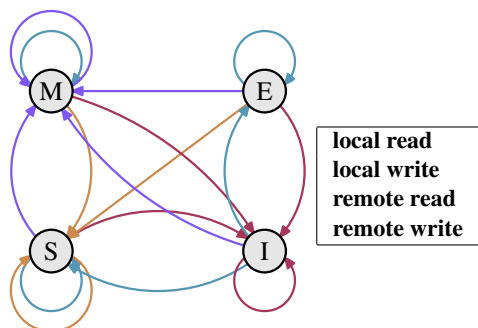


图 3.18: MESI 协议转换

状态改变是通过处理器监听或窥探其他处理器的工作来完成的，不需要太多的努力。处理器执行的某些操作是在外部引脚上宣布的，从而使

处理器的缓存处理对外部可见。相关缓存行的地址在地址总线上可见。在下面对状态及其转换的描述中（如图 3.18 所示），我们将指出何时涉及总线。

最初，所有缓存行都是空的，因此也是无效的。如果数据加载到缓存中进行写入，则缓存更改为 Modified。是否加载数据以读取新状态取决于另一个处理器是否也加载了高速缓存线。如果是这种情况，则新状态为共享状态，否则为独占状态。

如果在本地处理器上读取或写入修改后的高速缓存行，则指令可以使用当前高速缓存内容，并且状态不会更改。如果第二个处理器想要从缓存行中读取，第一个处理器必须将其缓存的内容发送到第二个处理器，然后它可以将状态更改为共享。发送到第二处理器的数据也由存储器控制器接收和处理，并将内容存储在存储器中。如果没有发生这种情况，则缓存行无法标记为共享。如果第二个处理器想要写入高速缓存行，则第一个处理器会发送高速缓存行内容并将本地高速缓存行标记为无效。这就是臭名昭著的“所有权请求”（RFO）操作。在最后一级缓存中执行此操作，就像 $I \rightarrow M$ 转换一样，成本相对较高。对于直写式缓存，我们还必须增加将新的缓存行内容写入下一个更高级别的缓存或主内存所需的时间，这进一步增加了成本。

如果缓存行处于共享状态并且本地处理器从中读取数据，则无需更改状态，并且可以从缓存中满足读取请求。如果缓存行是本地写入的，则缓存行也可以使用，但状态会更改为“已修改”。它还要求其他处理器中缓存行的所有其他可能副本都标记为无效。因此，必须通过 RFO 消息向其他处理器宣布写操作。如果第二个处理器请求读取高速缓存行，则不会发生任何事情。主存储器包含当前数据并且本地状态已经共享。如果第二个处理器想要写入高速缓存行 (RFO)，则该高速缓存行将被简单地标记为无效。无需进行总线操作。

独占状态与共享状态基本相同，但有一个关键区别：本地写操作 *not* 必须在总线上声明。已知本地缓存是唯一保存该特定缓存行的缓存。这是一个巨大的优势，因此处理器将尝试将尽可能多的缓存线保持在独占状态而不是共享状态。后者是当信息不可用时的后备方案。独占状态也可以完全省略，而不会导致功能问题。只是性能会受到影响，因为 $E \rightarrow M$ 转换比 $S \rightarrow M$ 转换快得多。

从状态转换的描述中应该可以清楚多处理器操作的特定成本在哪里。是的，填充缓存仍然很昂贵，但现在我们还必须留意 RFO 消息。每当需要发送这样的消息时，速度都会很慢。

有两种情况需要 RFO 消息：

- 线程从一个处理器迁移到另一个处理器，并且所有高速缓存行都必须移至新处理器一次。
- 两个不同的处理器确实需要高速缓存行。²¹

在多线程或多进程程序中总是需要同步；这种同步是使用内存来实现的。所以有一些有效的 RFO 消息。它们仍然必须尽可能少出现。不过，RFO 消息还有其他来源。在第 6 节中，我们将解释这些场景。高速缓存一致性协议消息必须在系统的处理器之间分发。在明确系统中的所有处理器都有机会回复消息之前，MESI 转换不会发生。这意味着回复所需的最长可能时间决定了一致性协议的速度。²²总线上的冲突是可能的，NUMA 系统中的延迟可能很高，当然，巨大的流量也会减慢速度。这些都是集中精力避免不必要交通的充分理由。

还有一个与使用多个处理器相关的问题。其影响是高度机器特定的，但原则上问题始终存在：FSB 是共享资源。在大多数机器中，所有处理器都通过一条总线连接到内存控制器（见图 2.1）。如果单个处理器可以使总线饱和（通常是这种情况），那么共享同一总线的两个或四个处理器将进一步限制每个处理器可用的带宽。

即使每个处理器都有自己的连接内存控制器的总线（如图 2.2 所示），仍然有连接内存模块的总线。通常这是一条总线，但即使在图 2.2 中的扩展模型中，对同一内存模块的并发访问也会限制带宽。

AMD 模型也是如此，每个处理器都可以有本地内存。所有处理器确实可以同时快速访问其本地内存，尤其是使用集成内存控制器时。但是多线程和多进程程序——至少有时

时间——必须访问相同的内存区域才能同步。

并发性受到可用于实现必要同步的有限带宽的严重限制。需要仔细设计程序，以尽量减少不同处理器和内核对相同内存位置的访问。以下测量将显示此效果以及多线程代码相关的其他缓存效果。

多线程访问 为了确保理解在不同处理器上同时使用相同高速缓存行所带来的问题的严重性，我们将在此处查看我们之前使用的同一程序的更多性能图。不过，这一次有多个线程同时运行。测量的是任何线程中最快的运行时间。这意味着所有线程完成后完整运行的时间甚至更长。使用的机器有四个处理器；测试最多使用四个线程。所有处理器共享一条通往内存控制器的总线，并且只有一条总线通往内存模块。

图 3.19 显示了 128 字节条目的顺序只读访问的性能（64 位机器上的 NPAD=15）。对于一个线程的曲线，我们可以预期得到类似于图 3.11 的曲线。测量结果适用于不同的机器，因此实际数字有所不同。

当然，该图中的重要部分是运行多个线程时的行为。请注意，在遍历链表时，不会修改任何内存，也不会尝试保持线程同步。尽管不需要 RFO 消息并且可以共享所有缓存行，但我们发现，当使用两个线程时，最快线程的性能下降高达 18%，当使用四个线程时，性能下降高达 34%。由于不需要在处理器之间传输缓存行，因此这种速度减慢完全是由两个瓶颈之一或两者造成的：从处理器到内存控制器的共享总线以及从内存控制器到内存模块的总线。一旦工作集大小大于本机中的 L3 缓存，所有三个线程都将预取新的列表元素。即使有两个线程，可用带宽也不足以线性扩展（即，运行多个线程不会造成任何损失）。

当我们修改内存时，事情会变得更加丑陋。图 3.20 显示了顺序增量测试的结果。该图的 Y 轴使用对数刻度。因此，不要被表面上微小的差异所迷惑。运行两个线程时我们仍然会受到大约 18% 的惩罚，而现在运行四个线程时会受到 93% 的惊人惩罚。这意味着当使用四个线程时，预取流量和写回流量几乎使总线饱和。

我们使用对数标度来显示 L1d 范围的结果。可以看到的是，一旦更多

²¹At a smaller level the same is true for two cores on the same processor. The costs are just a bit smaller. The RFO message is likely to be sent many times.

²²Which is why we see nowadays, for instance, AMD Opteron systems with three sockets. Each processor is exactly one hop away given that the processors only have three hyperlinks and one is needed for the Northbridge connection.

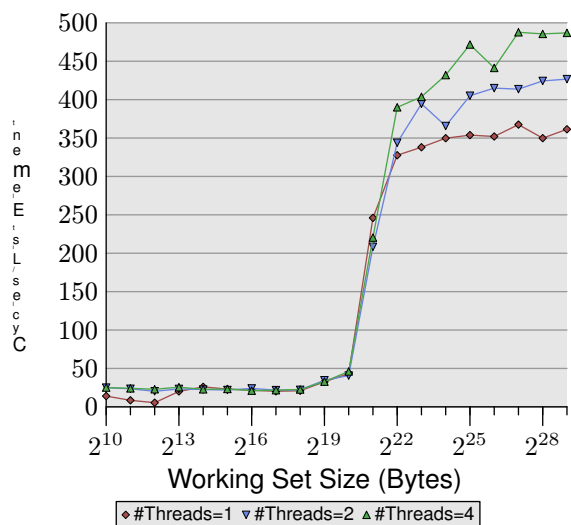


图 3.19: 顺序读取访问, 多线程

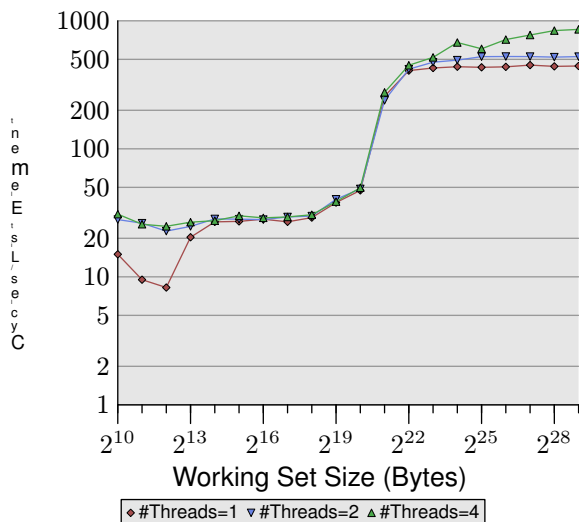


图 3.20: 顺序增量, 多线程

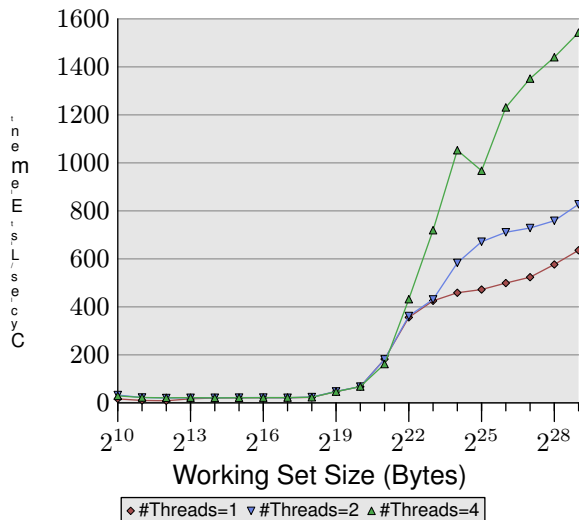


图 3.21: 随机 Addnextlast, 多线程

当一个线程正在运行时, L1d 基本上是无效的。仅当 L1d 不足以容纳工作集时, 单线程访问时间才会超过 20 个周期。当多个线程运行时, 即使工作集大小最小, 访问时间也会立即受到影响。

这里没有显示问题的一方面。用这个特定的测试程序很难测量。尽管测试会修改内存, 因此我们必须期待 RFO 消息, 但当使用多个线程时, 我们并没有看到 L2 范围的成本更高。该程序必须使用大量内存, 并且所有线程必须并行访问同一内存。如果没有大量的同步, 这将很难实现, 因为同步会占据执行时间。

最后, 在图 3.21 中, 我们得到了随机访问内存的 Addnextlast 测试的数字。提供此图主要是为了显示令人震惊的高数字。现在, 在极端情况下, 处理单个列表元素需要大约 1,500 个周期。使用更多线程就更值得怀疑了。我们可以在表格中总结多线程使用的效率。

#Threads	Seq Read	Seq Inc	Rand Add
2	1.69	1.69	1.54
4	2.98	2.07	1.65

表 3.3: 多线程的效率

该表显示了第 28 页上的三张图中最大工作集大小的多线程运行的效率。该数字显示了使用两个或四个线程时测试程序在最大工作集大小下所带来的最佳可能加速。对于两个线程, 加速比的理论极限是 2, 对于四个线程, 理论极限是 4。两个线程的数字还不错。但对于四个线程, 最后一次测试的数字表明, 扩展到两个线程以上几乎不值得。额外的好处微乎其微。如果我们以稍微不同的方式表示图 3.21 中的数据, 我们可以更容易地看到这一点。

图 3.22 中的曲线显示了加速因素, 即与单线程执行的代码相比的相对性能。我们必须忽略最小的尺寸, 测量不够准确。对于 L2 和 L3 缓存的范围, 我们可以看到我们确实实现了几乎线性的加速。我们几乎分别达到因子 2 和 4。但一旦 L3 缓存不足以容纳工作集, 数字就会崩溃。它们崩溃到两个线程和四个线程的加速相同的程度 (参见表 3.3 中的第四列)。这就是为什么很难找到带有四个以上 CPU 插槽且全部使用相同内存控制器的主板的原因之一。具有更多处理器的机器必须以不同的方式构建 (参见第 5 节)。

这些数字并不具有普遍性。在某些情况下, 即使适合最后一级缓存的工作集也不会

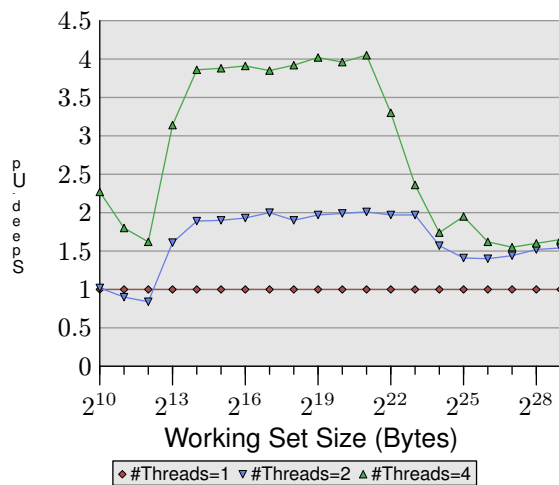


图 3.22: 通过并行加速

允许线性加速。事实上，这是常态，因为线程通常不像此测试程序中那样解耦。另一方面，可以使用大型工作集并仍然利用两个以上的线程。不过，这样做需要思考。我们将在第 6 节中讨论一些方法。

特殊情况：超线程 超线程（有时称为对称多线程，SMT）由 CPU 实现，是一种特殊情况，因为各个线程实际上不能同时运行。它们都共享除寄存器组之外的几乎所有处理资源。各个核心和 CPU 仍然并行工作，但每个核心上实现的线程受到此限制的限制。理论上每个核心可以有多个线程，但到目前为止，英特尔的 CPU 每个核心最多有两个线程。CPU 负责对线程进行时间复用。不过，仅此一点并没有多大意义。真正的优势在于，当当前运行的超线程延迟时，CPU 可以调度另一个超线程并利用算术逻辑单元 (ALU) 等可用资源。在大多数情况下，这是由内存访问引起的延迟。

如果两个线程在一个超线程核心上运行，则只有当两个线程的 *combined* 运行时间低于单线程代码的运行时间时，程序才会比单线程代码更高效。这可以通过重叠通常按顺序发生的不同内存访问的等待时间来实现。简单计算一下，就可以得出达到一定加速比对缓存命中率的最低要求。

程序的执行时间可以用只有一级缓存的简单模型来近似，如下所示（参见[16]）：

$$T_{\text{exe}} = N [(1 - F_{\text{mem}})T_{\text{proc}} + F_{\text{mem}}(G_{\text{hit}}T_{\text{cache}} + (1 - G_{\text{hit}})T_{\text{miss}})]$$

变量含义如下：

N = 指令数量。 F_{mem} = 访问内存的 N 部分。 G_{hit} = 命中缓存的负载分数。 T_{proc} = 每条指令的周期数。 T_{cache} = 缓存命中的周期数。 T_{miss} = 缓存未命中的周期数。 T_{exe} = 程序的执行时间。

为了使使用两个线程有意义，两个线程中每个线程的执行时间最多只能是单线程代码的一半。两边唯一的变量是缓存命中数。如果我们求解不使线程执行速度减慢 50% 或更多所需的最小缓存命中率的方程，我们将得到图 3.23 中的图表。

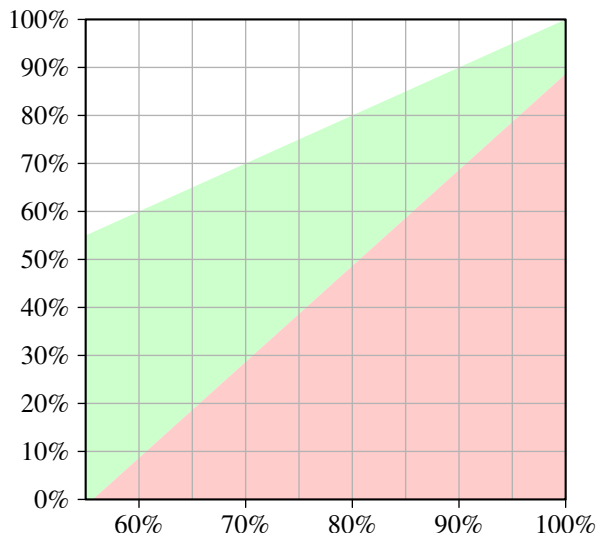


图 3.23: 加速的最低缓存命中率

在 X 轴上测量的输入是单线程代码的缓存命中率 G_{hit} 。Y 轴显示多线程代码的缓存命中率。该值永远不能高于单线程命中率，否则单线程代码也会使用该改进的代码。对于单线程命中率（在本例中）低于 55% 的情况，程序在所有情况下都可以从使用线程中受益。由于缓存未命中，CPU 或多或少足够空闲，可以运行第二个超线程。

绿色区域是目标。如果线程的速度减慢小于 50%，并且每个线程的工作负载减半，则组合运行时间可能会小于单线程运行时间。对于建模处理器（使用具有超线程的 P4 的编号），有一个程序

单线程代码的命中率为 60%，要求双线程程序的命中率至少为 10%。这通常是可行的。但如果单线程代码的命中率为 95%，那么多线程代码的命中率至少需要 80%。那更难了。特别是，这是超线程的问题，因为现在每个超线程可用的有效缓存大小（这里是 L1d，实际上还有 L2 等等）被削减了一半。两个超线程使用相同的缓存来加载数据。如果两个线程的工作集不重叠，则原始 95% 的命中率也会减半，因此远低于所需的 80%。

因此，超线程仅在有限的情况下有用。单线程代码的缓存命中率必须足够低，以便在考虑到上述等式和减小缓存大小的情况下，新的命中率仍能满足目标。只有这样，使用超线程才有意义。实际上结果是否更快取决于处理器是否能够充分地将一个线程中的等待时间与其他线程中的执行时间重叠。并行化代码的开销必须添加到新的总运行时间中，并且这种额外成本通常不能忽视。

在第 6.3.4 节中，我们将看到一种技术，其中线程紧密协作，并且通过公共缓存的紧密耦合实际上是一种优势。只要程序员愿意投入时间和精力来扩展他们的代码，这种技术就可以适用于许多情况。

应该清楚的是，如果两个超线程执行完全不同的代码（即，操作系统将这两个线程视为单独的处理器来执行单独的进程），则缓存大小确实会减少一半，这意味着缓存未命中的情况会显着增加。除非缓存足够大，否则这种操作系统调度实践是有问题的。除非机器的工作负载由通过其设计确实可以从超线程中受益的进程组成，否则最好在计算机的 BIOS 中关闭超线程。²³

3.3.5 其他细节

到目前为止，我们讨论的地址由三部分组成：标记、集合索引和缓存行偏移量。但实际使用的是哪个地址呢？如今所有相关处理器都为进程提供虚拟地址空间，这意味着有两种不同类型的地址：虚拟地址和物理地址。

虚拟地址的问题在于它们不是唯一的。随着时间的推移，虚拟地址可以引用不同的物理内存地址。不同进程中的相同地址也可能引用不同的物理地址。所以使用物理内存地址总是更好，对吧？

这里的问题是执行期间使用的虚拟地址必须在内存管理单元（MMU）的帮助下转换为物理地址。这是一个不平凡的操作。在执行指令的流水线中，物理地址可能仅在稍后阶段可用。这意味着缓存逻辑必须非常快速地确定内存位置是否被缓存。如果可以使用虚拟地址，则缓存查找可以在管道中更早地发生，并且在缓存命中的情况下，可以使内存内容可用。结果是更多的内存访问成本可能被管道隐藏。

处理器设计者目前正在对第一级高速缓存使用虚拟地址标记。这些缓存相当小，可以轻松清除。如果进程的页表树发生变化，至少需要部分清除缓存。如果处理器有一条指令允许指定已更改的虚拟地址范围，则可能可以避免完全刷新。鉴于 L1i 和 L1d 缓存（~ 3 个周期）的低延迟，使用虚拟地址几乎是强制性的。

对于较大的缓存，包括 L2、L3、...。需要缓存物理地址标记。这些缓存具有较高的延迟，并且虚拟物理地址转换可以及时完成。由于这些缓存较大（即，刷新时会丢失大量信息），并且由于主内存访问延迟而重新填充它们需要很长时间，因此经常刷新它们的成本很高。

一般来说，没有必要了解这些高速缓存中地址处理的细节。它们无法更改，并且所有影响性能的因素通常都应该避免或与高成本相关。缓存容量溢出是不好的，如果大多数使用的缓存线属于同一组，那么所有缓存都会尽早遇到问题。后者可以通过虚拟寻址的缓存来避免，但对于用户级进程来说，使用物理地址寻址的缓存是不可能避免的。人们可能需要记住的唯一细节是，如果可能的话，不要将相同的物理内存位置映射到同一进程中的两个或多个虚拟地址。

程序员对缓存相当不感兴趣的另一个细节是缓存替换策略。大多数缓存首先删除最近最少使用（LRU）元素。这始终是一个很好的默认策略。随着更大的关联性（由于添加了更多核心，关联性在未来几年确实可能会进一步增长），维护 LRU 列表变得越来越昂贵，我们可能会看到采用不同的策略。

至于缓存替换，程序员无能为力。如果缓存使用物理地址标签，则无法找出虚拟地址如何与缓存集关联。可能所有逻辑页中的缓存行都映射到相同的缓存集，

²³ Another reason to keep hyper-threads enabled is debugging. SMT is astonishingly good at finding some sets of problems in parallel code.

留下大部分缓存未使用。如果有的话，操作系统的工作就是安排这种情况不要经常发生。

随着虚拟化的出现，事情变得更加复杂。现在甚至操作系统也无法控制物理内存的分配。虚拟机监视器（VMM，又名 Hypervisor）负责物理内存分配。

程序员能做的最好的事情就是 a) 完全使用逻辑内存页面，b) 使用尽可能大的页面大小，以尽可能地使物理地址多样化。较大的页面尺寸还有其他好处，但这是另一个主题（参见第 4 节）。

3.4 指令缓存

不仅仅是处理器使用的数据被缓存；处理器执行的指令也被缓存。然而，这种缓存比数据缓存的问题要少得多。有以下几个原因：

- 执行的代码量取决于所需代码的大小。代码的大小通常取决于问题的复杂性。问题的复杂性是固定的。
- 虽然程序的数据处理是由程序员设计的，但程序的指令通常由编译器生成。编译器编写者了解良好代码生成的规则。
- 程序流程比数据访问模式更可预测。当今的 CPU 非常擅长检测模式。这有助于预取。
- 代码总是具有很好的空间和时间局部性。

程序员应该遵循一些规则，但这些规则主要包括如何使用工具的规则。我们将在第 6 节中讨论它们。这里我们只讨论指令缓存的技术细节。

自从 CPU 的核心时钟急剧增加以及高速缓存（甚至一级高速缓存）和核心之间的速度差异增大以来，CPU 就开始采用流水线设计。这意味着指令的执行是分阶段发生的。首先对指令进行解码，然后准备参数，最后执行指令。这样的管道可能会相当长（对于英特尔的 Netburst 架构，> 20 个阶段）。长管道意味着如果管道停顿（即流经其中的指令流被中断），则需要一段时间才能再次加速。例如，如果无法正确预测下一条指令的位置，或者加载下一条指令需要太长时间（例如，必须从内存中读取它时），就会发生管道停顿。

因此，CPU 设计人员在分支预测上花费了大量时间和芯片空间，以便尽可能减少管道停顿的发生。

在 CISC 处理器上，解码阶段也可能需要一些时间。x86 和 x86-64 处理器尤其受到影响。因此，近年来这些处理器不在 L1i 中缓存指令的原始字节序列，而是缓存解码后的指令。在这种情况下，L1i 称为“跟踪缓存”。跟踪缓存允许处理器在缓存命中的情况下跳过管道的第一步，这在管道停止时尤其有用。

如前所述，L2 以后的缓存是包含代码和数据的统一缓存。显然，这里的代码以字节序列形式缓存并且不被解码。

为了实现最佳性能，只有一些与指令缓存相关的规则：

1. 生成尽可能小的代码。当为了使用管道而进行软件管道化需要创建更多代码或者使用小代码的开销太高时，存在例外情况。
2. 帮助处理器做出良好的预取决策。这可以通过代码布局或显式预取来完成。

这些规则通常由编译器的代码生成强制执行。程序员可以做一些事情，我们将在第 6 节中讨论它们。

3.4.1 自修改代码

在早期的计算机时代，内存非常宝贵。人们竭尽全力减小程序的大小，为程序数据腾出更多空间。经常使用的一个技巧是随着时间的推移改变程序本身。这种自修改代码 (SMC) 偶尔仍然会被发现，目前主要是出于性能原因或安全漏洞。

一般应避免使用 SMC。尽管它通常可以正确执行，但也有一些边界情况没有正确执行，如果执行不正确，则会产生性能问题。显然，更改的代码不能保留在包含解码指令的跟踪缓存中。但即使由于代码根本没有执行（或已经有一段时间）而没有使用跟踪缓存，处理器也可能会出现问题。如果即将到来的指令在已经进入管道时被更改，则处理器必须丢弃大量工作并重新开始。甚至在某些情况下，处理器的大部分状态都必须被丢弃。

最后，由于处理器假设——为了简单起见，并且因为它在 99.9999999% 的情况下都是正确的——

由于代码页是不可变的，L1i 实现不使用 MESI 协议，而是使用简化的 SI 协议。这意味着如果检测到修改，则必须做出很多悲观的假设。

强烈建议尽可能避免使用 SMC。内存不再是那么稀缺的资源。最好编写单独的函数，而不是根据特定需要修改一个函数。也许有一天 SMC 支持可以成为可选的，我们可以检测到试图以这种方式修改代码的漏洞利用代码。如果绝对必须使用 SMC，则写入操作应绕过缓存，以免对 L1i 所需的 L1d 中的数据造成问题。有关这些说明的更多信息，请参阅第 6.1 节。

在 Linux 上，通常很容易识别包含 SMC 的程序。使用常规工具链构建时，所有程序代码都受到写保护。程序员必须在链接时执行重要的魔法来创建代码页可写的可执行文件。发生这种情况时，现代 Intel x86 和 x86-64 处理器具有专用性能计数器，用于计算自修改代码的使用情况。在这些计数器的帮助下，即使程序由于权限宽松而成功，也可以很容易地识别 SMC 程序。

3.5 缓存缺失因素

我们已经看到，当内存访问错过缓存时，成本就会飙升。有时这是无法避免的，了解实际成本以及可以采取哪些措施来缓解问题非常重要。

3.5.1 缓存和内存带宽

为了更好地了解处理器的功能，我们测量了最佳环境下的可用带宽。这种测量特别有趣，因为不同的处理器版本差异很大。这就是为什么本节充满了几台不同机器的数据。测量性能的程序使用 x86 和 x86-64 处理器的 SSE 指令一次加载或存储 16 个字节。正如我们的其他测试一样，工作集从 1kB 增加到 512MB，并测量每个周期可以加载或存储多少字节。

图 3.24 显示了 64 位 Intel Netburst 处理器上的性能。对于适合 L1d 的工作集大小，处理器能够在每个周期读取完整的 16 个字节，即每个周期执行一个加载指令（movaps 指令一次移动 16 个字节）。该测试不对读取的数据执行任何操作，我们仅测试读取指令本身。一旦 L1d 不再足够，性能就会急剧下降到每个周期不到 6 个字节。2¹⁸ 字节的步骤是由于 DTLB 缓存耗尽，这意味着每个新页面需要额外的工作。自从读书以来

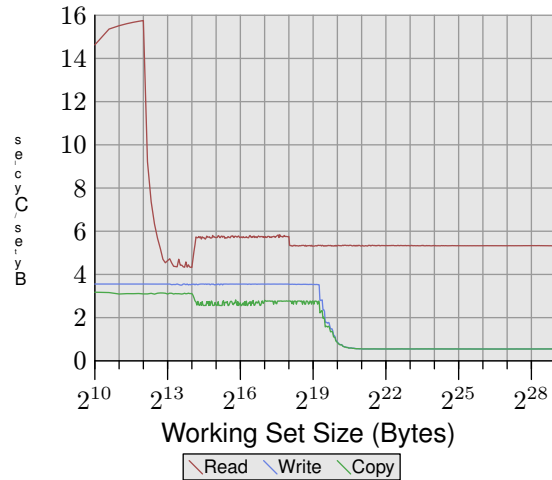


图 3.24: Pentium 4 带宽

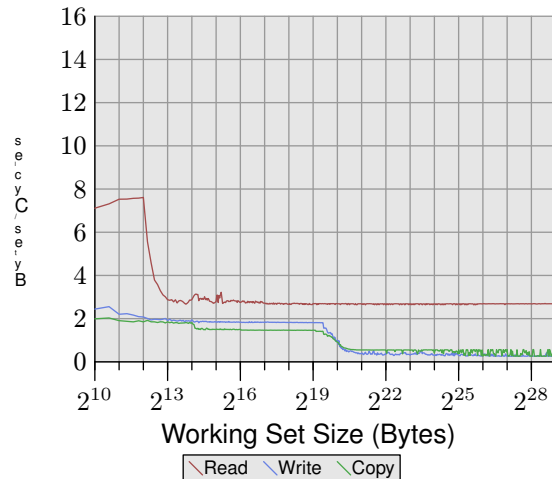


图 3.25: 具有 2 个超线程的 P4 带宽

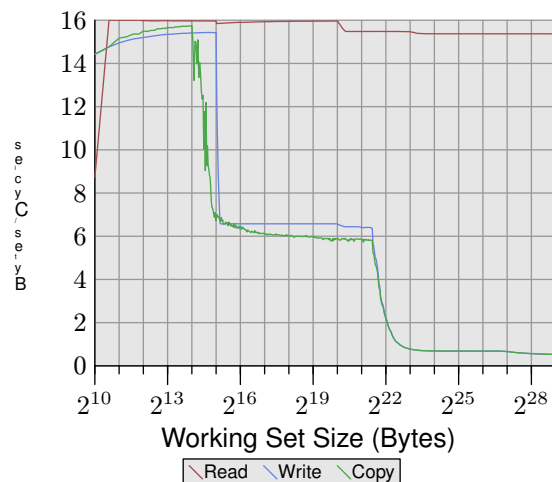


图 3.26: 核心 2 带宽

顺序预取可以完美地预测访问，并且对于所有大小的工作集，FSB 可以以每周期约 5.3 字节的速度传输内存内容。不过，预取的数据不会传播到 L1d 中。当然，这些数字在实际程序中永远无法实现。将它们视为实际限制。

比读取性能更惊人的是写入和复制性能。即使对于较小的工作集大小，写入性能也不会超过每个周期 4 个字节。这表明，在这些 Netburst 处理器中，Intel 选择对 L1d 使用 Write-Through 模式，其性能明显受到 L2 速度的限制。这也意味着复制测试（从一个内存区域复制到第二个不重叠的内存区域）的性能并没有明显变差。必要的读取操作要快得多，并且可能与写入操作部分重叠。写入和复制测量中最值得注意的细节是，一旦 L2 缓存不再足够，性能就会降低。性能下降到每个周期 0.5 个字节！这意味着写入操作比读取操作慢十倍。这意味着优化这些操作对于程序的性能更为重要。

在图 3.25 中，我们看到了同一处理器上的结果，但运行了两个线程，每个线程固定到处理器的两个超线程中。该图以与前一张图相同的比例显示，以说明差异，并且曲线有点不稳定，仅仅是因为测量两个并发线程的问题。结果符合预期。由于超线程共享除寄存器之外的所有资源，因此每个线程只有一半的可用缓存和带宽。这意味着即使每个线程必须等待很长时间并且可以奖励另一个线程执行时间，但这并没有任何区别，因为另一个线程也必须等待内存。这确实展示了超线程最糟糕的使用方式。

与图 3.24 和 3.25 相比，对于 Intel Core 2 处理器，图 3.26 和 3.27 中的结果看起来有很大不同。这是一个具有共享 L2 的双核处理器，其大小是 P4 机器上的 L2 的四倍。但这只能解释写入和复制性能延迟下降的原因。

还有其他更大的差异。整个工作集范围内的读取性能徘徊在每周期最佳 16 字节左右。 2^{20} 字节后读取性能下降也是由于工作集对于 DTLB 来说太大。实现这些高数字意味着处理器不仅能够预取数据并及时传输数据。这也意味着数据已预取到 L1d 中。

写入和复制性能也有显著差异。处理器没有直写策略；写入的数据存储在 L1d 中，并且仅在必要时才被逐出。这使得写入速度接近

每个周期最佳 16 字节。一旦 L1d 不再足够，性能就会显著下降。与 Netburst 处理器一样，写入性能明显较低。由于读取性能较高，这里的差异甚至更大。事实上，当 L2 不再足够时，速度差异会增加到 20 倍！这并不意味着 Core 2 处理器性能不佳。相反，它们的性能总是优于 Netburst 核心。

在图 3.27 中，测试运行两个线程，Core 2 处理器的两个内核各一个。两个线程访问相同的内存，但不一定完全同步。读取性能的结果与单线程情况没有什么不同。可以看到更多的抖动，这在任何多线程测试用例中都是预料之中的。

有趣的一点是适合 L1d 的工作集大小的写入和复制性能。从图中可以看出，性能与必须从主存储器读取数据相同。两个线程竞争相同的内存位置，并且必须发送缓存行的 RFO 消息。问题在于，即使两个核心共享缓存，这些请求也不会以 L2 缓存的速度处理。一旦 L1d 缓存不够用，修改过的条目就会从每个核心的 L1d 刷新到共享的 L2 中。此时，性能显著提高，因为现在 L1d 未命中由 L2 缓存满足，并且仅当数据尚未刷新时才需要 RFO 消息。这就是为什么我们看到这些大小的工作集速度降低了 50%。渐近行为符合预期：由于两个内核共享相同的 FSB，每个内核获得一半的 FSB 带宽，这意味着对于大型工作集，每个线程的性能大约是单线程情况的一半。

由于即使一个供应商的处理器版本之间也存在显著差异，因此当然也值得关注其他供应商处理器的性能。图 3.28 显示了 AMD 系列 10h Opteron 处理器的性能。该处理器具有 64kB L1d、512kB L2 和 2MB L3。L3 高速缓存在处理器的所有内核之间共享。性能测试结果如图 3.28 所示。

关于这些数字，人们注意到的第一个细节是，如果 L1d 缓存足够，处理器每个周期能够处理两条指令。每个周期的读取性能超过 32 字节，甚至写入性能也很高，每个周期 18.7 字节。不过，读取曲线很快变平，并且每个周期 2.3 个字节，相当低。用于此测试的处理器不会预取任何数据，至少效率不高。

另一方面，写入曲线根据各种缓存的大小来执行。L1d 的全尺寸可实现峰值性能，L2 降至每周期 6 字节，L3 降至每周期 2.8 字节，如果连 L3 都无法容纳所有数据，则最终每周期 0.5 字节。



图 3.27: 2 个线程的核心 2 带宽

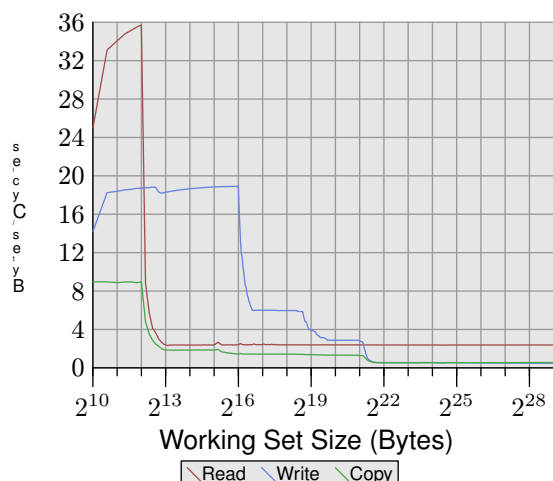


图 3.28: AMD 系列 10h Opteron 带宽

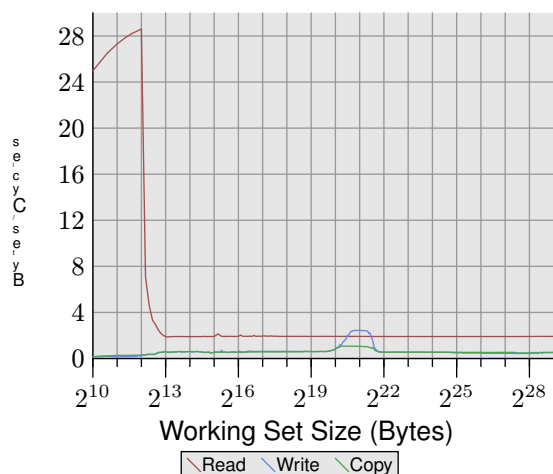


图 3.29: 2 线程的 AMD Fam 10h 带宽

数据。L1d 高速缓存的性能超过了（较旧的）Core 2 处理器，L2 访问速度同样快（Core 2 具有更大的高速缓存），而 L3 和主内存访问速度较慢。

复制性能不能优于读取或写入性能。这就是为什么我们看到曲线最初由读取性能主导，后来由写入性能主导。

Opteron 处理器的多线程性能如图 3.29 所示。读取性能基本上不受影响。每个线程的 L1d 和 L2 都像以前一样工作，并且在这种情况下 L3 缓存也没有很好地预取。这两个线程不会对 L3 施加过大的压力，以达到其目的。本次测试最大的问题是写入性能。线程共享的所有数据都必须经过 L3 缓存。这种共享似乎效率很低，因为即使 L3 缓存大小足以容纳整个工作集，其成本也明显高于 L3 访问。将此图与图 3.27 进行比较，我们可以看到 Core 2 处理器的两个线程以共享 L2 缓存的速度运行，以适应适当的工作集大小范围。Opteron 处理器仅在非常小的工作集大小范围内才能达到这种性能水平，即使在这里它也接近 *only* L3 的速度，而 L3 的速度比 Core 2 的 L2 慢。

3.5.2 关键字负载

内存以小于高速缓存行大小的块从主存储器传输到高速缓存。现在一次传输 64 个 *bits*，缓存行大小为 64 或 128 *bytes*。这意味着每个缓存行需要 8 或 16 次传输。

DRAM 芯片可以以突发模式传输这些 64 字节块。这可以填充高速缓存行，而无需来自内存控制器的任何进一步命令以及可能相关的延迟。如果处理器预取缓存行，这可能是最好的操作方式。

如果程序对数据或指令高速缓存的访问未命中（这意味着这是强制高速缓存未命中，因为数据是第一次使用，或者是容量高速缓存未命中，因为有限的高速缓存大小需要逐出高速缓存行），情况就不同了。程序继续运行所需的高速缓存行内的字可能不是高速缓存行中的第一个字。即使在突发模式和双倍数据速率传输下，各个 64 位块的到达时间也明显不同。每个块比前一个块晚 4 个 CPU 周期或更长时间到达。如果程序需要继续的字是高速缓存行的第八个字，则程序必须在第一个字到达后再等待 30 个周期或更长时间。

事情不一定非得这样。内存控制器可以自由地以不同的顺序请求高速缓存行的字。处理器可以传达程序正在等待哪个字，即 *critical word*,

并且内存控制器可以首先请求这个字。一旦该字到达，程序就可以继续，而缓存行的其余部分到达并且缓存尚未处于一致状态。这种技术称为关键词优先和早期重启。

如今的处理器实现了这种技术，但在某些情况下这是不可能的。如果处理器预取数据，则关键字是未知的。如果处理器在预取操作进行期间请求高速缓存行，则必须等待关键字到达，而不能影响顺序。

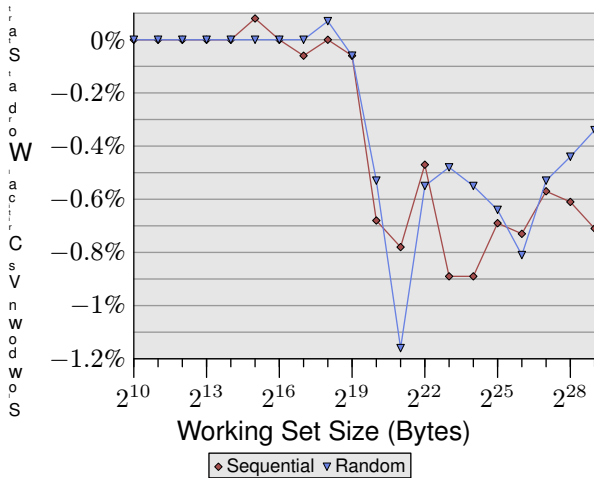


图 3.30: 缓存行末尾的关键字

即使进行了这些优化，关键字在缓存行上的位置也很重要。图 3.30 显示了顺序和随机访问的跟随测试。显示的是在第一个字中使用追踪中使用的指针与指针位于最后一个字中的情况相比运行测试的速度减慢。元素大小为 64 字节，对应于缓存行大小。这些数字相当嘈杂，但可以看出，一旦 L2 不足以容纳工作集大小，关键字位于末尾的情况下的性能就会慢约 0.7%。顺序访问似乎受到的影响更大一些。这与预取下一个缓存行时的上述问题是一致的。

3.5.3 缓存放置

高速缓存与超线程、内核和处理器的关系不受程序员控制。但程序员可以确定线程在哪里执行，然后缓存与所用 CPU 的关系就变得很重要。

这里我们不会详细讨论何时选择哪些核心来运行线程。我们只会描述程序员在设置线程亲和性时必须考虑的架构细节。

根据定义，超线程共享除寄存器集之外的所有内容。这包括 L1 缓存。这里就不多说了。乐趣始于处理器的各个核心。每个核心至少有自己的 L1 缓存。除此之外，今天没有太多共同的细节：

- 早期的多核处理器根本没有共享缓存。
- 后来的 Intel 型号为双核处理器共享二级缓存。对于四核处理器，我们必须为每对两个核心处理单独的二级缓存。没有更高级别的缓存。
- AMD 系列 10h 处理器具有独立的 L2 缓存和统一的 L3 缓存。

处理器供应商的宣传材料中已经写了很多关于各自型号的优势的文章。如果内核处理的工作集不重叠，则不使用共享缓存具有优势。这对于单线程程序来说效果很好。由于这在今天仍然是现实，所以这种方法的效果并不算太差。但总有一些重叠。缓存都包含公共运行时库中最常用的部分，这意味着浪费了一些缓存空间。

像英特尔双核处理器那样完全共享 L1 之外的所有缓存可以带来很大的优势。如果两个内核上工作的线程的工作集显着重叠，则可用缓存总量会增加，并且工作集可以更大，而不会降低性能。如果工作集不重叠，英特尔的高级智能缓存管理应该可以防止任何一个核心独占整个缓存。

不过，如果两个核心都将大约一半的缓存用于各自的工作集，则会出现一些摩擦。缓存必须不断权衡两个核心的缓存使用情况，并且作为此重新平衡的一部分执行的驱逐可能会选择不当。为了找出问题所在，我们查看另一个测试程序的结果。

测试程序有一个进程使用 SSE 指令不断读取或写入 2MB 内存块。选择 2MB 是因为这是该 Core 2 处理器 L2 缓存大小的一半。该进程固定到一个核心，而第二个进程则固定到另一个核心。第二个进程读取和写入可变大小的内存区域。该图显示每个周期读取或写入的字节数。显示了四个不同的图表，一个对应于读取和写入过程的每种组合。读/写图针对后台进程，该进程始终使用 2MB 工作集进行写入，而测量的进程则使用可变工作集进行读取。

该图有趣的部分是 2²⁰ 和 2²³ 字节之间的部分。如果两个核心的 L2 缓存完全独立，我们可以预期

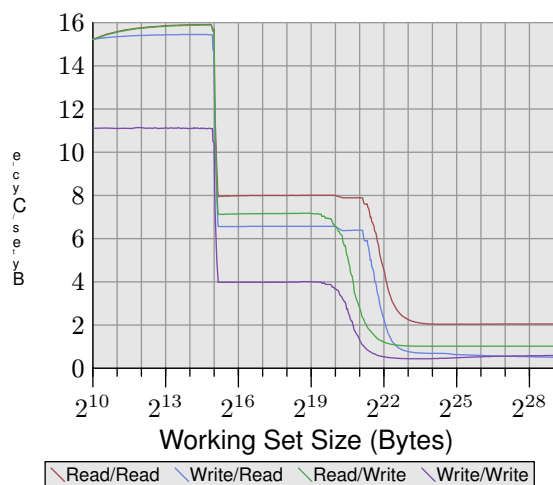


图 3.31: 两个进程的带宽

所有四个测试都会下降 2^{21} 和 2^{22} 字节之间，这意味着一旦 L2 缓存耗尽。正如我们在图 3.31 中看到的那样，情况并非如此。对于后台进程正在写入的情况，这是最明显的。在工作集大小达到 1MB 之前，性能开始恶化。这两个进程不共享内存，因此这些进程不会导致生成 RFO 消息。这些是纯粹的缓存驱逐问题。智能缓存处理存在问题，即每个核心的经验缓存大小接近 1MB，而不是可用的每个核心 2MB。人们只能希望，如果内核之间共享的缓存仍然是即将推出的处理器的一项功能，那么用于智能缓存处理的算法将得到修复。

在引入更高级别的缓存之前，拥有带有两个二级缓存的四核处理器只是一个权宜之计。与单独的插槽和双核处理器相比，这种设计没有提供显著的性能优势。两个内核通过同一总线进行通信，该总线在外部可见为 FSB。没有特殊的快速数据交换。

多核处理器缓存设计的未来将在于更多层。AMD 的 10h 处理器系列率先亮相。我们是否会继续看到较低级别的缓存被处理器的一部分核心共享还有待观察（在 2008 代处理器中，L2 缓存不被共享）。由于高速且频繁使用的缓存无法在多个核心之间共享，因此额外级别的缓存是必要的。性能将会受到影响。它还需要具有高关联性的非常大的缓存。缓存大小和关联性这两个数字都必须随着共享缓存的核心数量而变化。使用大型 L3 缓存和合理大小的 L2 缓存是一个合理的权衡。L3 缓存速度较慢，但理想情况下它不像 L2 缓存那样频繁使用。

对于程序员来说，所有这些不同的设计意味着做出调度决策时的复杂性。一个人必须

了解机器架构的工作负载和详细信息，以实现最佳性能。幸运的是，我们有确定机器架构的支持。这些接口将在后面的章节中介绍。

3.5.4 FSB 影响

FSB 对机器的性能起着核心作用。缓存内容只能在内存连接允许的情况下快速存储和加载。我们可以通过在两台仅内存模块速度不同的机器上运行一个程序来显示这一点。图 3.32 显示了 64 位机器上 NP AD=7 的 Addnext0 测试（将下一个元素 pad[0] 元素的内容添加到自己的 pad[0] 元素中）的结果。两台机器均配备 Intel Core 2 处理器，第一台使用 667MHz DDR2 模块，第二台使用 800MHz 模块（增加了 20%）。

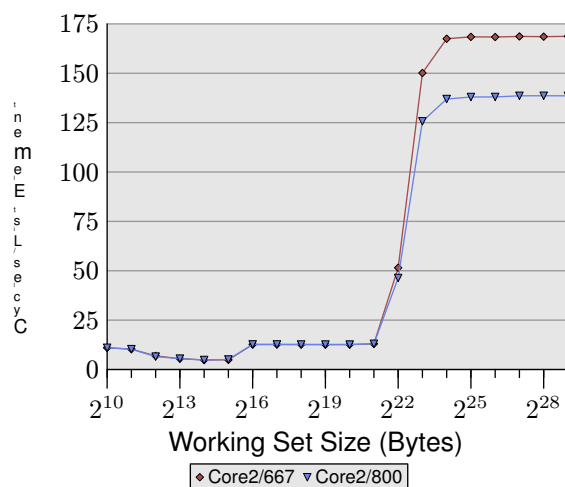


图 3.32: FSB 速度的影响

数字表明，当 FSB 确实因大型工作集大小而受到压力时，我们确实看到了巨大的好处。本次测试测得的最大性能提升为 18.2%，接近理论最大值。这表明更快的 FSB 确实可以带来丰厚的回报。工作集何时适合缓存并不重要（并且这些处理器具有 4MB L2）。必须记住，我们在这里测量的是一个程序。系统的工作集包括所有同时运行的进程所需的内存。这样，较小的程序就可以轻松超过 4MB 内存或更多。

如今，一些英特尔处理器支持高达 1,333MHz 的 FSB 速度，这意味着又提高了 60%。未来将会看到更高的速度。如果速度很重要并且工作集大小较大，那么快速 RAM 和高 FSB 速度当然物有所值。不过，我们必须小心，因为即使处理器可能支持更高的 FSB 速度，主板/北桥也可能不支持。检查规格至关重要。

4 虚拟内存

处理器的虚拟内存（VM）子系统实现提供给每个进程的虚拟地址空间。这使得每个进程都认为它在系统中是单独的。虚拟内存的优点列表已在其他地方详细描述，因此这里不再重复。相反，本节重点关注虚拟内存子系统的实际实现细节以及相关成本。

虚拟地址空间由CPU 的内存管理单元(MMU) 实现。操作系统必须填写页表数据结构，但大多数 CPU 会自行完成其余工作。这实际上是一个相当复杂的机制；理解它的最好方法是介绍用于描述虚拟地址空间的数据结构。

MMU 执行的地址转换的输入是虚拟地址。其价值通常很少（如果有的话）限制。虚拟地址在 32 位系统上是 32 位值，在 64 位系统上是 64 位值。在某些系统上，例如 x86 和 x86-64，所使用的地址实际上涉及另一级间接：这些体系结构使用的段只会导致将偏移量添加到每个逻辑地址。我们可以忽略地址生成的这一部分，它是微不足道的，程序员不必关心内存处理性能。²⁴

4.1 最简单的地址转换

有趣的部分是将虚拟地址转换为物理地址。MMU 可以逐页重新映射地址。就像寻址高速缓存行一样，虚拟地址被分为不同的部分。这些部分用于索引各种表，这些表用于构建最终的物理地址。对于最简单的模型，我们只有一层表。

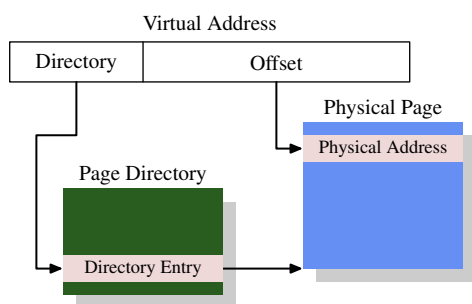


图 4.1: 1 级地址转换

图 4.1 显示了虚拟地址的不同部分是如何使用的。顶部部分用于选择页面目录中的条目；该目录中的每个条目都可以

由操作系统单独设置。页目录项决定物理内存页的地址；页目录中的多个条目可以指向同一物理地址。存储单元的完整物理地址是通过将页目录中的页地址与虚拟地址中的低位相结合来确定的。页目录项还包含一些有关该页的附加信息，例如访问权限。

页目录的数据结构存储在主存储器中。操作系统必须分配连续的物理内存，并将该内存区域的基地址存储在一个特殊的寄存器中。然后，虚拟地址的适当位被用作页目录的索引，页目录实际上是目录条目的数组。

举一个具体的例子，这是 x86 机器上 4MB 页面使用的布局。虚拟地址的偏移部分大小为 22 位，足以寻址 4MB 页中的每个字节。虚拟地址的其余 10 位选择页目录中 1024 个条目之一。每个条目包含 4MB 页的 10 位基地址，与偏移量组合形成完整的 32 位地址。

4.2 多级页表

4MB 页面不是标准，它们会浪费大量内存，因为操作系统必须执行的许多操作都需要与内存页面对齐。对于 4kB 页（32 位机器上的标准，并且仍然经常在 64 位机器上），虚拟地址的偏移量部分的大小仅为 12 位。这留下 20 位作为页目录的选择器。具有 2^{20} 条目的表是不切实际的。即使每个条目只有 4 个字节，表的大小也将是 4MB。由于每个进程都可能拥有自己不同的页目录，系统的大部分物理内存将与这些页目录相关联。

解决方案是使用多级页表。然后该级别形成一个巨大的、稀疏的页目录；实际未使用的地址空间区域不需要分配内存。因此，这种表示形式更加紧凑，使得内存中的许多进程的页表成为可能，而不会过多影响性能。

如今，最复杂的页表结构包含四个级别。图 4.2 显示了此类实现的原理图。在本例中，虚拟地址至少分为五个部分。其中四个部分是各个目录的索引。4 级目录是使用 CPU 中的专用寄存器来引用的。4 级到 2 级目录的内容是对下一级目录的引用。如果一个目录条目被标记为空，那么它显然不需要指向任何较低的目录。这样页表树就可以稀疏且紧凑。1 级目录的条目如图 4.1 所示，是部分物理地址，加上访问权限等辅助数据。

²⁴Segment limits on x86 are performance-relevant but that is another story.

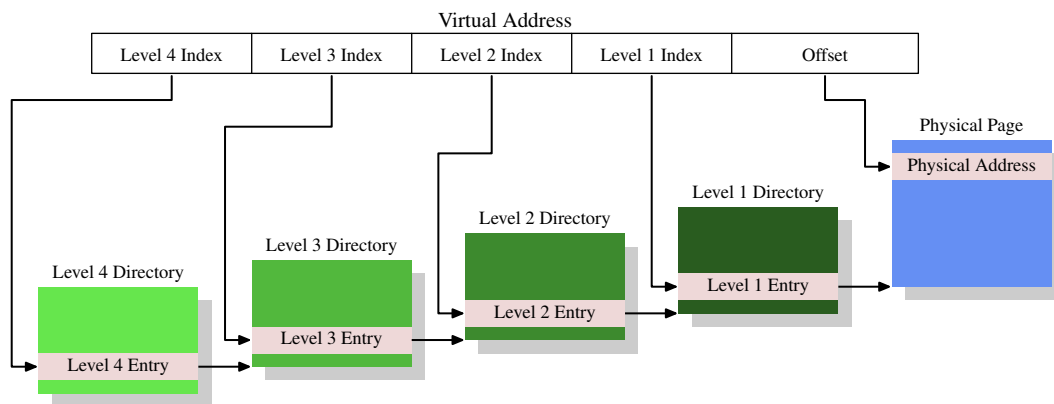


图 4.2：4 级地址转换

为了确定与虚拟地址相对应的物理地址，处理器首先确定最高级别目录的地址。该地址通常存储在寄存器中。然后，CPU 获取与该目录对应的虚拟地址的索引部分，并使用该索引来选择适当的条目。该条目是下一个目录的地址，使用虚拟地址的下部分进行索引。这个过程一直持续到到达一级目录，此时目录项的值就是物理地址的高位部分。物理地址是通过添加虚拟地址中的页偏移位来完成的。这个过程称为页面树遍历。一些处理器（如 x86 和 x86-64）在硬件中执行此操作，其他处理器则需要操作系统的帮助。

系统上运行的每个进程可能需要自己的页表树。部分共享树是可能的，但这是例外。因此，如果页表树所需的内存尽可能小，则有利于性能和可扩展性。理想的情况是将使用的内存紧密地放置在虚拟地址空间中；使用的实际物理地址并不重要。一个小程序可能只需要使用 2、3 和 4 级目录中的一个目录以及一些 1 级目录即可。在 x86-64 上，每个目录有 4kB 页面和 512 个条目，这允许对总共 4 个目录（每个级别一个）进行 2MB 寻址。2 至 4 级可使用 1 个目录寻址，1 级可使用 512 个目录寻址 1GB 连续内存。

不过，假设所有内存都可以连续分配就太简单了。出于灵活性原因，在大多数情况下，进程的堆栈和堆区域被分配在地址空间的几乎相反的两端。如果需要的话，这允许任一区域尽可能地增长。这意味着很可能需要两个 2 级目录以及相应的更多较低级目录。

但即便如此，也并不总是符合当前的做法。出于安全原因，可执行文件的各个部分（代码、数据、堆、堆栈、动态共享对象 (DSO)，又名

共享库）被映射到随机地址[9]。随机化延伸到各个部分的相对位置；这意味着进程中使用的各种内存区域广泛分布在整个虚拟地址空间中。通过对随机地址的位数应用一些限制，可以限制范围，但在大多数情况下，肯定不允许进程仅使用一两个目录运行 2 级和 3 级。

如果性能确实比安全性重要得多，则可以关闭随机化。然后，操作系统通常至少会在虚拟内存中连续加载所有 DSO。

4.3 优化页表访问

页表的所有数据结构都保存在主存中；这是操作系统构建和更新表的地方。创建进程或更改页表时，CPU 会收到通知。页表用于使用上述页表遍历将每个虚拟地址解析为物理地址。更重要的是：在解析虚拟地址的过程中，每一级至少使用一个目录。这需要多达四次内存访问（对于正在运行的进程的单次访问），速度很慢。可以将这些目录表条目视为普通数据并将它们缓存在 L1d、L2 等中，但这仍然太慢。

从虚拟内存诞生之初，CPU 设计人员就使用了不同的优化方法。一个简单的计算可以表明，仅将目录表条目保留在 L1d 和更高级别的缓存中会导致糟糕的性能。每个绝对地址计算都需要与页表深度相对应的多次 L1d 访问。这些访问无法并行，因为它们依赖于先前查找的结果。在具有四个页表级别的机器上，仅此一项就至少需要 12 个周期。再加上 L1d 未命中的概率，结果是指令管道无法隐藏的。额外的 L1d 访问还会窃取缓存的宝贵带宽。

因此，不只是缓存目录表条目，而是缓存物理页地址的完整计算。出于与代码和数据缓存工作相同的原因，这种缓存地址计算是有效的。由于虚拟地址的页偏移部分在物理页地址的计算中不发挥任何作用，因此仅将虚拟地址的其余部分用作高速缓存的标记。根据页面大小，这意味着数百或数千条指令或数据对象共享相同的标签，因此共享相同的物理地址前缀。

存储计算值的缓存称为转换后备缓冲区 (TLB)。它通常是一个小缓存，因为它必须非常快。现代CPU 提供多级TLB缓存，就像其他缓存一样；更高级别的缓存更大且速度更慢。L1TLB 的小尺寸通常可以通过使用 LRU 驱逐策略使缓存完全关联来弥补。最近，该缓存的大小一直在增长，并在此过程中更改为设置关联。因此，每当必须添加新条目时，它可能不是被逐出并替换的最旧条目。

如上所述，用于访问TLB的标签是虚拟地址的一部分。如果标签在缓存中匹配，则通过将虚拟地址的页面偏移量与缓存值相加来计算最终的物理地址。这是一个非常快的过程；它必须如此，因为物理地址必须可用于每条使用绝对地址的指令，并且在某些情况下，可用于使用物理地址作为密钥的 L2 查找。如果 TLB 查找失败，处理器必须执行页表遍历；这可能是相当昂贵的。

如果地址位于另一页上，则通过软件或硬件预取代码或数据可能会隐式预取 TLB 的条目。硬件预取不允许这样做，因为硬件可能会启动无效的页表遍历。因此，程序员不能依赖硬件预取来预取 TLB 条目。它必须使用预取指令显式地完成。TLB 与数据和指令缓存一样，可以出现在多个级别。正如数据缓存一样，TLB 通常以两种形式出现：指令 TLB (ITLB) 和数据 TLB (DTLB)。与其他缓存一样，较高级别的 TLB (例如 L2TLB) 通常是统一的。

4.3.1 使用 TLB 的注意事项

TLB 是处理器核心全局资源。在处理器核心上执行的所有线程和进程都使用相同的 TLB。由于虚拟地址到物理地址的转换取决于安装的页表树，因此如果页表发生更改，CPU 不能盲目地重用缓存的条目。每个进程都有一个不同的页表树（但不是同一进程中的线程），内核和 VMM（管理程序）（如果存在）也是如此。这是

进程的地址空间布局也可能发生变化。有两种方法可以处理这个问题：

- 每当页表树发生更改时，TLB 就会被刷新。
- TLB 条目的标记被扩展为额外且唯一地标识它们引用的页表树。

在第一种情况下，每当执行上下文切换时，TLB 就会被刷新。由于在大多数操作系统中，从一个线程/进程切换到另一个线程/进程需要执行一些内核代码，因此 TLB 刷新仅限于离开（有时进入）内核地址空间。在虚拟化系统上，当内核必须调用 VMM 以及返回时也会发生这种情况。如果内核和/或 VMM 不必使用虚拟地址，或者可以重用与进行系统/VM 调用的进程或内核相同的虚拟地址（即，地址空间重叠），则仅当处理器在离开内核或 VMM 时恢复执行不同的进程或内核时，才需要刷新 TLB。

刷新 TLB 有效但昂贵。例如，当执行系统调用时，内核代码可能会被限制为几千条指令，这些指令可能会涉及少数新页面（或一个大页面，就像某些体系结构上的 Linux 的情况一样）。这项工作将仅替换与所触及的页面一样多的 TLB 条目。对于具有 128 个 ITLB 和 256 个 DTLB 条目的 Intel Core2 架构，完全刷新将意味着（分别）超过 100 个和 200 个条目将被不必要地刷新。当系统调用返回到同一进程时，所有那些刷新的 TLB 条目都可以再次使用，但它们将会消失。对于内核或 VMM 中常用的代码也是如此。尽管内核和 VMM 的页表通常不会更改，但每次进入内核时，TLB 都必须从头开始填充，因此，理论上，TLB 条目可以保留很长时间。这也解释了为什么当今处理器中的 TLB 缓存并不大：程序很可能不会运行足够长的时间来填充所有这些条目。

当然，CPU 架构师也逃不过这一事实。优化缓存刷新的一种可能性是单独使 TLB 条目无效。例如，如果内核代码和数据落入特定的地址范围，则只有落入该地址范围的页面必须从 TLB 中逐出。这只需要比较标签，因此不是很昂贵。如果部分地址空间发生更改（例如通过调用 `munmap`），此方法也很有用。

更好的解决方案是扩展用于 TLB 访问的标记。如果除了虚拟地址部分之外，还为每个页表树（即进程的地址空间）添加一个唯一标识符，则 TLB 根本不必完全刷新。内核、VMM 和

各个进程都可以有唯一的标识符。该方案的唯一问题是可用于 TLB 标记的位数受到严格限制，而地址空间的数量却不受限制。这意味着一些标识符的重用是必要的。当这种情况发生时，TLB 必须被部分刷新（如果可能的话）。所有具有重用标识符的条目都必须被刷新，但希望这是一个小得多的集合。

当系统上运行多个进程时，这种扩展的 TLB 标记在虚拟化领域之外具有优势。如果每个可运行进程的内存使用（以及 TLB 条目使用）受到限制，则当再次调度该进程时，该进程最近使用的 TLB 条目很可能仍在 TLB 中。但还有两个额外的优点：

1. 特殊地址空间，例如内核和VMM使用的地址空间，通常只进入很短的时间；之后控制通常返回到发起该条目的地址空间。如果没有标签，则会执行一到两次 TLB 刷新。使用标记，调用地址空间的缓存翻译将被保留，并且由于内核和 VMM 地址空间根本不经常更改 TLB 条目，因此仍然可以使用先前系统调用等的翻译。

2. 当在同一进程的两个线程之间切换时，根本不需要 TLB 刷新。不过，如果没有扩展 TLB 标记，进入内核就会破坏第一个线程的 TLB 条目。

一段时间以来，一些处理器已经实现了这些扩展标签。AMD 通过 Pacifica 虚拟化扩展引入了 1 位标签扩展。在虚拟化环境中，这个 1 位地址空间 ID (ASID) 用于区分 VMM 的地址空间和来宾域的地址空间。这使得操作系统可以避免每次进入 VMM 时刷新客户的 TLB 条目（例如，处理页面错误），或者在控制权返回客户时刷新 VMM 的 TLB 条目。该架构将来将允许使用更多位。其他主流处理器可能会效仿并支持此功能。

4.3.2 影响TLB性能

有几个因素会影响 TLB 的性能。首先是页面的大小。显然，页面越大，其中容纳的指令或数据对象就越多。因此，较大的页面大小会减少所需的地址转换总数，这意味着 TLB 缓存中需要的条目较少。现在大多数架构都允许使用多种不同的页面大小。某些尺寸可以同时使用。例如，x86/x86-64 处理器的正常页面大小为 4kB，但它们也可以分别使用 4MB 和 2MB 页面。IA-64 和 PowerPC 允许 64kB 等大小作为基本页面大小。

不过，使用大页面会带来一些问题。用于大页面的内存区域在物理内存中必须是连续的。如果将物理内存管理的单位大小提高到虚拟内存页的大小，则浪费的内存量将会增加。所有类型的内存操作（例如加载可执行文件）都需要与页边界对齐。这意味着，平均而言，每个映射都会浪费物理内存中一半的页面大小。这种浪费很容易累积起来；因此，它对物理内存分配的合理单元大小设置了上限。

将单元大小增加到 2MB 来容纳 x86-64 上的大页面当然是不切实际的。这个尺寸太大了。但这反过来又意味着每个大页面必须由许多较小的页面组成。这些小页面在 *physical* 内存中必须是连续的。以 4kB 的单位页大小分配 2MB 的连续物理内存可能具有挑战性。它需要找到一个有 512 个连续页面的空闲区域。在系统运行一段时间并且物理内存变得碎片化后，这可能会非常困难（或不可能）。

因此，在 Linux 上，有必要在系统启动时使用特殊的 Hugetlbfs 文件系统来分配这些大页面。保留固定数量的物理页作为大虚拟页专用。这限制了可能并不总是被使用的资源。这也是一个有限的池；增加它通常意味着重新启动系统。尽管如此，在性能很重要、资源充足且繁琐的设置不构成太大阻碍的情况下，大页面仍然是一种选择。数据库服务器就是一个例子。

增加最小虚拟页面大小（相对于可选的大页面）也有其问题。内存映射操作（例如加载应用程序）必须符合这些页面大小。不可能有更小的映射。对于大多数体系结构来说，可执行文件的各个部分的位置具有固定的关系。如果页面大小增加到超出了构建可执行文件或 DSO 时考虑的范围，则无法执行加载操作。记住这一限制很重要。图 4.3 显示了如何确定 ELF 二进制文件的对齐要求。它被编码在 ELF 程序头中。在此示例中，对于 x86-64 二进制文件，该值为 $200000_{16} = 2,097,152 = 2\text{MB}$ ，对应于处理器支持的最大页面大小。

使用较大页面大小还有第二个效果：页表树的级数减少。由于与页偏移相对应的虚拟地址部分增加，因此需要通过页目录处理的位已经不多了。这意味着，在 TLB 未命中的情况下，必须完成的工作量会减少。

除了使用大页面大小之外，还可以减少

```
$ eu-readelf -l /bin/ls
Program Headers:
  Type   Offset      VirtAddr           PhysAddr          FileSiz  MemSiz   Flg Align
...
LOAD    0x000000    0x0000000000400000 0x0000000000400000 0x0132ac 0x0132ac R E 0x200000
LOAD    0x0132b0    0x00000000006132b0 0x00000000006132b0 0x001a71 0x001a71 RW 0x200000
...
```

图 4.3: 指示对齐要求的 ELF 程序头

将同时使用的数据移动到更少的页所需的 TLB 条目数。这类似于我们上面讨论的缓存使用的一些优化。只是现在所需的对齐量很大。鉴于 TLB 条目的数量非常小，这可能是一个重要的优化。

4.4 虚拟化的影响

操作系统镜像的虚拟化将变得越来越普遍；这意味着在图片中添加了另一层内存处理。进程（基本上是监狱）或操作系统容器的虚拟化不属于这一类，因为只涉及一个操作系统。Xen 或 KVM 等技术可以在有或没有处理器帮助的情况下实现独立操作系统映像的执行。在这些情况下，只有一个软件可以直接控制对物理内存的访问。

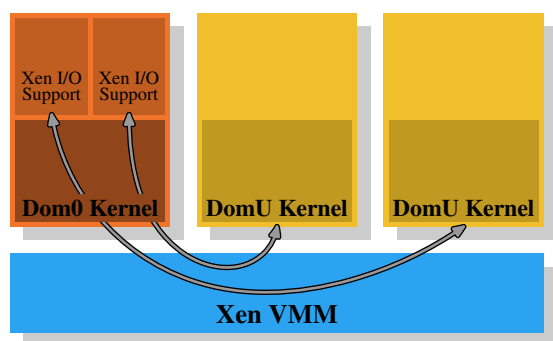


图 4.4: Xen 虚拟化模型

对于 Xen（参见图 4.4），Xen VMM 就是这样一个软件。不过，VMM 本身并不实现许多其他硬件控制。与其他早期系统（以及 Xen VMM 的第一个版本）上的 VMM 不同，内存和处理器之外的硬件由特权 Dom0 域控制。目前，这与非特权 DomU 内核基本上是相同的内核，并且就内存处理而言，它们没有区别。这里重要的是，VMM 将物理内存分发给 Dom0 和 DomU 内核，然后它们本身会实现通常的内存处理，就好像它们直接在处理器上运行一样。

为了实现完成虚拟化所需的域分离，内存

Dom0 和 DomU 内核中的物理处理确实 *not* 可以不受限制地访问物理内存。VMM 不会通过分配单独的物理页并让客户操作系统处理寻址来分配内存；而是通过分配单独的物理页来分配内存。这不会提供任何针对错误或恶意来宾域的保护。相反，VMM 为每个来宾域创建自己的页表树，并使用这些数据结构分配内存。好处是可以控制对页表树管理信息的访问。如果代码没有适当的权限，它就无法执行任何操作。

无论是否使用半虚拟化或硬件（也称为全）虚拟化，Xen 提供的虚拟化都会利用此访问控制。来宾域为每个进程构建页表树的方式有意与半虚拟化和硬件虚拟化非常相似。每当客户操作系统修改其页表时，VMM 就会被调用。然后，VMM 使用来宾域中的更新信息来更新其自己的影子页表。这些是硬件实际使用的页表。显然，这个过程的开销相当大：页表树的每次修改都需要调用 VMM。虽然如果没有虚拟化，内存映射的更改并不便宜，但现在它们变得更加昂贵。

考虑到从客户操作系统到 VMM 及其本身的更改已经相当昂贵，额外的成本可能非常大。这就是为什么处理器开始具有额外的功能来避免创建影子页表。这不仅是因为速度问题，而且还减少了 VMM 的内存消耗。Intel 有扩展页表 (EPT)，AMD 将其称为嵌套页表 (NPT)。基本上，这两种技术都有客户操作系统的页表从“客户虚拟地址”生成“主机虚拟地址”。然后，必须使用每个域的 EPT/NPT 树将主机虚拟地址进一步转换为实际物理地址。这将允许以几乎与非虚拟化情况相同的速度进行内存处理，因为用于内存处理的大多数 VMM 条目都已被删除。它还减少了 VMM 的内存使用，因为现在每个域（而不是进程）只需维护一棵页表树。

附加地址转换步骤的结果也存储在 TLB 中。这意味着 TLB 不存储虚拟物理地址，而是存储查找的完整结果。已经解释过

AMD 的 Pacifica 扩展引入了 ASID 以避免每个条目上的 TLB 刷新。在处理器扩展的初始版本中，ASID 的位数为 1；这足以区分 VMM 和客户操作系统。Intel 拥有虚拟处理器 ID (VPID)，其用途相同，只是数量更多。但是每个来宾域的 VPID 都是固定的，因此它也不能用于标记单独的进程并避免该级别的 TLB 刷新。

每次地址空间修改所需的工作量是虚拟化操作系统的一个问题。然而，基于 VMM 的虚拟化还存在另一个固有的问题：没有办法绕过两层内存处理。但内存处理很困难（特别是在考虑像 NUMA 这样的复杂性时，请参阅第 5 节）。使用单独的 VMM 的 Xen 方法使得最佳（甚至良好）处理变得困难，因为内存管理实现的所有复杂性，包括发现内存区域等“琐碎”的事情，都必须在 VMM 中重复。操作系统具有成熟且优化的实现；人们确实想避免重复它们。

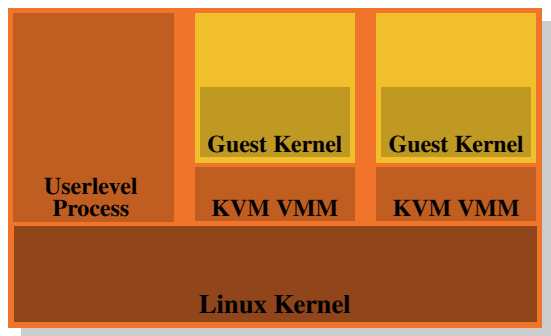


图 4.5: KVM 虚拟化模型

这就是为什么采用 VMM/Dom0 模型得出结论是一个如此有吸引力的选择。图 4.5 显示了 KVM Linux 内核扩展如何尝试解决该问题。没有单独的 VMM 直接在硬件上运行并控制所有来宾；相反，普通的 Linux 内核会接管此功能。这意味着 Linux 内核中完整且复杂的内存处理功能用于管理系统的内存。来宾域与普通用户级进程一起运行，创建者称之为“来宾模式”。虚拟化功能（半虚拟化或全虚拟化）由 KVM VMM 控制。这只是另一个用户级进程，它恰好使用内核实现的特殊 KVM 设备来控制来宾域。

与 Xen 模型的单独 VMM 相比，该模型的优点在于，即使在使用来宾操作系统时仍然有两个内存处理程序在工作，但只需要一个实现，即 Linux 内核中的实现。没有必要在 Xen VMM 等另一段代码中复制相同的功能。这

可以减少工作量、错误，并且可能减少两个内存处理程序接触时的摩擦，因为 Linux 客户机中的内存处理程序与在裸硬件上运行的外部 Linux 内核中的内存处理程序做出相同的假设。

总的来说，程序员必须意识到，使用虚拟化时，缓存未命中（指令、数据或 TLB）的成本甚至比不使用虚拟化时更高。任何减少这项工作的优化都会在虚拟化环境中获得更大的回报。随着时间的推移，处理器设计人员将通过 EPT 和 NPT 等技术越来越减少这种差异，但这种差异永远不会完全消失。

5 NUMA 支持

在第 2 节中，我们看到，在某些机器上，访问物理内存特定区域的成本根据访问来源的不同而有所不同。此类硬件需要操作系统和应用程序特别小心。我们将从 NUMA 硬件的一些细节开始，然后介绍 Linux 内核为 NUMA 提供的一些支持。

5.1 NUMA 硬件

非统一内存架构变得越来越普遍。在最简单的 NUMA 形式中，处理器可以拥有本地内存（见图 2.3），它的访问成本比其他处理器的本地内存要便宜。此类 NUMA 系统的成本差异并不高，即 NUMA 系数较低。

NUMA 还特别用于大型机器。我们已经描述了让多个处理器访问同一内存的问题。对于商用硬件，所有处理器将共享相同的北桥（暂时忽略 AMD Opteron NUMA 节点，它们有自己的问题）。这使得北桥成为严重的瓶颈，因为 *all* 内存流量是通过它路由的。当然，大型机器可以使用定制硬件来代替北桥，但是，除非使用的内存芯片具有多个端口，即它们可以从多条总线上使用——仍然存在瓶颈。多端口 RAM 的构建和支持非常复杂且昂贵，因此很少使用。

复杂性的下一步是 AMD 使用的模型，其中互连机制（AMD 的超级传输，是他们从 Digital 获得许可的技术）为未直接连接到 RAM 的处理器提供访问。除非想要任意增加直径（即任意两个节点之间的最大距离），否则以这种方式形成的结构的尺寸是有限的。

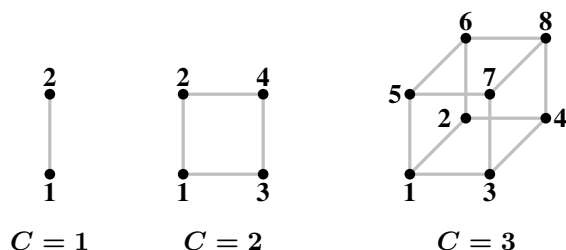


图 5.1: 超立方体

用于连接节点的有效拓扑是超立方体，它将节点数量限制为 2^C ，其中 C 是每个节点具有的互连接口的数量。对于所有具有 2^n CPU 和 n 互连的系统来说，超立方体具有最小的直径。图 5.1 显示了前三个超立方体。每个超立方体都有一个直径

C 的绝对最小值。AMD 的第一代 Opteron 处理器的每个处理器具有三个超传输链路。至少其中一个处理器必须有一个连接到一个链路的南桥，这意味着目前可以直接有效地实现具有 $C = 2$ 的超立方体。下一代将在某个时刻有四个链接，此时 $C = 3$ 个超立方体将成为可能。

但这并不意味着不能支持更大的处理器积累。有些公司开发了允许使用更大组处理器的交叉开关（例如 Newisys 的 Horus）。但这些交叉开关增加了 NUMA 系数，并且在一定数量的处理器上它们不再有效。

下一步意味着连接 CPU 组并为所有 CPU 实现共享内存。所有此类系统都需要专门的硬件，并且绝不是商品系统。此类设计存在多个复杂程度。IBM x445 和类似的机器仍然非常接近商用机器。它们可以作为带有 x86 和 x86-64 处理器的普通 4U、8 路机器购买。然后可以将其中两台（有时最多四台）连接起来，作为具有共享内存的单台机器工作。所使用的互连引入了操作系统和应用程序必须考虑的重要 NUMA 因素。

另一方面，像 SGI 的 Altix 这样的机器是专门为互连而设计的。SGI 的 NUMalink 互连结构非常快，同时具有低延迟；这两个属性都是高性能计算（HPC）的要求，特别是在使用消息传递接口（MPI）时。当然，缺点是这种复杂性和专业化的成本非常昂贵。它们使相当低的 NUMA 系数成为可能，但由于这些机器可以拥有的 CPU 数量（数千个）和互连容量有限，NUMA 系数实际上是动态的，并且可能根据工作负载达到不可接受的水平。

更常用的解决方案是使用高速网络连接许多商品机器以形成集群。不过，这些不是 NUMA 机器；它们是 NUMA 机器。它们不实现共享地址空间，因此不属于此处讨论的任何类别。

5.2 操作系统对 NUMA 的支持

为了支持 NUMA 机器，操作系统必须考虑内存的分布式特性。例如，如果一个进程在给定的处理器上运行，分配给该进程地址空间的物理 RAM 理想情况下应该来自本地内存。否则，每条指令都必须访问远程存储器以获取代码和数据。有一些特殊情况需要考虑，这些情况仅出现在 NUMA 机器中。DSO 的文本段通常在机器的内存中只出现一次

物理内存。但是，如果 DSO 被所有 CPU 上的进程和线程使用（例如，像 libc 这样的基本运行时库），则意味着除了少数处理器之外的所有处理器都必须具有远程访问权限。理想情况下，操作系统会将此类 DSO “镜像”到每个处理器的物理 RAM 中并使用本地副本。这是一种优化，而不是要求，并且通常很难实现。它可能不受支持或仅以有限的方式支持。

为了避免使情况变得更糟，操作系统不应将进程或线程从一个节点迁移到另一个节点。操作系统应该已经尝试避免在普通多处理器计算机上迁移进程，因为从一个处理器迁移到另一个处理器意味着缓存内容会丢失。如果负载分配需要将进程或线程从处理器上迁移出来，操作系统通常可以选择一个具有足够剩余容量的任意新处理器。在 NUMA 环境中，新处理器的选择受到一些限制。新选择的处理器不应比旧处理器对进程正在使用的内存具有更高的访问成本；这限制了目标列表。如果没有符合该标准的空闲处理器，操作系统别无选择，只能迁移到内存访问成本更高的处理器。

在这种情况下，有两种可能的前进方式。首先，人们可以希望这种情况是暂时的，并且进程可以迁移回更适合的处理器。或者，操作系统还可以将进程的内存迁移到更靠近新使用的处理器的物理页。这是一个相当昂贵的操作。可能需要复制大量内存，尽管不一定是一步完成。当这种情况发生时，必须至少暂时停止该过程，以便正确迁移对旧页面的修改。要使页面迁移高效且快速，还有一整套其他要求。简而言之，操作系统应该避免它，除非确实有必要。

一般来说，不能假设 NUMA 计算机上的所有进程都使用相同数量的内存，这样，随着进程在处理器之间的分布，内存使用量也均匀分布。事实上，除非机器上运行的应用程序非常特殊（在 HPC 领域很常见，但在外部并不常见），否则内存使用将非常不平等。一些应用程序将使用大量内存，而另一些则几乎不需要。如果内存始终分配给发起请求的处理器本地，那么迟早会导致问题。系统最终将耗尽运行大型进程的节点的本地内存。

为了应对这些严重问题，默认情况下，内存不是专门在本地节点上分配的。为了利用系统的所有内存，默认策略是对内存进行条带化。这保证了系统所有内存的平等使用。副作用是，可以在处理器之间自由迁移进程，因为平均而言，对所有使用的内存的访问成本不会改变。对于较小的 NUMA 因子，条带化是可以接受的，但仍然不是最佳的（参见第 5.4 节中的数据）。

这是一种悲观情绪，有助于系统避免严重问题，并使其在正常运行下更具可预测性。但在某些情况下，它确实会显著降低整体系统性能。这就是 Linux 允许每个进程选择内存分配规则的原因。一个进程可以为自己及其子进程选择不同的策略。我们将在第 6 节中介绍可用于此目的的接口。

5.3 公开信息

内核通过 sys 伪文件系统 (sysfs) 发布有关以下处理器缓存的信息

/sys/devices/system/cpu/cpu*/缓存

在第 6.2.1 节中，我们将看到可用于查询各种缓存大小的接口。这里重要的是缓存的拓扑结构。上面的目录包含子目录（名为 index*），其中列出了有关 CPU 拥有的各种缓存的信息。就拓扑而言，文件类型、级别和 shared_cpu_map 是这些目录中的重要文件。对于 Intel Core 2 QX6700，信息如表 5.1 所示。

		type	level	shared_cpu_map
cpu0	index0	Data	1	00000001
	index1	Instruction	1	00000001
	index2	Unified	2	00000003
cpu1	index0	Data	1	00000002
	index1	Instruction	1	00000002
	index2	Unified	2	00000003
cpu2	index0	Data	1	00000004
	index1	Instruction	1	00000004
	index2	Unified	2	0000000c
cpu3	index0	Data	1	00000008
	index1	Instruction	1	00000008
	index2	Unified	2	0000000c

表 5.1: Core 2 CPU 缓存的 sysfs 信息

这个数据的含义如下：

- 每个核心²⁵具有三个缓存：L1i、L1d、L2。
- L1d 和 L1i 缓存不与任何其他核心共享 - 每个核心都有自己的一组缓存。这由只有一个设置位的 shared_cpu_map 中的位图来指示。
- cpu0 和 cpu1 上的 L2 缓存是共享的，cpu2 和 cpu3 上的 L2 也是共享的。

如果 CPU 有更多的缓存级别，就会有更多的 index* 目录。

对于四路双核 Opteron 机器，缓存信息如表 5.2 所示。可以看出这些

²⁵ cpu0 到 cpu3 是核心的知识来自另一个地方，稍后将对此进行解释。

		type	level	shared_cpu_map
cpu0	index0	Data	1	00000001
	index1	Instruction	1	00000001
	index2	Unified	2	00000001
cpu1	index0	Data	1	00000002
	index1	Instruction	1	00000002
	index2	Unified	2	00000002
cpu2	index0	Data	1	00000004
	index1	Instruction	1	00000004
	index2	Unified	2	00000004
cpu3	index0	Data	1	00000008
	index1	Instruction	1	00000008
	index2	Unified	2	00000008
cpu4	index0	Data	1	00000010
	index1	Instruction	1	00000010
	index2	Unified	2	00000010
cpu5	index0	Data	1	00000020
	index1	Instruction	1	00000020
	index2	Unified	2	00000020
cpu6	index0	Data	1	00000040
	index1	Instruction	1	00000040
	index2	Unified	2	00000040
cpu7	index0	Data	1	00000080
	index1	Instruction	1	00000080
	index2	Unified	2	00000080

表 5.2: Opteron CPU 缓存的 sysfs 信息

处理器还具有三个高速缓存: L1i、L1d、L2。没有一个核心共享任何级别的缓存。该系统有趣的部分是处理器拓扑。如果没有这些附加信息,人们就无法理解缓存数据。sys 文件系统在以下文件中公开此信息

/sys/devices/system/cpu/cpu*/拓扑

表 5.3 显示了 SMP Opteron 机器的层次结构中的有趣文件。

	physical_package_id	core_id	core_siblings	thread_siblings
cpu0	0	0	00000003	00000001
cpu1		1	00000003	00000002
cpu2	1	0	0000000c	00000004
cpu3		1	0000000c	00000008
cpu4	2	0	00000030	00000010
cpu5		1	00000030	00000020
cpu6	3	0	000000c0	00000040
cpu7		1	000000c0	00000080

表 5.3: Opteron CPU 拓扑的 sysfs 信息

将表 5.2 和表 5.3 结合起来我们可以看出

- 没有一个 CPU 具有超线程 (thread_-兄弟位图有一位设置),
- 该系统实际上共有四个处理器

(physical_package_id 0 到 3),

- 每个处理器有两个核心, 并且
- 没有一个核心共享任何缓存。

这正是对应于早期 Opteron 的情况。

到目前为止提供的数据中完全缺少有关该计算机上 NUMA 性质的信息。任何 SMP Opteron 机器都是 NUMA 机器。对于这些数据, 我们必须查看 NUMA 机器上存在的 sys 文件系统的另一部分, 即下面的层次结构

/sys/设备/系统/节点

此目录包含系统上每个 NUMA 节点的子目录。在特定于节点的目录中有许多文件。前两个表中描述的 Opteron 机器的重要文件及其内容如表 5.4 所示。

	cpumap	distance
node0	00000003	10 20 20 20
node1	0000000c	20 10 20 20
node2	00000030	20 20 10 20
node3	000000c0	20 20 20 10

表 5.4: Opteron 节点的 sysfs 信息

这些信息将所有其他信息联系在一起; 现在我们对机器的架构有了完整的了解。我们已经知道该机器有四个处理器。每个处理器都构成自己的节点, 从节点 * 目录中 cpumap 文件的值中设置的位可以看出。这些目录中的距离文件包含一组值, 每个节点一个, 代表各个节点的内存访问成本。在此示例中, 所有本地内存访问的成本为 10, 对任何其他节点的所有远程访问的成本为 20。²⁶ 这意味着, 即使处理器被组织为二维超立方体 (见图 5.1), 不直接连接的处理器之间的访问也不会更昂贵。成本的相对值应该可用作访问时间实际差异的估计。所有这些信息的准确性是另一个问题。

5.4 远程访问成本

不过, 距离是相关的。在 [1] AMD 中记录了四插槽计算机的 NUMA 成本。对于写操作, 数字如图 5.3 所示。写

²⁶This is, by the way, incorrect. The ACPI information is apparently wrong since, although the processors used have three coherent HyperTransport links, at least one processor must be connected to a Southbridge. At least one pair of nodes must therefore have a larger distance.

00400000 默认文件=/bin/cat 映射=3 N3=3 00504000 默认文件=/bin/cat 匿名=1 脏=1 映射=2 N3=2 00506000 默认堆匿名=3 脏=3 活动=0 N3=3 38a9000000 默认文件=/lib64/ld-2.4.so 映射=22 mapmax=47 N1=22 38a9119000 默认文件=/lib64/ld-2.4.so anon=1 脏=1 N3=1 38a911a000 默认文件=/lib64/ld-2.4.so anon=1 脏=1 N3=1 38a9200000 默认文件=/lib64/libc-2.4.so 映射=53 mapmax=52 N1=51 N2=2 38a933f000 默认文件=/lib64/libc-2.4.so 38a943f000 默认文件=/lib64/libc-2.4.so 匿名=1 脏=1 映射=3 mapmax=32 N1=2 N3=1 38a9443000 默认文件=/lib64/libc-2.4.so anon=1 脏=1 N3=1 38a9444000 默认 anon=4 脏=4 活动=0 N3=4 2b2bbcdce000 默认 anon=1 dirty=1 N3=1 2b2bbcdce4000 默认 anon=2 dirty=2 N3=2 2b2bbcdce6000 默认文件=/usr/lib/locale/locale-archive 映射=11 mapmax=8 N0=11 7ffedcc7000 默认堆栈匿名=2 脏=2 N3=2

图 5.2: /proc/PID/numa_maps 的内容

比读取慢，这并不奇怪。有趣的部分是 1 跳和 2 跳情况的成本。这两种 1 跳案例的成本实际上略有不同。详细信息请参见[1]。从这张图表中我们需要记住的事实是，2 跳读取和写入分别比 0 跳读取慢 30% 和 49%。2 跳写入比 0 跳写入慢 32%，比 1 跳写入慢 17%。处理器和内存节点的相对位置会产生很大的差异。AMD 的下一代处理器将在每个处理器上配备四个相干的 HyperTransport 链路。在这种情况下，四孔机器的直径将为 1。对于八个插槽，同样的问题又会出现，而且更加严重，因为具有八个节点的超立方体的直径是三。

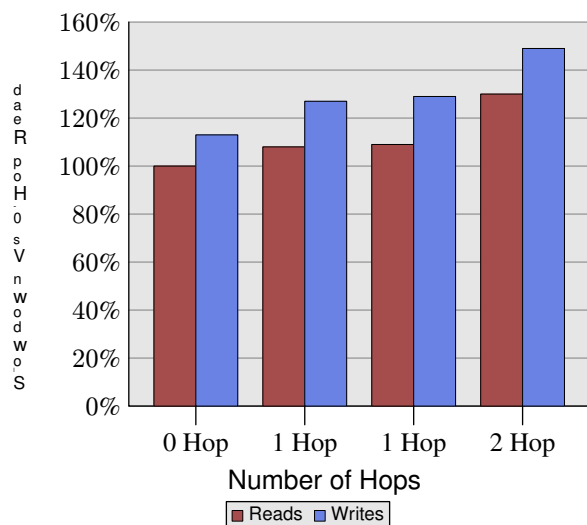


图 5.3: 多个节点的读/写性能

所有这些信息都可用，但使用起来很麻烦。在第 6.5 节中，我们将看到一个有助于更轻松地访问和使用此信息的界面。

系统提供的最后一条信息位于

进程本身的状态。可以确定内存映射文件、写入时复制 (COW)²⁷ 页和匿名内存如何分布在系统中的节点上。对于每个进程，内核都会提供一个伪文件/proc/PID/numa_maps，其中PID是进程的ID，如图5.2所示。文件中的重要信息是 N0 到 N3 的值，它们指示为节点 0 到 3 上的内存区域分配的页数。可以很好地猜测该程序是在节点 3 的核心上执行的。程序本身和脏页都在该节点上分配。只读映射，例如 ld-2.4.so 和 libc-2.4.so 的第一个映射以及共享文件 locale-archive 分配在其他节点上。

正如我们在图 5.3 中看到的，当跨节点执行时，1 跳和 2 跳读取的读取性能分别下降 9% 和 30%。对于执行来说，需要这样的读取，并且如果 L2 缓存丢失，每个缓存行都会产生这些额外的成本。如果内存远离处理器，则针对超出缓存大小的大型工作负载测量的所有成本都必须增加 9%/30%。

为了查看现实世界中的效果，我们可以按照第 3.5.1 节中的方式测量带宽，但这一次内存位于远程节点上，一跳之外。与使用本地内存的数据进行比较的测试结果如图 5.4 所示。这些数字在两个方向上都有一个大峰值，这是测量多线程代码问题的结果，可以忽略不计。该图中的重要信息是读取操作总是慢 20%。这比图 5.3 中的 9% 慢得多，这很可能不是不间断读/写操作的数字，并且可能指较旧的处理器版本。仅有的

²⁷Copy-On-Write is a method often used in OS implementations when a memory page has one user at first and then has to be copied to allow independent users. In many situations the copying is unnecessary, at all or at first, in which case it makes sense to only copy when either user modifies the memory. The operating system intercepts the write operation, duplicates the memory page, and then allows the write instruction to proceed.



图 5.4：在远程内存上操作

AMD 知道。

对于适合缓存的工作集大小，写入和复制操作的性能也会慢 20%。对于超过缓存大小的工作集，写入性能不会明显慢于本地节点上的操作。互连的速度足以跟上内存的速度。主要因素是等待主内存所花费的时间。

6 程序员能做什么

经过前面几节的描述，很明显，程序员有很多很多的机会来影响程序的性能，无论是积极的还是消极的。这仅适用于与内存相关的操作。我们将从头开始介绍这些机会，从最低级别的物理 RAM 访问和 L1 缓存开始，一直到影响内存处理的操作系统功能。

6.1 绕过缓存

当生成数据并且没有（立即）再次使用数据时，内存存储操作首先读取完整的缓存行，然后修改缓存的数据，这一事实对性能不利。此操作将可能再次需要的数据从缓存中推出，以支持不会很快使用的数据。对于大型数据结构尤其如此，例如矩阵，先填充然后再使用。在填充矩阵的最后一个元素之前，巨大的大小会驱逐第一个元素，从而使写入的缓存无效。

对于这种情况和类似情况，处理器提供对 *non-temporal* 写入操作的支持。在这种情况下，非临时意味着数据不会很快被重用，因此没有理由缓存它。这些非临时写操作不会读取缓存行然后修改它；相反，新内容会直接写入内存。

这可能听起来很昂贵，但事实并非如此。处理器将尝试使用写组合（参见第 3.3.3 节）来填充整个缓存行。如果成功，则根本不需要内存读取操作。对于 x86 和 x86-64 架构，gcc 提供了许多内在函数：

```
#include <emmintrin.h> void _mm_stream_si32(int *p, int a)
; void _mm_stream_si128(int *p, __m128i a); void _mm_stream_pd(double *p, __m128d a); #include <xmmmintrin.h> void _mm_stream_pi(__m64 *p, __m64 a); void _mm_stream_ps(浮点 *p, __m128 a); #include <ammintrin.h> void _mm_stream_sd(double *p, __m128d a); void _mm_stream_ss(浮点 *p, __m128 a);
```

如果一次性处理大量数据，这些指令的使用效率最高。数据从内存加载，经过一个或多个步骤处理，然后写回内存。数据“流”过处理器，因此得名内在函数。

内存地址必须分别与 8 或 16 字节对齐。在使用多媒体扩展的代码中，可以替换正常的 `_mm_store_*` 内在函数

与这些非时间版本。在 A.1 节的矩阵乘法代码中，我们不这样做，因为写入的值会在短时间内重用。这是一个使用流指令没有用的示例。有关此代码的更多信息，请参阅第 6.2.1 节。

处理器的写入组合缓冲区只能将部分写入高速缓存行的请求保留这么长时间。通常需要依次发出修改单个高速缓存行的所有指令，以便真正进行写组合。如何执行此操作的示例如下：

```
#include <emmintrin.h>
无效 setbytes(char *p, int c)
{ __m128i i = _mm_set_epi8(c, c, c, c, c, c, c, c, c, c, c, c,
c, c, c); _mm_stream_si128((__m128i *)&p[0], i); _mm_stream_si128((__m128i *)&p[16], i); _mm_stream_si128((__m128i *)&p[32], i); _mm_stream_si128((__m128i *)&p[48], i);
}
```

假设指针 p 已适当对齐，则调用此函数会将寻址缓存行的所有字节设置为 c。写组合逻辑将看到四个生成的 movntdq 指令，并且仅在执行完最后一条指令后才向存储器发出写命令。总而言之，此代码序列不仅避免了在写入缓存行之前读取缓存行，还避免了可能很快不需要的数据污染缓存。这在某些情况下可以带来巨大的好处。使用此技术的日常代码的一个示例是 C 运行时中的 memset 函数，对于大块，它应该使用与上面类似的代码序列。

一些架构提供专门的解决方案。PowerPC 架构定义了 dcbz 指令，可用于清除整个高速缓存行。该指令并没有真正绕过高速缓存，因为为结果分配了高速缓存行，但没有从内存中读取数据。它比非临时存储指令受到更多限制，因为缓存行只能设置为全零，并且会污染缓存（如果数据是非临时的），但不需要写组合逻辑来实现结果。

为了查看非时间指令的实际作用，我们将查看一个新的测试，该测试用于测量对组织为二维数组的矩阵的写入。编译器在内存中布置矩阵，以便最左边（第一个）索引寻址所有元素在内存中按顺序布置的行。右侧（第二个）索引寻址行中的元素。测试程序以两种方式迭代矩阵：首先增加内循环中的列号，然后增加内循环中的行索引。这意味着我们得到了be-

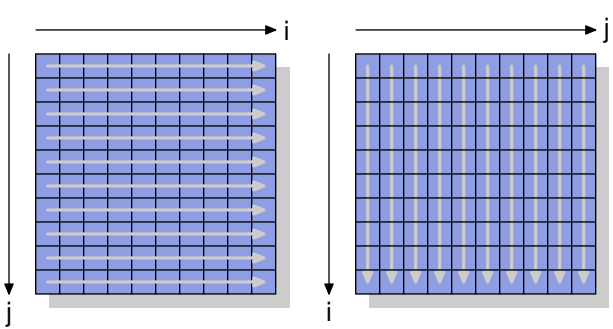


图 6.1：矩阵访问模式

行为如图 6.1 所示。我们测量初始化 3000 × 3000 矩阵所需的时间。为了了解内存的行为方式，我们使用不使用缓存的存储指令。在 IA-32 处理器上，“非时间提示”用于此目的。为了进行比较，我们还测量了普通商店的运营情况。结果见表6.1。

	Inner Loop Increment	
	Row	Column
Normal	0.048s	0.127s
Non-Temporal	0.048s	0.160s

表 6.1：时序矩阵初始化

对于使用缓存的正常写入，我们看到了预期的结果：如果按顺序使用内存，我们会得到更好的结果，整个操作需要 0.048 秒，转换为约 750MB/秒，而随机访问或多或少需要 0.127 秒（约 280MB/秒）。矩阵足够大，以至于缓存基本上无效。

我们在这里主要感兴趣的部分是绕过缓存的写入。令人惊讶的是，这里的顺序访问与使用缓存的情况一样快。出现此结果的原因是处理器正在执行如上所述的写入组合。此外，非临时写入的 memory ordering 规则也放宽了：程序需要显式插入内存屏障（x86 和 x86-64 处理器的 sfence 指令）。这意味着处理器可以更自由地写回数据，从而尽可能地使用可用带宽。

在内循环中按列访问的情况下，情况有所不同。未缓存访问的结果明显慢于缓存访问的情况（0.16 秒，约 225MB/秒）。在这里我们可以看到不可能进行写入组合，并且每个存储单元必须单独寻址。这需要不断地在 RAM 芯片中选择新的行，并产生所有相关的延迟。结果比缓存运行的结果差 25%。

在读取方面，直到最近，除了使用非临时访问（NTA）预取指令的弱提示之外，处理器还缺乏支持。对于读取来说，没有相当于写组合的方法，这对于不可缓存的内存（例如内存映射 I/O）尤其不利。Intel 通过 SSE4.1 扩展引入了 NTA 负载。它们是使用少量的流加载缓冲区来实现的；每个缓冲区包含一个高速缓存行。给定高速缓存行的第一条 `movntdqa` 指令会将高速缓存行加载到缓冲区中，可能会替换另一个高速缓存行。对同一高速缓存行的后续 16 字节对齐访问将从加载缓冲区以很少的成本提供服务。除非有其他原因这样做，否则缓存行不会被加载到缓存中，从而能够加载大量内存而不污染缓存。编译器为此指令提供了一个内在函数：

```
#include <smmintrin.h> __m128i _mm_stream_load_si128 (__m128i *p);
```

这个内在函数应该多次使用，并将 16 字节块的地址作为参数传递，直到读取每个缓存行。只有这样才能启动下一个缓存行。由于有一些流式读取缓冲区，因此可以同时从两个内存位置读取。

我们应该从这个实验中得到的结论是，现代 CPU 非常好地优化未缓存的写入，甚至最近甚至可以优化读取访问，只要它们是顺序的。当处理只使用一次的大型数据结构时，这些知识会非常有用。其次，缓存可以帮助掩盖随机内存访问的部分（但不是全部）成本。由于 RAM 访问的实现，本示例中的随机访问速度慢了 70%。在实现发生变化之前，应尽可能避免随机访问。

在关于预取的部分中，我们将再次讨论非时间标志。

6.2 缓存访问

希望提高程序性能的程序员会发现最好关注影响一级缓存的更改，因为这些更改可能会产生最佳结果。我们将首先讨论它，然后再将讨论扩展到其他级别。显然，所有针对一级缓存的优化也会影响其他缓存。所有内存访问的主题都是相同的：提高局部性（空间和时间）并对齐代码和数据。

6.2.1 优化 1 级数据缓存访问

在第 3.3 节中，我们已经看到有效使用 L1d 缓存可以在多大程度上提高性能

曼斯。在本节中，我们将展示哪些类型的代码更改可以帮助提高性能。继续上一节，我们首先关注顺序访问内存的优化。从 3.3 节的数字可以看出，当顺序访问内存时，处理器会自动预取数据。

使用的示例代码是矩阵乘法。我们使用两个 1000×1000 双元素的方阵。对于那些忘记了数学的人，给定两个矩阵 A 和 B ，其中元素为 a_{ij} 和 b_{ij} ，元素为 $0 \leq i, j < N$ ，其乘积为

$$(AB)_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{kj} = a_{i1} b_{1j} + a_{i2} b_{2j} + \cdots + a_{i(N-1)} b_{(N-1)j}$$

其直接的 C 实现如下所示：

```
for (i = 0; i < N; ++i) for (j = 0; j < N; ++j) for (k = 0; k < N; ++k) res[i][j] += mul1[i][k] * mul2[k][j];
```

两个输入矩阵是 `mul1` 和 `mul2`。假设结果矩阵 `res` 被初始化为全零。这是一个很好且简单的实现。但很明显，我们确实遇到了图 6.1 中解释的问题。当按顺序访问 `mul1` 时，内部循环会增加 `mul2` 的行号。这意味着 `mul1` 的处理方式类似于图 6.1 中的左侧矩阵，而 `mul2` 的处理方式类似于右侧矩阵。这不可能是好事。

有一种可能的补救措施可以轻松尝试。由于矩阵中的每个元素都会被多次访问，因此在使用第二个矩阵 `mul2` 之前重新排列（用数学术语“转置”）可能是值得的。

$$(AB)_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{jk}^T = a_{i1} b_{j1}^T + a_{i2} b_{j2}^T + \cdots + a_{i(N-1)} b_{j(N-1)}^T$$

转置后（传统上用上标“T”表示），我们现在按顺序迭代两个矩阵。就 C 代码而言，现在看起来像这样：

```
双 tmp[N][N]; for (i = 0; i < N; ++i) for (j = 0; j < N; ++j) tmp[i][j] = mul2[j][i]; for (i = 0; i < N; ++i) for (j = 0; j < N; ++j) for (k = 0; k < N; ++k) res[i][j] += mul1[i][k] * tmp[j][k];
```

我们创建一个临时变量来包含转置矩阵。这需要接触额外的内存，但这一成本有望得到恢复，因为每列 1000 次非顺序访问的成本更高（至少在现代硬件上）。是时候进行一些性能测试了。时钟速度为 26 66MHz 的 Intel Core 2 上的结果为（以时钟周期为单位）：

	Original	Transposed
Cycles	16,765,297,870	3,922,373,010
Relative	100%	23.4%

通过矩阵的简单变换我们可以实现76.6%的加速！复制操作不仅仅是编造的。1000 次非顺序访问确实很痛苦。

下一个问题是这是否是我们能做的最好的事情。无论如何，我们当然需要一种不需要额外副本的替代方法。我们并不总是能够执行复制：矩阵可能太大或可用内存太小。

寻找替代实现应该从仔细检查所涉及的数学和原始实现执行的操作开始。简单的数学知识使我们能够看到，只要每个加数只出现一次，结果矩阵的每个元素的加法执行顺序就无关紧要。²⁸这种理解使我们能够寻找对原始代码的内部循环中执行的加法重新排序的解决方案。

现在让我们检查一下原始代码执行中的实际问题。访问 mul2 元素的顺序是：(0,0), (1,0),...。。。。(N-1,0), (0,1), (1,1), ..。。元素 (0,0) 和 (0,1) 位于同一缓存行中，但是当内部循环完成一轮时，该缓存行早已被逐出。对于此示例，对于三个矩阵中的每一个，每轮内部循环都需要 1000 个高速缓存行（对于 Core 2 处理器为 64 字节）。这加起来远远超过可用的 32k L1d。

但是，如果我们在执行内部循环时一起处理中间循环的两次迭代会怎么样？在本例中，我们使用缓存行中的两个双精度值，保证位于 L1d 中。我们将 L1d 失效率降低了一半。这当然是一个改进，但是，根据缓存行的大小，它仍然可能不如我们所能得到的那么好。Core 2 处理器的 L1d 高速缓存行大小为 64 字节。实际值可以通过以下方式查询

系统配置（_SC_LEVEL1_DCACHE_LINESIZE）

在运行时或使用命令行中的 getconf 实用程序，以便可以针对特定的缓存行大小来编译程序。sizeof(double) 为 8

²⁸We ignore arithmetic effects here which might change the occurrence of overflows, underflows, or rounding.

这意味着，为了充分利用缓存行，我们应该展开中间循环 8 次。继续这个想法，为了有效地使用 res 矩阵，即同时写入 8 个结果，我们也应该展开外循环 8 次。我们假设此处缓存行大小为 64，但该代码在具有 32 字节缓存行的系统上也能正常运行，因为两个缓存行也都得到了 100% 的利用。一般来说，最好在编译时使用 getconf 实用程序对缓存行大小进行硬编码，如下所示：

```
gcc -DCLS=$(getconf LEVEL1_DCACHE_LINESIZE) ...
```

如果二进制文件应该是通用的，则应使用最大的缓存行大小。对于非常小的 L1d，这可能意味着并非所有数据都适合缓存，但此类处理器无论如何都不适合高性能程序。我们得到的代码看起来像这样：

```
#define SM (CLS / sizeof (double))

for (i = 0; i < N; i += SM)
  for (j = 0; j < N; j += SM)
    for (k = 0; k < N; k += SM)
      for (i2 = 0, rres = &res[i][j],
           rmul1 = &mul1[i][k]; i2 < SM;
           ++i2, rres += N, rmul1 += N)
        for (k2 = 0, rmul2 = &mul2[k][j];
             k2 < SM; ++k2, rmul2 += N)
          for (j2 = 0; j2 < SM; ++j2)
            rres[j2] += rmul1[k2] * rmul2[j2];
```

这看起来相当可怕。在某种程度上是这样，但这只是因为它包含了一些技巧。最明显的变化是我们现在有六个嵌套循环。外层循环以 SM 为间隔进行迭代（缓存行大小除以 sizeof(double)）。这将乘法分解为几个较小的问题，这些问题可以通过更多的缓存局部性来处理。内部循环迭代外部循环缺失的索引。再次出现三个循环。这里唯一棘手的部分是 k2 和 j2 循环的顺序不同。这样做是因为，在实际计算中，只有一个表达式依赖于 k2，但有两个表达式依赖于 j2。

这里剩下的复杂性是由于 gcc 在优化数组索引方面不是很聪明。附加变量 rres、rmul1 和 rmul2 的引入通过将公共表达式尽可能地从内部循环中拉出来优化代码。C 和 C++ 语言的默认别名规则无法帮助编译器做出这些决定（除非使用限制，否则所有指针访问都是潜在的别名来源）。这就是为什么 Fortran 仍然是数字编程的首选语言：它使编写快速代码变得更容易。²⁹

²⁹In theory the restrict keyword introduced into the C language in the 1999 revision should solve the problem. Compilers have not caught up yet, though. The reason is mainly that too much incorrect code exists which would mislead the compiler and cause it to generate incorrect object code.

	Original	Transposed	Sub-Matrix	Vectorized
Cycles	16,765,297,870	3,922,373,010	2,895,041,480	1,588,711,750
Relative	100%	23.4%	17.3%	9.47%

表 6.2: 矩阵乘法时序

所有这些工作的成效如表 6.2 所示。通过避免复制，我们的性能又提高了 6.1%。另外，我们不需要任何额外的内存。输入矩阵可以任意大，只要结果矩阵也适合内存即可。这是我们现在已经实现的通用解决方案的要求。

表 6.2 中还有一栏没有解释。如今大多数现代处理器都包含对矢量化特殊支持。这些特殊指令通常被称为多媒体扩展，允许同时处理 2、4、8 或更多值。这些通常是 SIMD（单指令、多数据）操作，并通过其他操作进行增强以获得正确形式的数据。Intel 处理器提供的 SSE2 指令可以在一次操作中处理两个 double 值。指令参考手册列出了提供对这些 SSE2 指令的访问的内部函数。如果使用这些内在函数，程序的运行速度会再快 7.3%（相对于原始程序）。结果是程序的运行时间仅为原始代码的 10%。转化为人们认可的数字后，我们从 318 MFLOPS 提高到 3.35 GFLOPS。由于我们在这里只对记忆效应感兴趣，因此程序代码被推到 A.1 节中。

需要注意的是，在上一个版本的代码中，我们仍然存在 mul2 的一些缓存问题；预取仍然不起作用。但如果不转置矩阵就无法解决这个问题。也许缓存预取单元会变得更智能，能够识别模式，然后就不需要进行额外的更改。不过，单线程代码在 2.66 GHz 处理器上的 3.19 GFLOPS 还不错。

我们在矩阵乘法示例中优化的是加载的缓存行的使用。始终使用高速缓存行的所有字节。我们只是确保在缓存行被清空之前使用它们。这当然是一个特例。

更常见的是具有填充一个或多个高速缓存行的数据结构，其中程序在任何时候仅使用几个成员。在图 3.11 中，我们已经看到了仅使用少量成员时大结构尺寸的影响。

图 6.2 显示了使用现在众所周知的程序执行的另一组基准测试的结果。这次添加了同一列表元素的两个值。在一种情况下，两个元素都在同一个缓存行中；在另一种情况下，一个元素位于列表元素的第一个缓存行中，第二个元素位于最后一个缓存行中。该图显示了正在经历的放缓。

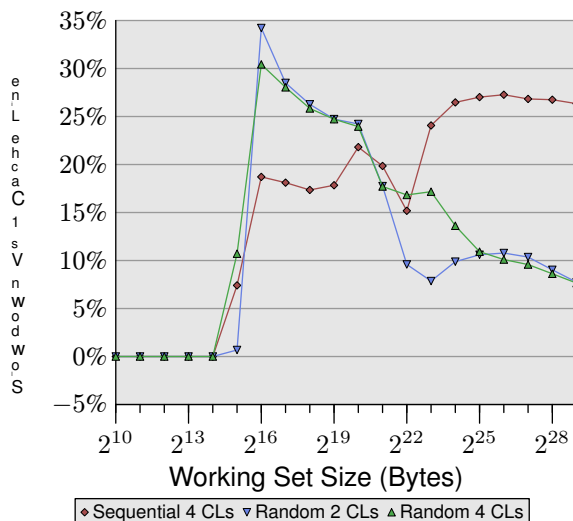


图 6.2: 分布在多个缓存行上

毫不奇怪，在所有情况下，如果工作集适合 L1d，则不会产生负面影响。一旦 L1d 不再足够，就会通过在进程中使用两个缓存线而不是一个来支付惩罚。红线显示了列表在内存中按顺序排列时的数据。我们看到通常的两步模式：当二级缓存足够时，大约有 17% 的惩罚；当必须使用主内存时，大约有 27% 的惩罚。

在随机存储器访问的情况下，相关数据看起来有点不同。适合 L2 的工作集的减速在 25% 到 35% 之间。除此之外，它会下降到 10% 左右。这并不是因为惩罚变小，而是因为实际的内存访问成本不成比例地增加。数据还表明，在某些情况下，元素之间的距离确实很重要。随机 4 CL 曲线显示出更高的惩罚，因为使用了第一和第四缓存行。

查看数据结构与缓存行相比的布局的一个简单方法是使用 pahole 程序（参见[4]）。该程序检查二进制文件中定义的数据结构。编写一个包含此定义的程序：

```
结构体 foo { int a;长
填充[7]; 整数b; };
```

```

struct foo {
    int                a;                /*      0      4 */

    /* XXX 4 bytes hole, try to pack */

    long int           fill[7];          /*      8      56 */
    /* --- cacheline 1 boundary (64 bytes) --- */
    int                b;                /*     64      4 */
}; /* size: 72, cachelines: 2 */
/* sum members: 64, holes: 1, sum holes: 4 */
/* padding: 4 */
/* last cacheline: 8 bytes */

```

图 6.3: pahole 运行的输出

当在 64 位机器上编译时，pahole 的输出包含（除其他外）如图 6.3 所示的输出。这个输出告诉了我们很多信息。首先，它表明数据结构使用了多个缓存行。该工具假定当前使用的处理器的缓存行大小，但可以使用命令行参数覆盖该值。特别是在结构的大小刚刚超过缓存行的限制并且分配了许多这种类型的对象的情况下，寻求一种压缩该结构的方法是有意义的。也许一些元素可以有更小的类型，或者某些字段实际上是可以使用单独的位来表示的标志。

在本例中，压缩很容易并且由程序暗示。输出显示第一个元素后面有一个四个字节的空洞。该孔是由结构和填充元件的对齐要求引起的。很容易看出，元素 b 的大小为 4 个字节（由行末尾的 4 表示），完全适合间隙。这种情况下的结果是间隙不再存在并且数据结构适合一个高速缓存行。Pahole 工具可以自行执行此优化。如果使用 --reorganize 参数并且在命令行末尾添加结构名称，则该工具的输出是优化的结构和缓存行使用。除了移动元素来填充间隙之外，该工具还可以优化位字段并组合填充和孔。欲了解更多详细信息，请参阅[4]。

当然，最理想的情况是有一个刚好足以容纳尾部元件的孔。为了使这种优化有用，需要对对象本身与缓存行对齐。我们稍后会谈到这一点。

pahole 输出还可以轻松查看元素是否必须重新排序，以便那些一起使用的元素也存储在一起。使用pahole工具，可以轻松确定哪些元素位于同一缓存行上，以及何时必须重新调整元素才能实现这一点。这不是一个自动过程，但该工具可以提供很大帮助。

各个结构元素的位置及其使用方式也很重要。正如我们在第 3.5.2 节中看到的，关键字位于高速缓存行后期的代码的性能更差。这意味着亲

语法应该始终遵循以下两条规则：

1. 始终将最有可能是关键词的结构元素移动到结构的开头。
2. 访问数据结构时，如果访问顺序不受情况限制，请按照结构中定义的顺序访问元素。

对于小型结构，这意味着元素应该按照它们可能被访问的顺序排列。这必须以灵活的方式处理，以便也可以应用其他优化，例如填充孔。对于更大的数据结构，每个缓存行大小的块应该按照规则进行排列。

但是，如果对象本身未按预期对齐，则不值得花时间重新排序元素。对象的对齐方式由数据类型的对齐要求决定。每个基本类型都有其自己的对齐要求。对于结构化类型，其任何元素的最大对齐要求决定了结构的对齐方式。这几乎总是小于缓存行大小。这意味着即使结构体的成员排列成适合同一缓存行，分配的对象也可能没有与缓存行大小匹配的对齐方式。有两种方法可以确保对象具有在设计结构布局时使用的对齐方式：

- 可以根据明确的对齐要求来分配对象。对于动态分配，对 malloc 的调用只会分配与最苛刻的标准类型（通常是 long double）的对齐方式匹配的对象。不过，可以使用 posix_memalign 来请求更高的对齐方式。

```

#include <stdlib.h> int posix_memalign(void **
    memptr, size_t 对齐, size_t 大小);

```

该函数将指向新分配的内存的指针存储在 `mem_ptr` 指向的指针变量中。内存块的大小为 `size` 字节，并在对齐字节边界上对齐。

对于编译器分配的对象（在 `.data`、`.bss` 等以及堆栈上），可以使用变量属性：

```
struct strtype 变量 __attribute__((aligned(64)))  
;
```

在这种情况下，无论 `strtype` 结构的对齐要求如何，变量都会在 64 字节边界对齐。这适用于全局变量和自动变量。

对于数组，此方法并不像人们预期的那样工作。只有数组的第一个元素才会对齐，除非每个数组元素的大小是对齐值的倍数。这也意味着每个变量都必须进行适当的注释。`posix_memalign` 的使用也不是完全免费的，因为对齐要求通常会导致碎片和/或更高的内存消耗。

- 用户定义类型的对齐要求可以通过使用类型属性来更改：

```
struct strtype { ...成员... } __attribute__((aligned(64)));
```

这将导致编译器以适当的对齐方式分配所有对象，包括数组。不过，程序员必须注意为动态分配的对象请求适当的对齐。这里必须再次使用 `posix_memalign`。使用 `gcc` 提供的 `alignof` 运算符并将值作为第二个参数传递给 `posix_memalign` 是很容易的。

本节前面提到的多媒体扩展几乎总是要求内存访问对齐。即，对于 16 字节内存访问，地址应该是 16 字节对齐的。x86 和 x86-64 处理器具有特殊的内存操作变体，可以处理未对齐的访问，但速度较慢。对于大多数 RISC 架构来说，这种硬对齐要求并不是什么新鲜事，因为大多数 RISC 架构都需要对所有内存访问进行完全对齐。即使架构支持未对齐的访问，有时也会比使用适当的对齐慢，特别是如果未对齐导致加载或存储使用两个缓存行而不是一个。

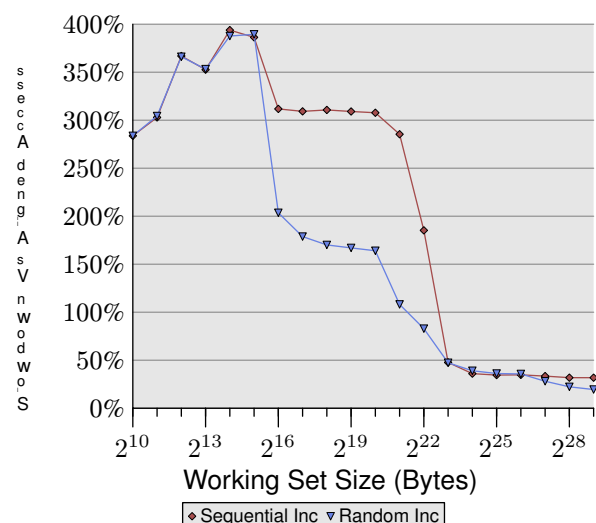


图 6.4: 未对齐访问的开销

图 6.4 显示了未对齐内存访问的影响。现在众所周知的测试是在访问内存时（顺序或随机）增加数据元素，一次使用对齐的列表元素，一次使用故意未对齐的元素。该图显示了由于未对齐的访问而引起的程序速度减慢。顺序访问情况的影响比随机情况更为显着，因为在后一种情况下，未对齐访问的成本被通常较高的内存访问成本部分隐藏。在顺序情况下，对于适合 L2 缓存的工作集大小，速度减慢约为 300%。这可以通过 L1 缓存有效性降低来解释。一些增量操作现在涉及两个缓存行，并且开始对列表元素进行工作现在通常需要读取两个缓存行。L1 和 L2 之间的连接实在是太拥挤了。

对于非常大的工作集大小，未对齐访问的影响仍然是 20% 到 30%——考虑到这些大小的对齐访问时间很长，这个影响很大。该图应该表明必须认真对待对齐。即使架构支持未对齐访问，也不能认为“它们与对齐访问一样好”。

不过，这些对齐要求会产生一些影响。如果自动变量有对齐要求，编译器必须确保在所有情况下都满足它。这不是一件小事，因为编译器无法控制调用站点及其处理堆栈的方式。这个问题可以通过两种方式处理：

1. 生成的代码主动对齐堆栈，必要时插入间隙。这需要代码来检查对齐、创建对齐以及稍后撤消对齐。
2. 要求所有调用者堆栈对齐。

所有常用的应用程序二进制接口 (ABI) 都遵循第二条路线。如果调用者违反规则并且被调用者需要对齐, 程序可能会失败。不过, 保持对齐的完整性并不是免费的。

函数中使用的堆栈帧的大小不一定是对齐的倍数。这意味着如果从此堆栈帧调用其他函数, 则需要填充。最大的区别在于, 在大多数情况下, 编译器知道堆栈帧的大小, 因此, 它知道如何调整堆栈指针以确保从该堆栈帧调用的任何函数的对齐。事实上, 大多数编译器都会简单地将堆栈帧大小向上舍入并完成它。

如果使用可变长度数组 (VLA) 或 `alloca`, 则这种简单的对齐方式是不可能的。在这种情况下, 堆栈帧的总大小仅在运行时才知道。在这种情况下可能需要主动对齐控制, 从而使生成的代码 (稍微) 变慢。

在某些架构上, 只有多媒体扩展需要严格对齐; 这些架构上的堆栈始终针对普通数据类型进行最低限度对齐, 通常分别针对 32 位和 64 位架构分别进行 4 或 8 字节对齐。在这些系统上, 强制对齐会产生不必要的成本。这意味着, 在这种情况下, 如果我们知道从不依赖严格对齐要求, 我们可能希望摆脱它。不执行多媒体操作的尾部函数 (不调用其他函数的函数) 不需要对齐。仅调用不需要对齐的函数的函数也不会。如果可以识别足够大的函数集, 程序可能希望放宽对齐要求。对于 x86 二进制文件, `gcc` 支持宽松的堆栈对齐要求:

`-mpreferred-堆栈边界=2`

如果此选项的值为 N , 则堆栈对齐要求将设置为 2^N 字节。因此, 如果使用值 2, 则堆栈对齐要求将从默认值 (16 字节) 减少到仅 4 字节。在大多数情况下, 这意味着不需要额外的对齐操作, 因为正常的堆栈推送和弹出操作无论如何都在四字节边界上工作。这个特定于机器的选项可以帮助减少代码大小并提高执行速度。但它不能应用于许多其他架构。即使对于 x86-64, 它通常也不适用, 因为 x86-64 ABI 要求浮点参数在 SSE 寄存器中传递, 并且 SSE 指令需要完整的 16 字节对齐。然而, 只要该选项可用, 它就会产生显着的变化。

结构元素的有效放置和对齐并不是数据结构影响缓存效率的唯一方面。如果使用结构数组, 则整个结构定义都会影响性能。记住

结果如图 3.11 所示: 在这种情况下, 数组元素中未使用的数据量不断增加。结果是预取的效率越来越低, 程序对于大型数据集的效率也越来越低。

对于大型工作集, 尽可能使用可用的缓存非常重要。为了实现这一点, 可能需要重新排列数据结构。虽然程序员更容易将概念上属于同一数据结构的所有数据放在一起, 但这可能不是实现最大性能的最佳方法。假设我们有一个如下的数据结构:

```
struct order { 双倍价格;布尔已付
;  const char *买家[5];长买家 ID
;  };
```

进一步假设这些记录存储在一个大数组中, 并且经常运行的作业将所有未偿账单的预期付款相加。在这种情况下, 用于 `buyer` 和 `buyer_id` 字段的内存不一定会加载到缓存中。从图 3.11 中的数据来看, 该程序的性能将比其实际性能差 5 倍。

最好将订单数据结构分成两部分, 将前两个字段存储在一个结构中, 将其他字段存储在其他地方。这种变化肯定会增加程序的复杂性, 但性能的提高可能证明这种成本是合理的。

最后, 让我们考虑另一种缓存使用优化, 它虽然也适用于其他缓存, 但主要体现在 L1d 访问中。如图 3.8 所示, 缓存关联性的增加有利于正常操作。缓存越大, 关联性通常越高。L1d 缓存太大而无法实现完全关联, 但又不足以具有与 L2 缓存相同的关联性。如果工作集中的许多对象落入同一个缓存集中, 这可能会出现。如果这导致由于过度使用集合而被逐出, 则即使大部分缓存未使用, 程序也可能遇到延迟。这些缓存未命中有时称为 *conflict misses*。由于 L1d 寻址使用虚拟地址, 因此这实际上是程序员可以控制的。如果一起使用的变量也存储在一起, 那么它们落入同一组的可能性就会最小化。图 6.5 显示了问题出现的速度。

在图中, 现在熟悉的 Follow³⁰ with NPAD=15 测试是通过特殊设置进行测量的。X 轴是两个列表元素之间的距离, 以空为单位测量

³⁰The test was performed on a 32-bit machine, hence NPAD=15 means one 64-byte cache line per list element.

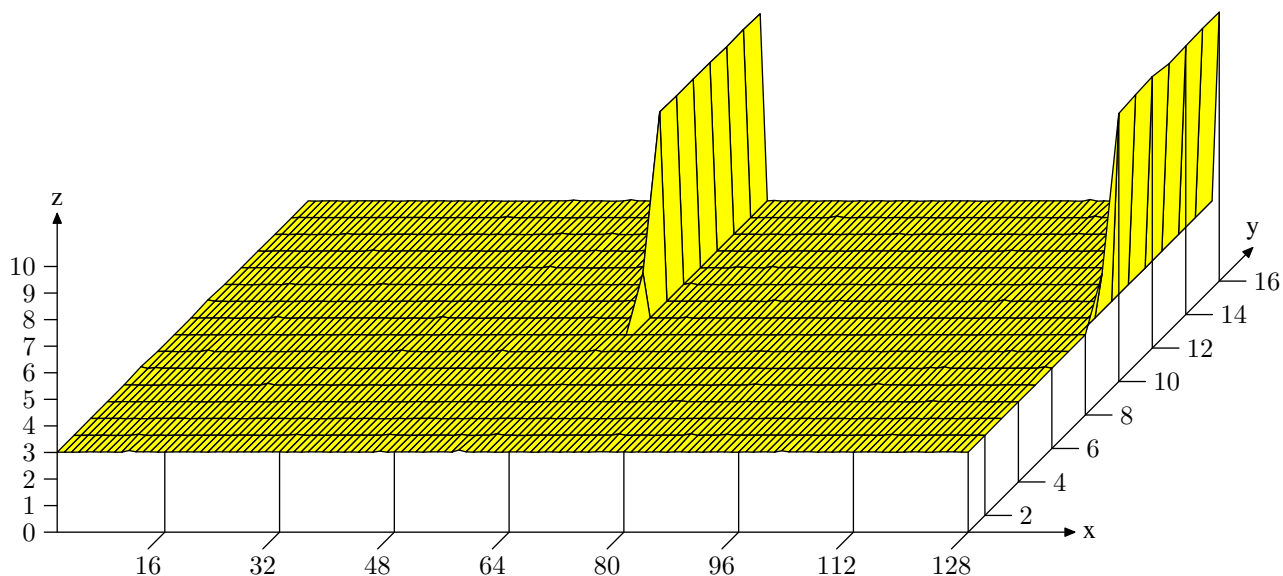


图 6.5: 缓存关联性效果

列表元素。换句话说，距离 2 意味着下一个元素的地址比前一个元素的地址晚 128 个字节。所有元素都以相同的距离布置在虚拟地址空间中。Y 轴显示列表的总长度。仅使用 1 到 16 个元素，这意味着总工作集大小为 64 到 1024 字节。z 轴显示遍历每个列表元素所需的平均周期数。

图中显示的结果应该不足为奇。如果使用很少的元素，则所有数据都适合 L1d，并且每个列表元素的访问时间仅为 3 个周期。对于列表元素的几乎所有排列都是如此：虚拟地址很好地映射到 L1d 插槽，几乎没有冲突。有两个（在此图中）特殊距离值的情况有所不同。如果距离是 4096 字节的倍数（即 64 个元素的距离）并且列表的长度大于 8，则每个列表元素的平均循环数会急剧增加。在这些情况下，所有条目都在同一集合中，一旦列表长度大于关联性，条目就会从 L1d 中刷新，并且必须在下一轮中从 L2 重新读取。这导致每个列表元素大约需要 10 个周期的成本。

通过该图，我们可以确定所使用的处理器具有关联性为 8 的 L1d 缓存，总大小为 32kB。这意味着如有必要，可以使用测试来确定这些值。对于 L2 高速缓存也可以测量到相同的效果，但这里的情况更为复杂，因为 L2 高速缓存是使用物理地址进行索引的，而且它要大得多。

程序员希望将这些数据视为集合关联性值得关注的一个指标。在现实世界中，将数据布置在二次幂的边界上的情况经常发生，但这正是很容易导致上述情况的情况

影响和性能下降。未对齐的访问会增加冲突未命中的可能性，因为每次访问可能需要额外的缓存行。



图 6.6: AMD 上 L1d 的 Bank 地址

如果执行此优化，则也可以进行其他相关优化。至少 AMD 的处理器将 L1d 实现为多个单独的存储体。L1d 每个周期可以接收两个数据字，但前提是两个字都存储在不同的存储体中或存储在具有相同索引的存储体中。组地址被编码在虚拟地址的低位中，如图 6.6 所示。如果一起使用的变量也存储在一起，那么它们位于不同库或具有相同索引的同一库的可能性很高。

6.2.2 优化 1 级指令缓存访问

为良好的 L1i 使用准备代码需要与良好的 L1d 使用类似的技术。但问题是，程序员通常不会直接影响 L1i 的使用方式，除非他/她用汇编语言编写代码。如果使用编译器，程序员可以通过指导编译器创建更好的代码布局来间接确定 L1i 的使用。

代码的优点是跳转之间是线性的。在这些时期，处理器可以有效地预取内存。跳跃扰乱了这张漂亮的图片，因为

- 跳跃目标可能不是静态确定的；

- 即使它是静态的，如果错过所有缓存，内存获取也可能需要很长时间。

这些问题会造成执行停滞，并可能严重影响性能。这就是为什么当今的处理器在分支预测（BP）上投入巨资的原因。高度专业化的 BP 单元尝试尽可能早地确定跳转目标，以便处理器可以开始将新位置的指令加载到高速缓存中。他们使用静态和动态规则，并且越来越擅长确定执行模式。

尽快将数据放入缓存对于指令缓存来说更为重要。正如第 3.1 节中提到的，指令在执行之前必须进行解码，为了加快速度（在 x86 和 x86-64 上很重要），指令实际上以解码的形式缓存，而不是以从内存读取的字节/字形式缓存。

为了实现 L1i 的最佳使用，程序员应该至少注意代码生成的以下方面：

- 1. 尽可能减少代码占用。这必须与循环展开和内联等优化相平衡。
- 2. 代码执行应该是线性的，没有气泡。³¹
- 3. 在有意义时对齐代码。

现在我们将研究一些可用于根据这些方面优化程序的编译器技术。

编译器可以选择启用优化级别；还可以单独启用特定的优化。许多在高优化级别启用的优化（gcc 的 -O2 和 -O3）处理循环优化和函数内联。一般来说，这些都是很好的优化。如果以这些方式优化的代码占程序总执行时间的很大一部分，则可以提高整体性能。特别是，函数内联允许编译器一次优化更大的代码块，从而能够生成更好地利用处理器管道架构的机器代码。当程序的较大部分可以被视为单个单元时，代码和数据的处理（通过死代码消除或值范围传播等）效果更好。

代码大小越大意味着 L1i（以及 L2 和更高级别）缓存的压力越大。此 *can* 会导致性能下降。较小的代码可以更快。幸运的是 gcc 有一个优化选项来指定这一点。如果使用 -Os，编译器将优化代码大小。优化

³¹Bubbles describe graphically the holes in the execution in the pipeline of a processor which appear when the execution has to wait for resources. For more details the reader is referred to literature on processor design.

已知会增加代码大小的功能已被禁用。使用此选项通常会产生令人惊讶的结果。特别是如果编译器不能真正利用循环展开和内联，那么这个选项是一个巨大的胜利。

内联也可以单独控制。编译器具有指导内联的启发式和限制；这些限制可以由程序员控制。-finline-limit 选项指定函数必须有多大才被视为内联太大。如果在多个位置调用一个函数，则将其内联到所有位置会导致代码大小爆炸。但还有更多。假设函数 *inlcand* 在两个函数 *f1* 和 *f2* 中被调用。函数 *f1* 和 *f2* 本身按顺序调用。

start f1	start inlcand
code f1	code inlcand
inlined inlcand	end inlcand
more code f1	
end f1	start f1
	code f1
start f2	end f1
code f2	
inlined inlcand	start f2
more code f2	code f2
end f2	end f2

表 6.3: 内联与非内联

表 6.3 显示了在两个函数中没有内联和内联的情况下生成的代码的样子。如果函数 *inlcand* 内联在 *f1* 和 *f2* 中，则生成的代码的总大小为 *size f1* + *size f2* + 2 × *size inlcand*。如果没有发生内联，则总大小会小于 *inlcand* 的大小。如果 *f1* 和 *f2* 很快被调用，则需要多少 L1i 和 L2 缓存。另外：如果 *inlcand* 没有内联，代码可能仍然在 L1i 中，并且不必再次解码。另外：分支预测单元可能会更好地预测跳转，因为它已经看到了代码。如果编译器默认的内联函数大小上限对于程序来说不是最佳选择，则应降低该上限。

不过，在某些情况下，内联总是有意义的。如果一个函数只被调用一次，那么它也可能被内联。这使编译器有机会执行更多优化（例如值范围传播，这可能会显着改进代码）。这种内联可能会受到选择限制的阻碍。对于这种情况，gcc 有一个选项来指定函数始终是内联的。添加 *always_inline* 函数属性会指示编译器完全按照名称所暗示的那样进行操作。

在相同的上下文中，如果一个函数尽管足够小但永远不应该被内联，则可以使用 *noinline* 函数属性。即使对于小函数，如果经常从不同的地方调用这些函数，使用此属性也是有意义的。

地方。如果 L1i 内容可以重用并且总体占用空间减少，这通常可以弥补额外函数调用的额外成本。如今分支预测单元非常好。如果内联可以导致更积极的优化，那么情况就会有所不同。这是必须根据具体情况来决定的事情。

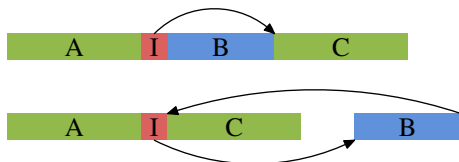
如果始终使用内联代码，则 `always_inline` 属性效果很好。但如果情况并非如此呢？如果内联函数只是偶尔被调用怎么办：

```
void fct(void) { ... 代码块 A ...  
if (条件) infct() ... 代码块 C ... }
```

为这样的代码序列生成的代码通常与源的结构匹配。这意味着首先是代码块 A，然后是条件跳转，如果条件计算结果为 false，则向前跳转。接下来是为内联 `infct` 生成的代码，最后是代码块 C。这看起来很合理，但有一个问题。

如果条件经常为假，则执行不是线性的。中间有一大块未使用的代码，这不仅会由于预取而污染 L1i，还会导致分支预测出现问题。如果分支预测错误，则条件表达式的效率可能会非常低。

这是一个普遍问题，并不是内联函数特有的。每当使用条件执行并且它是不平衡的（即，表达式更经常导致一个结果而不是另一个结果）时，就有可能出现错误的静态分支预测，从而在管道中出现气泡。可以通过告诉编译器将不常执行的代码移出主代码路径来防止这种情况。在这种情况下，为 if 语句生成的条件分支将跳转到不按顺序的位置，如下图所示。



上半部分代表简单的代码布局。如果区域 B，例如从上面的内联函数 `infct` 生成的，通常不会被执行，因为条件 I 跳过它，处理器的预取将拉入包含很少使用的块 B 的缓存行。使用块重新排序可以改变这一点，结果可以在图的下半部分看到。这

经常执行的代码在内存中是线性的，而很少执行的代码则被移动到不会损害预取和 L1i 效率的地方。

gcc 提供了两种方法来实现这一点。首先，编译器可以在重新编译代码时考虑分析输出，并根据分析来布局代码块。我们将在第 7 节中看到它是如何工作的。第二种方法是通过显式分支预测。gcc 识别 `__builtin_expect`：

```
长 __builtin_expect(长 EXP, 长 C);
```

此构造告诉编译器表达式 EXP 最有可能具有值 C。返回值是 EXP。 `__builtin_expect` 用于条件表达式。在几乎所有情况下，它都会在布尔表达式的上下文中使用，在这种情况下，定义两个辅助宏更方便：

```
#定义不太可能(expr) __builtin_expect(!(expr), 0) #定义可能(expr) __builtin_expect(!(expr), 1)
```

然后可以使用这些宏，如下所示

```
如果（可能（a > 1））
```

如果程序员使用这些宏，然后使用 `-freorder-blocks` 优化选项，gcc 将重新排序块，如上图所示。此选项通过 `-O2` 启用，但通过 `-Os` 禁用。还有另一个 gcc 选项可以重新排序块（`-freorder-blocks-and-partition`），但它的用处有限，因为它不适用于异常处理。

小循环还有另一个很大的优点，至少在某些处理器上是这样。Intel Core 2 前端有一个称为循环流检测器 (LSD) 的特殊功能。如果一个循环不超过 18 条指令（其中没有一条是对子例程的调用），只需要最多 4 次解码器读取 16 字节，最多有 4 个分支指令，并且执行次数超过 64 次，则该循环有时会被锁定在指令队列中，因此在再次使用该循环时可以更快地使用。例如，这适用于通过外循环多次进入的小内循环。即使没有这种专门的硬件，紧凑循环也有优势。

内联并不是 L1i 优化的唯一方面。另一个方面是对齐，就像数据一样。有明显的区别：代码大部分是线性的 blob，不能任意放置在地址空间中，并且在编译器生成代码时不能直接受到程序员的影响。不过，有些方面是程序员可以控制的。

对齐每条指令没有任何意义。目标是使指令流是顺序的。因此，协调只有在战略位置才有意义。为了决定在哪里添加对齐，有必要了解其优点是什么。在缓存行³²的开头有一条指令意味着缓存行的预取被最大化。对于指令来说，这也意味着解码器更有效。很容易看出，如果执行高速缓存行末尾的指令，处理器必须准备好读取新的高速缓存行并解码指令。有些事情可能会出错（例如缓存行未命中），这意味着缓存行末尾的指令平均执行效率不如开头的指令。

将此与后续推论相结合，如果控制权只是转移到有问题的指令（因此预取无效），那么问题是最严重的，我们得出最终结论，其中代码对齐最有用：

- 在函数的开头；
- 在只能通过跳跃到达的基本块的开头；
- 在某种程度上，在循环的开始。

在前两种情况下，对齐的成本很小。执行在新的位置继续进行，如果我们选择将其放置在缓存行的开头，我们会优化预取和解码。³³编译器通过插入一系列无操作指令来填充对齐代码所产生的间隙来完成这种对齐。这种“死代码”占用一点空间，但通常不会损害性能。

第三种情况略有不同：对齐每个循环的开头可能会产生性能问题。问题是循环的开始通常按顺序跟随其他代码。如果情况不太幸运，前一条指令和对齐的循环开头之间将会存在间隙。与前两种情况不同，这个缺口不可能完全消失。执行完前一条指令后，必须执行循环中的第一条指令。这意味着，在前一条指令之后，要么必须有许多无操作指令来填补间隙，要么必须有一个无条件跳转到循环的开头。这两种可能性都不是免费的。特别是如果循环本身不经常执行，则通过对齐循环，无操作或跳转可能会花费不止一次保存。

程序员可以通过三种方式影响代码的对齐。显然，如果代码写成

³²For some processors cache lines are not the atomic blocks for instructions. The Intel Core 2 front end issues 16 byte blocks to the decoder. They are appropriately aligned and so no issued block can span a cache line boundary. Aligning at the beginning of a cache line still has advantages since it optimizes the positive effects of prefetching.

³³For instruction decoding processors often use a smaller unit than cache lines, 16 bytes in case of x86 and x86-64.

汇编程序中的函数和其中的所有指令都可以显式对齐。汇编器为所有体系结构提供 `.align` 伪操作来执行此操作。对于高级语言，必须告知编译器对齐要求。与数据类型和变量不同，这在源代码中是不可能的。相反，使用编译器选项：

`-falign-函数=N`

此选项指示编译器将所有函数对齐到下一个大于 `N` 的二次方边界。这意味着会创建最多 `N` 个字节的间隙。对于小函数，使用较大的 `N` 值是一种浪费。对于很少执行的代码也是如此。后者在包含流行和不那么流行的界面的库中经常发生。明智地选择选项值可以通过避免对齐来加快速度或节省内存。通过使用 `1` 作为 `N` 值或使用 `-fno-align-functions` 选项来关闭所有对齐。

上述第二种情况的对齐（未按顺序到达的基本块的开始）可以使用不同的选项进行控制：

`-falign-jumps=N`

所有其他细节都是相同的，关于内存浪费的相同警告也适用。

第三种情况也有自己的选择：

`-falign-loops=N`

同样的细节和警告同样适用。除此之外，如前所述，对齐会产生运行时成本，因为如果顺序到达对齐地址，则必须执行无操作或跳转指令。

gcc 知道另一种控制对齐的选项，这里提到它只是为了完整性。`-falign-labels` 对齐代码中的每个标签（基本上是每个基本块的开头）。除了少数特殊情况外，这会减慢代码速度，因此不应使用。

6.2.3 优化 2 级及更高级别缓存访问

关于 1 级缓存优化的所有内容也适用于 2 级及更高级别的缓存访问。最后一级缓存还有两个额外的方面：

- 缓存未命中总是非常昂贵的。虽然 L1 未命中（希望如此）频繁地命中 L2 和更高级别的缓存，从而限制了惩罚，但最后一级缓存显然没有回退措施。

- L2 及更高级别的缓存通常由多个内核和/或超线程共享。因此，每个执行单元可用的有效高速缓存大小通常小于总高速缓存大小。

为了避免高速缓存未命中的高成本，工作集大小应与高速缓存大小相匹配。如果只需要一次数据，这显然是没有必要的，因为无论如何缓存都是无效的。我们讨论的是多次需要数据集的工作负载。在这种情况下，使用太大而无法放入高速缓存的工作集将导致大量高速缓存未命中，即使预取成功执行，也会减慢程序速度。

即使数据集太大，程序也必须执行其工作。程序员的工作就是以最小化缓存未命中的方式完成工作。对于最后一级缓存，这是可能的——就像对于一级缓存一样——通过以较小的片段来完成工作。这与第 50 页上的优化矩阵乘法非常相似。不过，一个区别是，对于最后一级缓存，所处理的数据块可以更大。如果还需要 L1 优化，代码会变得更加复杂。想象一下矩阵乘法，其中数据集（两个输入矩阵和输出矩阵）不能一起放入最后一级缓存。在这种情况下，同时优化 L1 和最后一级缓存访问可能是合适的。

L1 高速缓存行大小通常在许多代处理器中保持不变；即使不是，差异也会很小。假设更大的尺寸也没有什么大问题。在高速缓存大小较小的处理器上，将使用两个或多个高速缓存行而不是一个。无论如何，对缓存行大小进行硬编码并为其优化代码是合理的。

对于更高级别的缓存，如果程序应该是通用的，则情况并非如此。这些缓存的大小可能相差很大。八个或更多的因数并不少见。不可能假设较大的缓存大小为默认值，因为这意味着代码在除具有最大缓存的机器之外的所有机器上性能都很差。相反的选择也很糟糕：假设最小的缓存意味着丢弃 87% 或更多的缓存。这很糟糕；从图 3.14 中我们可以看到，使用大缓存会对程序的速度产生巨大的影响。

这意味着代码必须动态调整自身以适应缓存行大小。这是针对该程序的优化。我们在这里只能说，程序员应该正确计算程序的需求。不仅需要数据集本身，更高级别的缓存也可用于其他目的；例如，所有执行的指令都是从缓存加载的。如果使用库函数，则此缓存使用量可能会达到相当大的数量。这些库函数可能还需要自己的数据，这进一步减少了可用内存。

一旦我们有了内存需求的公式，我们就可以将其与缓存大小进行比较。如前所述，缓存可能与多个其他核心共享。目前³⁴无需硬编码知识即可获取正确信息的唯一方法是通过 /sys 文件系统。在表 5.2 中我们看到了内核发布的有关硬件的内容。程序必须找到目录：

```
/sys/devices/system/cpu/cpu*/缓存
```

用于最后一级缓存。这可以通过该目录中级别文件中的最高数值来识别。当目录被识别时，程序应该读取该目录中大小文件的内容，并将数值除以文件 `shared_cpu_map` 中位掩码中设置的位数。

以这种方式计算出的值是安全下限。有时，程序对其他线程或进程的行为了解更多。如果这些线程被调度在共享缓存的核心或超线程上，并且已知缓存使用量不会耗尽总缓存大小的一部分，则计算出的限制可能太低而无法实现最佳。是否应该使用超过公平份额的部分实际上取决于具体情况。程序员必须做出选择或者必须允许用户做出决定。

6.2.4 优化 TLB 使用

TLB 使用的优化有两种。第一个优化是减少程序必须使用的页面数量。这会自动减少 TLB 未命中的情况。第二个优化是通过减少必须分配的高级目录表的数量来使 TLB 查找更便宜。表越少意味着使用的内存越少，这可以导致目录查找的缓存命中率更高。

第一个优化与页面错误的最小化密切相关。我们将在 7.5 节中详细介绍该主题。虽然页面错误通常是一次性成本，但考虑到 TLB 缓存通常很小并且经常刷新，TLB 未命中是一种永久的惩罚。页错误比 TLB 未命中的成本要高几个数量级，但是，如果程序运行的时间足够长并且程序的某些部分执行得足够频繁，则 TLB 未命中的成本甚至可能超过页错误成本。因此，不仅从页面错误的角度考虑页面优化，而且从 TLB 未命中的角度考虑页面优化也很重要。不同之处在于，页错误优化仅需要对代码和数据进行页范围分组，而 TLB 优化则要求在任何时间点使用尽可能少的 TLB 条目。

第二个 TLB 优化更难控制。必须使用的页目录的数量取决于进程的虚拟地址空间中使用的地址范围的分布。地址空间中不同的位置意味着更多的目录。

³⁴There definitely will sometime soon be a better way!

复杂的是地址空间布局随机化 (ASLR) 正是导致这些情况。堆栈、DSO、堆和可能的可执行文件的加载地址在运行时被随机化, 以防止机器的攻击者猜测函数或变量的地址。

仅当最大性能至关重要时, 才应关闭 ASLR。额外目录的成本足够低, 除了少数极端情况外, 在所有情况下都不需要执行此步骤。内核可以随时执行的一种可能的优化是确保单个映射不会跨越两个目录之间的地址空间边界。这将以最小的方式限制 ASLR, 但不足以大幅削弱它。

程序员直接受此影响的唯一方式是显式请求地址空间区域时。当使用 `mmap` 和 `MAP_FIXED` 时会发生这种情况。以这种方式分配新的地址空间区域是非常危险的, 而且很少有人这样做。不过, 这是可能的, 并且, 如果使用它并且可以自由选择地址, 那么程序员应该了解最后一级页目录的边界并适当地选择所请求的地址。

6.3 预取

预取的目的是隐藏内存访问的延迟。当今处理器的命令管道和无序 (OOO) 执行功能可以隐藏一些延迟, 但最多只能针对命中缓存的访问。为了弥补主内存访问的延迟, 命令队列必须非常长。一些没有 OOO 的处理器尝试通过增加内核数量来进行补偿, 但这是一个糟糕的交易, 除非所有使用的代码都是并行的。

预取可以进一步帮助隐藏延迟。处理器自行执行预取, 由某些事件触发 (硬件预取) 或由程序明确请求 (软件预取)。

6.3.1 硬件预取

CPU 启动硬件预取的触发因素通常是按特定模式出现的一系列两次或多次高速缓存未命中。这些缓存未命中可能是针对后续或之前的缓存行。在旧的实现中, 仅识别相邻高速缓存线的高速缓存未命中。对于现代硬件, 步幅也可以被识别, 这意味着跳过固定数量的缓存行被识别为一种模式并进行适当的处理。

如果每次缓存未命中都会触发硬件预取, 则会对性能产生不利影响。随机内存访问模式 (例如全局变量) 非常常见, 并且由此产生的预取大部分会浪费 FSB 带宽。这就是为什么要启动预取, 至少需要两次缓存未命中。当今的处理器

所有人都期望存在多个内存访问流。处理器尝试自动将每个缓存未命中分配给此类流, 如果达到阈值, 则启动硬件预取。如今的 CPU 可以跟踪 8 到 16 个独立的流以用于更高级别的缓存。

负责模式识别的单元与各自的缓存相关联。L1d 和 L1i 高速缓存可以有一个预取单元。L2 缓存及更高级别的缓存很可能有一个预取单元。L2 和更高预取单元与使用相同缓存的所有其他内核和超线程共享。因此, 八到十六个独立流的数量很快减少。

预取有一个很大的弱点: 它不能跨越页面边界。当人们认识到 CPU 支持请求调页时, 原因应该是显而易见的。如果允许预取器跨越页面边界, 则访问可能会触发操作系统事件以使页面可用。这本身可能很糟糕, 尤其是对于性能而言。更糟糕的是预取器不知道程序或操作系统本身的语义。因此, 它可能会预取在现实生活中永远不会被请求的页面。这意味着预取器将运行到处理器之前以可识别模式访问的内存区域的末尾。这不仅有可能, 而且很有可能。如果作为预取的副作用, 处理器触发了对此类页面的请求, 那么如果此类请求永远不会发生, 那么操作系统甚至可能完全脱离轨道。

因此, 重要的是要认识到, 无论预取器在预测模式方面有多好, 程序都会在页面边界处遇到缓存未命中, 除非它显式预取或从新页面读取。这是优化数据布局的另一个原因, 如第 6.2 节所述, 通过保留不相关的数据来最大程度地减少缓存污染。

由于页面限制, 当今的处理器没有非常复杂的逻辑来识别预取模式。由于 4k 页面大小仍然占主导地位, 所以有意义的就这么多了。多年来, 识别跨度的地址范围一直在增加, 但超出当今常用的 512 字节窗口可能没有多大意义。目前预取单元无法识别非线性访问模式。这些模式很可能是真正随机的, 或者至少是足够不重复的, 因此尝试识别它们是没有意义的。

如果硬件预取被意外触发, 那么我们能做的就只有这么多了。一种可能性是尝试检测此问题并稍微更改数据和/或代码布局。这可能会被证明是困难的。可能有特殊的本地化解决方案, 例如在 x86 和 x86-64 处理器上使用 `ud2` 指令³⁵。该指令本身不能执行, 用在间接跳转指令之后; 它用作指令的信号

³⁵Or non-instruction. It is the recommended undefined opcode.

fetcher 告诉处理器不应浪费精力解码后续内存，因为执行将在不同的位置继续。不过，这是一个非常特殊的情况。在大多数情况下，人们必须忍受这个问题。

可以完全或部分禁用整个处理器的硬件预取。在 Intel 处理器上，为此使用模型特定寄存器 (MSR) (IA32_MISC_ENABLE，许多处理器上的位 9；位 19 仅禁用相邻缓存行预取)。在大多数情况下，这必须在内核中发生，因为它是特权操作。如果分析显示系统上运行的重要应用程序由于硬件预取而遭受带宽耗尽和过早缓存驱逐，则可以使用此 MSR。

6.3.2 软件预取

硬件预取的优点是程序不必调整。正如刚才所描述的，缺点是访问模式必须很简单，并且预取不能跨页边界发生。由于这些原因，我们现在有了更多的可能性，其中最重要的是软件预取。软件预取确实需要通过插入特殊指令来修改源代码。一些编译器支持编译指示以或多或少地自动插入预取指令。在 x86 和 x86-64 上，通常使用英特尔编译器内部函数插入这些特殊指令的约定：

```
#include <xmmintrin.h>
enum _mm_hint
{
    _MM_HINT_T0 = 3,
    _MM_HINT_T1 = 2,
    _MM_HINT_T2 = 1,
    _MM_HINT_NTA = 0
};
void _mm_prefetch(void *p,
                  enum _mm_hint h);
```

程序可以在程序中的任何指针上使用 `_mm_prefetch` 内在函数。大多数处理器（当然是所有 x86 和 x86-64 处理器）都会忽略无效指针导致的错误，这使程序员的工作变得更加轻松。如果传递的指针引用有效内存，则将指示预取单元将数据加载到缓存中，并在必要时逐出其他数据。绝对应该避免不必要的预取，因为这可能会降低缓存的有效性并消耗内存带宽（可能是两个缓存行，以防被逐出的缓存行脏）。

与 `_mm_prefetch` 内在函数一起使用的不同提示是实现定义的。这意味着每个处理器版本可以以（略有）不同的方式实现它们-

完全地。一般来说，`_MM_HINT_T0` 将数据提取到所有级别的缓存（对于包含式缓存）和最低级别缓存（对于独占式缓存）。如果数据项位于更高级别的缓存中，则会将其加载到 L1d 中。`_MM_HINT_T1` 提示将数据拉入 L2，而不是拉入 L1d。如果有 L3 缓存，`_MM_HINT_T2` 提示可以为其执行类似的操作。不过，这些细节还没有明确规定，需要针对实际使用的处理器进行验证。一般来说，如果要立即使用数据，则使用 `_MM_HINT_T0` 是正确的做法。当然这需要 L1d 缓存大小足够大以容纳所有预取的数据。如果立即使用的工作集的大小太大，则将所有内容预取到 L1d 中是一个坏主意，应该使用其他两个提示。

第四个提示，`_MM_HINT_NTA`，允许告诉处理器特殊对待预取的缓存行。NTA 代表非时间对齐，我们已经在 6.1 节中解释过。该程序告诉处理器，应尽可能避免该数据污染缓存，因为该数据仅使用很短的时间。因此，处理器可以在加载时避免将数据读入较低级高速缓存以实现包容性高速缓存实现。当数据从 L1d 逐出时，数据不需要被推送到 L2 或更高级别，而是可以直接写入内存。如果给出此提示，处理器设计人员可能还可以采用其他技巧。程序员必须小心使用此提示：如果直接工作集大小太大并强制逐出加载了 NTA 提示的缓存行，则会从内存中重新加载。

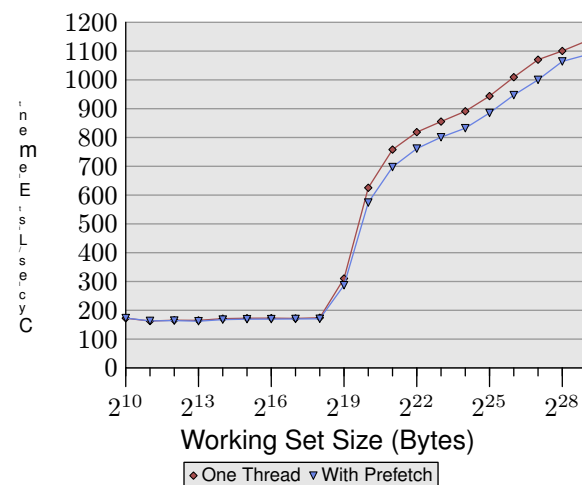


图 6.7: 预取平均值，NPAD=31

图 6.7 显示了使用现在熟悉的指针追踪框架进行测试的结果。该列表在内存中随机排列。与之前测试的不同之处在于，程序实际上在每个列表节点上花费了一些时间（大约 160 个周期）。从图 3.15 中的数据可以看出，一旦工作集大小大于最后一级缓存，程序的性能就会受到严重影响。

现在我们可以尝试通过在计算之前发出预取请求来改善这种情况。即，在循环的每一轮中，我们预取一个新元素。必须仔细选择列表中预取节点和当前正在处理的节点之间的距离。假设每个节点在 160 个周期内处理，并且我们必须预取两个缓存行 (NPA D=31)，则五个列表元素的距离就足够了。

图 6.7 中的结果表明预取确实有帮助。只要工作集大小不超过最后一级缓存的大小（机器有 512kB = 2¹⁹B 的 L2），数字就相同。预取指令不会增加可测量的额外负担。一旦超过 L2 大小，预取就会节省 50 到 60 个周期，最高可达 8%。使用预取并不能隐藏所有的缺点，但它确实有一点帮助。

AMD 在其 Opteron 系列 10h 系列中实现了另一条指令：prefetchw。迄今为止，该指令在 Intel 方面还没有等效指令，并且无法通过内在函数获得。prefetchw 指令告诉 CPU 将缓存行预取到 L1，就像其他预取指令一样。不同之处在于缓存行立即进入“M”状态。如果稍后没有写入缓存行，这将是一个缺点。如果有一个或多个写入，它们将被加速，因为写入不必改变缓存状态——这是在预取缓存行时发生的。这对于竞争的高速缓存行尤其重要，其中对另一个处理器高速缓存中的高速缓存行的简单读取将首先将两个高速缓存中的状态更改为“S”。

预取比我们在这里实现的区区 8% 具有更大的优势。但众所周知，要做到正确是非常困难的，特别是如果同一个二进制文件应该在各种机器上表现良好的话。CPU 提供的性能计数器可以帮助程序员分析预取。可以计数和采样的事件包括硬件预取、软件预取、有用/已用软件预取、各个级别的缓存未命中等等。在第 7.1 节中，我们将介绍一些此类事件。所有这些计数器都是特定于机器的。

在分析程序时，应该首先查看缓存未命中情况。当找到大量高速缓存未命中源时，应尝试为有问题的内存访问添加预取指令。这应该一次在一个地方完成。应通过观察测量有用预取指令的性能计数器来检查每次修改的结果。如果这些计数器没有增加，预取可能是错误的，没有足够的时间从内存加载，或者预取从高速缓存中逐出仍然需要的内存。

现在的 gcc 能够在一种情况下自动发出预取指令。如果循环遍历数组，则可以使用以下选项：

-f预取循环数组

编译器将确定预取是否有意义，如果有意义，它应该向前看多远。对于小数组，这可能是一个缺点，如果编译时不知道数组的大小，结果可能会更糟。gcc 手册警告说，好处很大程度上取决于代码的形式，在某些情况下，代码实际上可能运行得更慢。程序员必须谨慎使用此选项。

6.3.3 特殊类型的预取：推测

现代处理器的 OOO 执行能力允许在指令相互不冲突的情况下移动指令。举例（本次以 IA-64 为例）：

```
st8      [r4] = 12
add      r5 = r6, r7;;
st8      [r18] = r5
```

该代码序列将 12 存储在寄存器 r4 指定的地址处，将寄存器 r6 和 r7 的内容相加，并将其存储在寄存器 r5 中。最后，它将总和存储在寄存器 r18 指定的地址处。这里的要点是，add 指令可以在第一条 st8 指令之前或同时执行，因为不存在数据依赖性。但是，如果必须加载其中一个加数，会发生什么情况呢？

```
st8      [r4] = 12
ld8      r6 = [r8];;
add      r5 = r6, r7;;
st8      [r18] = r5
```

额外的 ld8 指令从寄存器 r8 指定的地址加载值。该加载指令和后面的添加指令之间存在明显的依赖关系（这就是指令后面有;;的原因，感谢提问）。这里关键的是新的 ld8 指令（与 add 指令不同）不能移到第一个 st8 之前。处理器在指令解码期间无法足够快地确定存储和加载是否冲突，即 r4 和 r8 是否可能具有相同的值。如果它们确实具有相同的值，则 st8 指令将确定加载到 r6 中的值。更糟糕的是，如果负载错过缓存，ld8 还可能带来很大的延迟。IA-64 架构支持这种情况下的推测负载：

```
ld8.a r6 = [r8]; [...其他说明...] st8 [r4] = 12
```

```
ld8.c.clr r6 = [r8];;添加 r5 = r6, r7;;
st8 [r18] = r5
```

新的 ld8.a 和 ld8.c.clr 指令属于同一组，并取代先前代码序列中的 ld8 指令。ld8.a 指令是推测负载。该值不能直接使用，但处理器可以开始工作。当到达 ld8.c.clr 指令时，内容可能已经被加载（假定间隙中有足够数量的指令）。该指令的参数必须与 ld8.a 指令的参数匹配。如果前面的 st8 指令没有覆盖该值（即 r4 和 r8 相同），则无需执行任何操作。推测负载完成其工作，并且负载的延迟被隐藏。如果存储和加载确实发生冲突，ld8.c.clr 会从内存中重新加载值，我们最终会得到正常 ld8 指令的语义。

投机负载尚未（尚未？）广泛使用。但正如示例所示，这是隐藏延迟的一种非常简单而有效的方法。预取基本上是等效的，并且对于寄存器很少的处理器来说，推测加载可能没有多大意义。推测加载具有（有时很大）优势，即将值直接加载到寄存器中，而不是加载到可能会再次被逐出的高速缓存行中（例如，当线程被取消调度时）。如果推测可用，则应使用它。

6.3.4 辅助线程

当尝试使用软件预取时，经常会遇到代码复杂性的问题。如果代码必须迭代数据结构（在我们的例子中是列表），则必须在同一循环中实现两个独立的迭代：执行工作的正常迭代和向前看的第二次迭代以使用预取。这很容易变得足够复杂，很可能会出现错误。

此外，有必要确定未来的展望。太少，内存将无法及时加载。太远了，刚刚加载的数据可能会再次被驱逐。另一个问题是预取指令虽然不会阻塞并等待内存加载，但需要时间。该指令必须被解码，如果解码器太忙（例如，由于编写/生成的代码良好），这可能会很明显。最后，增加了循环的代码大小。这降低了 L1i 效率。如果试图通过连续发出多个预取请求来避免部分成本（如果第二次加载不依赖于第一次加载的结果），则会遇到未完成预取请求数量的问题。

另一种方法是完全分开执行正常操作和预取。使用两个普通线程可以发生这种情况。线程必须 ob-

进行调度，以便预取线程填充两个线程都访问的缓存。有两个特殊的解决方案值得一提：

- 在同一核心上使用超线程（请参见第 29 页）。在这种情况下，预取可以进入 L2（甚至 L1 d）。
- 使用比 SMT 线程更“笨”的线程，SMT 线程除了预取和其他简单操作外什么也不做。这是处理器制造商可能探索的一个选项。

超线程的使用特别有趣。正如我们在第 29 页上看到的，如果超线程执行独立代码，则缓存共享就会出现。相反，如果将一个线程用作预取辅助线程，则这不是问题。相反，由于预加载了最低级缓存，因此这是期望的效果。此外，由于预取线程大多处于空闲状态或等待内存，因此如果其他超线程本身不必访问主内存，则它的正常操作不会受到太大干扰。后者正是预取辅助线程所阻止的。

唯一棘手的部分是确保辅助线程不会提前运行太远。它不能完全污染缓存，以便再次驱逐最旧的预取值。在 Linux 上，使用 futex 系统调用 [7] 可以轻松完成同步，或者使用 POSIX 线程同步原语（成本稍高）来完成同步。

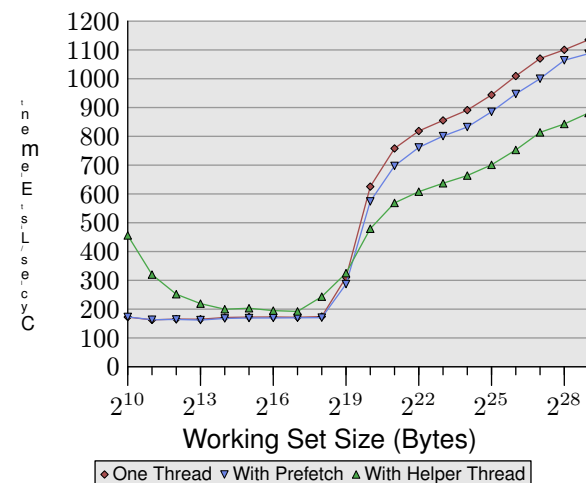


图 6.8：使用辅助线程的平均值，NPAD=31

该方法的优点如图 6.8 所示。这与图 6.7 中的测试相同，只是添加了附加结果。新测试创建了一个额外的辅助线程，该线程提前运行大约 100 个列表条目，并读取（不仅是预取）每个列表元素的所有缓存行。在这种情况下，每个列表元素有两个缓存行（32 位机器上的 NPAD=31 具有 64 字节缓存行大小）。

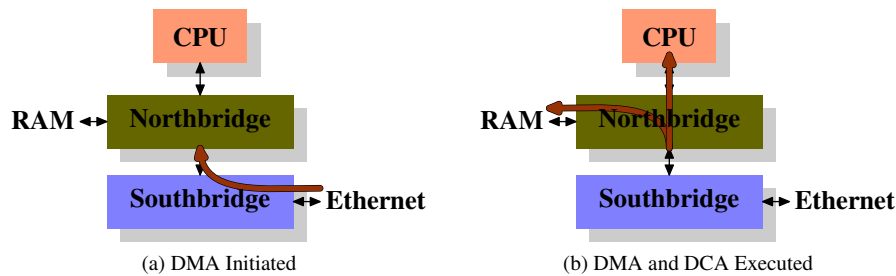


图 6.9: 直接缓存访问

这两个线程被调度在同一核心的两个超线程上。测试机只有一个核心，但如果有多个核心，结果应该大致相同。我们将在第 6.4.3 节中介绍的关联函数用于将线程绑定到适当的超线程。

要确定操作系统知道哪两个（或更多）处理器是超线程，可以使用 libNUMA 中的 NUMA_cpu_level_mask 接口（请参阅附录 D）。

```
#include <libNUMA.h> ssize_t NUMA_cpu_level_mask(size_t destsize,
cpu_set_t *dest, size_t srcsize, const cpu_set_t *src, unsigned int level);
```

该接口可用于确定 CPU 通过高速缓存和内存连接时的层次结构。这里感兴趣的是级别 1，它对应于超线程。要在两个超线程上调度两个线程，可以使用 libNUMA 函数（为简洁起见，删除了错误处理）：

```
cpu_set_t self; NUMA_cpu_self_current_mask(sizeof(self), &self);
cpu_set_t hts; NUMA_cpu_level_mask(sizeof(hts), &hts, sizeof(self), &self, 1);
CPU_XOR(&hts, &hts, &self);
```

执行此代码后，我们有两个 CPU 位集。self 可用于设置当前线程的亲中性，hts 中的掩码可用于设置辅助线程的亲中性。理想情况下，这应该在创建线程之前发生。在 6.4.3 节中我们将介绍设置亲和力的接口。如果没有可用的超线程，NUMA_cpu_level_mask 函数将返回 1。这可以用作避免这种优化的标志。

该基准测试的结果可能会令人惊讶（或者也许不会）。如果工作集适合 L2，则开销

辅助线程的减少会使性能降低 10% 到 60%（主要是在较低端，再次忽略最小工作集大小，噪音太高）。这应该是预料之中的，因为如果所有数据都已在二级缓存中，则预取辅助线程仅使用系统资源而不参与执行。

然而，一旦 L2 大小不够用，图像就会发生变化。预取辅助线程有助于将运行时间减少约 25%。我们仍然看到一条上升的曲线，仅仅是因为预取的处理速度不够快。不过，主线程执行的算术运算和辅助线程的内存加载操作确实是相互补充的。资源冲突极小，从而产生了这种协同效应。

该测试的结果应该可以应用于许多其他情况。超线程通常由于缓存污染而无用，但在这些情况下却大放异彩，应该加以利用。附录 D 中引入的 NUMA 库使查找同级线程变得非常容易（请参阅该附录中的示例）。如果该库不可用，sys 文件系统允许程序查找同级线程（请参阅表 5.3 中的 thread_siblings 列）。一旦该信息可用，程序只需定义线程的亲中性，然后以两种模式运行循环：正常操作和预取。预取的内存量应取决于共享缓存的大小。在此示例中，L2 大小是相关的，程序可以使用以下命令查询大小

```
sysconf (_SC_LEVEL2_CACHE_SIZE)
```

是否必须限制辅助线程的进度取决于程序。一般来说，最好确保存在某种同步，因为调度细节可能会导致性能显著下降。

6.3.5 直接缓存访问

现代操作系统中缓存未命中的来源之一是传入数据流量的处理。现代硬件，如网络接口卡 (NIC) 和磁盘控制器，能够将接收或读取的数据直接写入

内存而不涉及CPU。这对于我们今天拥有的设备的性能至关重要，但它也会引起问题。假设有一个来自网络的传入数据包：操作系统必须通过查看数据包的标头来决定如何处理它。NIC 将数据包放入内存中，然后通知处理器有关到达的信息。处理器没有机会预取数据，因为它不知道数据何时到达，甚至可能不知道数据的确切存储位置。结果是读取标头时缓存未命中。

Intel 在其芯片组和 CPU 中添加了技术来缓解这个问题 [14]。这个想法是用数据包的数据填充 CPU 的缓存，CPU 将收到有关传入数据包的通知。数据包的有效负载在这里并不重要，通常，该数据将由内核或用户级别的更高级别功能处理。数据包标头用于决定数据包的处理方式，因此立即需要该数据。

网络 I/O 硬件已经具有 DMA 来写入数据包。这意味着它直接与可能集成在北桥中的内存控制器通信。内存控制器的另一面是通过 FSB 与处理器的接口（假设内存控制器未集成到 CPU 本身中）。

直接缓存访问 (DCA) 背后的想法是扩展 NIC 和内存控制器之间的协议。在图 6.9 中，第一幅图显示了带有北桥和南桥的常规机器中 DMA 传输的开始。NIC 连接到南桥（或者是南桥的一部分）。它启动 DMA 访问，但提供有关数据包标头的新信息，该信息应推送到处理器的缓存中。

传统的行为是在第二步中简单地通过连接到内存来完成 DMA 传输。对于设置了 DCA 标志的 DMA 传输，北桥另外在带有特殊的新 DCA 标志的 FSB 上发送数据。处理器始终监听 FSB，如果它识别出 DCA 标志，它会尝试将定向到处理器的数据加载到最低缓存中。事实上，DCA 标志是一个提示；处理器可以随意忽略它。DMA 传输完成后，处理器会收到信号。

操作系统在处理数据包时，首先必须确定它是什么类型的数据包。如果不忽略 DCA 提示，操作系统必须执行加载来识别最有可能命中缓存的数据包。将每个数据包数百个周期的节省乘以每秒可处理数万个数据包，节省的金额加起来非常可观，尤其是在延迟方面。

如果没有 I/O 硬件（本例中为 NIC）、芯片组和 CPU 的集成，这样的优化是不可能的。因此，如果需要这项技术，就必须确保明智地选择平台。

6.4 多线程优化

当谈到多线程时，缓存使用的三个不同方面都很重要：

- 并发性
- 原子性
- 带宽

这些方面也适用于多进程情况，但由于多个进程（大部分）是独立的，因此对其进行优化并不那么容易。可能的多进程优化是可用于多线程场景的优化的子集。因此，我们将在这里专门讨论后者。

在这种情况下，并发性是指进程一次运行多个线程时所经历的内存效应。线程的一个属性是它们都共享相同的地址空间，因此都可以访问相同的内存。在理想情况下，线程大多数时间使用的内存区域是不同的，在这种情况下，这些线程仅轻度耦合（例如，公共输入和/或输出）。如果多个线程使用相同的数据，则需要协调；这就是原子性发挥作用的时候。最后，根据机器架构，处理器可用的可用内存和处理器间总线带宽是有限的。我们将在下面的章节中分别处理这三个方面——尽管它们当然是紧密相连的。

6.4.1 并发优化

首先，在本节中，我们将讨论两个实际上需要矛盾优化的独立问题。多线程应用程序在其某些线程中使用公共数据。正常的缓存优化要求将数据保存在一起，以便减少应用程序的占用空间，从而在任何时候最大化适合缓存的内存量。

不过，这种方法存在一个问题：如果多个线程写入内存位置，则每个内核的 L1d 中的缓存行必须处于“E”（独占）状态。这意味着会发送大量 RFO 消息，在最坏的情况下，每次写访问都会发送一条 RFO 消息。所以正常的写入会突然变得非常昂贵。如果使用相同的内存位置，则需要同步（可能通过使用原子操作，这将在下一节中处理）。然而，当所有线程都使用不同的内存位置并且被认为是独立的时候，问题也很明显。

图 6.10 显示了这种“错误共享”的结果。测试程序（如 A.3 节所示）创建了许多线程，这些线程除了增加内存位置（5 亿次）之外什么也不做。测量的时间是从

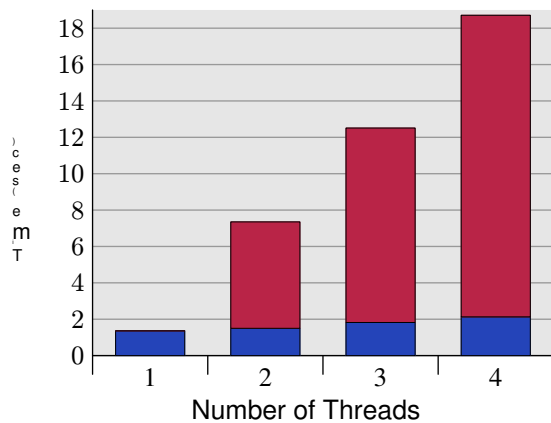


图 6.10: 并发缓存行访问开销

程序开始，直到等待最后一个线程后程序结束。这些线程被固定到各个处理器上。该机有四个P4处理器。蓝色值表示分配给每个线程的内存分配位于单独的缓存行上的运行。红色部分是当线程的位置仅移动到一个缓存行时发生的惩罚。

蓝色测量值（使用单独的缓存线时所需的时间）与人们的预期相符。该程序可以在不影响多个线程的情况下进行扩展。每个处理器将其缓存行保留在自己的L1d中，并且不存在带宽问题，因为无需读取太多代码或数据（事实上，它们都已缓存）。测量到的轻微增加实际上是系统噪声，并且可能是一些预取效应（线程使用顺序缓存行）。

通过将每个线程使用单个缓存行与单独缓存行所需的时间相除计算得出的测量开销分别为390%、734%和1,147%。这些大数字乍一看可能令人惊讶，但当考虑所需的缓存交互时，它应该是显而易见的。高速缓存行在完成对高速缓存行的写入后立即从处理器的高速缓存中取出。除了在任何给定时刻拥有高速缓存行的处理器之外，所有处理器都会被延迟并且无法执行任何操作。每个额外的处理器只会导致更多的延迟。

从这些测量结果可以清楚地看出，在程序中必须避免这种情况。考虑到巨大的损失，这个问题在许多情况下都是显而易见的（至少分析将显示代码位置），但现代硬件存在一个陷阱。图 6.11 显示了在单处理器四核机器（Intel Core 2 QX 6700）上运行代码时的等效测量结果。即使该处理器有两个独立的L2，测试用例也没有显示出任何可扩展性问题。多次使用相同的缓存线时会产生轻微的开销，但它不会随着核心数量的增加而增加。³⁶如果

³⁶I cannot explain the lower number when all four cores are used but it is reproducible.

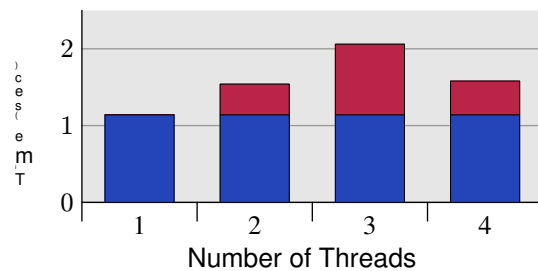


图 6.11: 开销，四核

如果使用了多个处理器，我们当然会看到与图 6.10 类似的结果。尽管多核处理器的使用越来越多，但许多机器将继续使用多个处理器，因此，正确处理这种情况很重要，这可能意味着在真实的 SMP 机器上测试代码。

对于这个问题有一个非常简单的“修复”：将每个变量放在自己的缓存行上。这就是与前面提到的优化发生冲突的地方，具体来说，应用程序的占用空间会增加很多。这是不可接受的；因此，有必要提出更智能的解决方案。

所需要的是确定哪些变量一次仅由一个线程使用，哪些变量曾经仅由一个线程使用，以及哪些变量有时会出现争用。针对每种情况都有不同的解决方案是可能且有用的。区分变量的最基本标准是：它们是否曾经被写入以及这种情况发生的频率。

从未写入的变量和仅初始化一次的变量基本上是常量。由于RFO消息仅用于写操作，因此可以在缓存中共享常量（“S”状态）。因此，不必对这些变量进行特殊处理；将它们组合在一起就好了。如果程序员使用const正确标记变量，工具链会将变量从普通变量移至.rodata（只读数据）或.data.rel.ro（重定位后只读）部分³⁷，无需其他特殊操作。如果由于某种原因，变量不能用const正确标记，程序员可以通过将它们分配到一个特殊的部分来影响它们的位置。

当链接器构造最终的二进制文件时，它首先附加所有输入文件中具有相同名称的部分；然后，这些部分按照链接描述文件确定的顺序排列。这意味着，通过将所有基本上不变但未标记为常量的变量移至特殊部分，程序员可以将所有这些变量组合在一起。它们之间不会有经常写入的变量。通过适当地对齐该部分中的第一个变量，

³⁷Sections, identified by their names are the atomic units containing code and data in an ELF file.

可以保证不出现虚假分享的情况。假设这个小例子：

```
int foo = 1; int bar __attribute__((section(".data.ro"))) = 2; int baz = 3; int xyzzy __attribute__((section(".data.ro"))) = 4;
```

如果编译，该输入文件定义四个变量。有趣的部分是变量 `foo` 和 `baz`、`bar` 和 `xyzzy` 分别分组在一起。如果没有属性定义，编译器将按照源代码中定义的顺序将所有四个变量分配到名为 `.data`³⁸ 的节中。按原样代码，变量 `bar` 和 `xyzzy` 被放置在名为 `.data.ro` 的节中。节名 `.data.ro` 或多或少是任意的。`.data` 前缀。保证 GNU 链接器将该部分与其他数据部分放在一起。

可以应用相同的技术来分离出大部分被读取但偶尔也被写入的变量。只需选择不同的部分名称即可。这种分离在某些情况下（例如 Linux 内核）似乎是有意义的。

如果一个变量仅被一个线程使用，则还有另一种方法来指定该变量。在这种情况下，使用线程局部变量是可能且有用的（参见[8]）。gcc 中的 C 和 C++ 语言允许使用 `__thread` 关键字将变量定义为每个线程。

```
int foo = 1; __thread int bar = 2; int baz = 3; __thread int xyzzy = 4;
```

变量 `bar` 和 `xyzzy` 没有分配在普通数据段中；相反，每个线程都有自己独立的区域来存储此类变量。变量可以有静态初始化器。所有线程局部变量都可以被所有其他线程寻址，但是，除非线程将指向线程局部变量的指针传递给其他线程，否则其他线程无法找到该变量。由于变量是线程局部的，错误共享不是问题——除非程序人为地造成问题。该解决方案很容易设置（编译器和链接器完成所有工作），但它有其成本。当创建线程时，它必须花费一些时间来设置线程局部变量，这需要时间和内存。此外，寻址线程局部变量通常比使用全局变量或自动变量更昂贵（如果可能的话，请参阅[8]了解如何自动最小化成本的说明）。

³⁸This is not guaranteed by the ISO C standard but it is how gcc works.

使用线程本地存储 (TLS) 的另一个缺点是，如果变量的使用转移到另一个线程，则旧线程中变量的当前值对新线程不可用。每个线程的变量副本都是不同的。通常这根本不是问题，如果是的话，转移到新线程需要协调，此时可以复制当前值。

更大的问题是可能浪费资源。如果任何时候只有一个线程使用该变量，则所有线程都必须付出内存方面的代价。如果线程不使用任何 TLS 变量，则 TLS 内存区域的惰性分配可以防止出现问题（应用程序本身中的 TLS 除外）。如果一个线程仅使用 DSO 中的一个 TLS 变量，则该对象中所有其他 TLS 变量的内存也将被分配。如果大规模使用 TLS 变量，这可能会增加。

一般来说，可以给出的最佳建议是

1. 至少将只读（初始化后）和读写变量分开。也许可以将这种分离扩展到以读取为主的变量作为第三类。
2. 将一起使用的读写变量分组到一个结构体中。使用结构是确保所有这些变量的内存位置以所有 gcc 版本一致翻译的方式靠近在一起的唯一方法。
3. 将经常被不同线程写入的读写变量移动到它们自己的缓存行上。这可能意味着在末尾添加填充以填充缓存行的剩余部分。如果与步骤 2 结合起来，这通常并不是真正的浪费。扩展上面的示例，我们最终可能会得到如下代码（假设 `bar` 和 `xyzzy` 一起使用）：

```
int foo = 1;
int baz = 3;
struct {
    struct all {
        int bar;
        int xyzzy;
    };
    char pad[CLSIZE - sizeof(struct all)];
} rwstruct __attribute__((aligned(CLSIZE))) =
{ { .bar = 2, .xyzzy = 4 } };
```

需要进行一些代码更改（对 `bar` 的引用必须替换为 `rwstruct.bar`，对于 `xyzzy` 也是如此），但仅此而已。编译器和链接器会完成剩下的所有工作。³⁹

4. 如果一个变量被多个线程使用，但每次使用都是独立的，则将该变量移入 TLS。

³⁹So far this code has to be compiled with `-fms-extensions` on the command line.

<pre>for (i = 0; i < N; ++i) __sync_add_and_fetch(&var, 1);</pre>	<pre>for (i = 0; i < N; ++i) __sync_fetch_and_add(&var, 1);</pre>	<pre>for (i = 0; i < N; ++i) { long v, n; do { v = var; n = v + 1; } while (!__sync_bool_compare_and_swap(&var, v, n)); }</pre>
1. Add and Read Result	2. Add and Return Old Value	3. Atomic Replace with New Value

图 6.12: 循环中的原子增量

6.4.2 原子性优化

如果多个线程同时修改同一内存位置，处理器不保证任何特定结果。这是一个经过深思熟虑的决定，旨在避免 99.999% 的情况下不必要的成本。例如，如果一个内存位置处于“S”状态，并且两个线程同时必须增加其值，则执行管道不必等待缓存行在“E”状态下可用，然后再从缓存中读取旧值来执行加法。相反，它读取缓存中当前的值，一旦缓存行在状态“E”下可用，新值就会被写回。如果两个线程中的两次缓存读取同时发生，则结果不符合预期；添加的一项将丢失。

对于可能发生并发操作的情况，处理器提供原子操作。例如，这些原子操作不会读取旧值，直到明确可以通过对内存位置的加法显示为原子的方式来执行加法。除了等待其他内核和处理器之外，一些处理器甚至向主板上的其他设备发出特定地址的原子操作信号。所有这些都会使原子操作变慢。

处理器供应商决定提供不同的原子操作集。早期的 RISC 处理器，与简化的“R”一致，提供了很少的原子操作，⁴⁰有时只提供原子位设置和测试。在频谱的另一端，我们有 x86 和 x86-64，它们提供了大量的原子操作。通常可用的原子操作可以分为四类：

位测试 这些操作以原子方式设置或清除位，并返回指示该位之前是否设置过的状态。

加载锁定/条件存储 (LL/SC)⁴¹ LL/SC 操作成对工作，其中特殊加载指令用于启动事务，并且仅当位置在此期间没有被修改时最终存储才会成功。商店操作指示成功或失败，因此程序可以在必要时重复其工作。

比较和交换 (CAS) 这是一个三元操作，仅当当前值与第三个参数值相同时，才会将作为参数提供的值写入地址（第二个参数）；

原子算术 这些运算仅在 x86 和 x86-64 上可用，它们可以在内存位置上执行算术和逻辑运算。这些处理器支持这些操作的非原子版本，但 RISC 架构不支持。因此，它们的可用性有限也就不足为奇了。

架构支持 LL/SC 或 CAS 指令，但不能同时支持两者。两种方法基本上是相同的。它们同样可以很好地实现原子算术运算，但 CAS 似乎是当今的首选方法。所有其他操作都可以使用它间接实现。例如，原子加法：

```
int 曲线; int 纽瓦尔; 做 {curval = var; newval = c
urval + 加数; while (CAS(&var, curval, newval));
```

CAS调用的结果表明操作是否成功。如果返回失败（非零值），则再次运行循环，执行加法，并再次尝试 CAS 调用。重复此操作直至成功。该代码值得注意的是，内存位置的地址必须在两个单独的指令中计算。⁴² 对于 LL/SC，代码看起来大致相同：

```
int 曲线; int 纽瓦尔; 做 {curval = L
L(var); newval = curval + 加数; while
(SC(var, newval));
```

⁴⁰HP Parisc still does not provide more...

⁴¹Some people use “linked” instead of “lock”, it is all the same.

⁴²The CAS opcode on x86 and x86-64 can avoid the load of the value in the second and later iterations but, on this platform, we can write the atomic addition in a simpler way, with a single addition opcode.

这里我们必须使用特殊的加载指令（LL），并且我们不必将内存位置的当前值传递给 SC，因为处理器同时知道内存位置是否已被修改。

最大的区别是 x86 和 x86-64，它们具有原子操作，在这里，选择正确的原子操作以获得最佳结果非常重要。图 6.12 显示了实现原子增量操作的三种不同方法。这三种架构在 x86 和 x86-64 上生成不同的代码，而在其他架构上代码可能相同。存在巨大的性能差异。下表显示了四个并发线程执行 100 万次增量的执行时间。该代码使用 gcc 的内置原语（`__sync_*`）。

1. Exchange Add	2. Add Fetch	3. CAS
0.23s	0.21s	0.73s

前两个数字相似；我们看到返回旧值的速度要快一些。重要的信息是突出显示的字段，即使用 CAS 时的成本。毫不奇怪，它要贵得多。造成这种情况的原因有几个：1. 有两个内存操作，2. CAS 操作本身更复杂，甚至需要条件操作，3. 整个操作必须在循环中完成，以防两个并发访问导致 CAS 调用失败。

现在读者可能会问一个问题：为什么有人会使用利用 CAS 的复杂且更长的代码？答案是：复杂性通常是隐藏的。如前所述，CAS 目前是所有有趣架构中的统一原子操作。所以有些人认为用 CAS 来定义所有原子操作就足够了。这使得程序更加简单。但正如数字所示，结果可能并非最优。CAS 解决方案的内存处理开销是巨大的。下面说明了两个线程的执行，每个线程都有自己的核心。

Thread #1	Thread #2	var Cache State
<code>v = var</code>		'E' on Proc 1
<code>n = v + 1</code>	<code>v = var</code>	'S' on Proc 1+2
<code>CAS(var)</code>	<code>n = v + 1</code>	'E' on Proc 1
	<code>CAS(var)</code>	'E' on Proc 2

我们看到，在这么短的执行时间内，cache line 状态至少发生了 3 次变化；其中两项变更是 RFO。此外，第二个 CAS 将失败，因此该线程必须重复整个操作。在此操作期间，同样的情况可能会再次发生。

相反，当使用原子算术运算时，处理器可以将执行加法（或其他操作）所需的加载和存储操作保留在一起。它可以确保并发发出的缓存行请求被阻塞，直到原子操作完成。

因此，示例中的每次循环迭代最多会产生一个 RFO 缓存请求，而不会产生其他任何结果。

所有这一切意味着，在可以使用原子算术和逻辑运算的级别上定义机器抽象至关重要。CAS 不应该被普遍用作统一机制。

对于大多数处理器来说，原子操作本身始终是原子的。只有在不需要原子性的情况下提供完全独立的代码路径才能避免它们。这意味着更多的代码、条件以及进一步跳转到适当的直接执行。

对于 x86 和 x86-64，情况有所不同：相同的指令可以以原子和非原子方式使用。为了使它们成为原子的，使用了一个特殊的指令前缀：锁前缀。这为原子操作打开了大门，以避免在给定情况下不需要原子性要求时的高成本。例如，库中的代码（如果需要的话始终必须是线程安全的）可以从中受益。编写代码时不需要任何信息，可以在运行时做出决定。诀窍是跳过锁定前缀。这个技巧适用于 x86 和 x86-64 处理器允许带有锁前缀的所有指令。

```
cmpl $0, multiple_threads je 1f 锁
```

```
1: 添加 $1, some_var
```

如果此汇编代码看起来很神秘，请不要担心，它很简单。第一条指令检查变量是否为零。在这种情况下，非零表示有多个线程正在运行。如果该值为零，则第二条指令跳转到标签 1。否则，执行下一条指令。这是棘手的部分。如果 je 指令不跳转，则以 lock 前缀执行 add 指令。否则执行时不带锁前缀。

添加一个潜在昂贵的操作，如条件跳转（在分支预测错误的情况下昂贵）似乎会适得其反。事实上，它可能是：如果大部分时间都在运行多个线程，性能会进一步下降，特别是在分支预测不正确的情况下。但是，如果在很多情况下只使用一个线程，则代码的速度会明显加快。使用 if-then-else 构造的替代方案在这两种情况下都会引入额外的无条件跳转，这可能会更慢。鉴于原子操作的成本约为 200 个周期，因此使用该技巧（或 if-then-else 块）的交叉点非常低。这绝对是一个需要牢记的技巧。不幸的是，这意味着不能使用 gcc 的 `__sync_*` 原语。

当使用许多线程时，并且它们不会通过在不同核心上使用相同的缓存线而导致缓存争用，仍然存在潜在的问题。每个处理器都有一个最大内存带宽，该带宽由该处理器上的所有内核和超线程共享。根据机器架构（例如图 2.1 中的架构），多个处理器可能共享到内存或北桥的同一总线。

处理器内核本身的运行频率是，即使在完美的条件下，全速运行时，与内存的连接也无法在不等待的情况下满足所有加载和存储请求。现在，将可用带宽进一步除以共享北桥连接的核心、超线程和处理器的数量，并行性突然成为一个大问题。高效程序的性能可能会受到可用内存带宽的限制。

图 3.32 显示提高处理器的 FSB 速度会有很大帮助。这就是为什么随着处理器上内核数量的增加，我们也会看到 FSB 速度的提高。尽管如此，如果程序使用大型工作集并且经过充分优化，这还远远不够。程序员必须准备好认识到由于带宽有限而导致的问题。

现代处理器的性能测量计数器可以观察 FSB 争用。在 Core 2 处理器上，NUS_BNR_DRV 事件计算内核因总线未就绪而必须等待的周期数。这表明总线被高度使用，并且从主存储器加载或存储到主存储器所花费的时间甚至比平时更长。Core 2 处理器支持更多事件，可以对特定总线操作进行计数，例如 RFO 或一般 FSB 利用率。在开发过程中研究应用程序可扩展性的可能性时，后者可能会派上用场。如果总线利用率已经接近 1.0，那么可扩展性机会就很小。

如果发现带宽问题，可以采取多种措施。它们有时是矛盾的，因此可能需要进行一些实验。一种解决方案是购买速度更快的计算机（如果有的话）。获得更高的 FSB 速度、更快的 RAM 模块以及可能的处理器本地内存可以（而且可能会）有所帮助。不过，这可能要花很多钱。如果所讨论的程序仅在一台（或几台机器）上需要，那么一次性的硬件费用可能比重新编写程序的费用要低。但总的来说，最好还是按照计划进行工作。

在优化程序代码本身以避免缓存未命中之后，实现更好的带宽利用率的唯一选择是将线程更好地放置在可用内核上。默认情况下，内核中的调度程序会根据自己的策略将线程分配给处理器。在以下情况下可以避免将线程从一个核心移动到另一个核心：

可能的。不过，调度程序并不真正了解工作负载的任何情况。它可以从缓存未命中等情况中收集信息，但这在许多情况下没有多大帮助。

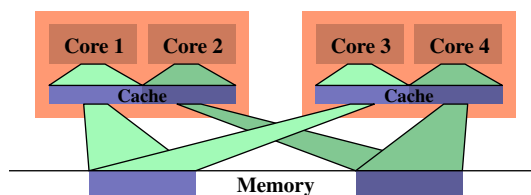


图 6.13：低效的调度

可能导致大量内存总线使用的一种情况是，两个线程被调度在不同的处理器（或不同缓存域中的核心）上并且它们使用相同的数据集。图 6.13 显示了这种情况。核心 1 和核心 3 访问相同的数据（访问指示器和内存区域用相同的颜色表示）。同样，核心 2 和核心 4 访问相同的数据。但线程被调度在不同的处理器上。这意味着每个数据集必须从内存中读取两次。这种情况可以更好地处理。

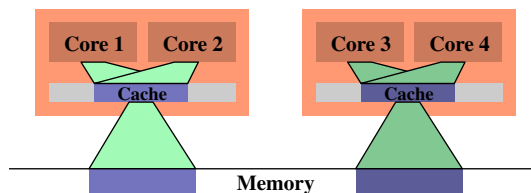


图 6.14：高效的调度

在图 6.14 中，我们看到了理想情况下它应该是什么样子。现在，使用的总缓存大小减少了，因为现在核心 1 和 2 以及核心 3 和 4 处理相同的数据。数据集只需从内存中读取一次。

这是一个简单的示例，但推而广之，它适用于许多情况。如前所述，内核中的调度程序无法洞察数据的使用情况，因此程序员必须确保调度有效完成。没有很多内核接口可用于满足此要求。事实上，只有一个：定义线程亲和性。

线程亲和性意味着将一个线程分配给一个或多个内核。然后，调度程序将（仅）在决定在哪里运行线程时在这些核心中进行选择。即使其他核心空闲，它们也不会被考虑。这听起来像是一个缺点，但这是必须付出的代价。如果太多线程专门在一组核心上运行，则其余核心可能大部分处于空闲状态，除了更改亲和性之外，我们无能为力。默认情况下，线程可以在任何核心上运行。

有许多接口可以查询和更改线程的亲和性：

```
#define _GNU_SOURCE #include <sched.h>
int sched_setaffinity(pid_t pid, size_t 大小, const cpu_set_t *cpuset);
int sched_getaffinity(pid_t pid, size_t 大小, cpu_set_t *cpuset);
```

这两个接口旨在用于单线程代码。pid 参数指定应该更改或确定哪个进程的亲和性。调用者显然需要适当的权限才能执行此操作。第二个和第三个参数指定内核的位掩码。第一个函数需要填充位掩码，以便它可以设置亲和性。第二个用所选线程的调度信息填充位掩码。这些接口在 <sched.h> 中声明。

cpu_set_t 类型也在该标头中定义，以及许多用于操作和使用该类型对象的宏。

```
#define _GNU_SOURCE #include <sched.h>
#define CPU_SETSIZE #define CPU_SET(cpu, cpusetp) #define CPU_CLR(cpu, cpusetp) #define CPU_ZERO(cpusetp) #define CPU_ISSET(cpu, cpusetp) #define CPU_COUNT(cpu, cpusetp)
```

CPU_SETSIZE 指定数据结构中可以表示多少个 CPU。其他三个宏操作 cpu_set_t 对象。应该使用 CPU_ZERO 来初始化一个对象；其他两个宏应该用于选择或取消选择各个内核。CPU_ISSET 测试特定处理器是否是该集合的一部分。CPU_COUNT 返回集合中选择的核数。cpu_set_t 类型为 CPU 数量的上限提供合理的默认值。随着时间的推移，它肯定会显得太小了；此时类型将被调整。这意味着程序必须始终牢记大小。上述便利宏根据 cpu_set_t 的定义隐式处理大小。如果需要更多的动态大小处理，则应使用一组扩展的宏：

```
#define _GNU_SOURCE #include <sched.h>
#define CPU_SET_S(cpu, setsize, cpusetp) #define CPU_CLR_S(cpu, setsize, cpusetp) #define CPU_ZERO_S(setsize, cpusetp) #define CPU_ISSET_S(cpu, setsize, cpusetp) #define CPU_COUNT_S(setsize, cpusetp)
```

这些接口需要一个带有尺寸的附加参数。为了能够动态分配大小的 CPU 集，提供了三个宏：

```
#define _GNU_SOURCE #include <sched.h>
#define CPU_ALLOC_SIZE(count) #define CPU_ALLOC(count) #define CPU_FREE(cpuset)
```

CPU_ALLOC_SIZE 宏的返回值是必须为可以处理 count 个 CPU 的 cpu_set_t 结构分配的字节数。要分配这样的块，可以使用 CPU_ALLOC 宏。以这种方式分配的内存必须通过调用 CPU_FREE 来释放。这些宏可能会在幕后使用 malloc 和 free，但这不一定必须保持这种方式。

最后，定义了一些对 CPU 集对象的操作：

```
#define _GNU_SOURCE #include <sched.h>
#define CPU_EQUAL(cpuset1, cpuset2) #define CPU_AND(destset, cpuset1, cpuset2) #define CPU_OR(destset, cpuset1, cpuset2) #define CPU_XOR(destset, cpuset1, cpuset2) #define CPU_EQUAL_S(setsize, cpuset1, cpuset2) #define CPU_AND_S(setsize, destset, cpuset1, cpuset2) #define CPU_OR_S(setsize, destset, cpuset1, cpuset2) #define CPU_XOR_S(setsize, destset, cpuset1, cpuset2)
```

这两组四个宏可以检查两个集合是否相等并对集合执行逻辑 AND、OR 和 XOR 运算。当使用某些 libNUMA 函数时，这些操作会派上用场（请参阅附录 D）。

进程可以使用 sched_getcpu 接口确定当前在哪个处理器上运行：

```
#define _GNU_SOURCE #include <sched.h>
int sched_getcpu(void);
```

结果就是该 CPU 在 CPU 集中的索引。由于调度的性质，这个数字并不总是 100% 正确。在返回结果和线程返回用户级别之间，线程可能已移动到不同的 CPU。程序总是必须考虑到这种不准确的可能性。无论如何，更重要的是允许线程运行的 CPU 集。可以使用 sched_getaffinity 检索该集合。该集合由子线程和进程继承。线程不能依赖该集合在整个生命周期内保持稳定。亲和性掩码可以从外部设置（请参阅原型中的 pid 参数）

多于)；Linux 还支持 CPU 热插，这意味着 CPU 可以从系统中消失，因此也可以从亲和性 CPU 集中消失。

在多线程程序中，各个线程通常没有 POSIX 定义的进程 ID，因此不能使用上述两个函数。相反，`<pthread.h>` 声明了四个不同的接口：

```
#define _GNU_SOURCE #include <pthread.h> int pthread_setaffinity_np(pthread_t th, size_t 大小, const cpu_set_t *cpuset); int pthread_getaffinity_np(pthread_t th, size_t 大小, cpu_set_t *cpuset); int pthread_attr_setaffinity_np(pthread_attr_t *at, size_t 大小, const cpu_set_t *cpuset); int pthread_attr_getaffinity_np(pthread_attr_t *at, size_t 大小, cpu_set_t *cpuset);
```

前两个接口基本上与我们已经见过的两个接口相同，只是它们在第一个参数中采用线程句柄而不是进程 ID。这允许寻址进程中的各个线程。这也意味着这些接口不能从另一个进程使用，它们严格供进程内使用。第三和第四接口使用线程属性。这些属性在创建新线程时使用。通过设置该属性，可以从一开始就在一组特定的 CPU 上调度线程。这么早就选择目标处理器（而不是在线程已经启动之后）可以在许多不同级别上发挥优势，包括（特别是）内存分配（请参阅第 6.5 节中的 NUMA）。

说到 NUMA，亲和接口在 NUMA 编程中也发挥着重要作用。我们很快就会回到这个案例。

到目前为止，我们已经讨论了两个线程的工作集重叠的情况，使得两个线程位于同一核心上是有意义的。相反的情况也可能成立。如果两个线程在不同的数据集上工作，将它们安排在同一个核心上可能会出现两个线程争夺相同的缓存，从而减少彼此对缓存的有效使用。其次，两个数据集必须加载到同一个缓存中；实际上，这增加了必须加载的数据量，因此可用带宽减少了一半。

这种情况下的解决方案是设置线程的亲和性，使它们不能被调度在同一个核心上。这与之前的情况相反，因此在进行任何更改之前了解尝试优化的情况非常重要。

优化缓存共享以优化带宽是在

这实际上是 NUMA 编程的一个方面，下一节将对此进行介绍。只需将“内存”的概念扩展到高速缓存即可。一旦缓存级别的数量增加，这一点将变得更加重要。因此，NUMA 支持库中提供了多核调度的解决方案。请参阅附录 D 中的代码示例，了解确定亲和性掩码的方法，而无需对系统详细信息进行硬编码或深入了解 `/sys` 文件系统。

6.5 NUMA 编程

对于 NUMA 编程，到目前为止所说的有关缓存优化的所有内容也适用。差异仅在该水平以下开始。访问地址空间的不同部分时，NUMA 会引入不同的成本。通过统一的内存访问，我们可以优化以最大限度地减少页面错误（请参阅第 7.5 节），但仅此而已。所有页面都是平等创建的。

NUMA 改变了这一点。访问成本可能取决于所访问的页面。不同的访问成本也增加了优化内存页面局部性的重要性。NUMA 对于大多数 SMP 机器来说是不可避免的，因为带有 CSI 的 Intel（针对 x86、x86-64 和 IA-64）和 AMD（针对 Opteron）都使用它。随着每个处理器的内核数量不断增加，我们可能会看到正在使用的 SMP 系统急剧减少（至少在数据中心和对 CPU 使用率要求极高的办公室之外）。大多数家用机器只需一个处理器就可以了，因此不会出现 NUMA 问题。但这 a) 并不意味着程序员可以忽略 NUMA，b) 并不意味着不存在相关问题。

如果人们考虑对 NUMA 的推广，很快就会意识到这一概念也扩展到了处理器缓存。使用相同缓存的内核上的两个线程将比不共享缓存的内核上的线程协作速度更快。这并非捏造的案例：

- 早期的双核处理器没有 L2 共享。
- 例如，英特尔的 Core 2 QX 6700 和 QX 6800 四核芯片有两个独立的二级缓存。
- 正如早期推测的那样，随着芯片上的核心数量增多以及统一缓存的愿望，我们将拥有更多级别的缓存。

缓存形成自己的层次结构；线程在核心上的放置对于共享（或不共享）各种缓存变得很重要。这与 NUMA 面临的问题没有太大区别，因此，这两个概念可以统一。因此，即使是只对非 SMP 机器感兴趣的人也应该阅读本节。

在 5.3 节中，我们已经看到 Linux 内核提供了许多在 NUMA 编程中有用且需要的信息。然而，收集这些信息并不那么容易。当前可用的 NUMA 库

Linux 上完全不足以实现此目的。作者目前正在构建一个更合适的版本。

现有的 NUMA 库 libnuma 是 numactl 包的一部分，不提供对系统体系结构信息的访问。它只是可用系统调用的包装器以及一些常用操作的便捷接口。目前 Linux 上可用的系统调用有：

mbind 选择指定内存页的绑定。

set_mempolicy 设置默认内存绑定策略。

get_mempolicy 获取默认内存绑定策略。

migrate_pages 将给定节点集上进程的所有页面迁移到不同的节点集。

move_pages 将选定的页面移动到给定节点或请求有关页面的节点信息。

这些接口在 libnuma 库附带的 <numaif.h> 标头中声明。在我们深入了解更多细节之前，我们必须了解内存策略的概念。

6.5.1 内存策略

定义内存策略背后的想法是允许现有代码在 NUMA 环境中相当好地工作，而无需进行重大修改。该策略由于进程继承，这使得使用 numactl 工具成为可能。该工具可用于启动具有给定策略的程序等。

Linux 内核支持以下策略：

MPOL_BIND 内存仅从给定的节点集分配。如果不可能，分配就会失败。

MPOL_PREFERRED 内存优选地从给定的节点集分配。如果失败，则考虑其他节点的内存。

MPOL_INTERLEAVE 内存从指定节点平均分配。对于基于 VMA 的策略，可以通过虚拟内存区域中的偏移量来选择节点；对于基于任务的策略，可以通过自由运行的计数器来选择节点。

MPOL_DEFAULT 根据区域的默认值选择分配。

该列表似乎递归地定义了策略。这句话只对了一半。事实上，内存策略形成了一个层次结构（见图 6.15）。如果 VMA 策略涵盖某个地址，则使用该策略。一种特殊的政策用于

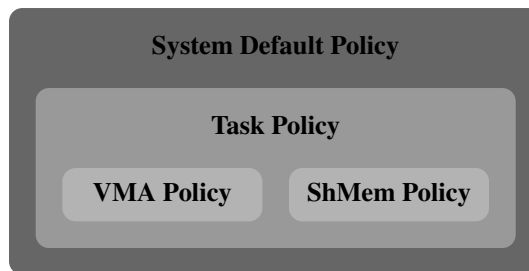


图 6.15: 内存策略层次结构

共享内存段。如果不存在针对特定地址的策略，则使用任务的策略。如果也不存在，则使用系统的默认策略。

系统默认为请求内存的线程分配本地内存。默认情况下不提供任何任务和 VMA 策略。对于具有多个线程的进程，本地节点是“主”节点，即第一个运行该进程的节点。上述系统调用可用于选择不同的策略。

6.5.2 指定策略

set_mempolicy 调用可用于设置当前线程（内核中的任务）的任务策略。只有当前线程受到影响，而不是整个进程。

```
#include <numaif.h> long set_mempolicy(int mode, unsigned long *nodemask, unsigned long maxnode);
```

模式参数必须是上一节中介绍的 MPOL_* 常量之一。nodemask 参数指定用于将来分配的内存节点，maxnode 是 nodemask 中的节点数（即位）。如果模式为 MPOL_DEFAULT，则无需指定内存节点，并且节点掩码参数将被忽略。如果将空指针作为 MPOL_PREFERRED 的节点掩码传递，则选择本地节点。否则，MPOL_PREFERRED 使用最低节点号以及节点掩码中设置的相应位。

设置策略不会对已分配的内存产生任何影响。页面不会自动迁移；只有未来的分配受到影响。请注意内存分配和地址空间保留之间的区别：使用 mmap 建立的地址空间区域通常不会自动分配。内存区域上的第一次读或写操作将分配适当的页面。如果策略在对同一地址空间区域的不同页面的访问之间发生变化，或者如果策略允许从不同节点分配内存，则看似统一的地址空间区域可能会分散在许多内存节点上。

6.5.3 交换和策略

如果物理内存耗尽，系统必须删除干净页面并保存脏页面进行交换。Linux 交换实现在将页面写入交换时会丢弃节点信息。这意味着当页面被重用并在所使用的节点中进行分页时，将从头开始选择。线程的策略可能会导致选择靠近执行处理器的节点，但该节点可能与之前使用的节点不同。

这种变化的关联意味着节点关联不能由程序存储为页面的属性。该关联可能会随着时间而改变。对于与其他进程共享的页面，也可能发生这种情况，因为进程需要它（请参阅下面对 `mbind` 的讨论）。如果一个节点空间不足而其他节点仍有可用空间，则内核本身可以迁移页面。

因此，用户级代码了解的任何节点关联只能在短时间内保持真实。它更像是一个暗示，而不是绝对信息。每当需要准确的知识时，就应该使用 `get_mempolicy` 接口（参见第 6.5.5 节）。

6.5.4 VMA 政策

要为某个地址范围设置 VMA 策略，必须使用不同的接口：

```
#include <numaif.h>
long mbind(void *start, unsigned long len,
           int mode,
           unsigned long *nodemask,
           unsigned long maxnode,
           unsigned flags);
```

该接口为地址范围 `[start, start + len)` 注册一个新的 VMA 策略。由于内存处理在页上操作，因此起始地址必须页对齐。len 值向上舍入到下一个页面大小。

模式参数再次指定策略；这些值必须从第 6.5.1 节中的列表中选择。与 `set_mempolicy` 一样，`nodemask` 参数仅用于某些策略。其处理方式是相同的。

`mbind` 接口的语义取决于 `flags` 参数的值。默认情况下，如果 `flags` 为零，系统调用将设置地址范围的 VMA 策略。现有映射不受影响。如果这还不够，当前有三个标志可以修改此行为；它们可以单独或一起选择：

MPOL_MF_STRICT 如果并非所有页面都位于 `nodemask` 指定的节点上，则对 `mbind` 的调用将失败。在

如果该标志与 **MPOL_MF_MOVE** 和/或 **MPOL_MF_MOVEALL** 一起使用，则如果无法移动任何页面，则调用将失败。

MPOL_MF_MOVE 内核将尝试移动在节点上分配的地址范围内的任何页面，该节点不在由节点掩码指定的集合中。默认情况下，仅移动当前进程页表专用的页面。

MPOL_MF_MOVEALL 与 **MPOL_MF_MOVE** 类似，但内核将尝试移动所有页，而不仅仅是当前进程页表使用的页。此操作具有系统范围的影响，因为它也会影响其他进程的内存访问（这些进程可能不属于同一用户）。因此，**MPOL_MF_MOVEALL** 是一个特权操作（需要 **CAP_NICE** 功能）。

请注意，仅在 2.6.16 Linux 内核中添加了对 **MPOL_MF_MOVE** 和 **MPOL_MF_MOVEALL** 的支持。

当必须在实际分配任何页面之前指定新保留地址范围的策略时，调用不带任何标志的 `mbind` 非常有用。

```
void *p = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_ANON, -1, 0);
if (p != MAP_FAILED) mbind(p, len, 模式, 节点掩码, maxnode, 0);
```

该代码序列保留了 `len` 个字节的地址空间范围，并指定应使用引用节点掩码中内存节点的策略模式。除非 **MAP_POPULATE** 标志与 `mmap` 一起使用，否则在 `mbind` 调用时不会分配任何内存，因此，新策略适用于该地址空间区域中的所有页面。

单独的 **MPOL_MF_STRICT** 标志可用于确定 `mbind` 的 `start` 和 `len` 参数描述的地址范围内的任何页面是否分配在除 `nodemask` 指定的节点之外的节点上。分配的页面没有改变。如果所有页面都分配在指定节点上，则地址空间区域的 VMA 策略将根据模式进行更改。

有时需要重新平衡内存，在这种情况下，可能需要将一个节点上分配的页面移动到另一个节点。使用 **MPOL_MF_MOVE** 设置调用 `mbind` 会尽最大努力实现这一目标。仅考虑移动仅由进程的页表树引用的页面。可以有多个以线程或其他进程形式存在的用户共享页表树的该部分。不可能影响碰巧映射相同数据的其他进程。这些页不共享页表条目。

如果在传递给 mbind 的 flags 参数中同时设置了 MPOL_MF_STRICT 和 MPOL_MF_MOVE 位，则内核将尝试移动未在指定节点上分配的所有页面。如果不可能，调用将会失败。这样的调用可能有助于确定是否存在可以容纳所有页面的节点（或节点集）。可以连续尝试几种组合，直到找到合适的节点。

除非运行当前进程是计算机的主要目的，否则很难证明 MPOL_MF_MOVEALL 的使用是合理的。原因是即使出现在多个页表中的页面也会被移动。这很容易对其他进程产生负面影响。因此应谨慎使用此操作。

6.5.5 查询节点信息

get_mempolicy 接口可用于查询有关给定地址的 NUMA 状态的各种事实。

```
#include <numaif.h> long get_mempolicy(int *policy, const unsigned long *nmask, unsigned long maxnode, void *addr, int flags);
```

当使用 flags 参数为零调用 get_mempolicy 时，有关地址 addr 的策略的信息将存储在策略指向的字中以及 nmask 指向的节点的位掩码中。如果 addr 落入指定了 VMA 策略的地址空间区域，则返回有关该策略的信息。否则，将返回有关任务策略或系统默认策略（如有必要）的信息。

如果在 flags 中设置了 MPOL_F_NODE 标志，并且管理 addr 的策略是 MPOL_INTERLEAVE，则策略指向的字中存储的值是下一次分配将要发生的节点的索引。该信息可能用于设置将在新分配的内存上工作的线程的亲合性。这可能是一种成本较低的实现邻近性的方法，特别是在线程尚未创建的情况下。

MPOL_F_ADDR 标志可用于检索另一个完全不同的数据项。如果使用此标志，则存储在策略指向的字中的值是内存节点的索引，在该内存节点上分配了包含 addr 的页面的内存。该信息可用于做出有关可能的页面迁移的决策，决定哪个线程可以最有效地在内存位置上工作，等等。

线程使用的 CPU（以及内存节点）比其内存分配更不稳定。在没有明确请求的情况下，内存页仅被移入

极端情况。作为重新平衡 CPU 负载的结果，可以将线程分配给另一个 CPU。因此，有关当前 CPU 和节点的信息可能是短暂的。调度程序将尝试将线程保留在同一个 CPU 上，甚至可能保留在同一个内核上，以最大限度地减少冷缓存造成的性能损失。这意味着查看当前 CPU 和节点信息很有用；我们必须避免假设这种关联不会改变。

libNUMA 提供了两个接口来查询给定虚拟地址空间范围的节点信息：

```
#include <libNUMA.h> int NUMA_mem_get_node_idx(void *addr); int NUMA_mem_get_node_mask(void *addr, size_t 大小, size_t __dest_size, memnode_set_t *dest);
```

NUMA_mem_get_node_mask 在 dest 中设置所有内存节点的位，根据管理策略，在 [addr, addr+size) 范围内的页面已分配（或将要分配）。NUMA_mem_get_node 仅查看地址 addr 并返回已（或将）分配该地址的内存节点的索引。这些接口比 get_mempolicy 使用起来更简单，可能应该是首选。

可以使用 sched_getcpu 查询线程当前使用的 CPU（参见第 6.4.3 节）。使用此信息，程序可以使用 libNUMA 中的 NUMA_cpu_to_memnode 接口确定 CPU 本地的内存节点：

```
#include <libNUMA.h> int NUMA_cpu_to_memnode(size_t cpusetsize, const cpu_set_t *cpuset, size_t memnodesize, memnode_set_t *memnodeset);
```

调用此函数将设置（在第四个参数指向的内存节点集中）与第二个参数指向的集合中任何 CPU 本地的内存节点相对应的所有位。就像 CPU 信息本身一样，只有在机器的配置发生变化（例如，CPU 被移除和添加）之前，该信息才是正确的。

memnode_set_t 对象中的位可用于调用 get_mempolicy 等低级函数。使用 lib-NUMA 中的其他函数更方便。反向映射可通过以下方式获得：

```
#include <libNUMA.h>
```

```
int NUMA_memnode_to_cpu(size_t memnodesize,
                        const memnode_set_t *
                        memnodeset,
                        size_t cpusetsize,
                        cpu_set_t *cpuset);
```

结果 `cpuset` 中设置的位是任何内存节点本地 CPU 的位，并在 `memnodeset` 中设置相应的位。对于这两种接口，程序员必须意识到信息可能会随着时间的推移而改变（特别是 CPU 热插时）。在许多情况下，在输入位集中设置单个位，但这也是有意义的，例如，将通过调用 `sched_getaffinity` 检索到的整个 CPU 集传递给 `NUMA_cpu_to_memnode` 以确定线程可以直接访问哪些内存节点。

6.5.6 CPU 和节点集

如果源不可用，通过更改代码以使用迄今为止描述的接口来调整 SMP 和 NUMA 环境的代码可能会非常昂贵（或不可能）。此外，系统管理员可能希望对用户和/或进程可以使用的资源施加限制。对于这些情况，Linux 内核支持所谓的 CPU 集。这个名称有点误导，因为内存节点也被覆盖。它们也与 `cpu_set_t` 数据类型无关。

目前，CPU 集的接口是一个特殊的文件系统。它通常不被安装（至少到目前为止）。这可以改变

```
mount -t cpuset 无 /dev/cpuset
```

挂载点 `/dev/cpuset` 当然当时必须存在。该目录的内容是默认（根）CPU 集的描述。它最初包括所有 CPU 和所有内存节点。该目录中的 `cpus` 文件显示 CPU 集中的 CPU，`mems` 文件显示内存节点，任务文件显示进程。

要创建新的 CPU 集，只需在层次结构中的某个位置创建一个新目录即可。新的 CPU 集将继承父代的所有设置。然后，可以通过将新值写入新目录中的 `cpus` 和 `mems` 伪文件来更改新 CPU 集的 CPU 和内存节点。

如果进程属于 CPU 集，则 CPU 和内存节点的设置将用作亲和性和内存策略位掩码的掩码。这意味着程序无法在亲和性掩码中选择任何不在进程正在使用的 CPU 集的 `cpus` 文件中的 CPU（即它在任务文件中列出的位置）。内存策略和 `mems` 文件的节点掩码也是如此。

除非掩码后位掩码为空，否则程序不会遇到任何错误，因此 CPU 集是控制程序执行的几乎不可见的手段。此方法对于具有大量 CPU 和/或内存节点的机器尤其有效。将进程移至新的 CPU 组就像将进程 ID 写入相应 CPU 组的任务文件一样简单。

CPU 集的目录包含许多其他文件，可用于指定详细信息，例如内存压力下的行为以及对 CPU 和内存节点的独占访问。有兴趣的读者可以参考内核源代码树中的 `Documentation/cpusets.txt` 文件。

6.5.7 显式 NUMA 优化

如果所有节点上的所有线程都需要访问相同的内存区域，则所有本地内存亲和性规则都无济于事。当然，可以简单地将线程数量限制为直接连接到内存节点的处理器可支持的数量。不过，这并没有利用 SMP NUMA 机器的优势，因此不是一个真正的选择。

如果相关数据是只读的，有一个简单的解决方案：复制。每个节点都可以获得自己的数据副本，因此不需要节点间访问。执行此操作的代码如下所示：

```
void *local_data(void) {
    static void *data[NNODES];
    int node =
        NUMA_memnode_self_current_idx();
    if (node == -1)
        /* Cannot get node, pick one. */
        node = 0;
    if (data[node] == NULL)
        data[node] = allocate_data();
    return data[node];
}

void worker(void) {
    void *data = local_data();
    for (...)
        compute using data
}
```

在此代码中，函数工作人员通过调用 `local_data` 获取指向数据本地副本的指针来进行准备。然后它继续使用该指针的循环。`local_data` 函数保存已分配的数据副本的列表。每个系统的内存节点数量有限，因此包含指向每个节点内存副本的指针的数组的大小也受到限制。`libNUMA` 中的 `NUMA_memnode_system_count` 函数返回此数字。如果给定节点的内存尚未分配给当前节点（由输入数据中的空指针识别）

dex 由 NUMA_memnode_self_current_idx 调用返回)，分配一个新副本。

重要的是要认识到，如果在 getcpu 系统调用⁴³之后线程被调度到连接到不同内存节点的另一个 CPU 上，则不会发生任何可怕的情况。它只是意味着使用 worker 中的数据变量的访问访问另一个内存节点上的内存。这会减慢程序的速度，直到重新计算数据，但仅此而已。内核将始终避免无故重新平衡每个 CPU 运行队列。如果发生这样的转移，通常是有充分理由的，而且在不久的将来不会再发生。

当相关内存区域可写时，事情会变得更加复杂。在这种情况下，简单的复制将不起作用。根据具体情况，可能有多种可能的解决方案。

例如，如果可写内存区域用于累积结果，则可以首先为每个内存节点创建一个单独的区域来累积结果。然后，当这项工作完成时，所有每个节点的内存区域都会被组合起来以获得总结果。即使工作永远不会真正停止，这种技术也可以发挥作用，但需要中间结果。这种方法的要求是结果的积累是无状态的，即它不依赖于先前收集的结果。

不过，直接访问可写内存区域总是更好。如果对内存区域的访问次数很大，那么强制内核将有问题的内存页迁移到本地节点可能是个好主意。如果访问次数确实很高，并且不同节点上的写入不会同时发生，这可能会有所帮助。但请注意，内核无法创造奇迹：页面迁移是一个复制操作，因此它并不便宜。这笔费用必须摊销。

6.5.8 利用所有带宽

图 5.4 中的数字表明，当缓存无效时访问远程内存并不比访问本地内存慢多少。这意味着程序可以通过将不必再次读取的数据写入连接到另一个处理器的内存来节省本地内存的带宽。DRAM 模块连接的带宽和互连的带宽大多是独立的，因此并行使用可以提高整体性能。

这是否真的可能取决于许多因素。人们确实必须确保缓存无效，否则与远程访问相关的速度下降是可以测量的。另一个大问题是远程节点是否对自己的内存带有任何需求——

宽度。在采取该方法之前必须详细审查这种可能性。理论上，使用处理器可用的所有带宽可以产生积极的效果。一个系列 10h Opteron 处理器可以直接连接到最多四个其他处理器。如果系统的其余部分配合使用，利用所有额外的带宽，或许再加上适当的预取（尤其是 prefetchw）可能会带来改进。

⁴³The user-level sched_getcpu interface is implemented using the getcpu system call which should not be used directly and has a different interface.

7 个内存性能工具

有多种工具可以帮助程序员了解程序的性能特征、缓存和内存使用等。现代处理器具有可以使用的性能监控硬件。有些事件很难准确测量，因此还有模拟的空间。当涉及到更高级别的功能时，有特殊的工具可以监视流程的执行。我们将介绍大多数 Linux 系统上可用的一组常用工具。

7.1 内存操作分析

分析内存操作需要硬件的协作。可以单独在软件中收集一些信息，但这要么是粗粒度的，要么仅仅是模拟。模拟示例将在第 7.2 节和第 7.5 节中给出。在这里，我们将重点关注可测量的记忆效应。

对 Linux 上的性能监控硬件的访问由 `oprofile` 提供。`Oprofile` 提供连续分析功能，如 [2] 中首先描述的；它通过易于使用的界面执行统计、系统范围的分析。`Oprofile` 绝不是使用处理器性能测量功能的唯一方法；Linux 开发人员正在开发 `pfmon`，它可能在某个时候得到足够广泛的部署，值得在这里进行描述。

即使使用了可选的 GUI，`oprofile` 提供的界面也是简单且最小的，但也相当低级。用户必须在处理器可以记录的事件中进行选择。处理器的体系结构手册描述了事件，但通常需要有关处理器本身的广泛知识才能解释数据。另一个问题是对收集到的数据的解释。性能测量计数器是绝对值并且可以任意增长。对于给定的计数器来说，多高算过高？

此问题的部分答案是避免查看绝对值，而是将多个计数器相互关联。处理器可以监控多个事件；然后可以检查所收集的绝对值的比率。这给出了很好的、可比较的结果。除数通常是处理时间、时钟周期数或指令数的度量。作为对程序性能的初步尝试，仅将这两个数字本身联系起来是有用的。

图 7.1 显示了各种工作集大小的简单随机“跟随”测试用例的每条指令周期 (CPI)。大多数 Intel 处理器收集此信息的事件名称为 `CPU_CLK_UNHALTED` 和 `INST_RETIRED`。顾名思义，前者计算 CPU 的时钟周期，后者计算指令数。我们看到一张类似于我们使用的每个列表元素测量周期的图片。对于较小的工作集大小，该比率为 1.0 甚至更低。这些

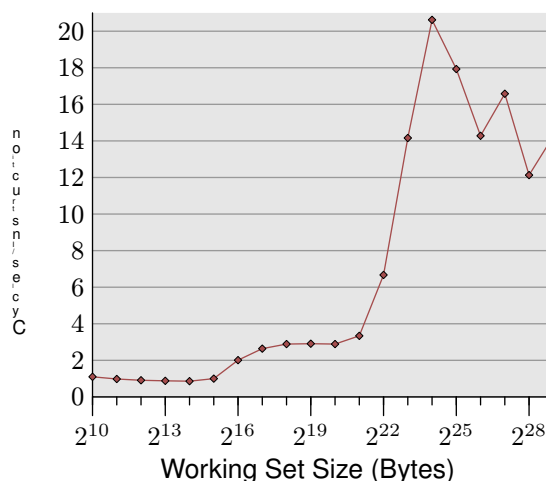


图 7.1: 每条指令的周期（遵循随机）

测量是在 Intel Core 2 处理器上进行的，该处理器是多标量的，可以同时处理多个指令。对于不受内存带宽限制的程序，该比率可能明显低于 1.0，但在本例中，1.0 就相当不错了。

一旦 L1d 不再大到足以容纳工作集，CPI 就会跳至略低于 3.0。请注意，CPI 比率是对所有指令（而不仅仅是内存指令）访问 L2 的惩罚进行平均。使用列表元素数据的周期，可以计算出每个列表元素需要多少条指令。如果连二级缓存都不够用，CPI 比率会跳到 20 以上。这是预期的结果。

但性能测量计数器应该可以更深入地了解处理器中发生的情况。为此，我们需要考虑处理器的实现。在本文档中，我们关注缓存处理细节，因此我们必须查看与缓存相关的事件。这些事件、它们的名称以及它们的计数都是特定于处理器的。这是目前 `oprofile` 难以使用的地方，无论用户界面如何简单：用户必须自己计算出性能计数器的详细信息。在附录 B 中，我们将看到一些处理器的详细信息。

对于 Core 2 处理器，要查找的事件是 `L1D_REPL`、`DTLB_MISSES` 和 `L2_LINES_IN`。后者可以测量所有未命中以及由指令而不是硬件预取引起的未命中。随机“跟随”测试的结果如图 7.2 所示。

所有比率均使用退役指令数 (`INST_RETIRED`) 计算。这意味着要计算所有加载和存储指令的高速缓存未命中率，必须从 `INST_RETIRED` 值中减去相当大的数字，这意味着存储器操作的实际高速缓存未命中率甚至高于图中所示的数字。

自 L2 失误以来，L1d 的失误比其他所有失误都要高

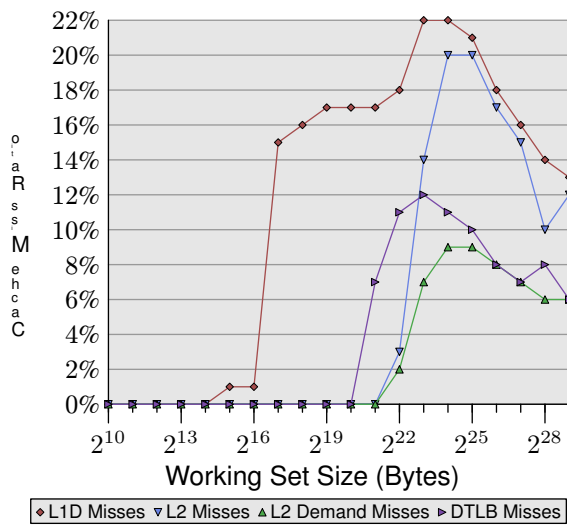


图 7.2: 测量的缓存未命中（遵循随机）

意味着，对于 Intel 处理器，由于使用包容性缓存而导致 L1d 未命中。处理器有 32k 的 L1d，因此我们看到，正如预期的那样，L1d 速率在大约该工作集大小时从零上升（除了列表数据结构之外，还有其他缓存用途，这意味着增加发生在 16k 和 32k 标记之间）。有趣的是，对于高达 64k 的工作集大小，硬件预取可以将丢失率保持在 1%。此后，L1d 率飙升。

L2 缺失率保持为零，直到 L2 耗尽；由于 L2 的其他用途而导致的少数失误不会对数字产生太大影响。一旦超过 L2 的大小（ 2^{21} 字节），丢失率就会上升。值得注意的是，L2 需求错过率不为零。这表明硬件预取器没有加载稍后指令所需的所有缓存行。这是预期的，访问的随机性阻止了完美的预取。将其与图 7.3 中顺序读取的数据进行比较。

在该图中我们可以看到，L2 需求错过率基本上为零（注意该图的比例与图 7.2 不同）。对于顺序访问情况，硬件预取器工作得很好：几乎所有 L2 缓存未命中都是由预取器引起的。L1d 和 L2 未命中率相同的事实表明，所有 L1d 高速缓存未命中均由 L2 高速缓存处理，没有进一步的延迟。这是所有计划的理想情况，但当然很难实现。

两张图中的第四行是 DTLB 未命中率（Intel 对于代码和数据有单独的 TLB，DTLB 是数据 TLB）。对于随机访问情况，DTLB 丢失率非常高并且会导致延迟。有趣的是，DTLB 处罚是在 L2 失手之前设定的。对于顺序访问情况，DTLB 成本基本上为零。

回到矩阵乘法的例子-

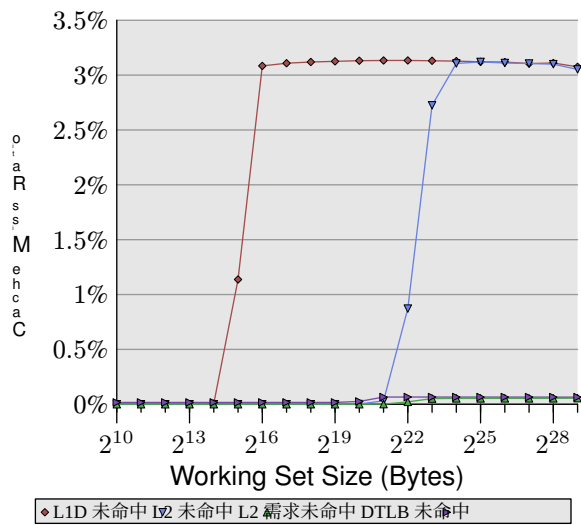


图 7.3: 测量的缓存未命中（遵循顺序）

通过 6.2.1 和 A.1 节中的示例代码，我们可以使用另外三个计数器。SSE_HIT_PRE、SSE_PRE_MISS 和 LOAD_PRE_EXEC 计数器可用于查看软件预取的有效性。如果运行 A.1 节中的代码，我们会得到以下结果：

Description	Ratio
Useful NTA prefetches	2.84%
Late NTA prefetches	2.65%

低有用的 NTA（非时间对齐）预取比率表明许多预取指令是针对已加载的高速缓存行执行的，因此不需要任何工作。这意味着处理器浪费时间来解码预取指令并查找缓存。不过，我们不能过于严厉地评判代码。很大程度上取决于所使用处理器的缓存大小；硬件预取器也发挥了作用。

较低的后期 NTA 预取比率具有误导性。该比率意味着所有预取指令中有 2.65% 发出得太晚。需要数据的指令在数据预取到高速缓存之前执行。必须记住，只有 $2.84\% + 2.65\% = 5.5\%$ 的预取指令是有用的。在有用的 NTA 预取指令中，48% 没有及时完成。因此，代码可以进一步优化：

- 大多数预取指令是不需要的。
- 可以调整预取指令的使用以更好地匹配硬件。

留给读者作为练习来确定可用硬件的最佳解决方案。确切的硬件规格起着重要作用。在 Core 2 处理器上，SSE 算术运算的延迟为 1 个周期。年长的

```
$ \time ls /etc
[...]
```

0.00user 0.00system 0:00.02elapsed 17%CPU (0avgtext+0avgdata 0maxresident)k
0inputs+0outputs (1major+335minor)pagefaults 0swaps

图 7.4: 时间实用程序的输出

版本的延迟为 2 个周期，这意味着硬件预取器和预取指令有更多时间来引入数据。

要确定哪里可能需要预取或不需要预取，可以使用 `opannotate` 程序。它列出了程序的源代码或汇编代码，并显示了识别事件的指令。请注意，模糊性有两个来源：

1. `Oprofile` 执行随机分析。仅记录每个第 N 个事件（其中 N 是每个事件的阈值，具有强制的最小值），以避免过多减慢系统的运行速度。可能有一些行会导致 100 个事件，但它们可能不会显示在报告中。
2. 并非所有事件都被准确记录。例如，记录特定事件时的指令计数器可能不正确。处理器是多标量的，因此很难给出 100% 正确的答案。不过，某些处理器上的一些事件是准确的。

带注释的列表不仅仅用于确定预取信息。每个事件都用指令指针记录；因此还可以查明程序中的其他热点。作为许多 `INST_RETIRED` 事件源的位置经常执行，值得调整。报告许多高速缓存未命中的位置可能需要预取指令以避免高速缓存未命中。

无需硬件支持即可测量的一类事件是页面错误。操作系统负责解决页面错误，并且在这种情况下，它也会对页面错误进行计数。它区分两种页面错误：

次要页面错误 对于到目前为止尚未使用的匿名（即不受文件支持）页面、写时复制页面以及内容已在内存中某处的其他页面。

主要页面错误 解决它们需要访问磁盘以检索文件备份（或换出）的数据。

显然，主要页面错误比次要页面错误的代价要高得多。但后者也不便宜。无论哪种情况，都需要进入内核，必须找到新页面，必须清除该页面或用适当的数据填充该页面，并且

页表树必须相应修改。最后一步需要与读取或修改页表树的其他任务同步，这可能会带来进一步的延迟。

检索有关页面错误计数信息的最简单方法是使用时间工具。注意：使用真正的工具，而不是 `shell` 内置的。输出如图 7.4 所示。⁴⁴

这里有趣的部分是最后一行。时间工具报告 1 个主要页面错误和 335 个次要页面错误。确切的数字有所不同；特别是，立即重复运行可能会表明现在根本不存在重大页面错误。如果程序执行相同的操作，并且环境没有任何变化，则页面错误总数将保持稳定。

对于页面错误特别敏感的阶段是程序启动。每个被使用的页面都会产生一个页面错误；可见的效果（特别是对于 GUI 应用程序）是使用的页面越多，程序开始工作所需的时间就越长。在第 7.5 节中，我们将看到一个专门衡量这种影响的工具。

在底层，时间工具使用 `rusage` 功能。当父进程等待子进程终止时，`wait4` 系统调用将填充 `struct rusage` 对象；这正是时间工具所需要的。但进程也可能请求有关其自身资源使用情况（这就是名称 `rusage` 的来源）或其终止子进程的资源使用情况的信息。

```
#include <sys/resource.h> int getrusage(__rusage_who_t who, struct rusage *usage)
```

`who` 参数指定请求信息的进程。目前，仅定义了 `RUSAGE_SELF` 和 `RUSAGE_CHILDREN`。当每个子进程终止时，子进程的资源使用量就会累积。它是一个总值，而不是单个子进程的使用情况。存在允许请求特定于线程的信息的提案，因此我们很可能在不久的将来看到 `RUSAGE_THREAD`。`rusage` 结构体被定义为包含各种指标，包括执行时间、发送的 I/O 消息数量和

⁴⁴The leading backslash prevents the use of the built-in command.

```

==19645== I   refs:      152,653,497
==19645== I1  misses:      25,833
==19645== L2i misses:      2,475
==19645== I1  miss rate:      0.01%
==19645== L2i miss rate:      0.00%
==19645==
==19645== D   refs:      56,857,129 (35,838,721 rd + 21,018,408 wr)
==19645== D1  misses:      14,187 ( 12,451 rd + 1,736 wr)
==19645== L2d misses:      7,701 ( 6,325 rd + 1,376 wr)
==19645== D1  miss rate:      0.0% ( 0.0% + 0.0% )
==19645== L2d miss rate:      0.0% ( 0.0% + 0.0% )
==19645==
==19645== L2  refs:      40,020 ( 38,284 rd + 1,736 wr)
==19645== L2  misses:      10,176 ( 8,800 rd + 1,376 wr)
==19645== L2  miss rate:      0.0% ( 0.0% + 0.0% )

```

图 7.5: Cachegrind 摘要输出

使用的内存和页面错误数。后一个信息可在该结构的 `ru_minflt` 和 `ru_majflt` 成员中获得。

试图确定程序在何处由于页面错误而导致性能下降的程序员可以定期请求该信息，然后将返回值与之前的结果进行比较。

从外部来看，如果请求者具有必要的权限，这些信息也是可见的。伪文件 `/proc/<PID>/stat`，其中 `<PID>` 是我们感兴趣的进程的进程ID，包含第十到第十四字段中的页面错误编号。它们分别是进程及其子进程的累积次要和主要页面错误的对。

7.2 模拟CPU缓存

虽然缓存如何工作的技术描述相对容易理解，但要了解实际程序相对于缓存的行为却并不容易。程序员并不直接关心地址的值，无论它们是绝对的还是相对的。地址部分由链接器确定，部分由动态链接器和内核在运行时确定。生成的汇编代码预计适用于所有可能的地址，并且在源语言中，甚至不存在任何绝对地址值的提示。因此，了解程序如何使用内存可能非常困难。⁴⁵

CPU 级分析工具，例如 `oprofile`（如第 7.1 节所述）可以帮助了解缓存的使用。得到的数据与实际硬件相对应，如果不需要细粒度的收集，可以相对较快地收集。一旦需要更细粒度的数据，`oprofile` 就不再可用；线程必须经常被中断。此外，要查看程序在不同处理器上的内存行为，实际上必须拥有这样的机器并执行

对他们进行编程。这有时（通常）是不可能的。图 3.8 中的数据就是一个例子。要使用 `oprofile` 收集此类数据，一台机器必须拥有 24 台不同的机器，其中许多机器并不存在。

该图中的数据是使用缓存模拟器收集的。这个程序，`cachegrind`，使用了 `valgrind` 框架，该框架最初是为了检查程序中与内存处理相关的问题而开发的。`valgrind` 框架模拟程序的执行，在执行此操作时，它允许各种扩展（例如 `cachegrind`）挂接到执行框架中。`cachegrind` 工具利用它来拦截所有内存地址的使用；然后，它模拟具有给定大小、高速缓存行大小和关联性的 L1i、L1d 和 L2 高速缓存的操作。

要使用该工具，必须使用 `valgrind` 作为包装器来运行程序：

`valgrind --tool=cachegrind` 命令参数

在这种最简单的形式中，程序命令使用参数 `arg` 执行，同时使用与其运行的处理器相对应的大小和关联性来模拟三个缓存。当程序运行时，输出的一部分被打印到标准错误；它由总缓存使用的统计数据组成，如图 7.5 所示。给出了指令和内存引用的总数，以及它们为 L1i/L1d 和 L2 缓存产生的未命中次数、未命中率等。该工具甚至能够将 L2 访问拆分为指令和数据访问，并且所有数据缓存使用都分为读取和写入访问。

当模拟缓存的细节发生变化并对结果进行比较时，事情会变得更加有趣。通过使用 `--I1`、`--D1` 和 `--L2` 参数，可以指示 `cachegrind` 忽略处理器的缓存布局并使用命令行上指定的布局。例如：

```

valgrind --tool=cachegrind \
--L2=8388608,8,64 command arg

```

⁴⁵When programming close to the hardware this might be different, but this is of no concern to normal programming and, in any case, is only possible for special addresses such as memory-mapped devices.

Ir	I1mr	I2mr	Dr	D1mr	D2mr	Dw	D1mw	D2mw	file:function
53,684,905	9	8	9,589,531	13	3	5,820,373	14	0	???:_IO_file_xsputn@@GLIBC_2.2.5
36,925,729	6,267	114	11,205,241	74	18	7,123,370	22	0	???:vfprintf
11,845,373	22	2	3,126,914	46	22	1,563,457	0	0	???:__find_specmb
6,004,482	40	10	697,872	1,744	484	0	0	0	???:strlen
5,008,448	3	2	1,450,093	370	118	0	0	0	???:strcmp
3,316,589	24	4	757,523	0	0	540,952	0	0	???:_IO_padn
2,825,541	3	3	290,222	5	1	216,403	0	0	???:_itoa_word
2,628,466	9	6	730,059	0	0	358,215	0	0	???:_IO_file_overflow@@GLIBC_2.2.5
2,504,211	4	4	762,151	2	0	598,833	3	0	???:_IO_do_write@@GLIBC_2.2.5
2,296,142	32	7	616,490	88	0	321,848	0	0	dwarf_child.c:__libdw_find_attr
2,184,153	2,876	20	503,805	67	0	435,562	0	0	???:__dcigettext
2,014,243	3	3	435,512	1	1	272,195	4	0	???:_IO_file_write@@GLIBC_2.2.5
1,988,697	2,804	4	656,112	380	0	47,847	1	1	???:getenv
1,973,463	27	6	597,768	15	0	420,805	0	0	dwarf_getattnrs.c:dwarf_getattnrs

图 7.6: cg 注释输出

将模拟具有 8 路组关联性和 64 字节缓存行大小的 8 MB L2 缓存。请注意，--L2 选项出现在命令行中模拟的程序名称之前。

这并不是 Cachegrind 所能做的全部。在进程退出之前，它会写出一个名为cachegrind.out.XXXXXX的文件，其中XXXXXX是进程的PID。该文件包含有关每个函数和源文件中缓存使用的摘要信息和详细信息。可以使用 cg annotate 程序查看数据。

该程序产生的输出包含高速缓存使用摘要，该摘要也在进程终止时打印，以及程序每个函数中高速缓存行使用的详细摘要。生成每个函数的数据要求 cg annotate 能够将地址与函数匹配。这意味着应该提供调试信息以获得最佳结果。如果做不到这一点，ELF 符号表可以提供一点帮助，但由于内部符号未在动态符号表中列出，因此结果不完整。图 7.6 显示了与图 7.5 相同的程序运行的部分输出。

Ir、Dr 和 Dw 列显示缓存使用总量，而不是缓存未命中数，如下两列所示。该数据可用于识别产生最多高速缓存未命中的代码。首先，人们可能会专注于 L2 缓存未命中，然后继续优化 L1i/L1d 缓存未命中。

cg_annotate可以提供更详细的数据。如果给出了源文件的名称，它还会用与该行对应的缓存命中和未命中数来注释源文件的每一行（因此是程序的名称）。该信息允许程序员深入到缓存未命中中出现问题的确切行。程序界面有点原始：在撰写本文时，cachegrind 数据文件和源文件必须位于同一目录中。

在这一点上，应该再次注意：cachegrind 是一个模拟器，它的 *not* 使用来自处理器的测量值。亲中的实际缓存实现

处理器很可能完全不同。cachegrind 模拟最近最少使用（LRU）驱逐，这对于具有大关联性的缓存来说可能过于昂贵。此外，模拟没有考虑上下文切换和系统调用，这两者都会破坏 L2 的大部分，并且必须刷新 L1i 和 L1d。这导致缓存未命中总数低于实际情况。尽管如此，cachegrind 是一个很好的工具，可以用来了解程序的内存使用情况及其内存问题。

7.3 测量内存使用情况

了解程序分配了多少内存以及可能在哪里分配是优化其内存使用的第一步。幸运的是，有一些易于使用的程序可用，不需要重新编译或专门修改程序。

对于第一个工具，称为 Massif，不删除编译器可以自动生成的调试信息就足够了。它提供了随时间推移累积的内存使用情况的概述。图 7.7 显示了生成的输出的示例。与cachegrind（第7.2节）一样，massif是一个使用valgrind基础设施的工具。开始使用

valgrind --tool=massif 命令参数

其中命令arg是要观察的程序及其参数，该程序将被模拟，并且所有对内存分配函数的调用都会被识别。调用站点与时间戳值一起记录；新的分配大小将添加到整个程序的总数和特定调用站点的总数中。这同样适用于释放内存的函数，显然，释放块的大小是从分配的总和减去。然后可以使用该信息创建一个图表，显示程序生命周期内的内存使用情况，根据请求分配的位置分割每个时间值。

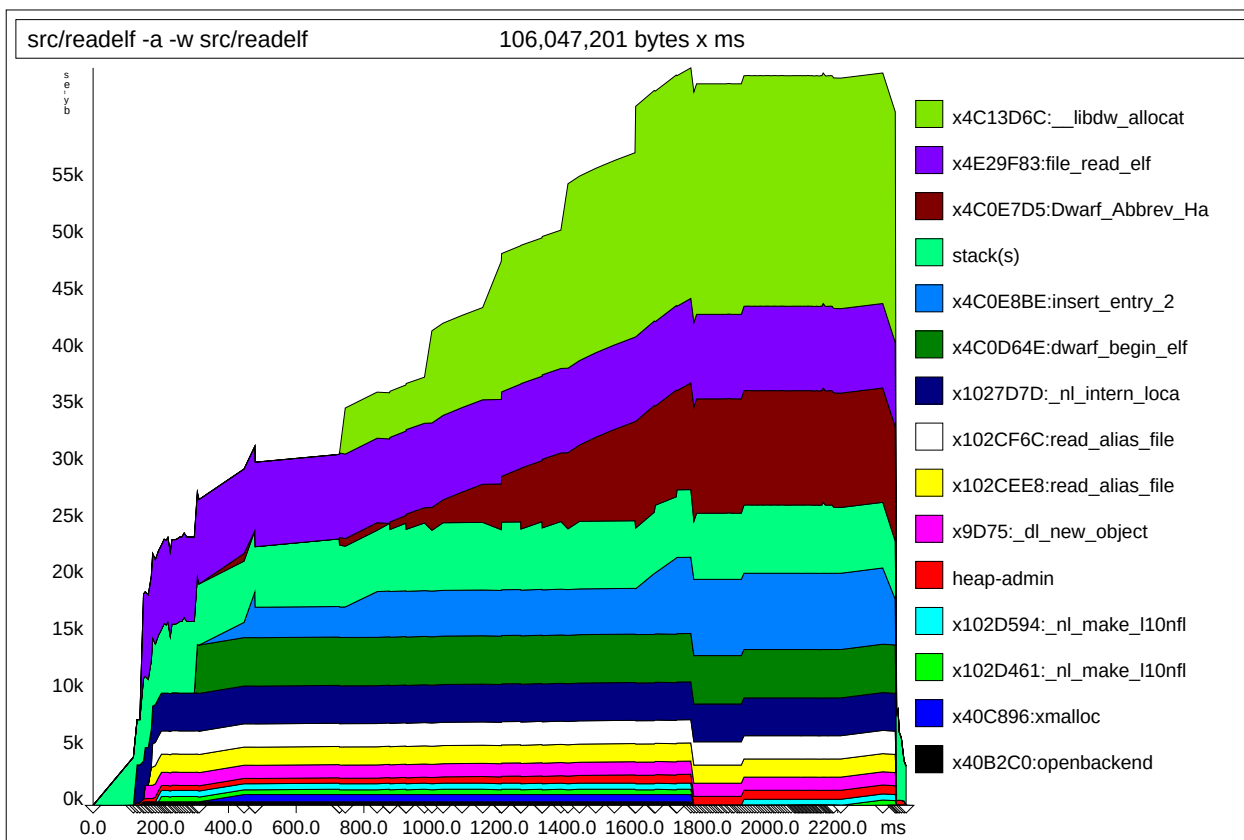


图 7.7: 地块输出

在进程终止之前，massif 创建两个文件：massif.XXX.XXX.txt 和 Massif.XXXXXX.ps；XXXXX 和之前一样是进程的 PID。 .txt 文件是所有调用站点的内存使用情况的摘要，.ps 文件如图 7.7 所示。

Massif 还可以记录程序的堆栈使用情况，这对于确定应用程序的总内存占用量很有用。但这并不总是可能的。在某些情况下（某些线程堆栈或使用 signal tstack 时），valgrind 运行时无法了解堆栈的限制。在这些情况下，将这些堆栈的大小添加到总数中也没有多大意义。还有其他几种情况是没有意义的。如果程序受此影响，massif 应使用附加选项 --stacks=no 启动。请注意，这是 valgrind 的一个选项，因此必须位于正在观察的程序名称之前。

一些程序提供自己的内存分配实现或围绕系统分配函数的包装函数。在第一种情况下，分配通常会被错过；在第二种情况下，记录的调用站点隐藏信息，因为仅记录包装器函数中的调用地址。因此，可以将附加函数添加到分配函数列表中。 --alloc-fn=xmalloc 参数指定 xmalloc 函数也是一个分配函数，这在 GNU 程序中经常出现。对 xmalloc 的调用会被记录，但不会记录 allo-

从 xmalloc 内部进行的阳离子调用。

第二个工具叫memusage；它是 GNU C 库的一部分。它是 Massif 的简化版本（但在 Massif 之前就已经存在很长时间了）。它仅记录堆的总内存使用量（包括可能的 mmap 调用等，如果给出 -m 选项）以及堆栈（可选）。结果可以显示为总内存使用量随时间变化的图表，或者显示为对分配函数的调用的线性图表。这些图表是由 memusage 脚本单独创建的，与 valgrind 一样，必须使用该脚本来启动应用程序：

memusage 命令 arg

必须使用 -p IMGFILE 选项来指定应在文件 IMGFILE 中生成图形。这是一个 PNG 文件。收集数据的代码是在实际程序本身中运行的，它不是像 valgrind 那样的模拟。这意味着 memusage 比 Massif 快得多，并且可以在 Massif 不起作用的情况下使用。除了总内存消耗之外，代码还记录分配大小，并且在程序终止时，它显示已使用分配大小的直方图。该信息被写入标准错误。

有时不可能（或不可行）调用应该直接观察的程序。一个例子是 gcc 的编译阶段，它是由 gcc 驱动程序启动的。在这种情况下，亲的名字

必须使用 `-n NAME` 参数将应观察的 `gram` 提供给 `me musage` 脚本。如果被观察的程序启动其他程序，则该参数也很有用。如果未指定程序名称，则将分析所有启动的程序。

Massif 和 memusage 这两个程序都有附加选项。发现自己需要更多功能的程序员应该首先查阅手册或帮助消息，以确保附加功能尚未实现。

现在我们知道如何捕获有关内存分配的数据，有必要讨论如何在内存和缓存使用的上下文中解释这些数据。高效动态内存分配的主要方面是线性分配和已使用部分的紧凑性。这又回到了提高预取效率和减少缓存未命中的目的。

必须读入任意数量的数据以供以后处理的程序可以通过创建一个列表来完成此操作，其中每个列表元素都包含一个新数据项。这种分配方法的开销可能是最小的（单链表的一个指针），但是使用数据时的缓存效应会显著降低性能。

例如，一个问题是无法保证顺序分配的内存存在内存中按顺序布置。造成这种情况的可能原因有很多：

- 由内存分配器管理的大内存块内的内存块实际上是从后到前返回的；
- 内存块耗尽，并在地址空间的不同部分开始新的内存块；
- 分配请求的大小不同，由不同的内存池提供服务；
- 多线程程序的各个线程中的交错分配。

如果必须预先分配数据以供以后处理，那么链表方法显然是一个坏主意。不能保证（甚至不可能）列表中的连续元素在内存中连续排列。为了确保连续分配，不得以小块的形式分配内存。必须使用另一层内存处理；程序员可以很容易地实现它。另一种方法是使用 GNU C 库中提供的 `obstack` 实现。该分配器从系统分配器请求大内存块，然后将任意大或小的内存块移出。这些分配始终是连续的，除非大内存块耗尽，而根据请求的分配大小，这种情况非常罕见。`Obstack` 并不能完全替代内存

分配器，它们释放对象的能力有限。有关详细信息，请参阅 GNU C 库手册。

那么，如何从图表中识别出建议使用 `obstacks`（或类似技术）的情况呢？如果不咨询来源，就无法识别可能的更改候选者，但图表可以提供搜索的入口点。如果许多分配是从同一位置进行的，这可能意味着批量分配可能会有所帮助。在图 7.7 中，我们可以在地址 `0x4c0e7d5` 的分配中看到这样一个可能的候选者。从运行开始约 800 毫秒到运行开始 1,800 毫秒，这是唯一增长的区域（顶部绿色区域除外）。而且，斜率并不陡，这意味着我们有大量相对较小的分配。这确实是使用 `obstacks` 或类似技术的候选者。

图表可以显示的另一个问题是分配总数很高时。如果图表不是随时间线性绘制，而是随调用次数线性绘制（memusage 的默认设置），则这一点尤其容易看出。在这种情况下，图表中的缓坡意味着大量的小分配。memusage 不会说明分配发生在哪里，但与 Massif 的输出进行比较可以说明这一点，或者程序员可能会立即识别出来。应合并许多小分配以实现线性内存使用。

但对于后一类情况，还有另一个同样重要的方面：许多分配也意味着管理数据的更高开销。这本身可能不是那么有问题。名为“heap-admin”的红色区域代表了 Massif 图中的这种开销，而且它非常小。但是，根据 malloc 实现，此管理数据与数据块一起分配在同一内存中。对于 GNU C 库中当前的 malloc 实现来说，情况是这样的：每个分配的块至少有一个 2 字头（32 位平台为 8 字节，64 位平台为 16 字节）。此外，由于内存管理方式（将块大小四舍五入为特定倍数），块大小通常比所需的要大一些。

这一切都意味着程序使用的内存与分配器仅用于管理目的的内存散布在一起。我们可能会看到这样的事情：



每个块代表一个存储字。在这个小内存区域中，我们有四个分配的块。由于块头和填充而产生的开销为 50%。由于标头的放置，这自动意味着处理器的有效预取率也降低了 50%。如果按顺序处理块（以最大限度地利用预取），处理器会将所有标头和填充字读入高速缓存，即使它们从未被读取过。

应该由应用程序本身读取或写入。只有运行时使用标头字，并且运行时仅在块被释放时发挥作用。

此时，人们可能会争辩说，应该改变实施方式，将管理数据放在其他地方。这确实在某些实现中完成了，并且可能被证明是一个好主意。不过，有很多方面需要牢记，安全并不是最不重要的。无论我们将来是否会看到变化，填充问题都永远不会消失（当忽略标题时，填充问题占示例中数据的 16%）。只有程序员直接控制分配才能避免这种情况。当对齐要求发挥作用时，仍然可能存在漏洞，但这也是程序员可以控制的。

7.4 改进分支预测

在第 6.2.2 节中，提到了通过分支预测和块重新排序来改进 L1i 使用的两种方法：通过 `__builtin_expect` 的静态预测和配置文件引导优化（PGO）。正确的分支预测会对性能产生影响，但这里我们感兴趣的是内存使用的改进。

`__builtin_expect`（或者更好的是可能和不可能宏）的使用很简单。定义被放置在中央头文件中，编译器负责其余的工作。不过，有一个小问题：程序员很容易在真正不太可能的情况下使用“可能”，反之亦然。即使有人使用像 `oprofile` 这样的工具来测量错误的分支预测并且 L1i 错过了这些问题也很难被发现。

不过，有一种简单的方法。A.2 节中的代码显示了可能和不可能宏的另一种定义，它们在运行时主动测量静态预测是否正确。然后，程序员或测试人员可以检查结果并进行调整。这些测量实际上并没有考虑程序的性能，它们只是测试程序员所做的静态假设。可以在上面引用的部分中找到更多详细信息以及代码。

如今，PGO 与 `gcc` 一起使用非常容易。不过，这是一个三步过程，并且必须满足某些要求。首先，所有源文件都必须使用附加的 `-fprofile-generate` 选项进行编译。必须将此选项传递给所有编译器运行以及链接程序的命令。可以混合使用和不使用此选项编译的目标文件，但 PGO 对于那些未启用它的人来说没有任何好处。

编译器生成一个行为正常的二进制文件，只是它更大且速度更慢，因为它记录（和存储）有关是否采用分支的信息。编译器还为每个输入文件生成一个扩展名为 `.gcno` 的文件。该文件包含

与代码中的分支相关的信息。必须保存起来供以后使用。

一旦程序二进制文件可用，就应该使用它来运行一组有代表性的工作负载。无论使用什么工作负载，最终的二进制文件都将被优化以很好地完成这项任务。程序的连续运行是可能的，并且通常是必要的；所有运行都将生成相同的输出文件。在程序终止之前，程序运行期间收集的数据将写入扩展名为 `.gcda` 的文件中。这些文件在包含源文件的目录中创建。该程序可以从任何目录执行，并且可以复制二进制文件，但包含源代码的目录必须可用且可写。同样，为每个输入源文件创建一个输出文件。如果程序运行多次，则必须在源目录中找到上一次运行的 `.gcda` 文件，否则运行的数据无法累积在一个文件中。

运行一组有代表性的测试后，就可以重新编译应用程序了。编译器必须能够在保存源文件的同一目录中找到 `.gcda` 文件。文件无法移动，因为编译器找不到它们，并且文件的嵌入式校验和不再匹配。对于重新编译，请将 `-fprofile-generate` 参数替换为 `-fprofile-use`。重要的是，源代码不会以任何可能改变生成代码的方式发生变化。这意味着：更改空格和编辑注释是可以的，但添加更多分支或基本块会使收集的数据无效，编译将失败。

这就是程序员所要做的全部工作；这是一个相当简单的过程。最重要的是要选择有代表性的测试来进行测量。如果测试工作负载与程序的实际使用方式不匹配，则执行的优化实际上可能弊大于利。因此，图书馆使用 PGO 是否常常很困难？库可用于许多（有时是截然不同的）场景。除非用例确实相似，否则通常最好完全依赖使用 `__builtin_expect` 的静态分支预测。

关于 `.gcno` 和 `.gcda` 文件的几句话。这些是二进制文件，不能立即用于检查。不过，可以使用 `gcov` 工具（它也是 `gcc` 软件包的一部分）来检查它们。该工具主要用于覆盖率分析（因此得名），但使用的文件格式与 PGO 相同。`gcov` 工具为每个带有执行代码的源文件生成扩展名为 `.gcov` 的输出文件（这可能包括系统标头）。这些文件是根据给 `gcov` 的参数、分支计数器、概率等进行注释的源列表。

0 0x3000000000 C	0 0x3000000B50: (within /lib64/ld-2.5.so)
1 0x 7FF000000 D	3320 0x3000000B53: (within /lib64/ld-2.5.so)
2 0x3000001000 C	58270 0x3000001080: _dl_start (in /lib64/ld-2.5.so)
3 0x3000219000 D	128020 0x30000010AE: _dl_start (in /lib64/ld-2.5.so)
4 0x300021A000 D	132170 0x30000010B5: _dl_start (in /lib64/ld-2.5.so)
5 0x3000008000 C	10489930 0x3000008B20: _dl_setup_hash (in /lib64/ld-2.5.so)
6 0x3000012000 C	13880830 0x3000012CC0: _dl_sysdep_start (in /lib64/ld-2.5.so)
7 0x3000013000 C	18091130 0x3000013440: brk (in /lib64/ld-2.5.so)
8 0x3000014000 C	19123850 0x3000014020: strlen (in /lib64/ld-2.5.so)
9 0x3000002000 C	23772480 0x3000002450: dl_main (in /lib64/ld-2.5.so)

图 7.8: pagein 工具的输出

7.5 页面错误优化

在支持请求分页的 Linux 等操作系统上, mmap 调用仅修改页表。它确保对于文件支持的页面, 可以找到底层数据, 对于匿名内存, 它确保在访问时提供用零初始化的页面。mmap 调用时没有分配实际内存。⁴⁶

分配部分发生在第一次访问内存页时, 无论是通过读取或写入数据, 还是通过执行代码。为了响应随后发生的页面错误, 内核使用页表树进行控制并确定页面上必须存在的数据。这种页面错误的解决方案并不便宜, 但进程使用的每个页面都会发生这种情况。

为了最大限度地减少页面错误的成本, 必须减少已使用页面的总数。优化代码大小将对此有所帮助。为了降低特定代码路径(例如启动代码)的成本, 还可以重新排列代码, 以便在该代码路径中最小化触摸页面的数量。然而, 确定正确的顺序并不容易。

作者基于 valgrind 工具集编写了一个工具, 用于测量发生的页面错误。不是页面错误的数量, 而是页面错误发生的原因。pagein 工具发出有关页面错误的顺序和时间的信息。输出写入名为 pagein.<PID> 的文件, 如图 7.8 所示。第二列指定调入页的地址。第三列指示是代码页还是数据页, 分别包含“C”或“D”。第四列指定自第一次缺页以来经过的周期数。该行的其余部分是 valgrind 尝试查找导致页面错误的地址的名称。地址值本身是正确的, 但如果没有可用的调试信息, 名称并不总是准确的。

在图 7.8 的示例中, 执行从地址 3000000B50₁₆ 开始, 这迫使系统分页到地址 3000000000₁₆ 处的页面。不久之后, 此后的页面也被引入; 调用的函数

该页面是 _dl_start。初始代码访问页 7FF000000₁₆ 上的变量。这发生在第一个页面错误之后的 3,320 个周期, 并且很可能是程序的第二条指令(仅在第一条指令之后的三个字节)。如果人们查看该程序, 就会注意到这种内存访问有一些特殊之处。该指令是一条调用指令, 它不会显式加载或存储数据。不过, 它确实将返回地址存储在堆栈上, 这正是这里发生的情况。但这不是进程的官方堆栈, 而是 valgrind 应用程序的内部堆栈。这意味着在解释 pagein 的结果时, 重要的是要记住 valgrind 引入了一些伪影。

pagein 的输出可用于确定哪些代码序列在程序代码中理想情况下应相邻。快速浏览一下 /lib64/ld-2.5.so 代码可以看出, 第一条指令立即调用函数 _dl_start, 并且这两个位置位于不同的页面上。重新排列代码以将代码序列移动到同一页上可以避免(或至少延迟)页面错误。到目前为止, 确定最佳代码布局应该是一个繁琐的过程。由于按照设计, 页面的第二次使用不会被记录, 因此需要通过反复试验来查看更改的效果。使用调用图分析, 可以猜测可能的调用序列; 这可能有助于加快函数和变量的排序过程。

在非常粗略的层面上, 可以通过查看构成可执行文件或 DSO 的目标文件来查看调用序列。从一个或多个入口点(即函数名称)开始, 可以计算依赖链。无需太多努力, 这在目标文件级别就可以很好地工作。在每一轮中, 确定哪些目标文件包含所需的函数和变量。种子集必须明确指定。然后确定这些目标文件中所有未定义的引用, 并将它们添加到所需符号集中。重复直到该组稳定。

该过程的第二步是确定订单。各种目标文件必须组合在一起以填充尽可能少的页面。作为额外的好处, 任何功能都不应跨越页面边界。所有这一切的一个复杂之处在于, 为了最好地安排目标文件, 必须知道链接器稍后将做什么。重要的事实

⁴⁶If you want to say “Wrong!” wait a second, it will be qualified later that there are exceptions.

链接器将按照目标文件在输入文件（例如档案）和命令行中出现的顺序将目标文件放入可执行文件或 DSO 中。这给了程序员足够的控制权。

对于那些愿意投入更多时间的人来说，在使用 `-finstrument-functions` 选项调用时，通过 `__cyg_profile_func_enter` 和 `__cyg_profile_func_exit` 挂钩 gcc 插入，使用自动调用跟踪进行了重新排序的成功尝试[17]。有关这些 `__cyg_*` 接口的更多信息，请参阅 gcc 手册。通过创建程序执行的跟踪，程序员可以更准确地确定调用链。[17] 中的结果是，仅通过对功能进行重新排序，启动成本就降低了 5%。主要好处是减少了页面错误的数量，但 TLB 缓存也发挥了作用——考虑到在虚拟化环境中，TLB 未命中的代价显著增加，这一作用变得越来越重要。

通过将分页工具的分析与调用序列信息相结合，应该可以优化程序的某些阶段（例如启动），以最大限度地减少页面错误的数量。

Linux 内核提供了两种额外的机制来避免页面错误。第一个是 `mmap` 的标志，它指示内核不仅修改页表，而且实际上预先对映射区域中的所有页进行故障排除。这是通过简单地将 `MAP_POPULATE` 标志添加到 `mmap` 调用的第四个参数来实现的。这将导致 `mmap` 调用的成本显著增加，但是，如果立即使用该调用映射的所有页面，则好处可能会很大。程序不会出现许多页面错误（由于同步要求等产生的开销，每一个页面错误都相当昂贵），程序将有一个更昂贵的 `mmap` 调用。但是，在调用后不久（或永远）不会使用大部分映射页面的情况下，使用此标志有缺点。映射的、未使用的页面显然浪费时间和内存。立即预先出现故障并且稍后才使用的页面也可能堵塞系统。内存在使用之前就被分配，这可能会导致内存短缺。另一方面，在最坏的情况下，页面只是被重新用于新的目的（因为它还没有被修改），这并不是那么昂贵，但仍然与分配一起增加了一些成本。

`MAP_POPULATE` 的粒度太粗了。还有第二个可能的问题：这是一个优化；所有页面确实都被映射进来并不重要。如果系统太忙而无法执行该操作，则可以放弃预故障。一旦页面被真正使用，程序就会出现页面错误，但这并不比人为造成资源稀缺更糟糕。另一种方法是将 `POSIX_MADV_WILLNEED` 建议与 `posix_madvise` 函数一起使用。这是向操作系统暗示，在不久的将来，程序将需要调用中描述的页面。内核是

可以随意忽略该建议，但它也可以预置故障页面。这里的优点是粒度更细。任何映射地址空间区域中的单个页面或页面范围都可以预先发生故障。对于包含大量运行时不使用的数据的内存映射文件，这比使用 `MAP_POPULATE` 具有巨大的优势。

除了这些最小化页面错误数量的主动方法之外，还可以采用硬件设计人员流行的更被动的方法。DSO 占用地址空间中的相邻页，每个页范围用于代码和数据。页面大小越小，容纳 DSO 所需的页面就越多。反过来，这也意味着更多的页面错误。这里重要的是，反之亦然。对于较大的页面大小，映射（或匿名内存）所需的页面数量会减少；页面错误的数量也会随之下降。

大多数架构支持 4k 的页面大小。在 IA-64 和 PPC64 上，64k 的页面大小也很流行。这意味着分配内存的最小单位是 64k。该值必须在编译内核时指定，并且不能动态更改（至少目前不能）。多页面大小架构的 ABI 旨在允许运行具有任一页面大小的应用程序。运行时将进行必要的调整，正确编写的程序不会注意到任何事情。较大的页面大小意味着部分使用的页面会造成更多浪费，但在某些情况下，这是可以的。

大多数体系结构还支持 1MB 或更大的非常大的页面大小。此类页面在某些情况下也很有用，但以这么大的单位分配所有内存是没有意义的。物理内存的浪费实在是太大了。但非常大的页面有其优点：如果使用巨大的数据集，将它们存储在 x86-64 上的 2 MB 页面中，与使用 4k 页面使用相同数量的内存相比，需要减少 511 个页面错误（每个大页面）。这可以产生很大的不同。解决方案是有选择地请求内存分配，仅对于请求的地址范围，使用巨大的内存页，对于同一进程中的所有其他映射，使用正常的页大小。

不过，巨大的页面尺寸是有代价的。由于用于大页面的物理内存必须是连续的，一段时间后，由于内存碎片，可能无法分配此类页面。人们正在致力于内存碎片整理和碎片避免，但它非常复杂。对于大页面（例如 2MB）来说，所需的 512 个连续页面总是很难获得，除非有一次：系统启动时。这就是为什么当前的大页面解决方案需要使用特殊的文件系统，`hugetlbfs`。这个伪文件系统是根据系统管理员的请求分配的，方法是写入应保留的大页数

`/proc/sys/vm/nr_hugepages`

如果找不到足够的连续内存，此操作可能会失败。如果使用虚拟化，情况会变得特别有趣。使用VMM模型的虚拟化系统不直接管理物理内存，因此无法自行分配hugetlbfs。它必须依赖于VMM，并且不保证支持该功能。对于KVM模型，运行KVM模块的Linux内核可以执行Hugetlbfs分配，并可能将因此分配的页面子集传递来宾域之一。

后来，当程序需要大页面时，有多种可能性：

- 程序可以使用带有 SHM_HUGETLB 标志的 System V 共享内存接口。
- Hugetlbfs 类型的文件系统实际上可以被挂载，然后程序可以在挂载点下创建一个文件，并使用 mmap 将一页或多页映射为匿名内存。

在第一种情况下，不需要安装hugetlbfs。请求一个或多个大页面的代码可能如下所示：

```
key_t k = ftok("/some/key/file", 42); int id = shmget(k,
长度, SHM_HUGETLB | IPC_CREAT | SHM_R | SHM_W); void *a = shmat(id, NULL, 0);
```

该代码序列的关键部分是 SHM_HUGETLB 标志的使用以及正确的 LENGTH 值的选择，该值必须是系统大页面大小的倍数。不同的架构有不同的价值。System V 共享内存接口的使用存在一个令人讨厌的问题，即依赖于关键参数来区分（或共享）映射。ftok 接口很容易产生冲突，这就是为什么，如果可能的话，最好使用其他机制。

如果挂载hugetlbfs文件系统的要求不是问题，最好使用它而不是System V共享内存。使用特殊文件系统的唯一真正问题是内核必须支持它，并且还没有标准化的安装点。一旦文件系统被挂载，例如在/dev/hugetlb，程序就可以轻松使用它：

```
int fd = open("/dev/hugetlb/file1", O_RDWR | O_CREAT, 0700); void *a = mmap(NULL, LENGTH, PROT_READ | PROT_WRITE, fd, 0);
```

通过在开放调用中使用相同的文件名，多个进程可以共享相同的大页面并进行协作。也可以使页面可执行，在这种情况下，还必须在 mmap 调用中设置 PROT_EXEC 标志。与 System V 共享内存示例一样，LENGTH 的值必须是系统大页面大小的倍数。

防御性编写的程序（所有程序都应该如此）可以使用如下函数在运行时确定挂载点：

```
char *hugetlbfs_mntpoint(void) { char *结果 = NULL; 文件 *fp = setmntent(_PATH_MOUNTED, "r"); if (fp != NULL) { struct mntent *m; while ((m = getmntent(fp)) != NULL) if (strcmp(m->mnt_fsname, "hugetlbfs") == 0) { 结果 = strdup(m->mnt_dir); 休息; } endmntent(fp); } 返回结果; }
```

有关这两种情况的更多信息可以在作为内核源代码树一部分的hugetlbpage.txt 文件中找到。该文件还描述了 IA-64 所需的特殊处理。

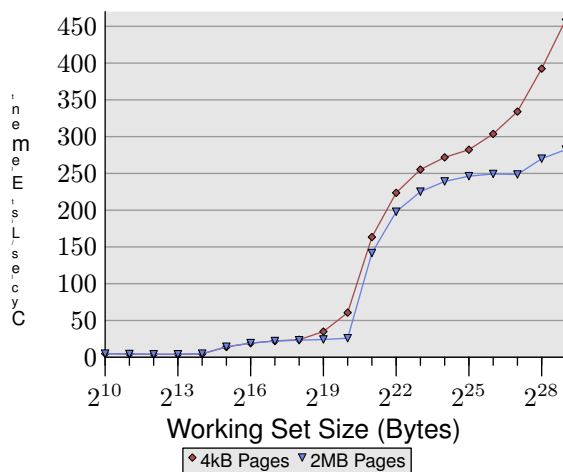


图 7.9：遵循大页面，NPAD=0

为了说明大页面的优点，图 7.9 显示了对 NPAD=0 运行随机 Follow 测试的结果。这与图 3.15 中显示的数据相同，但是，这一次，我们还使用分配在大页中的内存来测量数据。可以看出，性能优势是巨大的。对于 2²⁰ 字节，使用大页面的测试速度快了 57 %。这是因为这个尺寸

仍然完全适合单个 2MB 页，因此不会发生 DTLB 缺失。

在此之后，奖金最初会较小，但随着工作集大小的增加而再次增长。对于 512MB 工作集大小，大页面测试速度提高了 38%。大页测试的曲线在 250 个周期左右达到稳定状态。除了 2^{27} 字节的工作集之外，数字再次显着上升。出现稳定状态的原因是 2MB 页的 64 个 TLB 条目覆盖 2^{27} 字节。

正如这些数字所示，使用大工作集大小的成本很大一部分来自 TLB 未命中。使用本节中描述的接口可以获得巨大的回报。图中的数字很可能是上限，但即使是现实世界的程序也显示出显着的加速。数据库由于使用大量数据，因此属于当今使用大页的程序之一。

目前还没有办法使用大页面来映射文件支持的数据。人们对实现此功能很感兴趣，但迄今为止提出的建议都涉及显式使用大页面，并且它们依赖于 `hugefbfs` 文件系统。这是不可接受的：在这种情况下大页面的使用必须是透明的。内核可以轻松确定哪些映射较大并自动使用大页面。一个大问题是内核并不总是知道使用模式。如果本来可以映射为大页的内存后来需要 4k 页粒度（例如，因为使用 `mprotect` 更改了部分内存范围的保护），则将浪费大量宝贵的资源，特别是线性物理内存。因此，这种方法要成功实施肯定还需要一段时间。

8 即将推出的技术

在前面有关多处理器处理的部分中，我们已经看到，如果扩大 CPU 或内核的数量，必然会出现严重的性能问题。但这种扩大规模正是未来所期望的。处理器将获得越来越多的核心，程序必须更加并行才能利用 CPU 增加的潜力，因为单核性能将不会像以前那样快速提升。

8.1 原子操作的问题

同步访问共享数据结构传统上通过两种方式完成：

- 通过互斥，通常通过使用系统运行时的功能来实现；
- 通过使用无锁数据结构。

无锁数据结构的问题在于处理器必须提供可以原子地执行整个操作的原语。这种支持是有限的。在大多数体系结构上，支持仅限于原子读取和写入一个字。有两种基本方法可以实现这一点（参见第 6.4.2 节）：

- 使用原子比较和交换（CAS）操作；
- 使用加载锁定/存储条件（LL/SC）对。

可以很容易地看出如何使用 LL/SC 指令来实现 CAS 操作。这使得 CAS 操作成为大多数原子操作和无锁数据结构的构建块。

一些处理器，尤其是 x86 和 x86-64 架构，提供了一组更为复杂的原子操作。其中许多是针对特定目的的 CAS 操作的优化。例如，可以使用 CAS 和 LL/SC 操作来实现以原子方式向内存位置添加值，但 x86/x86-64 处理器上对原子增量的本机支持速度更快。对于程序员来说，了解这些操作以及在编程时使它们可用的内在函数非常重要，但这并不是什么新鲜事。

这两种架构的非凡扩展在于它们具有双字 CAS (DCAS) 操作。这对于某些应用程序很重要，但不是全部（参见[5]）。作为如何使用 DCAS 的示例，让我们尝试编写一个基于无锁数组的堆栈/LIFO 数据结构。使用 gcc 内部函数的第一次尝试如图 8.1 所示。

这段代码显然不是线程安全的。不同线程并发访问会修改全局变量 `top`

```

struct elem {
    data_t d;
    struct elem *c;
};
struct elem *top;
void push(struct elem *n) {
    n->c = top;
    top = n;
}
struct elem *pop(void) {
    struct elem *res = top;
    if (res != NULL)
        top = res->c;
    return res;
}

```

图 8.1: 非线程安全的 LIFO

不考虑其他线程的修改。元素可能会丢失，或者移除的元素可能会神奇地重新出现。可以使用互斥，但这里我们将尝试仅使用原子操作。

解决该问题的第一次尝试是在安装或删除列表元素时使用 CAS 操作。生成的代码如图 8.2 所示。

```

#define CAS __sync_bool_compare_and_swap
struct elem {
    data_t d;
    struct elem *c;
};
struct elem *top;
void push(struct elem *n) {
    do
        n->c = top;
        while (!CAS(&top, n->c, n));
}
struct elem *pop(void) {
    struct elem *res;
    while ((res = top) != NULL)
        if (CAS(&top, res, res->c))
            break;
    return res;
}

```

图 8.2: 使用 CAS 的 LIFO

乍一看，这似乎是一个可行的解决方案。top 永远不会被修改，除非它与操作开始时位于 LIFO 顶部的元素相匹配。但我们必须考虑各个级别的并发性。处理数据结构的另一个线程可能被安排在最糟糕的时刻。所谓的 ABA 问题就是这样的例子。考虑一下如果在 pop 中的 CAS 操作之前调度第二个线程并执行以下操作，会发生什么情况：

```

1.l = pop() 2.push(n
newelem) 3.push(l)

```

此操作的最终效果是 LIFO 的前一个顶部元素回到顶部，但第二个元素不同。回到第一个线程，因为顶部元素没有改变，所以 CAS 操作会成功。但 res->c 值不正确。它是指向原始 LIFO 的第二个元素的指针，而不是 newelem。结果就是这个新元素丢失了。

在文献 [10] 中，您可以找到使用某些处理器上的功能来解决此问题的建议。具体来说，这是关于 x86 和 x86-64 处理器执行 DCAS 操作的能力。这在图 8.3 中代码的第三个版本中使用。

```

#define CAS __sync_bool_compare_and_swap struct elem
{ data_t d;结构元素 *c; };struct lifo { struct elem *top; size
_t 代; } l; void Push(struct elem *n) { struct lifo 旧的,
新的; 做{旧= l; new.top = n->c = old.top;新代 = 旧代 +
1; while (!CAS(&l, 旧的, 新的)); }struct elem *pop(void) {
struct lifo 旧的, 新的; 做{旧= l; if (old.top == NULL)
返回 NULL; new.top = old.top->c;新代 = 旧代 + 1; while
(!CAS(&l, 旧的, 新的));返回旧的.top; }

```

图 8.3: 使用双字 CAS 的 LIFO

与其他两个示例不同，这是（当前）伪代码，因为 gcc 不理解 CAS 内在函数中结构的使用。无论如何，这个例子应该足以理解该方法。生成计数器被添加到指向 LIFO 顶部的指针。由于它在每次操作（push 或 pop）时都会更改，因此上述 ABA 问题不再是问题。当第一个线程通过实际交换顶部指针来恢复其工作时，生成计数器已增加了三倍。CAS 操作将失败，并且在下一轮循环中，正确的第一个和第二个

LIFO 的元素已确定并且 LIFO 未损坏。瞧a。

这真的是解决方案吗？[10]的作者确实让它听起来像这样，值得赞扬的是，应该提到的是，可以为 LIFO 构建允许使用上面代码的数据结构。但总的来说，这种方法和前一种方法一样注定会失败。我们仍然存在并发问题，只是现在在不同的地方。让我们假设一个线程执行 pop 并在测试 `old.top == NULL` 后被中断。现在，第二个线程使用 pop 并接收 LIFO 的前一个第一个元素的所有权。它可以用它做任何事情，包括更改所有值，或者在动态分配元素的情况下释放内存。

现在第一个线程恢复。旧变量仍然由 LIFO 的前一个顶部填充。更具体地说，顶部成员指向第二个线程弹出的元素。在 `new.top = old.top->c` 中，第一个线程取消引用元素中的指针。但 this 指针引用的元素可能已被释放。该部分地址空间可能无法访问，并且进程可能会崩溃。对于通用数据类型实现来说这是不允许的。解决这个问题的任何方法都是非常昂贵的：永远不能释放内存，或者至少在释放内存之前必须验证没有线程再引用该内存。鉴于无锁数据结构应该更快、更并发，这些额外的要求完全破坏了任何优势。在支持它的语言中，通过垃圾收集进行内存处理可以解决问题，但这需要付出代价。

对于更复杂的数据结构，情况往往更糟。上面引用的同一篇论文还描述了 FIFO 实现（在后续论文中进行了改进）。但这段代码有同样的问题。由于现有硬件（x86，x86-64）⁴⁷上的CAS操作仅限于修改内存中连续的两个字，因此它们在其他常见情况下根本没有帮助。例如，在双链表中的任何位置原子地添加或删除元素是不可能的。

问题是，通常会涉及多个内存地址，只有这些地址的值没有同时改变时，整个操作才能成功。这是数据库处理中众所周知的概念，这正是解决这一困境的最有希望的建议之一的来源。

8.2 事务内存

Herlihy 和 Moss 在 1993 年发表的开创性论文 [13] 中建议在硬件中实现内存操作的事务，因为仅靠软件无法有效地处理该问题。当时，数字设备公司已经在与可扩展性问题作斗争。

⁴⁷As a side note, the developers of the IA-64 did *not* include this feature. They allow *comparing* two words, but replacing only one.

他们的高端硬件配备了几十个处理器。其原理与数据库事务相同：事务的结果立即变得可见，或者事务被中止并且所有值保持不变。

这就是内存发挥作用的地方，也是上一节费心开发使用原子操作的算法的原因。在许多情况下，事务内存意味着原子操作的替代和扩展，特别是对于无锁数据结构。将交易系统集成到处理器中听起来是一件非常复杂的事情，但事实上，大多数处理器在某种程度上已经有了类似的东西。

一些处理器实现的LL/SC操作形成一个事务。SC 指令根据内存位置是否被触及来中止或提交事务。事务内存是这个概念的延伸。现在，参与事务的不是一对简单的指令，而是多条指令。要了解其工作原理，值得首先了解如何实现 LL/SC 指令。⁴⁸

8.2.1 加载锁定/存储条件实现

如果发出 LL 指令，则内存位置的值将加载到寄存器中。作为该操作的一部分，该值被加载到 L1d 中。只有当这个值没有被篡改时，后面的 SC 指令才能成功。处理器如何检测到这一点？回顾一下图 3.18 中 MESI 协议的描述，答案应该显而易见。如果另一个处理器更改了内存位置的值，则必须撤销第一个处理器的 L1d 中的值的副本。当 SC 指令在第一个处理器上执行时，它会发现必须再次将值加载到 L1d 中。这是处理器必须已经检测到的。

关于上下文切换（同一处理器上可能的修改）以及在另一个处理器上写入后意外重新加载缓存行，还有一些细节需要解决。这不是策略（上下文切换时的缓存刷新）和额外标志或 LL/SC 指令的单独缓存行无法解决的。一般来说，LL/SC 的实现几乎是免费的，与 MESI 等缓存一致性协议的实现一样。

8.2.2 事务内存操作

为了使事务内存普遍有用，事务一定不能用第一条存储指令完成。相反，实现应该允许一定数量的加载和存储操作；这意味着我们需要单独的提交和中止指令。稍后我们会看到我们还需要一条允许检查的指令 -

⁴⁸This does not mean it is actually implemented like this.

查询事务的当前状态以及事务是否已中止。

需要实现三种不同的内存操作：

- 读取内存
- 读取稍后写入的内存
- 写入内存

当查看 MESI 协议时，应该清楚这种特殊的第二种类型的读取操作如何有用。正常读取可以通过处于“E”和“S”状态的缓存行来满足。第二种类型的读取操作需要处于状态“E”的缓存行。从下面的讨论中可以看出为什么需要第二种类型的内存读取，但是，为了更完整的描述，感兴趣的读者可以参考从[13]开始的有关事务内存的文献。

此外，我们还需要事务处理，主要包括我们在数据库事务处理中已经熟悉的提交和中止操作。不过，还有一项操作在理论上是可选的，但对于使用事务内存编写健壮的程序是必需的。该指令让线程测试事务是否仍在正轨上并且可以（也许）稍后提交，或者事务是否已经失败并且在任何情况下都将被中止。

我们将讨论这些操作实际上如何与 CPU 缓存交互以及它们如何与总线操作匹配。但在此之前，我们先看一下一些使用事务内存的实际代码。这有望使本节的其余部分更容易理解。

8.2.3 使用事务内存的示例代码

对于这个例子，我们重新审视我们正在运行的例子，并提供一个使用事务内存原语的 LIFO 实现。

该代码看起来与非线程安全代码非常相似，这是一个额外的好处，因为它使使用事务内存编写代码变得更容易。代码的新部分是 LTX、ST、COMMIT 和 VALIDATE 操作。这四个操作是请求访问事务内存的方式。其实还有一个操作，LT，这里没有用到。LT 请求非独占读访问，LTX 请求独占读访问，ST 存储到事务内存中。VALIDATE 操作检查事务是否仍按计划提交。如果该事务仍然正常，则返回 true。如果事务已被标记为中止，则该事务实际上将被中止，并返回一个指示此情况的值。下一条事务内存指令将启动一个新事务。因此，代码使用新的 if 块，以防事务仍在继续。

```
结构体 elem { data_t d; 结构元素 *c; }; 结构元素*顶部; void Push(struct elem *n) { while (1) { n->c = LTX(top); } ST(&顶部, n); 如果 (提交 () ) 返回; ...延迟... } struct elem *pop(void) { while (1) { struct elem *res = LTX(top); if (VALIDATE()) { if (res != NULL) ST(&top, res->c); if (COMMIT()) 返回资源; } ... 延迟 ... } }
```

图 8.4：使用事务内存的 LIFO

COMMIT 操作完成事务；如果成功完成，操作将返回 true。这意味着程序的这一部分已经完成，线程可以继续前进。如果操作返回假值，这通常意味着必须重复整个代码序列。这就是外部 while 循环在这里所做的事情。不过，这并不是绝对必要的，在某些情况下，放弃工作是正确的做法。

LT、LTX 和 ST 操作的有趣之处在于，它们可能会失败，而无需以任何直接方式发出故障信号。程序请求此信息的方式是通过 VALIDATE 或 COMMIT 操作。对于加载操作，这可能意味着实际加载到寄存器中的值可能是假的；这就是为什么在上面的示例中需要在取消引用指针之前使用 VALIDATE。在下一节中，我们将了解为什么这是一个明智的实现选择。一旦事务内存实际上被广泛使用，处理器可能会实现一些不同的东西。不过，[13] 的结果表明了我们在这一章所描述的内容。

推送功能可以概括为：通过读取链表头部的指针来启动事务。读取请求独占所有权，因为稍后在函数中写入此变量。如果另一个线程已经启动了事务，则加载将失败并将死产事务标记为中止；在这种情况下，实际加载的值可能是垃圾。无论其状态如何，该值都存储在下一个字段中

新名单成员。这很好，因为该成员尚未使用，并且仅由一个线程访问。然后将指向列表头部的指针指定为指向新元素的指针。如果事务仍然OK，则本次写入可以成功。这是正常情况，只有当线程使用提供的push和pop函数之外的其他代码来访问此指针时，它才会失败。如果事务在 ST 执行时已经中止，则什么也不做。最后，线程尝试提交事务。如果成功，工作就完成了；其他线程现在可以开始它们的事务。如果交易失败，则必须从头开始重复。然而，在此之前，最好插入一个延迟。如果不这样做，线程可能会在繁忙的循环中运行（浪费能源，使 CPU 过热）。

pop 函数稍微复杂一些。它还首先读取包含列表头部的变量，请求独占所有权。然后代码立即检查 LTX 操作是否成功。如果没有，本轮除了推迟下一轮之外什么也不做。如果成功读取top指针，则说明其状态良好；我们现在可以取消引用该指针。请记住，这正是使用原子操作的代码的问题；使用事务内存，可以毫无问题地处理这种情况。以下 ST 操作仅在 LIFO 不为空时执行，就像原始的线程不安全代码一样。最后事务被提交。如果成功，该函数将返回指向头的旧指针；否则我们会延迟并重试。此代码中一个棘手的部分是要记住，如果事务已经失败，VALIDATE 操作将中止事务。下一个事务内存操作将启动一个新事务，因此，我们必须跳过函数中的其余代码。

当事务内存的实现在硬件中可用时，延迟代码的工作原理将是值得关注的。如果做得不好，系统性能可能会受到严重影响。

8.2.4 事务内存总线协议

现在我们已经了解了事务内存背后的基本原理，我们可以深入了解实现的细节。请注意，这是基于实际硬件的 *not*。它基于事务内存的原始设计和有关缓存一致性协议的知识。省略了一些细节，但仍然应该可以深入了解性能特征。

尽管有这个名称，事务内存实际上并不是作为单独的内存实现的。考虑到需要线程地址空间中任何位置的事务，这没有任何意义。相反，它是作为第一级缓存处理的一部分来实现的。理论上，该实现可以在正常的 L1d 中发生，但是，正如[13]所指出的，这不是一个好主意。我们更有可能看到事务缓存以标准方式实现

L1d 的等位基因。所有访问都将以与使用 L1d 相同的方式使用更高级别的缓存。事务缓存可能比 L1d 小得多。如果它是完全关联的，那么它的大小由事务可以包含的操作数量决定。实现可能会受到架构和/或特定处理器版本的限制。人们可以很容易地想象一个具有 16 个元素甚至更少的事务缓存。在上面的例子中，我们只需要一个内存位置；具有较大事务工作集的算法变得非常复杂。我们有可能会看到处理器在任一时间支持多个活动事务。缓存中的元素数量随后会成倍增加，但仍然足够小，足以实现完全关联。

事务缓存和 L1d 是独占的。这意味着缓存行最多位于其中一个缓存中，但绝不会同时位于两个缓存中。事务缓存中的每个槽在任何时候都处于四种 MESI 协议状态之一。除此之外，槽还有交易状态。状态如下（名称根据[13]）：

EMPTY 缓存槽不包含任何数据。MESI 状态始终为“I”。

NORMAL 缓存槽包含已提交的数据。数据也可能存在于 L1d 中。MESI 状态可以是“M”、“E”和“S”。允许“M”状态的事实意味着事务提交确实 *not* 强制将数据写入主内存（除非内存区域被声明为未缓存或直写）。这可以显著帮助提高性能。

XABORT 缓存槽包含中止时要丢弃的数据。这显然与XCOMMIT相反。事务期间创建的所有数据都保存在事务缓存中，在提交之前不会将任何内容写入主内存。这限制了最大事务大小，但这意味着，除了事务缓存之外，没有其他内存必须知道单个内存位置的 XCOMMIT/XABORT 对偶性。可能的 MESI 状态为“M”、“E”和“S”。

XCOMMIT 缓存槽包含在提交时被丢弃的数据。这是处理器可以实现的一种可能的优化。如果使用事务操作更改内存位置，则不能直接删除旧内容：如果事务失败，则需要恢复旧内容。MESI 状态与 XABORT 相同。与 XABORT 的一个区别是，如果事务缓存已满，任何处于“M”状态的 XCOMMIT 条目都可以写回内存，然后对于所有状态都将被丢弃。

当LT操作开始时，处理器在高速缓存中分配两个槽。首先寻找操作地址的 NORMAL 槽来选择受害者，即

缓存命中。如果找到这样的条目，则定位第二个槽，复制值，一个条目被标记为 XABORT，另一个条目被标记为 XCOMMIT。

如果该地址尚未被缓存，则定位空缓存槽。如果找不到，则寻找 NORMAL 插槽。如果 MESI 状态为“M”，则必须将旧内容刷新到内存中。如果也没有可用的 NORMAL 槽，则可能会损害 XCOMMIT 条目。不过，这可能是一个实现细节。事务的最大大小由事务缓存的大小决定，并且由于事务中每个操作所需的槽数量是固定的，因此在必须逐出 XCOMMIT 条目之前可以限制事务数量。

如果在事务高速缓存中未找到该地址，则在总线上发出 TREAD 请求。这就像正常的 READ 总线请求一样，但它表明这是针对事务缓存的。就像正常的 READ 请求一样，所有其他处理器中的缓存首先有机会响应。如果没有，则从主存储器中读取该值。MESI 协议确定新缓存行的状态是“E”还是“S”。当缓存行当前被另一个处理器或内核上的活动事务使用时，TREAD 和 READ 之间的区别就会发挥作用。在这种情况下，T-READ 操作明显失败，没有数据传输。生成 TREAD 总线请求的事务被标记为失败，并且操作中使用的值（通常是简单的寄存器加载）未定义。回顾该示例，我们可以看到，如果正确使用事务内存操作，此行为不会导致问题。在使用事务中加载的值之前，必须使用 VALIDATE 对其进行验证。几乎在任何情况下，这都是额外的负担。正如我们在使用原子操作创建 FIFO 实现的尝试中所看到的，我们添加的检查是一个缺失的功能，该功能将使无锁代码正常工作。

LTX 操作与 LT 几乎相同。唯一的区别是总线操作是 T RFO 而不是 T-READ。T RFO 与普通 RFO 总线消息一样，请求高速缓存行的独占所有权。结果缓存行的状态是“E”。与 TREAD 总线请求一样，T RFO 可能会失败，在这种情况下，使用的值也未定义。如果缓存行已经在本地事务缓存中且处于“M”或“E”状态，则无需执行任何操作。如果本地事务缓存中的状态为“S”，则总线请求必须发出以使所有其他副本无效。

ST 操作与 LTX 类似。该值首先在本地事务缓存中独占可用。然后 ST 操作将该值复制到缓存中的第二个槽中，并将该条目标记为 XCOMMIT。最后，另一个槽被标记为 XABORT 并将新值写入其中。如果事务已经中止，或者由于隐式 LTX 失败而新近中止，则不会写入任何内容。

VALIDATE 和 COMMIT 操作都不会自动执行

显式地、隐式地创建总线操作。这是事务内存相对于原子操作的巨大优势。通过原子操作，可以通过将更改的值写回主内存来实现并发。如果您到目前为止已经阅读过这篇文档，您应该知道它有多昂贵。对于事务内存，不会强制访问主内存。如果缓存没有 EMPTY 插槽，则必须逐出当前内容，并且对于处于“M”状态的插槽，必须将内容写入主内存。这与常规缓存没有什么不同，并且可以在没有特殊原子性保证的情况下执行写回。如果缓存大小足够，内容可以保存很长时间。如果事务在同一内存位置上一遍又一遍地执行，速度的提高可能是天文数字，因为在一种情况下，我们在每一轮中都有一到两次主内存访问，而对于事务内存，所有访问都会命中事务缓存，其速度与 L1d 一样快。

事务中止时执行的所有 VALIDATE 和 COMMIT 操作都是将标记为 XABORT 的缓存槽标记为空，并将 XCOMMIT 槽标记为 NORMAL。类似地，当 COMMIT 成功完成事务时，XCOMMIT 槽被标记为空，XABORT 槽被标记为 NORMAL。这些是对事务缓存的非常快的操作。不会向其他想要执行事务的处理器发出明确的通知；这些处理器只需要继续尝试。有效地做到这一点是另一回事。在上面的示例代码中，我们只是在适当的位置添加了...延迟...。我们可能会看到实际的处理器以有用的方式支持延迟。

总而言之，事务内存操作仅在启动新事务以及将尚未在事务高速缓存中的新高速缓存行添加到仍然成功的事务中时才会导致总线操作。中止事务中的操作不会导致总线操作。由于多个线程尝试使用相同的内存，因此不会出现缓存线乒乓现象。

8.2.5 其他考虑因素

在第 6.4.2 节中，我们已经讨论了如何使用 x86 和 x86-64 上可用的锁前缀来避免在某些情况下对原子操作进行编码。然而，当使用多个线程且不争用相同内存时，所提出的技巧就不够了。在这种情况下，就不必要使用原子操作了。有了事务内存，这个问题就消失了。仅当不同 CPU 同时或连续使用内存时，才会发出昂贵的 RFO 总线消息；只有在需要时才会出现这种情况。几乎不可能做得更好。

细心的读者可能会对延迟感到好奇。预计最坏的情况是什么？如果具有活动事务的线程被取消调度，或者如果它接收到信号并且可能被终止，或者决定

使用 `siglongjmp` 跳转到外部作用域？答案是：交易将被中止。每当线程进行系统调用或接收信号（即发生环级别更改）时，都可以中止事务。执行系统调用或处理信号时，中止事务也可能是操作系统职责的一部分。我们必须等到实现可用才能看到实际做了什么。

这里应该讨论的事务性记忆的最后一个是人们即使在今天也可能想要思考的。事务缓存与其他缓存一样，在缓存行上运行。由于事务缓存是独占缓存，事务和非事务操作使用相同的缓存行将是一个问题。因此，重要的是

- 将非事务性数据移出缓存行
- 对于单独事务中使用的数据有单独的缓存行

第一点并不新鲜，今天的原子操作也将得到同样的努力。第二个问题更严重，因为由于相关的高成本，现在对象几乎不会与缓存行对齐。如果所使用的数据以及使用原子操作修改的字位于同一高速缓存行上，则需要少一个高速缓存行。这不适用于互斥（互斥对象应始终具有自己的缓存行），但肯定可以出现原子操作与其他数据一起使用的情况。对于事务内存，将缓存行用于两个目的很可能是致命的。对数据⁴⁹的每次正常访问都会从事务缓存中删除缓存行，从而中止进程中的事务。数据对象的缓存对齐将来不仅是性能问题，而且是正确性问题。

事务存储器实现可能会使用更精确的记账，因此不会受到对作为事务一部分的高速缓存行上的数据的正常访问的影响。不过，这需要付出更多的努力，因为 MESI 协议信息已经不够了。

8.3 增加延迟

关于内存技术未来发展的一件事几乎可以肯定：延迟将继续增加。我们已经在第 2.2.4 节中讨论过，即将推出的 DDR3 内存技术将比当前的 DDR2 技术具有更高的延迟。如果部署 FB-DRAM，它也可能具有更高的延迟，特别是当 FB-DRAM 模块采用菊花链方式连接时。传递请求和结果并不是免费的。

⁴⁹From the cache line in question. Access to arbitrary other cache lines does not influence the transaction.

延迟的第二个来源是 NUMA 的使用增加。如果 AMD 的 Opteron 拥有多个处理器，那么它们就是 NUMA 机器。有一些本地内存通过自己的内存控制器连接到 CPU，但在 SMP 主板上，其余内存必须通过 Hypertransport 总线访问。英特尔的 CSI 技术将使用几乎相同的技术。由于每个处理器的带宽限制以及服务（例如）多个 10Gb/s 以太网卡的要求，即使每个插槽的核心数量增加，多插槽主板也不会消失。

延迟的第三个来源是协处理器。我们原以为，在 20 世纪 90 年代初不再需要商品处理器的数学协处理器之后，我们就摆脱了它们，但它们正在卷土重来。英特尔的 Geneseo 和 AMD 的 Torrenza 是该平台的扩展，允许第三方硬件开发商将其产品集成到主板中。也就是说，协处理器不必位于 PCIe 卡上，而是放置在距离 CPU 更近的位置。这给了他们更多的带宽。

IBM 在 Cell CPU 上走了一条不同的路线（尽管像英特尔和 AMD 这样的扩展仍然是可能的）。除了 PowerPC 内核之外，Cell CPU 还包含 8 个协同处理单元 (SPU)，它们是主要用于浮点计算的专用处理器。

协处理器和 SPU 的共同点是，它们的内存逻辑很可能比实际处理器还要慢。这在一定程度上是由必要的简化引起的：所有缓存处理、预取等都很复杂，特别是当也需要缓存一致性时。高性能程序将越来越依赖协处理器，因为性能差异可能很大。Cell CPU 的理论峰值性能为 210 GFLOPS，而高端 CPU 的理论峰值性能为 50-60 GFLOPS。如今使用的图形处理单元 (GPU、显卡上的处理器) 甚至可以达到更高的数量（超过 500 GFLOPS），并且无需太多努力，这些就可以集成到 Geneseo/Torrenza 系统中。

由于所有这些发展，程序员必须得出这样的结论：预取将变得更加重要。对于协处理器来说，这绝对至关重要。对于 CPU，尤其是内核越来越多的 CPU，有必要让 FSB 始终保持忙碌状态，而不是批量堆积请求。这需要通过有效使用预取指令让 CPU 尽可能多地了解未来的流量。

8.4 向量运算

当今主流处理器中的多媒体扩展仅以有限的方式实现矢量运算。向量指令的特点是作为一条指令（单指令多数据，SIMD）的结果执行大量操作。与标量运算相比，这可以是

谈到了多媒体指令，但这与 Cray-1 等矢量计算机或 IBM 3090 等机器的矢量单元相去甚远。

为了补偿一条指令执行的有限数量的操作（大多数机器上有四个浮点操作或两个双精度操作），必须更频繁地执行周围的循环。A.1 节中的示例清楚地表明了这一点，每个缓存行都需要 SM 迭代。

通过更宽的向量寄存器和运算，可以减少循环迭代的次数。这不仅仅带来了指令解码等方面的改进；在这里我们对记忆效应更感兴趣。通过加载或存储更多数据的单个指令，处理器可以更好地了解应用程序的内存使用情况，而不必尝试将各个指令的行为中的信息拼凑在一起。此外，提供不影响高速缓存的加载或存储指令变得更加有用。对于 x86 CPU 中 SSE 寄存器的 16 字节宽加载，使用未缓存的加载是一个坏主意，因为以后访问同一缓存行必须再次从内存加载数据（以防缓存未命中）。另一方面，如果向量寄存器足够宽以容纳一个或多个缓存行，则未缓存的加载或存储不会产生负面影响。对不适合缓存的数据集执行操作变得更加实用。

拥有大的向量寄存器并不一定意味着指令的延迟会增加；向量指令不必等到所有数据被读取或存储。如果向量单元可以识别代码流，则它可以从已读取的数据开始。这意味着，例如，如果要加载向量寄存器，然后将所有向量元素乘以标量，则一旦加载向量的第一部分，CPU 就可以开始乘法运算。这只是矢量单元复杂程度的问题。这表明，理论上，向量寄存器可以变得非常宽，并且今天设计的程序可能会考虑到这一点。实际上，由于处理器用于多进程和多线程操作系统，因此向量寄存器的大小受到限制。因此，上下文切换时间（包括存储和加载寄存器值）非常重要。

对于更宽的向量寄存器，存在操作的输入和输出数据无法在内存中顺序排列的问题。这可能是因为矩阵是稀疏的、通过列而不是行来访问矩阵以及许多其他因素。对于这种情况，向量单元提供了以非顺序模式访问内存的方法。单个向量加载或存储可以被参数化并指示从地址空间中的许多不同位置加载数据。使用当今的多媒体指令，这是根本不可能的。这些值必须一一显式加载，然后煞费苦心地组合到一个向量寄存器中。

过去的矢量单位有不同的模式

允许最有用的访问模式：

- 使用 *striding*，程序可以指定两个相邻向量元素之间的间隙有多大。所有元素之间的间隙必须相同，但这可以轻松地允许用一条指令将矩阵的列读入向量寄存器，而不是每行一条指令。
- 使用间接，可以创建任意访问模式。加载或存储指令接收指向数组的指针，该数组包含必须加载的实际内存位置的地址或偏移量。

目前尚不清楚我们是否会在未来版本的主流处理器中看到真正的向量运算的复兴。也许这项工作将被交给协处理器处理。无论如何，如果我们能够访问向量运算，正确组织执行此类运算的代码就显得尤为重要。代码应该是独立且可替换的，并且接口应该足够通用以有效地应用向量运算。例如，接口应该允许添加整个矩阵，而不是对行、列甚至元素组进行操作。构建块越大，使用向量运算的机会就越大。

在[11]中，作者热情呼吁向量运算的复兴。他们指出了许多优点并试图揭穿各种神话。不过，他们描绘的形象过于简单化。如上所述，大的寄存器集意味着高上下文切换时间，这在通用操作系统中必须避免。查看 IA-64 处理器在上下文切换密集型操作时存在的问题。如果涉及中断，向量运算的长执行时间也是一个问题。如果引发中断，处理器必须停止当前工作并开始处理中断。之后，它必须恢复执行被中断的代码。在工作中中断指令通常是一个大问题；这并非不可能，但很复杂。对于长时间运行的指令，这必须发生，或者指令必须以可重新启动的方式实现，否则中断反应时间太长。后者是不可接受的。

就内存访问的对齐而言，向量单元也很宽容，这塑造了所开发的算法。当今的一些处理器（尤其是 RISC 处理器）需要严格的对齐，因此扩展到全向量运算并非易事。向量操作有很大的潜在优势，特别是当支持跨步和间接时，因此我们希望在未看到这个功能。

这是第 6.2.1 节中矩阵乘法的完整基准程序。有关所使用的内在函数的详细信息，读者可以参阅英特尔的参考手册。

```
#include <stdlib.h> #include <stdio.h> #include <emmintrin.h> #define N 1000 double res[N][N] __attribute__
e__ ((aligned (64))); 双 mul1[N][N] __attribute__ ((对齐 (64))); 双 mul2[N][N] __attribute__ ((对齐 (64))); #
define SM (CLS / sizeof (double)) int main (void) { // ... 初始化 mul1 和 mul2 int i, i2, j, j2, k, k2; 双*限制rr
es; 双*限制rmul1; 双*限制 rmul2; 对于 (i = 0; i < N; i += SM) 对于 (j = 0; j < N; j += SM) 对于 (k = 0;
k < N; k += SM) 对于 (i2 = 0, rres = &res[i][j], rmul1 = &mul1[i][k]; i2 < SM; rres += N, rmul1 += N) { _
mm_prefetch (&rmul1[8], _MM_HINT_NTA); for (k2 = 0, rmul2 = &mul2[k][j]; k2 < SM; ++k2, rmul2 +=
N) { __m128d m1d = _mm_load_sd (&rmul1[k2]); m1d = _mm_unpacklo_pd (m1d, m1d); for (j2 = 0; j2 <
SM; j2 += 2) { __m128d m2 = _mm_load_pd (&rmul2[j2]); __m128d r2 = _mm_load_pd (&rres[j2]); _mm
_store_pd (&rres[j2], _mm_add_pd (_mm_mul_pd (m2, m1d), r2)); } } } // ...使用 res 矩阵 return 0; }
```

循环的结构与第 6.2.1 节中的最终版本几乎相同。一个很大的变化是加载 `rmul1[k2]` 值已从内部循环中取出，因为我们必须创建一个两个元素具有相同值的向量。这就是 `_mm_unpacklo_pd()` 内在函数的作用。

唯一值得注意的是，我们显式地对齐了三个数组，以便我们期望在同一缓存行中的值实际上可以在那里找到。

A.2 调试分支预测

如果按照建议，使用第 6.2.2 节中的可能和不可能的定义，则很容易使用调试模式来检查假设是否确实正确。宏的定义可以替换为：

```
#ifndef 已调试
# 定义不太可能(expr) __builtin_expect (!!(expr), 0) # 定义可能(expr) __builtin
_expect (!!(expr), 1)
# 别的
asm (".section Predict_data, \"aw\"; .previous\n" ".section Predict_line, \"
a\"; .previous\n" ".section Predict_file, \"a\"; .previous");

# ifdef __x86_64__
# 定义 debugpred__(e, E) \({ long int _e = !!(e); \ asm volatile (".pushsection Predict_data\n" \ "..predictcn
t%=: .quad 0; .quad 0\n" \ ".section Predict_line; .quad %c1\n" \ ".section Predict_file; .quad %c2; .popsecti
on\n" \ "addq $1,..predictcnt%= (,%0,8) " \ : : "r" (_e == E), "i" (__LINE__), "i" (__FILE__)); \ __builtin_e
xpect (_e, E); \ })

# elif 定义了 __i386__
# 定义 debugpred__(e, E) \({ long int _e = !!(e); \ asm volatile (".pushsection Predict_data\n" \ "..predictcn
t%=: .long 0; .long 0\n" \ ".section Predict_line; .long %c1\n" \ ".section Predict_file; .long %c2; .popsectio
n\n" \ "incl ..predictcnt%= (,%0,4) " \ : : "r" (_e == E), "i" (__LINE__), "i" (__FILE__));

# else # 错误 “debugpred__ 定义缺失” # endif # 定义不太可能(ex
pt) debugpred__ ((expr), 0) # 定义可能(expr) debugpred__ ((expr), 1)
#endif
```

创建 ELF 文件时，这些宏使用 GNU 编译器和链接器提供的许多功能。DEBUGPRED 情况下的第一个 asm 语句定义了三个附加部分；它主要为汇编器提供有关如何创建节的信息。所有部分在运行时都可用，并且 Predict_data 部分是可写的。所有节名称都是有效的 C 标识符，这一点很重要。原因很快就会清楚。

可能和不可能宏的新定义引用了特定于机器的 debugpred__ 宏。该宏有以下任务：

1. 在predict_data 部分分配两个字来包含正确和错误预测的计数。这两个字段通过使用 %=: 获得唯一的名称。前导点确保符号不会污染符号表。
2. 在predict_line 部分分配一个字来包含可能或不可能使用宏的行号。

⁵⁰至少使用 GNU 工具链。

3. 在predict_file 部分分配一个字来包含指向可能或不可能使用的宏的文件名的指针。

4. 根据表达式 e 的实际值递增为此宏创建的“正确”或“不正确”计数器。我们在这里不使用原子操作，因为它们的速度要慢得多，并且在不太可能发生碰撞的情况下绝对精度并不那么重要。如果需要的话，很容易改变。

汇编器手册中描述了 .pushsection 和 .popsection 伪操作。感兴趣的读者被要求借助手册和一些试验和错误来探索这些定义的细节。

这些宏自动且透明地负责收集有关正确和错误分支预测的信息。缺少的是获得结果的方法。最简单的方法是为对象定义一个析构函数并在那里打印结果。这可以通过如下定义的函数来实现：

```
extern long int __start_predict_data; extern long int __stop_predict_data; extern long int __start_predict_line; extern const char *__start_predict_file; static void __attribute__((析构函数)) predprint(void) { long int *s = &__start_predict_data; 长整型 *e = &__stop_predict_data; 长整型 *sl = &__start_predict_line; const char **sf = &__start_predict_file; while (s < e) { printf("%s:%ld: 不正确的=%ld, 正确=%ld%s\n", *sf, *sl, s[0], s[1], s[0] > s[1] ? " <==== 警告" : ""); ++sl; ++sf; s += 2; } }
```

在这里，节名是有效的 C 标识符这一事实就发挥了作用；如果需要，GNU 链接器使用它自动定义该节的两个符号。__start_XYZ 符号对应于 XYZ 节的开头，而 __stop_XYZ 是 XYZ 节之后的第一个字节的位置。这些符号使得在运行时迭代该部分内容成为可能。请注意，由于节的内容可以来自链接器在链接时使用的所有文件，因此编译器和汇编器没有足够的信息来确定节的大小。只有使用这些神奇的链接器生成的符号才可以迭代节内容。

不过，该代码并不只迭代一个部分；涉及三个部分。因为我们知道，每向 Predict_data 部分添加两个单词，我们就会向 Predict_line 和 Predict_file 部分各添加一个单词，因此我们不必检查这两个部分的边界。我们只是携带指针并一致递增它们。

该代码为代码中出现的每个预测打印出一行。它突出显示了预测不正确的那些用途。当然，这可以改变，并且可以将调试模式限制为仅标记错误预测多于正确预测的条目。这些都是变革的候选人。有些细节使问题变得复杂；例如，如果分支预测发生在一个在多个地方使用的宏内部，则在做出最终判断之前必须同时考虑所有宏的使用。

最后两点评论：此调试操作所需的数据并不小，而且在 DSO 的情况下，数据昂贵（必须重新定位 Predict_file 部分）。因此，不应在生产二进制文件中启用调试模式。最后，每个可执行文件和 DSO 都会创建自己的输出，在分析数据时必须牢记这一点。

A.3 测量缓存行共享开销

本节包含测试程序，用于测量在同一高速缓存行上使用变量与在不同高速缓存行上使用变量的开销。

```
#include <error.h> #include
<pthread.h> #include <stdlib.
h>

#define N (原子? 10000000: 500000000)

静态 int 原子; 静态无符号 nthreads; 静态无符号显示
; 静态长**读; 静态pthread_barrier_t b; 静态 void * tf(
void *arg) { long *p = arg; if (原子) for (int n = 0; n < N;
++n) __sync_add_and_fetch(p, 1);否则 for (int n = 0; n <
N; ++n) { *p += 1; asm 易失性("" : "m" (*p));返回NULL
; }int main(int argc, char *argv[]) { if (argc < 2) disp = 0;
否则 disp = atol(argv[1]); if (argc < 3) nthreads = 2;否则 nt
hreads = atol(argv[2]) ?: 1; if (argc < 4) 原子 = 1;否则原子
= atol(argv[3]); pthread_barrier_init(&b, NULL, nthread
```

```
s) ;
```

```

    无效*p; posix_memalign(&p, 64, (nthreads * disp ? 1) * sizeof(long));长 *mem = p; pthread_t th[n线程];
pthread_attr_t a; pthread_attr_init(&a); cpu_set_t c; for (unsigned i = 1; i < nthreads; ++i) { CPU_ZERO(&c);
CPU_SET(i, &c); pthread_attr_setaffinity_np(&a, sizeof(c), &c); mem[i * disp] = 0; pthread_create(&th[i], &a,
tf, &mem[i * disp]); CPU_ZERO(&c); CPU_SET(0, &c); pthread_setaffinity_np(pthread_self(), sizeof(c), &c);
内存[0] = 0; tf(&mem[0]); if ((disp == 0 && mem[0] != nthreads * N) || (disp != 0 && mem[0] != N) ) erro
r(1,0,"mem[0] 错误: %ld 而不是 %d", mem[0], disp == 0 ? nthreads * N : N); for (无符号 i = 1; i < nthreads;
++i) { pthread_join(th[i], NULL); if (disp != 0 && mem[i * disp] != N) error(1,0,"mem[%u] 错误: %ld 而不
是 %d", i, mem[i * disp], N);返回0; }

```

这里提供的代码主要是为了说明如何编写一个测量缓存行开销等影响的程序。有趣的部分是 `tf`。编译器已知的 `__sync_add_and_fetch` 内在函数生成原子添加指令。在第二个循环中，我们必须“消耗”增量的结果（通过内联 `asm` 语句）。`asm` 没有引入任何实际代码；相反，它会阻止编译器将增量操作移出循环。

第二个有趣的部分是程序将线程固定到特定的处理器上。该代码假设处理器编号为 0 到 3，如果机器有四个或更多逻辑处理器，通常会出现这种情况。该代码可以使用 `libNUMA` 的接口来确定可用处理器的数量，但该测试程序应该可以广泛使用，而无需引入这种依赖性。用这样或那样的方法来修复是很容易的。

B 一些 OProfile 技巧

以下内容并不是关于如何使用 `oprofile` 的教程。有关于该主题的完整文档。相反，它旨在提供一些更高级别的提示，告诉您如何查看程序以找到可能的问题点。但在此之前我们至少必须有一个简单的介绍。

B.1 Oprofile 基础知识

Oprofile 的工作分为两个阶段：收集和分析。收集由内核执行；它不能在用户级别完成，因为测量使用 CPU 的性能计数器。这些计数器需要访问 MSR，而 MSR 又需要特权。

每个现代处理器都提供自己的一组性能计数器。在某些架构上，所有处理器实现都提供计数器的子集，而其他计数器的子集因版本而异。这使得提供有关 `oprofile` 使用的一般建议变得困难。目前还没有可以隐藏这些细节的计数器的更高级别的抽象。

处理器版本还控制在任意时间可以跟踪的事件数量以及组合。这使情况变得更加复杂。

如果用户知道有关性能计数器的必要详细信息，则可以使用 `opcontrol` 程序来选择应计数的事件。对于每个事件，必须指定“溢出数”（在 CPU 被中断以记录事件之前必须发生的事件数）、是否应为用户级别和/或内核对事件进行计数，以及最后一个“单元掩码”（它选择性能计数器的子功能）。

要计算 x86 和 x86-64 处理器上的 CPU 周期，必须发出以下命令：

```
opcontrol --事件 CPU_CLK_UNHALTED:30000:0:1:1
```

数字 30000 是超限数字。选择合适的值对于系统的行为和收集的数据很重要。要求接收有关事件每次发生的数据是一个坏主意。对于许多事件，这会使机器陷入停顿，因为它所做的只是收集事件溢出的数据；这就是 `oprofile` 强制使用最小值的原因。每个事件的最小值都不同，因为不同的事件在正常代码中触发的概率不同。

选择非常高的数字会降低轮廓的分辨率。每次溢出时，`oprofile` 都会记录当时执行的指令的地址；对于 x86 和 PowerPC，在某些情况下，它也可以记录回溯。⁵¹ 使用粗分辨率，热点可能无法获得代表性的命中次数；一切都与概率有关，这就是为什么 `oprofile` 被称为概率分析器。溢出数越低，对系统速度的影响就越大，但分辨率越高。

如果要对特定程序进行分析，并且系统不用于生产，则使用尽可能低的溢出值通常是最有用的。可以使用以下命令查询每个事件的确切值

```
opcontrol --列表事件
```

如果分析的程序与另一个进程交互，并且速度减慢会导致交互出现问题，这可能会出现。如果一个进程有一些实时要求，而这些要求在经常中断时无法满足，也可能导致问题。在这种情况下，必须找到中间立场。如果要长时间对整个系统进行分析，情况也是如此。低超限数量意味着大幅放缓。无论如何，`oprofile` 与任何其他分析机制一样，都会带来不确定性和不准确性。

分析必须使用 `opcontrol --start` 启动，并可以使用 `opcontrol --stop` 停止。当 `oprofile` 处于活动状态时，它会收集数据；这些数据首先在内核中收集，然后批量发送到用户级守护程序，在其中解码并写入文件系统。使用 `opcontrol --dump` 可以请求将内核中缓冲的所有信息释放到用户级别。

收集的数据可以包含来自不同性能计数器的事件。这些数字都是并行保存的，除非用户选择在单独的 `oprofile` 运行之间擦除存储的数据。可以在不同场合积累同一事件的数据。如果在不同的分析运行期间遇到事件，则数字会被添加（如果这是用户选择的数字）。

数据收集过程的用户级部分对数据进行多路分解。每个文件的数据单独存储。这是

⁵¹Backtrace 支持有望在某个时候适用于所有架构。

甚至可以区分各个可执行文件使用的 DSO，甚至可以区分各个线程的数据。由此产生的数据可以使用 `oparchive` 进行归档。该命令生成的文件可以传输到另一台机器并在那里执行分析。

使用 `opreport` 程序，我们可以根据分析结果生成报告。使用 `opannotate` 可以查看各种事件发生的位置：哪一条指令以及（如果数据可用）在哪一行源代码中。这使得找到热点变得容易。计算 CPU 周期将指出花费最多时间的地方（这包括缓存未命中），而计算退休指令则可以找到大多数执行指令的位置 - 两者之间存在很大差异。

对地址的一次点击通常没有任何意义。统计分析的一个副作用是，仅执行几次甚至仅一次的指令可能会被归因于命中。在这种情况下，有必要通过重复来验证结果。

B.2 看起来如何

`oprofile` 会话看起来就像这样简单：

```
$ opcontrol -i cachebench $ opcontrol -e INST_RETIRED:6000:0:0:
1 --start $ ./cachebench ... $ opcontrol -h
```

请注意，这些命令（包括实际程序）均以 `root` 身份运行。此处以 `root` 身份运行程序只是为了简单起见；该程序可以由任何用户执行，并且 `oprofile` 会识别它。下一步是分析数据。通过 `opreport` 我们看到：

```
CPU: 核心 2, 速度 1596 MHz (估计) 单位掩码为 0x00 (无单位掩码) 的 INST_RETIRED.ANY_P 事件计数 (退役指令数) 计数 6000 INST_RETIRED:6000 | 样本 | % | ----- 116452 100.000 cachebench
```

这意味着我们收集了一堆事件；现在可以使用 `opannotate` 更详细地查看数据。我们可以看到程序中记录最多事件的位置。 `opannotate --source` 输出的一部分如下所示：

```
      :static void :inc (struct l *l, unsigned n) :{ while (n-- > 0) /* inc 总计: 139
80 11.7926 */ : { 5 0.0042 : ++l->pad[0].l; 13974 11.7875: l = l->n; 1 8.4e-04: asm 易失性
( “” : : “r” (l) ) ; : } ;}
```

这是测试的内部功能，花费了很大一部分时间。我们看到样本分布在循环的所有三行中。主要原因是采样相对于记录的指令指针并不总是 100% 准确。CPU 乱序执行指令；重建精确的执行顺序以产生正确的指令指针是很困难的。最新的 CPU 版本尝试对少数几个事件执行此操作，但一般来说，这是不值得的，或者根本不可能。在大多数情况下，这并不重要。即使存在一组正态分布的样本，程序员也应该能够确定发生了什么。

B.3 开始分析

当开始分析一段代码时，我们当然可以开始查看程序中花费最多时间的地方。该代码当然应该尽可能地优化。但接下来会发生什么呢？程序在哪里花费了 *unnecessary* 时间？这个问题不太容易回答。

这种情况的问题之一是绝对值并不能说明真实情况。程序中的一个循环可能需要大部分时间，这很好。不过，CPU 利用率高的可能原因有很多。但更常见的是，CPU 使用率在整个程序中分布得更均匀。在这种情况下，绝对值指向很多地方，这是没有用的。

在许多情况下，查看两个事件的比率会很有帮助。例如，如果无法测量函数执行的频率，则函数中错误预测的分支数量可能毫无意义。是的，绝对值与程序的性能相关。每次调用的错误预测比率对于函数的代码质量更有意义。Intel 的 x86 和 x86-64 优化手册 [15] 描述了应该研究的比率（Core 2 事件引用文档中的附录 B.7）。以下是一些与内存处理相关的比率。

Instruction Fetch Stall	<code>CYCLES_L1I_MEM_STALLED / CPU_CLK_UNHALTED.CORE</code>	Ratio of cycles during which in instruction decoder is waiting for new data due to cache or ITLB misses.
ITLB Miss Rate	<code>ITLB_MISS_RETIRED / INST_RETIRED.ANY</code>	ITLB misses per instruction. If this ratio is high the code is spread over too many pages.
L1I Miss Rate	<code>L1I_MISSES / INST_RETIRED.ANY</code>	L1i misses per instruction. The execution flow is unpredictable or the code size is too large. In the former case avoiding indirect jumps might help. In the latter case block reordering or avoiding inlining might help.
L2 Instruction Miss Rate	<code>L2_IFETCH.SELF.I_STATE / INST_RETIRED.ANY</code>	L2 misses for program code per instruction. Any value larger than zero indicates code locality problems which are even worse than L1i misses.
Load Rate	<code>L1D_CACHE_LD.MESI / CPU_CLK_UNHALTED.CORE</code>	Read operations per cycle. A Core 2 core can service one load operation. A high ratio means the execution is bound by memory reads.
Store Order Block	<code>STORE_BLOCK.ORDER / CPU_CLK_UNHALTED.CORE</code>	Ratio if stores blocked by previous stores which miss the cache.
L1d Rate Blocking Loads	<code>LOAD_BLOCK.L1D / CPU_CLK_UNHALTED.CORE</code>	Loads from L1d blocked by lack of resources. Usually this means too many concurrent L1d accesses.
L1D Miss Rate	<code>L1D_REPL / INST_RETIRED.ANY</code>	L1d misses per instruction. A high rate means that prefetching is not effective and L2 is used too often.
L2 Data Miss Rate	<code>L2_LINES_IN.SELF.ANY / INST_RETIRED.ANY</code>	L2 misses for data per instruction. If the value is significantly greater than zero, hardware and software prefetching is ineffective. The processor needs more (or earlier) software prefetching help.
L2 Demand Miss Rate	<code>L2_LINES_IN.SELF.DEMAND / INST_RETIRED.ANY</code>	L2 misses for data per instruction for which the hardware prefetcher was not used at all. That means, prefetching has not even started.
Useful NTA Prefetch Rate	<code>SSE_PRE_MISS.NTA / SSS_PRE_EXEC.NTA</code>	Ratio of useful non-temporal prefetch relative to the total number of all non-temporal prefetches. A low value means many values are already in the cache. This ratio can be computed for the other prefetch types as well.
Late NTA Prefetch Rate	<code>LOAD_HIT_PRE / SSS_PRE_EXEC.NTA</code>	Ratio of load requests for data with ongoing prefetch relative to the total number of all non-temporal prefetches. A high value means the software prefetch instruction is issued too late. This ratio can be computed for the other prefetch types as well.

对于所有这些比率，程序应该在运行时指示 `oprofile` 来测量这两个事件。这保证了两个计数具有可比性。在划分之前，必须确保考虑到可能不同的溢出值。最简单的方法是通过将每个事件计数器乘以溢出值来确保这一点。

这些比率对于整个程序、可执行文件/DSO 级别、甚至函数级别都是有意义的。对程序的研究越深入，值中包含的错误就越多。

要理解这些比率，需要的是基线值。这并不像看起来那么容易。不同类型的代码具有不同的特性，在一个程序中不好的比率值在另一个程序中可能是正常的。

尽管这不是高效编程所必需的知识，但描述可用内存类型的一些更多技术细节可能会很有用。具体来说，我们在这里对“注册”与“未注册”以及 ECC 与非 ECC DRAM 类型的差异感兴趣。

术语“注册”和“缓冲”在 de- 时间同义使用。

划线 DRAM 类型，该类型在 DRAM 模块上具有一个附加组件：缓冲器。所有 DDR 内存类型都可以采用注册和非注册形式。对于未注册的模块，内存控制器直接连接到模块上的所有芯片。图 C.1 显示了设置。

这对电气要求相当高。内存控制器必须能够处理所有内存芯片的容量（图中所示的数量超过六个）。如果内存控制器 (MC) 有限制，或者要使用许多内存模块，则此设置并不理想。

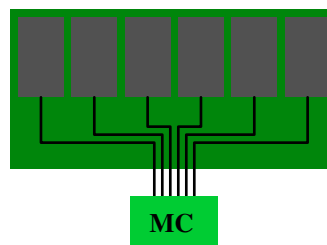


图 C.1: 未注册的 DRAM 模块

缓冲（或注册）内存改变了情况：而不是直接将 DRAM 模块上的 RAM 芯片连接到内存后，它们会连接到缓冲器，然后缓冲器又连接到内存控制器。这显着降低了电气连接的复杂性。内存控制器驱动 DRAM 模块的能力增加了与节省的连接数量相对应的系数。

有了这些优点，问题是：为什么不是所有 DRAM 模块都带有缓冲功能？有几个原因。显然，缓冲模块稍微复杂一些，因此也更昂贵。不过，成本并不是唯一的因素。缓冲器稍微延迟了来自 RAM 芯片的信号；这

延迟必须足够高，以确保来自 RAM 芯片的所有信号都得到缓冲。结果是 DRAM 模块的延迟增加。这里值得一提的最后一个因素是额外的电气元件会增加能源成本。由于缓冲器必须以总线频率运行，因此该组件的能耗可能很大。

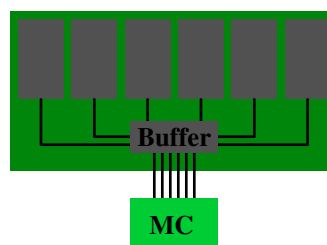


图 C.2: 注册的 DRAM 模块

由于使用 DDR2 和 DDR3 模块的其他因素，每个存储体通常不可能拥有两个以上的 DRAM 模块。内存控制器的引脚数量限制了存储体的数量（在商用硬件中为两个）。大多数内存控制器能够驱动四个 DRAM 模块，因此，未注册的模块就足够了。在内存要求较高的服务器环境中，情况可能有所不同。

某些服务器环境的另一个方面是它们不能容忍错误。由于 RAM 单元中的电容器所保留的电荷极少，因此可能会出现错误。人们经常开玩笑说宇宙辐射，但这确实是有可能的。与 alpha 衰变和其他自然现象一起，它们会导致 RAM 单元的内容从 0 变为 1 的错误，反之亦然。使用的内存越多，发生此类事件的概率就越高。

如果无法接受此类错误，则可以使用 ECC（纠错码）DRAM。纠错码使硬件能够识别不正确的单元内容，并在某些情况下纠正错误。在过去，奇偶校验只能识别错误，并且当检测到错误时必须停止机器。相反，使用 ECC 可以自动纠正少量错误位。但是，如果错误数量太多，则无法正确执行内存访问，并且机器仍然会停止。不过，对于工作中的 DRAM 模块来说，这种情况不太可能发生，因为同一模块上必定会发生多个错误。

当我们谈论 ECC 内存时，我们实际上并不完全正确。执行错误检查的不是内存；而是内存。相反，它是内存控制器。DRAM 模块只是提供更多存储空间并随真实数据一起传输额外的非数据位。通常，ECC 存储器每 8 个数据位存储一位附加位。为什么使用 8 位将在稍后解释。

将数据写入内存地址后，内存控制器会动态计算新内容的 ECC，然后将该数据和 ECC 发送到内存总线。读取时，接收到数据加上 ECC，内存控制器计算数据的 ECC，并将其与从 DRAM 模块发送的 ECC 进行比较。如果 ECC 匹配，则一切正常。如果它们不匹配，内存控制器会尝试纠正错误。如果无法进行此纠正，则会记录错误并且机器可能会停止。

Data Bits W	SEC		SEC/DED	
	ECC Bits E	Overhead	ECC Bits E	Overhead
4	3	75.0%	4	100.0%
8	4	50.0%	5	62.5%
16	5	31.3%	6	37.5%
32	6	18.8%	7	21.9%
64	7	10.9%	8	12.5%

表 C.1: ECC 和数据位关系

目前正在使用多种纠错技术，但对于 DRAM ECC，通常使用汉明码。汉明码最初用于对四个数据位进行编码，能够识别和纠正一个翻转位（SEC，单纠错）。该机制可以轻松扩展到更多数据位。数据位数 W 和错误代码位数 E 之间的关系由等式描述

$$E = \lceil \log_2(W + E + 1) \rceil$$

迭代求解该方程可得到表 C.1 第二列中所示的值。通过附加位，我们可以使用简单的奇偶校验位来识别两个翻转位。这称为 SEC/DED，单错误纠正/双错误检测。通过这个附加位，我们得到表 C.1 第四列中的值。 $W = 64$ 的开销足够低，并且数字 (64, 8) 是 8 的倍数，因此这是 ECC 的自然选择。在大多数模块上，每个 RAM 芯片产生 8 位，因此任何其他组合都会导致解决方案效率较低。

	7	6	5	4	3	2	1
ECC Word	D	D	D	P	D	P	P
P ₁ Parity	D	-	D	-	D	-	P
P ₂ Parity	D	D	-	-	D	P	-
P ₄ Parity	D	D	D	P	-	-	-

汉明码计算很容易用代码来演示

使用 $W = 4$ 和 $E = 3$ 。我们在编码字中的重要位置计算奇偶校验位。表 C.2 给出了其原理。在对应于 2 的幂的位位置处添加奇偶校验位。第一个奇偶校验位 P_1 的奇偶校验和包含每隔一个位。第二奇偶校验位 P_2 的奇偶校验和包含数据位 1、3 和 4（此处编码为 3、6 和 7）。类似地计算 P_4 。

表 C.2: 汉明生成矩阵构造

奇偶校验位的计算可以使用矩阵乘法来更优雅地描述。我们构造一个矩阵 $G = [I|A]$

其中 I 是单位矩阵， A 是奇偶校验生成矩阵，我们可以从表 C.2 中确定。

$$G = \left[\begin{array}{cccc|ccc} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{array} \right]$$

A 的列是根据 P_1 、 P_2 和 P_4 计算中使用的位构造的。如果我们现在将每个输入数据项表示为 4 维向量 d ，我们可以计算 $r = d \cdot G$ 并获得 7 维向量 r 。这是在 ECC DDR 情况下存储的数据。

为了解码数据，我们构建了一个新的矩阵 $H = [A^T|I]$ ，其中 A^T 是通过计算 G 得到的转置奇偶校验生成矩阵。这意味着：

$$H = \left[\begin{array}{cccc|ccc} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{array} \right]$$

$H \cdot r$ 的结果显示存储的数据是否有缺陷。如果不是这种情况，则乘积为 3 维向量 $(0\ 0\ 0)^T$ 。否则，当将乘积的值解释为数字的二进制表示形式时，它表示具有翻转位的列号。

例如，假设 $d = (1\ 0\ 0\ 1)$ 。这导致

$$r = (1 \ 0 \ 0 \ 1 \ 0 \ 0 \ 1)$$

使用与 H 的乘法执行测试结果为

$$s = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

现在，假设我们存储的数据已损坏并从内存中读回 $r' = (1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1)$ 。在这种情况下我们得到

$$s' = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$$

该向量不是空向量，当解释为数字时， s' 的值为 5。这是我们在 r' (中翻转的位数，开始从 1) 开始计数。内存控制器可以纠正该位，并且程序不会注意到存在问题。

处理 DED 部分的额外位只是稍微复杂一些。通过付出更多的努力，就有可能创建可以纠正两个翻转位甚至更多位的代码。决定是否需要的是概率和风险。一些内存制造商表示，256MB RAM 每 750 小时就会出现一次错误。通过将内存量加倍，时间减少了 75%。如果内存足够，短时间内出现错误的可能性可能会很大，因此需要 ECC RAM。时间框架甚至可能太短，以至于 SEC/DED 的实施不够充分。

服务器主板无需实现更多纠错功能，而是能够在给定时间范围内自动读取所有内存。这意味着，无论处理器是否实际请求内存，内存控制器都会读取数据，如果 ECC 检查失败，则将更正后的数据写回内存。只要在读取所有内存并将其写回所需的时间范围内发生少于两个内存错误的概率是可以接受的，SEC/DED 纠错就是一个完全合理的解决方案。

与注册 DRAM 一样，我们必须问一个问题：为什么 ECC DRAM 不是标准？这个问题的答案与有关注册 RAM 的等效问题相同：额外的 RAM 芯片会增加成本，而奇偶校验计算会增加延迟。未注册的非 ECC 内存可以明显更快。由于寄存式和非 ECC DRAM 问题的相似性，人们通常只能找到寄存式 ECC DRAM，而不是寄存式非 ECC DRAM。

还有另一种方法可以克服内存错误。一些制造商提供通常被错误地称为“内存 RAID”的技术，其中数据冗余地分布在多个 DRAM 模块或至少 RAM 芯片上。具有此功能的主板可以使用未注册的 DRAM 模块，但内存总线上流量的增加可能会抵消 ECC 和非 ECC DRAM 模块访问时间的差异。

D libNUMA简介

尽管程序员需要优化线程调度、适当分配内存等的许多信息都是可用的，但这些信息获取起来很麻烦。从长远来看，现有的 NUMA 支持库（libnuma，位于 RHEL/Fedora 系统上的 numactl 软件包中）无法提供足够的功能。

作为回应，作者提出了一个新的库，它提供 NUMA 所需的所有功能。由于内存和缓存层次结构处理的重叠，该库对于具有多线程和多核处理器的非 NUMA 系统也很有用——几乎所有当前可用的机器。

迫切需要这个新库的功能来遵循本文档中给出的建议。这是这里提到它的唯一原因。该库（截至撰写本文时）尚未完成，未经过审查，未完善，也未（广泛）分发。未来可能会发生重大变化。目前可在

<http://people.redhat.com/drepper/libNUMA.tar.bz2>

该库的接口很大程度上依赖于 /sys 文件系统导出的信息。如果未安装此文件系统，许多功能将直接失败或提供不准确的信息。如果进程是在 chroot 监狱中执行的，记住这一点尤其重要。

该库的接口头当前包含以下定义：

```
typedef memnode_set_t;

#define MEMNODE_ZERO_S(setsize, memnodesetp) #define MEMNODE
_SET_S(node, setsize, memnodesetp) #define MEMNODE_CLR_S(node, s
etsize, memnodesetp) #define MEMNODE_ISSET_S(node, setsize, memno
desetp) #define MEMNODE_COUNT_S(setsize, memnodesetp)

#define MEMNODE_EQUAL_S(setsize, memnodesetp1, memnodesetp2)

#define MEMNODE_AND_S(setsize, destset, srcset1, srcset2) #define MEMNODE
_OR_S(setsize, destset, srcset1, srcset2) #define MEMNODE_XOR_S(setsize, dests
et, srcset1, srcset2)

#define MEMNODE_ALLOC_SIZE(count) #def
ine MEMNODE_ALLOC(count) #define MEM
NODE_FREE(memnodeset)

int NUMA_cpu_system_count(void); int NUMA_cpu_system_mask(size_t destsize, cpu
_set_t *dest);

int NUMA_cpu_self_count(void); int NUMA_cpu_self_mask(size_t destsize, cpu_se
t_t *dest);

int NUMA_cpu_self_current_idx(void); int NUMA_cpu_self_current_mask(size_t destsize, cpu_
set_t *dest);

ssize_t NUMA_cpu_level_mask(size_t destsize, cpu_set_t *dest, size_t srcsize, const cpu_set_t *
src, unsigned int level);

int NUMA_memnode_system_count(void); int NUMA_memnode_system_mask(size_t destsize, memnode_set_t
*dest); int NUMA_memnode_self_mask(size_t destsize, memnode_set_t *dest); int NUMA_memnode_self_curr
ent_idx(void); int NUMA_memnode_self_current_mask(size_t destsize, memnode_set_t *dest); int NUMA_cpu_
to_memnode(size_t cpusetsize, const cpu_set_t *cpuset, size_t __memnodesize, memnode_set_t *memnodeset); i
nt NUMA_memnode_to_cpu(size_t memnodesize, const memnode_set_t *memnodeset,
```

```
size_t cpusetsize, cpu_set_t *cpuset);

int NUMA_mem_get_node_idx(void *addr); int NUMA_mem_get_node_mask(void *addr, size_t size, size_t destsize, memnode_set_t *dest);
```

MEMNODE_* 宏在形式和功能上与第 6.4.3 节中介绍的 CPU_* 宏类似。宏没有非 _S 变体，它们都需要大小参数。memnode_set_t 类型与 cpu_set_t 等效，但这次用于内存节点。请注意，内存节点的数量不需要与 CPU 的数量有任何关系，反之亦然。每个内存节点可能有多个 CPU，甚至根本没有 CPU。因此，动态分配的内存节点位集的大小不应由 CPU 的数量决定。

相反，应该使用 NUMA_memnode_system_count 接口。它返回当前注册的节点数。随着时间的推移，这个数字可能会增加或减少。但通常情况下，它会保持不变，因此对于调整内存节点位集的大小来说是一个很好的值。分配与 CPU_* 宏类似，使用 MEMNODE_ALLOC_SIZE、MEMNODE_ALLOC 和 MEMNODE_FREE 进行。

作为与 CPU_* 宏的最后并行，该库还提供宏来比较内存节点位集是否相等并执行逻辑运算。

NUMA_cpu_* 函数提供处理 CPU 集的功能。在某种程度上，这些接口只是使现有功能以新名称可用。NUMA_cpu_system_count 返回系统中的 CPU 数量，NUMA_CPU_system_mask 变体返回具有适当位集功能的位掩码，该功能在其他情况下不可用。

NUMA_cpu_self_count 和 NUMA_cpu_self_mask 返回有关当前线程当前允许运行的 CPU 的信息。NUMA_cpu_self_current_idx 返回当前使用的 CPU 的索引。由于内核可以做出的调度决策，这些信息在返回时可能已经过时了；它总是必须被假设为不准确的。NUMA_cpu_self_current_mask 返回相同的信息并设置位集中的适当位。

NUMA_memnode_system_count 已经被引入。NUMA_memnode_system_mask 是填充位集的等效函数。NUMA_memnode_self_mask 根据直接连接到线程当前可以运行的任何 CPU 的内存节点填充一个位集。

NUMA_memnode_self_current_idx 和 NUMA_memnode_self_current_mask 函数返回更专业的信息。返回的信息是连接到线程当前正在运行的处理器的内存节点。就像 NUMA_cpu_self_current_* 函数一样，当函数返回时，这些信息可能已经过时了；它只能用作提示。

NUMA_cpu_to_memnode 函数可用于将一组 CPU 映射到一组直接连接的内存节点。如果 CPU 集中只设置了一位，就可以确定每个 CPU 属于哪个内存节点。目前，Linux 不支持单个 CPU 属于多个内存节点；从理论上讲，这种情况将来可能会发生变化。要在另一个方向上映射，可以使用 NUMA_memnode_to_cpu 函数。

如果内存已经分配，有时了解其分配位置很有用。这是 NUMA_mem_get_node_idx 和 NUMA_mem_get_node_mask 允许程序员确定的。前一个函数返回内存节点的索引，在该内存节点上分配了与参数指定的地址相对应的页面，或者如果该页面尚未分配，则将根据当前安装的策略进行分配。第二个函数可以执行整个地址范围的工作；它以位集的形式返回信息。该函数的返回值是使用的不同内存节点的数量。

在本节的其余部分中，我们将看到这些接口的用例的一些示例。在所有情况下，我们都会跳过错误处理以及 CPU 和/或内存节点数量分别对于 cpu_set_t 和 memnode_set_t 类型来说太大的情况。让代码变得健壮是留给读者的练习。

确定给定 CPU 的线程兄弟姐妹

要调度辅助线程或因在给定 CPU 的线程上调度而受益的其他线程，可以使用如下代码序列。

```
cpu_set_t 当前; CPU_ZERO(&cur); CPU_SET(cpunr, &cur); cpu_set_t 超值; NUMA_cpu_level_mask(sizeof(hyperths), &hyperths, sizeof(cur), &cur, 1); CPU_CLR(cpunr, &hyperths);
```

该代码首先为 cpunr 指定的 CPU 生成一个位集。然后，该位集与指定我们正在寻找超线程的第五个参数一起传递给 NUMA_cpu_level_mask。结果在 hyperths 位集中返回。剩下要做的就是清除原来 CPU 对应的位。

确定给定 CPU 的核心兄弟姐妹

如果两个线程不应调度在两个超线程上，但可以从缓存共享中受益，则需要确定处理器的其他核心。下面的代码序列就可以解决这个问题。

```
cpu_set_t 当前; CPU_ZERO(&cur); CPU_SET(cpunr, &cur); cpu_set_t 超值; int nhts = NUMA_cpu_level_mask(sizeof(hyperths), &hyperths, sizeof(cur), &cur, 1); cpu_set_t coreths; int ncs = NUMA_cpu_level_mask(sizeof(coreths), &coreths, sizeof(cur), &cur, 2); CPU_XOR(&coreths, &coreths, &hyperths); ncs -= nhts;
```

代码的第一部分与确定超线程的代码相同。这并非巧合，因为我们必须将给定 CPU 的超线程与其他内核区分开来。这是在第二部分中实现的，再次调用 NUMA_cpu_level_mask，但这次的级别为 2。剩下要做的就是从结果中删除给定 CPU 的所有超线程。变量 nhts 和 ncs 用于跟踪各个位集中的位数。

生成的掩码可用于调度另一个线程。如果不需要显式调度其他线程，则可以将有关要使用的核心的决定留给操作系统。否则，可以迭代运行以下代码：

```
while (ncs > 0) { size_t idx = 0; while (!CPU_ISSET(idx, &ncs)) ++idx; CPU_ZERO(&cur); CPU_SET(idx, &cur); nhts = NUMA_cpu_level_mask(sizeof(hyperths), &hyperths, sizeof(cur), &cur, 1); CPU_XOR(&coreths, &coreths, hyperths); ncs -= nhts; ... 在 CPU idx 上调度线程 ... }
```

在每次迭代中，循环都会从剩余的已使用内核中选择一个 CPU 编号。然后它计算该 CPU 的所有超线程。然后从可用内核的位集中减去（使用 CPU_XOR）得到的位集。如果 XOR 运算没有删除任何内容，则确实有问题。ncs 变量已更新，我们已为下一轮做好准备，但不是在做出调度决策之前。最后，根据程序的要求，可以使用 idx、cur 或 hyperths 中的任何一个来调度线程。通常，最好给操作系统留下尽可能多的自由，因此，使用 hyperths 位集，以便操作系统可以选择最佳的超线程。

E指数

阿比,

. 54 访问模式,

. 4 激活, .

. 9 地址多路分解, 6 地

址窥探, 16 地

址空间布局随机化,

59 别名, 15 别名,

. 50 阿尔法衰变,

瓶颈, 105 算术逻辑单元,

. 3 分支预测,

. 29 应用程序二进制接口,

14, 56 连拍速度, 54 算术逻辑

单元, 8 29 ASID,

. 1, 13 访问成本,

. 40 ASLR, . 16 关联性,

. 18 一致性, 59 异步 D

RAM, . 16, 25f. 污染, 8

. 60 更换策略,

. 14 标签,

. 15 电容器,

. 5 充

电曲线, 6 容量

高速缓存未命中, 34 C

AS, 68, 89 CAS,

. 7

ff. 延迟,

. 8 单元 CPU,

. 95 CL,

. 8 个协处理器,

. 95 列地址选

择, 48 DDR, DDR1,

. 7 命令率,

. 8 商品硬件, . 10 DDR2

. 3 通用系统接口,

. 8, 10 DDR3, 4

比较和交换, 68, 89 强制缓存未

命中, 34 冲突未命中,

Ulrich Drenper, 54 宇宙射线,

. 57 LSD,

DMA, 3, 12 双错误检测,

. 106 双泵,

. 10 DRAM,

. 3, 5 动态 RAM,

. 5 ECC 动

态随机存取存储器,

. 105 纠错码,

. 05 独占高速缓存,

. 16 虚假分享,

. 65 FB-DRAM,

. 11 福特兰

语言, 50 前端总线,

. 8 前端总线, .

. 3、8f、11、14

、70 全关联高速缓存,

. 17 全双工

总线, 12 冠

状病毒, 85 杰纳西奥,

. 95 获取conf,

. 50 汉明码,

. 106 硬件预取,

. 60 辅助线程,

. 63 巨大_TLB,

. 88

巨大的LBFS,

. 88 超级运输,

. 43 超线程,

. 29, 63 管理程序,

. 31, 39 包容性高速缓存

, 16, 78 延迟,

. 8、23、30、60、95 泄漏,

. 5 法学学士/SC,

. 68, 89 加载

锁定/条件存储,

. 68、89 环路流检测器,

. 57 LSD,

. 57 地图固定

操作控制,
.102 报告,
.103 OProfile,
太平洋,
102 无序,
40 页表行走,
60
.37 并行端口,
.3 亚太
旅游协会,
.3 PCI Express, .
.
.3 PCI-E,
.3 PGO,
四泉浦,85 物理地址,
内存,6 预故障,
——3 分布式充电,
——3
8 RAS, 预取触发器,
.60 预取,7ft. RAS 到 CAS 延迟
.14, 20, 60 配置文件指南优化充电,
.859
刷新,
SATA,6 注册 DRAM,
.3 SC_LEVEL1_DCA
CHE_LINESIZE,105 所有权请求,
.50 个 SDRAM,
.26 RFO,
.3, 8, 10 自修改代码,
.26, 65 行地址选择
.31 检测放大器,
.7
.6 串行端口,
.
.3 设置关联,
.18 SIMD,
.51, 95 单纠错,
.105 SMC,
.31 S
MP,
.16 表面贴装技术,
.
.29 软件预取,
.61 南桥,
标签,
.3 空间局部性,
.15 时间局部性,
.13 推测,
.13 TLB,

托伦萨,
.95 跟踪高速缓存,
.31 事务内存
,
.91 晶体管,
.5 转
换后备缓冲区,
未注册的 DRAM,39 tRAS,
.105 US
B, 9 tRCD,
向量运算,3, 128 tRP,9
5 虚拟地址,
.6 虚拟机监视器,
.31 VM
M,
我们,
.31, 39 冯诺依曼架构,9 写使能,14
.9 写
入组合,25, 48
.

F 参考书目

- [1] *Performance Guidelines for AMD Athlon™ 64 and AMD Opteron™ ccNUMA Multiprocessor Systems*。Advanced Micro Devices, 2006 年 6 月。URL http://www.amd.com/us-en/assets/content_type/white_papers_and_tech_docs/40555.pdf。5.4
- [2] Jennifer M. Anderson、Lance M. Berc、Jeffrey Dean、Sanjay Ghemawat、Monika R. Henzinger、Shun-Tak A. Leung、Richard L. Sites、Mark T. Vandevoorde、Carl A. Waldspurger 和 William E. Weihl。连续分析：所有循环都去哪儿了。 *Proceedings of the 16th ACM Symposium of Operating Systems Principles*, 第 1-14 页, 1997 年 10 月。URL <http://citeseer.ist.psu.edu/anderson97continuous.html>。7.1
- [3] 维诺德·库普、布鲁斯·雅各布、布莱恩·戴维斯和特雷弗·马奇。工作站环境中的高性能 DRAM。 *IEEE Transactions on Computers*, 50(11): 1133–1153, 2001 年 11 月。URL <http://citeseer.ist.psu.edu/476689.html>。2.1.2、2.2、2.2.1、2.2.3、10
- [4] 阿纳尔多·卡瓦略·德·梅洛。七个小矮人：gdb 之外的调试信息。在 *Proceedings of the Linux Symposium*, 2007 年。URL <https://ols2006.108.redhat.com/2007/Reprints/melo-Reprint.pdf>。6.2.1
- [5] Simon Doherty、David L. Detlefs、Lindsay Grove、Christine H. Flood、Victor Luchangco、Paul A. Martin、Mark Moir、Nir Shavit 和 Jr. Guy L. Steele。DCAS 并不是非阻塞算法设计的灵丹妙药。 *SPAA '04: Proceedings of the Sixteenth Annual ACM Symposium on Parallelism in Algorithms and Architectures*, 第 216-224 页, 美国纽约州纽约市, 2004 年。ACM 出版社。ISBN 1-58113-840-7。URL <http://research.sun.com/scalable/pubs/SPAA04.pdf>。8.1
- [6] M.道勒。DDR-2 简介：DDR 内存替代品。 <http://www.pcstats.com/articleview.cfm?articleID=1573>, 2004 年 5 月。2.2.1
- [7] 乌尔里希·德雷珀。Futexes 很棘手, 2005 年 12 月。URL <http://people.redhat.com/drepper/futex.pdf>。6.3.4
- [8] 乌尔里希·德雷珀。线程本地存储的 ELF 处理。技术报告, Red Hat, Inc., 2003 年。URL <http://people.redhat.com/drepper/tls.pdf>。6.4.1
- [9] 乌尔里希·德雷珀。红帽企业 Linux 中的安全增强, 2004 年。URL <http://people.redhat.com/drepper/nonselsec.pdf>。4.2
- [10] 多米尼克·福伯、扬·奥拉雷和斯蒂芬·莱茨。并发访问共享对象的无锁技术。GMEM, 编辑, *Actes des Journees d'Informatique Musicale JIM2002, Marseille*, 第 143–150 页, 2002 年。URL <http://www.grame.fr/pub/fober-JIM2002.pdf>。8.1, 8.1
- [11] 乔·吉比斯和大卫·帕特森。拥抱并扩展 20 世纪指令集架构。 *Computer*, 40(4): 68–75, 2007 年 4 月。8.4
- [12] 大卫·戈德堡。每个计算机科学家都应该了解的浮点运算知识。 *ACM Computing Surveys*, 23(1):5–48, 1991 年 3 月。URL <http://citeseer.ist.psu.edu/goldberg91what.html>。1
- [13] 莫里斯·赫利希和 J.艾略特·B.莫斯。事务内存：对无锁数据结构的架构支持。在 *Proceedings of 20th International Symposium on Computer Architecture*, 1993 年。URL <http://citeseer.ist.psu.edu/herlihy93transactional.html>。8.2、8.2.2、8.2.3、8.2.4
- [14] 拉姆·胡加哈利 (Ram Huggahalli)、拉维·艾耶 (Ravi Iyer) 和斯科特·泰特里克 (Scott Tetrick)。高带宽网络 I/O 的直接缓存访问, 2005 年。URL <http://www.stanford.edu/group/comparch/papers/huggahalli05.pdf>。6.3.5
- [15] *Intel® 64 and IA-32 Architectures Optimization Reference Manual*。英特尔公司, 2007 年 5 月。URL <http://www.intel.com/design/processor/manuals/248966.pdf>。B.3
- [16] 威廉·马戈、保罗·彼得森和桑吉夫·沙阿。超线程技术：对计算密集型工作负载的影响。 *Intel Technology Journal*, 6(1), 2002。URL <ftp://download.intel.com/technology/itj/2002/volume06issue01/art06computeintense/vol6iss1art06>。3.3.4
- [17] 考尔安·麦克纳马拉。控制符号排序。 <http://blogs.linux.ie/caolan/2007/04/24/controlling-symbol-ordering/>, 2007 年 4 月。7.5
- [18] *Double Data Rate (DDR) SDRAM MT46V*。美光科技, 2003 年。修订版 L 6/06 EN。2.2.2、10
- [19] 乔恩·“汉尼”·斯托克斯。Ars Technica RAM 指南, 第二部分：异步和同步 DRAM。 <http://arstechnica.com/paedia/r/ramguide/ramguide.part2-1.html>, 2004。2.2 [20] 维基百科。静态随机存取存储器。 <http://en.wikipedia.org/wiki/静态随机存取存储器>, 2006 年。2.1.1

G 修订历史

2007-6-24 First internal draft.

2007-8-11 Add first set of Johnray's edits.

2007-9 to 2007-11 Lots of language cleanup from Jonathan Corbet and team.

2007-11-21 Version 1.0.

Source File Identification	
Last Modification	D:20071121175205-08'00'
Source File Size	553180 Bytes
Source File MD5	0299BAC7C5B6A501077DEB5714C944DF
MPost Last Modification	D:20071104104838-08'00'
MPost Source File Size	138021 Bytes
MPost Source File MD5	7732297065D660A939A9D651A7992FA2